

Back-end knowledge base

Collected by László Kővári

Tartalomjegyzék

1. [Minimal API](#)
2. [EF Core Transaction Handling](#)
3. [Autofac Implementation](#)
4. [.Net 8 Clean Architecture](#)
5. [.Net Core Clean Architecture Infra Web stb](#)
6. [Autofac Benefits](#)
7. [.Net Clean Architecture Essence](#)
8. [Bounded Context](#)
9. [Hexagonal Onion Clean Architectures](#)
10. [Vertical sliced code Organisation pattern](#)
11. [Microsoft eShopOnContainers](#)
12. [.Net Reference Application Comparison](#)
13. [CQRS Command and Query responsibility Segregation by Greg Young 2010](#)
14. [MediatR](#)
15. [SOLID](#)
16. [Design Patterns](#)
17. [UnitOfWork](#)
18. [Yield](#)
19. [Covariance Contravariance](#)
20. [IEnumerable vs. List](#)
21. [Swagger és Kiota](#)
22. [EF Core Interceptors](#)
23. [EF Core Migration Parancsok](#)
24. [Keycloak](#)
25. [REST API használata .NET](#)
26. [JOIN left right inner outer in SQL and EF Core LINQ](#)
27. [gRPC service – gRPC Service létrehozása .NET 8-ban](#)
28. [xUnit Tesztesetek Példa](#)
29. [IEntityTypeConfiguration](#)
30. [EF Core Global Filter](#)
31. [File-scoped namespace](#)
32. [EF Core Joins](#)
33. [Controller return values](#)
34. [Get Set Init](#)
35. [Record vs Struct vs Class](#)
36. [Immutability C#-ban](#)
37. [Collection Framework – hash alapú gyűjtemények](#)
38. [Mit implementáljunk egy új C# class-ban \(Equals, GetHashCode, ToString stb.\)](#)
39. [EF Core relationship with Fluent API](#)
40. [EF Core enable Lazy Loading](#)
41. [Entity States](#)
42. [Add Attach Update](#)
43. [EF Core composite key](#)
44. [EF Core Concurrency control](#)
45. [EF Core TPH, TPT, TPC](#)
46. [EF Core transaction handling \(Unit of Work variáció\)](#)
47. [EF Core performance optimization strategies](#)
48. [EF Core In-Memory database](#)
49. [Middleware](#)
50. [JWT, OAuth, OpenId Connect](#)
51. [ASP.NET core Caching strategies](#)
52. [Extension method – klasszikus alap](#)
53. [.NET 7 Újdonságai C# 11](#)
54. [Extension method – minden, amit tudni érdemes](#)
55. [Mikor használjunk sync és mikor async API metódusokat](#)
56. [SQL lassú query – mit kell ellenőrizni](#)
57. [.NET 8 Újdonságai C# 12](#)
58. [.NET 9 Újdonságai C# 13](#)
59. [.NET 10 Újdonságai C# 14](#)
60. [Dependency Injection Lifetime-ok \(Transient, Scoped, Singleton\)](#)
61. [Options Pattern \(IOptions, IOptionsSnapshot, IOptionsMonitor\)](#)
62. [Globális kivételkezelés és ProblemDetails](#)
63. [Strukturált logolás – ILogger és Serilog](#)
64. [HttpClientFactory és Polly \(resilience\)](#)
65. [Háttérfolyamatok – IHostedService és BackgroundService](#)

66. [Nullable Reference Types](#)
67. [Async/await Best Practices és CancellationTokens](#)
68. [Health Checks](#)
69. [API Versioning](#)
70. [Unit Testing – Mocking Moq-kal](#)
71. [Alkalmazásszintű \(in-memory\) Concurrency Handling – szálbiztos programozás](#)
72. [xUnit Tesztelés Alapjai – napi szinten szükséges patternek](#)
73. [Teszt adat-előállítási patternek – Object Mother, Test Data Builder](#)
74. [CancellationToken használata API-kban – gyakorlati példa](#)
75. [Saga Orchestration](#)
76. [Outbox Pattern](#)
77. [Circuit Breaker Pattern](#)
78. [Repository Pattern](#)
79. [Service Collection Extension Pattern](#)
80. [REPR Pattern – Request-Endpoint-Response](#)
81. [Domain Events](#)
82. [Decorator Pattern \(Pipeline\)](#)
83. [Specification Pattern](#)
84. [Result Pattern / Railway-Oriented Programming](#)
85. [Strangler Fig Pattern](#)
86. [Backend for Frontend \(BFF\) és API Gateway](#)
87. [Anti-Corruption Layer](#)
88. [Idempotency Pattern API-kban](#)
89. [SignalR](#)
90. [Quartz.NET – Háttérfolyamat ütemezés](#)
91. [Apache Kafka .NET-ben – alapfogalmak és első producer/consumer](#)
92. [Kafka gyakorlati minták – rebalance, offset, DLQ, kulcsválasztás](#)
93. [RabbitMQ alapfogalmak – exchange, queue, binding, routing](#)
94. [Kafka vs. RabbitMQ – mikor melyiket](#)
95. [MassTransit – alapok, konfiguráció, consumer](#)
96. [MassTransit Saga State Machine](#)
97. [MassTransit hibakezelés – retry, redelivery, error queue, circuit breaker](#)
98. [Wolverine – kódközpontú messaging és mediátor](#)
99. [Kézbesítési szemantika és idempotens consumer \(Inbox Pattern\)](#)
100. [Üzenet-szerződések verziózása és séma-evolúció](#)
101. [Event-Driven integrációs stílusok](#)
102. [Event Sourcing .NET-ben \(Marten\)](#)
103. [Eventual consistency és az elosztott tranzakciók valósága](#)
104. [Vertical Slice Architecture a gyakorlatban](#)
105. [Modular Monolith – mikor NE microservice](#)
106. [.NET Aspire – helyi orchestration, service discovery, telemetria](#)
107. [OpenTelemetry – distributed tracing, metrikák, korreláció](#)
108. [Polly v8 és Microsoft.Extensions.Resilience](#)
109. [HybridCache és elosztott gyorsítótár](#)
110. [Rate limiting és Output Caching](#)
111. [Keyed Services és haladó DI \(.NET 8+\)](#)
112. [FluentValidation és validáció a pipeline-ban](#)
113. [Objektum-mapping: Mapperly vs. AutoMapper](#)
114. [Dapper és hibrid perzisztencia](#)
115. [Testcontainers – integrációs tesztek valós infrastruktúrával](#)
116. [WebApplicationFactory – API-szintű integrációs tesztek](#)
117. [Architektúra tesztek – a szabályok kikényszerítése](#)
118. [Contract testing – szerződéses tesztek szolgáltatások között](#)
119. [System.Text.Json haladó szint](#)
120. [Minimal API haladó szint – TypedResults, endpoint filter, route group](#)
121. [Authorization – policy, requirement, claim, multi-tenant hozzáférés](#)
122. [Konfiguráció, titokkezelés és Options-validáció](#)
123. [Feature flags – funkciókapcsolók](#)
124. [Multi-tenancy .NET-ben](#)

- | | |
|--|---|
| <ul style="list-style-type: none">125. Elosztott lock és háttérfolyamatok több példányban126. Channels és IAsyncEnumerable – streaming és belső producer-consumer127. Teljesítmény: Native AOT, trimming, benchmarkolás128. Docker multi-stage build és Kubernetes-alapok129. CI/CD és adatbázis-migráció éles környezetben130. Biztonsági alapok API-knál – gyakorlati ellenőrzőlista131. Konkurenciakezelés haladó szinten – adatbázis, throttling, ütközésfeloldás132. Async/await haladó szint – ValueTask, thread pool, timeout, fire-and-forget | <ul style="list-style-type: none">133. System.Text.Json vs. Newtonsoft.Json – melyiket mikor134. Enterprise szintű globális hibakezelés – teljes felállítás135. Correlation ID – kérés-követés végponttól végpontig136. Object Mother és Test Data Builder – haladó, kettős célú megvalósítás137. Testing pyramid felállítása és eldobható adatbázis tesztenként138. Middleware részletesen – pipeline, sorrend, saját middleware139. Azonosító-generálási stratégiák .NET-ben – melyiket mikor |
|--|---|

#1 Minimal API

Verzió: .NET 6-tól (Minimal API); TypedResults , endpoint filter, route group: .NET 7-től.

A .NET Minimal API egy könnyűsúlyú megközelítése a webes API-k létrehozásának, amely a .NET 6-tal debütált, és továbbfejlődött a .NET 8-ban. A minimal API-k egyszerű és gyors megoldást nyújtanak kisebb, kevésbé bonyolult alkalmazások fejlesztéséhez, ahol nincs szükség az összes ASP.NET Core MVC funkcióra.

Előnyei:

1. Egyszerűség: A Minimal API-k kevesebb sablonkódot igényelnek, és egyszerűbb konfigurációval rendelkeznek.
2. Gyors fejlesztés: Mivel kevesebb kóddal és konfigurációval rendelkezik, könnyen és gyorsan létrehozhatók API-k.
3. Jobb teljesítmény: kevesebb szolgáltatás és overhead szükséges kisebb projektek esetén.
4. Kis erőforrásigény: alkalmas mikroservice architektúrákhoz.
5. Egyszerűbb bevezetés tanuláshoz.

Hátrányai:

1. Korlátozott funkcionalitás: MVC/Razor Pages funkciók csak minimálisan támogatottak.
2. Nehéz karbantartás komplex projektekben.
3. Kevésbé moduláris.
4. Limitált testreszabási lehetőségek (pl. komplex routing).

Mikor használjuk a Minimal API-t?

- Kis vagy közepes méretű projektek, mikroservice-ek, egyszerű REST API-k.
- Egyszerű CRUD műveletek esetén.
- Gyors prototípusok vagy teszt API-k fejlesztésére.
- Amikor a kisebb erőforrásigény és a gyorsabb válaszidő fontos szempont.

Minimal API Implementáció példák (GET, POST, PUT, DELETE)

Projekt létrehozása:

```
dotnet new web -n MinimalApiExample
cd MinimalApiExample
```

Model létrehozása:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

In-memory adatok tárolása:

```
var products = new List<Product>
{
    new Product { Id = 1, Name = "Product 1", Price = 10.00M },
    new Product { Id = 2, Name = "Product 2", Price = 20.00M },
};
```

Program.cs – teljes CRUD implementáció:

```

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var products = new List<Product>
{
    new Product { Id = 1, Name = "Product 1", Price = 10.00M },
    new Product { Id = 2, Name = "Product 2", Price = 20.00M },
};

app.MapGet("/products", () => Results.Ok(products));

app.MapGet("/products/{id:int}", (int id) =>
{
    var product = products.FirstOrDefault(p => p.Id == id);
    return product != null ? Results.Ok(product) : Results.NotFound();
});

app.MapPost("/products", (Product newProduct) =>
{
    newProduct.Id = products.Max(p => p.Id) + 1;
    products.Add(newProduct);
    return Results.Created($"/products/{newProduct.Id}", newProduct);
});

app.MapPut("/products/{id:int}", (int id, Product updatedProduct) =>
{
    var product = products.FirstOrDefault(p => p.Id == id);
    if (product == null) return Results.NotFound();
    product.Name = updatedProduct.Name;
    product.Price = updatedProduct.Price;
    return Results.NoContent();
});

app.MapDelete("/products/{id:int}", (int id) =>
{
    var product = products.FirstOrDefault(p => p.Id == id);
    if (product == null) return Results.NotFound();
    products.Remove(product);
    return Results.NoContent();
});

app.Run();

```

REST API CRUD műveletek bemutatása

- GET /products – Az összes terméket visszaadja.
- GET /products/{id} – Egy adott terméket ad vissza az id alapján.
- POST /products – Új termék hozzáadása.
- PUT /products/{id} – Egy meglévő termék frissítése.
- DELETE /products/{id} – Egy termék törlése.

Futtatás:

Összefoglalás A Minimal API egy könnyű megközelítés a .NET-ben, amely gyors fejlesztést tesz lehetővé kevesebb konfigurációval. Előnye az egyszerűség, gyors fejlesztés és jobb teljesítmény kisebb projekteknél; hátránya a korlátozott funkcionalitás és a nagyobb projektek nehezebb karbantartása. Különösen hasznos egyszerű webes szolgáltatásokhoz vagy mikroservice-ekhez.

#2 EF Core Transaction Handling

Hatékony tranzakciókezelésre akkor van szükség, amikor több műveletet kell atomikusan végrehajtani: vagy minden művelet sikeresen végrehajtódik, vagy hiba esetén minden visszagörgetésre kerül. Ez különösen fontos, ha több adatbázis-művelet egymástól függ.

Mikor van szükség tranzakciókezelésre?

- Több művelet atomikus végrehajtása (pl. rendelés leadása + készlet frissítése).
- Összetett üzleti műveletek több entitáson/aggregátumon.
- Külső rendszerekkel történő integráció.

Példa: rendelési use case tranzakciókezeléssel (.NET 8, Clean Architecture, EF Core):

```

using Microsoft.EntityFrameworkCore;
using MyApp.Domain.Entities;
using MyApp.Domain.Repositories;
using MyApp.Application.Interfaces;

namespace MyApp.Application.UseCases
{
    public class PlaceOrderUseCase
    {
        private readonly IOrderRepository _orderRepository;
        private readonly IProductRepository _productRepository;
        private readonly IAppDbContext _dbContext;

        public PlaceOrderUseCase(IOrderRepository orderRepository, IProductRepository
productRepository, IAppDbContext dbContext)
        {
            _orderRepository = orderRepository;
            _productRepository = productRepository;
            _dbContext = dbContext;
        }

        public async Task ExecuteAsync(Order order)
        {
            using (var transaction = await _dbContext.BeginTransactionAsync())
            {
                try
                {
                    await _orderRepository.AddAsync(order);

                    foreach (var item in order.Items)
                    {
                        var product = await _productRepository.GetByIdAsync(item.ProductId);
                        product.DecreaseStock(item.Quantity);
                        await _productRepository.UpdateAsync(product);
                    }

                    await _dbContext.CommitTransactionAsync();
                }
                catch (Exception)
                {
                    await _dbContext.RollbackTransactionAsync();
                    throw;
                }
            }
        }
    }
}

```

IAppDbContext.cs:

```
using Microsoft.EntityFrameworkCore.Storage;

namespace MyApp.Application.Interfaces
{
    public interface IAppDbContext
    {
        Task<IDbContextTransaction> BeginTransactionAsync();
        Task CommitTransactionAsync();
        Task RollbackTransactionAsync();
    }
}
```

AppDbContext.cs (EF Core implementáció):

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Storage;
using MyApp.Application.Interfaces;

namespace MyApp.Infrastructure.Persistence
{
    public class AppDbContext : DbContext, IAppDbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

        public async Task<IDbContextTransaction> BeginTransactionAsync() => await
        Database.BeginTransactionAsync();
        public async Task CommitTransactionAsync() => await Database.CommitTransactionAsync();
        public async Task RollbackTransactionAsync() => await
        Database.RollbackTransactionAsync();
    }
}
```

Hogyan működik a tranzakció? A PlaceOrderUseCase egy tranzakció kezdeményezésével kezdi az üzleti műveletet, elmenti a rendelést, majd frissíti a készletet minden tételhez. Ha minden sikeres, commit történik; hiba esetén rollback biztosítja az adatbázis konzisztenciáját.

Mikor van szükség tranzakciókezelésre?

- Több adatbázisművelet egy üzleti logikán belül.
- Külső rendszerek hívása esetén (pl. fizetés feldolgozása).
- Többszörös írási műveletek elkerülése, inkonzisztencia megelőzése.

Összefoglalás Az EF Core tranzakciókezelés fontos szerepet játszik a .NET 8 Clean Architecture üzleti logikájának megfelelő működésében, biztosítva hogy minden adatbázisműveletet egy "csomagként" kezeljünk.

#3 Autofac Implementation

A .NET 8 Clean Architecture Core rétegében található elemek (Aggregates, ValueObjects, Entities, Domain Services, Domain Exceptions, Domain Events, Event Handlers) központi szerepet játszanak az üzleti logika felépítésében, és függetlenek minden technológiai részlettől.

Core réteg elemeinek szerepe:

1. Aggregates (Aggregátumok) – Egy összetett üzleti objektum, amely egy vagy több entitást tartalmaz és egységként viselkedik. Felelős az üzleti szabályok érvényesítéséért és az állapot konzisztenciáért. Pl.: Order aggregátum több OrderItem-et tartalmaz.
2. ValueObjects (Értékobjektumok) – Az értékeik alapján egyenlő típusok, egyedi azonosító nélkül (pl. Money).
3. Entities (Entitások) – Egyedi azonosítóval rendelkező objektumok, állapotuk idővel változik (pl. Customer).
4. Domain Services (Domain Szolgáltatások) – Üzleti logikát valósítanak meg, amely nem tartozik egy konkrét entitáshoz vagy aggregátumhoz.
5. Domain Exceptions (Domain Kivételek) – Üzleti szabályok megsértésekor fellépő hibák kezelésére szolgálnak.
6. Domain Events (Domain Események) – A rendszerben történő fontos üzleti változásokat képviselik.
7. Event Handlers (Eseménykezelők) – Reagálnak a domain eseményekre és végrehajtják a megfelelő műveleteket.

Az Autofac használata a Clean Architecture-ben

Az Autofac egy fejlett Dependency Injection (DI) konténer, amely rugalmasságot és gazdag funkcionalitást biztosít.

Telepítés:

```
dotnet add package Autofac
dotnet add package Autofac.Extensions.DependencyInjection
```

Integráció a .NET DI-vel (Program.cs):

```

using Autofac;
using Autofac.Extensions.DependencyInjection;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Host.UseServiceProviderFactory(new AutofacServiceProviderFactory());

builder.Host.ConfigureContainer<ContainerBuilder>(containerBuilder =>
{
    containerBuilder.RegisterModule(new ApplicationModule());
    containerBuilder.RegisterModule(new InfrastructureModule());
});

var app = builder.Build();
app.MapControllers();
app.Run();

```

Autofac modulok:

ApplicationModule.cs:

```

public class ApplicationModule : Module
{
    protected override void Load(ContainerBuilder builder)
    {
        builder.RegisterType<ProcessOrderUseCase>().As<IProcessOrderUseCase>();

        builder.RegisterAssemblyTypes(typeof(OrderPlacedEventHandler).Assembly)
            .AsClosedTypesOf(typeof(INotificationHandler<>))
            .InstancePerLifetimeScope();
    }
}

```

InfrastructureModule.cs:

```

public class InfrastructureModule : Module
{
    protected override void Load(ContainerBuilder builder)
    {
        builder.RegisterType<OrderRepository>().As<IOrderRepository>()
            .InstancePerLifetimeScope();
        builder.RegisterType<EmailService>().As<IEmailService>().InstancePerLifetimeScope();
    }
}

```

Domain Events és Event Handlers regisztrálása:

```

builder.RegisterAssemblyTypes(typeof(OrderPlacedEventHandler).Assembly)
    .AsClosedTypesOf(typeof(INotificationHandler<>))
    .InstancePerLifetimeScope();

```

Összegzés Az Autofac használata a Clean Architecture-ben nagy rugalmasságot biztosít a függőségek kezelésében, különösen több aggregátummal, domain szolgáltatással, eseménnyel és eseménykezelővel dolgozva. Jól strukturálhatók és modulárisan kezelhetők a különböző rétegek, ami segíti az alkalmazás skálázhatóságát és karbantarthatóságát.

#4 .Net 8 Clean Architecture

A Clean Architecture Core rétegében található elemek (Aggregates, ValueObjects, Entities, Domain Services, Domain Exceptions, Domain Events, Event Handlers) mindegyike fontos szerepet játszik a rendszer üzleti logikájának szervezésében, különösen vertikális szeletek használatakor.

1. Aggregates

```
public class Order : AggregateRoot
{
    public int Id { get; private set; }
    public Customer Customer { get; private set; }
    public List<OrderItem> Items { get; private set; }

    public void AddItem(OrderItem item) => Items.Add(item);
}
```

2. ValueObjects

```
public class Address : ValueObject
{
    public string Street { get; private set; }
    public string City { get; private set; }
    public string PostalCode { get; private set; }

    protected override IEnumerable<object> GetEqualityComponents()
    {
        yield return Street;
        yield return City;
        yield return PostalCode;
    }
}
```

3. Entities

```
public class OrderItem : Entity
{
    public string ProductName { get; private set; }
    public decimal Price { get; private set; }
    public int Quantity { get; private set; }
}
```

4. Domain Services

```
public class OrderPricingService
{
    public decimal CalculateTotal(Order order)
    {
        return order.Items.Sum(item => item.Price * item.Quantity);
    }
}
```

5. Domain Exceptions

```
public class OrderNotFoundException : DomainException
{
    public OrderNotFoundException(int orderId)
        : base($"Order with ID {orderId} was not found.") { }
}
```

6. Domain Events

```
public class OrderPlacedEvent : IDomainEvent
{
    public int OrderId { get; }
    public OrderPlacedEvent(int orderId) => OrderId = orderId;
}
```

7. Event Handlers

```
public class OrderPlacedEventHandler : INotificationHandler<OrderPlacedEvent>
{
    public Task Handle(OrderPlacedEvent notification, CancellationToken cancellationToken)
    {
        Console.WriteLine($"Order {notification.OrderId} has been placed.");
        return Task.CompletedTask;
    }
}
```

Több vertikális szelet esetén a domain rétegek struktúrája

Egy e-kereskedelmi rendszerben két fő modul: Orders és Products, saját domain részstruktúrákkal:

```
- src/
  - Core/
    - Orders/
      - Aggregates/Order.cs
      - ValueObjects/Address.cs
      - Entities/OrderItem.cs
      - Events/OrderPlacedEvent.cs
      - Services/OrderPricingService.cs
      - Exceptions/OrderNotFoundException.cs
    - Products/
      - Aggregates/Product.cs
      - ValueObjects/Price.cs
      - Entities/ProductCategory.cs
      - Events/ProductAddedEvent.cs
      - Services/ProductService.cs
      - Exceptions/ProductNotFoundException.cs
```

Struktúra jellemzői

1. Szeparáció: minden vertikális szelet saját domain logikával rendelkezik.
2. Aggregátumok különválasztása modulonként.
3. Modul-specifikus domain események.
4. Független domain szolgáltatások, amelyek csak a saját moduljukkal kapcsolatos logikát kezelnek.

#5 .Net Core Clean Architecture Infra Web stb

A .NET 8 Clean Architecture egy jól strukturált rétegezett architektúra, amely különválasztja az üzleti logikát a technikai részletektől és az input/output felülettől.

Core réteg elemei és szerepük Aggregates, ValueObjects, Entities, Domain Services, Domain Exceptions, Domain Events, Event Handlers – ugyanazok a fogalmak, mint fentebb: az adatkonzisztencia, az üzleti szabályok és a domain-eseménykezelés biztosítására szolgálnak.

Többi réteg és azok elemei

1. Application Layer (Use Cases) – Az üzleti logika operatív, Use Case szintű megvalósítása; interfészeket biztosít a domain szolgáltatások és az infrastruktúra között.
2. Infrastructure Layer – Technikai részletek: adatbázis-kezelés, külső szolgáltatások, tárolási mechanizmusok. Repositories, Service Implementations, Database Context.
3. API (Web Layer) – Külső kommunikációs felület (REST, gRPC). Controllers, Middlewares.

Láthatósági viszonyok

- Core réteg: nem függhet más rétegtől; mindenki használhatja.
- Application réteg: függ a Core-től, nem ismeri az Infrastructure-t.
- Infrastructure réteg: implementálja a Core és Application interfészeit.

- API réteg: az Application réteget használja, függhet minden más rétegtől.

Több vertikális szelet struktúrálása

```
- src/  
  - Core/Orders/... , Core/Products/...  
  - Application/Orders/UseCases/PlaceOrderUseCase.cs, Interfaces/IOrderRepository.cs  
  - Application/Products/UseCases/AddProductUseCase.cs, Interfaces/IProductRepository.cs  
  - Infrastructure/Persistence/Orders/OrderRepository.cs, Products/ProductRepository.cs  
  - Infrastructure/Services/EmailService.cs  
  - API/Controllers/OrdersController.cs, ProductsController.cs
```

Összegzés A Core réteg elemei központi szerepet játszanak az üzleti logika megvalósításában; az Application, Infrastructure és API rétegek külön funkciókat látnak el, biztosítva a jól strukturált, skálázható, karbantartható alkalmazást. Több vertikális szelet esetén az alkalmazás elkülönített modulokra bontható.

#6 Autofac Benefits

Miért jobb az Autofac a .NET beépített DI keretrendszerénél Clean Architecture esetén?

1. Gazdagabb funkciókészlet: lifetime scope, property injection is támogatott (a beépített DI csak constructor injection-t támogat).
2. Specifikus komponens feloldás (Keyed Services):

```
builder.RegisterType<SqlOrderRepository>().Keyed<IOrderRepository>("sql");  
builder.RegisterType<NoSqlOrderRepository>().Keyed<IOrderRepository>("nosql");  
  
var container = builder.Build();  
var sqlRepository = container.ResolveKeyed<IOrderRepository>("sql");
```

3. Interceptors és AOP támogatás natívan (logolás, tranzakciókezelés, cache-elés).
4. Modularitás és bővíthetőség:

```
public class ApplicationModule : Module  
{  
    protected override void Load(ContainerBuilder builder)  
    {  
        builder.RegisterType<ProcessOrderUseCase>().As<IProcessOrderUseCase>();  
        builder.RegisterType<OrderRepository>().As<IOrderRepository>();  
    }  
}
```

5. Dynamic Assembly Scanning:

```
builder.RegisterAssemblyTypes(typeof(ProcessOrderUseCase).Assembly)  
    .AsImplementedInterfaces();
```

6. Különböző élekciklusok pontos kezelése (pl. per-lifetime-scope).

7. Integráció más eszközökkel (MediatR, Quartz stb.).

Összegzés Rugalmasság (Keyed Services, interceptorok, fejlett élekcikluskezelés), modularitás (modulalapú regisztráció) és cross-cutting concerns kezelése (AOP) teszi az Autofacot vonzóvá nagyobb Clean Architecture projektekhez, míg egyszerűbb alkalmazásokhoz a beépített DI is elegendő lehet.

#7 .Net Clean Architecture Essence

A Clean Architecture Robert C. Martin (Uncle Bob) által bevezetett architektúra, amelynek célja moduláris, karbantartható, tesztelhető szoftver felépítése. Követi a Dependency Rule-t: a külső rétegek soha nem függhetnek a belső rétegektől.

1. Domain Layer (Üzleti logika)

```

namespace MyApp.Domain.Entities
{
    public class Order
    {
        public int OrderId { get; private set; }
        public Customer Customer { get; private set; }
        public List<OrderItem> Items { get; private set; }
        public decimal TotalAmount => Items.Sum(item => item.Price * item.Quantity);

        public Order(int orderId, Customer customer)
        {
            OrderId = orderId;
            Customer = customer;
            Items = new List<OrderItem>();
        }

        public void AddItem(OrderItem item) => Items.Add(item);
    }

    public class OrderItem
    {
        public string ProductName { get; private set; }
        public decimal Price { get; private set; }
        public int Quantity { get; private set; }

        public OrderItem(string productName, decimal price, int quantity)
        {
            ProductName = productName;
            Price = price;
            Quantity = quantity;
        }
    }

    public class Customer
    {
        public string Name { get; private set; }
        public string Email { get; private set; }

        public Customer(string name, string email)
        {
            Name = name;
            Email = email;
        }
    }
}

```

2. Application Layer (Use Case-ek)


```

namespace MyApp.Application.UseCases
{
    public class ProcessOrderUseCase
    {
        private readonly IOrderRepository _orderRepository;
        private readonly IEmailService _emailService;

        public ProcessOrderUseCase(IOrderRepository orderRepository, IEmailService emailService)
        {
            _orderRepository = orderRepository;
            _emailService = emailService;
        }

        public void Execute(int orderId)
        {
            var order = _orderRepository.GetOrderById(orderId);
            if (order == null) throw new Exception("Order not found");

            if (order.TotalAmount > 0)
            {
                _emailService.SendOrderConfirmation(order.Customer.Email);
                _orderRepository.MarkAsProcessed(order);
            }
        }
    }
}

```

3. Infrastructure Layer

```

using MyApp.Domain.Entities;
using MyApp.Domain.Repositories;

namespace MyApp.Infrastructure.Repositories
{
    public class OrderRepository : IOrderRepository
    {
        private readonly ApplicationDbContext _dbContext;
        public OrderRepository(ApplicationDbContext dbContext) => _dbContext = dbContext;

        public Order GetOrderById(int orderId) =>
            _dbContext.Orders.Include(o => o.Items).FirstOrDefault(o => o.OrderId == orderId);

        public void MarkAsProcessed(Order order)
        {
            order.MarkAsProcessed();
            _dbContext.SaveChanges();
        }
    }
}

```

```

namespace MyApp.Infrastructure.Services
{
    public class EmailService : IEmailService
    {
        public void SendOrderConfirmation(string email) { /* SMTP küldés */ }
    }
}

```

4. API Layer

```

using Microsoft.AspNetCore.Mvc;
using MyApp.Application.UseCases;

namespace MyApp.API.Controllers
{
    [ApiController]
    [Route("api/orders")]
    public class OrderController : ControllerBase
    {
        private readonly ProcessOrderUseCase _processOrderUseCase;
        public OrderController(ProcessOrderUseCase processOrderUseCase) => _processOrderUseCase
        = processOrderUseCase;

        [HttpPost("{orderId}/process")]
        public IActionResult ProcessOrder(int orderId)
        {
            try
            {
                _processOrderUseCase.Execute(orderId);
                return Ok("Order processed successfully");
            }
            catch (Exception ex)
            {
                return BadRequest(ex.Message);
            }
        }
    }
}

```

#8 Bounded Context

A Bounded Context (Domain-Driven Design alapelv) esetén minden context egy különálló üzleti logikát vagy folyamatot képvisel, explicit határokkal.

1. ACL (Anti-Corruption Layer) – biztosítja, hogy a külső rendszerek/más bounded context-ek modelljei ne "szennyezzék be" a belső domaint; a konverziós logika az Infrastructure rétegben található.
2. API – külső interfész (REST/GraphQL); csak az Application Services réteget látja.
3. Infrastructure – technikai megvalósítások (adatbázis, külső szolgáltatások); a Domain/Application interfészeit implementálja.

4. Entities – csak a Domain és Application Services rétegek láthatják.
5. Value Objects – csak a Domain rétegen belül érhetők el.
6. Aggregates – csak a Domain és Application Services rétegek láthatják.
7. Repositories – interfészeik a Domain rétegben, implementációjuk az Infrastructure rétegben.
8. Factories – összetett aggregátumok/entitások létrehozási logikája; a Domain része, de az Infrastructure is használhatja.
9. Application Services – domain logikát tesz elérhetővé szolgáltatások formájában; hozzáfér a Domain-hez, de nem látja közvetlenül az Infrastructure-t.
10. Interfaces Adapters – DTO-t alakítanak domain entitássá; API és Application Services rétegek használják.

Láthatósági viszonyok összefoglalása

1. API: látja az Application Services-t; nem látja közvetlenül a Domain-t, Infrastructure-t.
2. Application Services: látja a Domain-t, Repositories interfészeket; nem látja közvetlenül az Infrastructure implementációt.
3. Domain: látja az Entities, Value Objects, Aggregates, Repositories interfészeket, Factories-t; nem látja az Infrastructure implementációt.
4. Infrastructure: látja a Repositories/Factories interfészeket; nem látja közvetlenül a Domain-t vagy Application Services-t.
5. ACL: látja az Infrastructure-t, konvertálja a külső adatokat.

A cél, hogy az üzleti logika (Domain) és a technikai megvalósítás (Infrastructure) teljesen elkülönüljön, minden réteg csak azokkal kommunikáljon, amelyekre ténylegesen szüksége van.

#9 Hexagonal Onion Clean Architectures

Mindhárom megközelítés a domén logika és a technikai részletek különválasztására, a külső infrastruktúrától való függetlenségre és rétegezésre épít.

1. Hexagonal Architecture (Ports and Adapters) – Dr. Alistair Cockburn ([hivatkozás](#)) Felépítés: a domén logika közepén, "portok" definiálják az interfészeket, "adapterek" implementálják őket. Előnyök: kiváló tesztelhetőség, technológiai függetlenség, rugalmasság. Hátrányok: túlkomplikált lehet kisebb projekteknél, sok interfész/adapter.
2. Onion Architecture – Jeffrey Palermo ([hivatkozás](#)) Felépítés: több rétegű, legbelül a domén modellek, kifelé haladva repository-k, UI, alkalmazási szolgáltatások; a külső rétegek csak a belsőkre támaszkodhatnak. Előnyök: fókusz a domén logikán, SOLID elvek, könnyen tesztelhető. Hátrányok: bonyolultabb lehet sok réteggel.

3. Clean Architecture Felépítés: domén, use case-ek, felhasználói interfész, adatbázis; a belső rétegek teljesen függetlenek a külsőktől; a függőségek csak a composition rootnál kerülnek befecskendezésre. Előnyök: rugalmasság, tesztelhetőség, könnyen cserélhető infrastruktúra. Hátrányok: hosszabb fejlesztési idő, túlzottan bonyolult lehet kisebb projekteknél.

Összefoglalás A Hexagonal Architecture adapterekkel dolgozik, az Onion Architecture rétegeket alkalmaz, a Clean Architecture a rétegezést és az use case-eket hangsúlyozza. Mindegyik jól használható modern alkalmazásokban, ha a karbantarthatóság és technológiai függetlenség fontos.

#10 Vertical sliced code Organisation pattern

Eredeti forrás: [Jimmy Bogard – Vertical Slice Architecture](#)

A vertikálisan szeletelt kód szervezési minta (Vertical Slice Architecture) lényege: ne rétegek szerint (Controllers, Services, Repositories) rendezd a kódot, hanem funkciók szerint (Account/Create, Account/Delete, Order/GetDetails). Így egy adott funkcióhoz tartozó minden kódrészlet (API, validáció, handler, domain logika) egy helyen van.

#11 Microsoft eShopOnContainers

1. Milyen architektúrát valósít meg? Többféle architektúra-elvet kombinál: elsősorban Clean Architecture (részben Onion Architecture) elveit követi, miközben a mikroszolgáltatások miatt a Bounded Context / DDD koncepció is hangsúlyos.
2. Részei és funkcionalitásuk
 - Catalog API: termékkatalógus kezelése, keresés, kategorizálás.
 - Basket API: felhasználói kosár kezelése.
 - Ordering API: vásárlási folyamat és rendelések kezelése.
 - Identity API: hitelesítés és jogosultságkezelés (Identity Server 4).
 - Payment API: fizetési tranzakciók feldolgozása.
 - Shipping API: szállítási folyamatok, nyomon követés.
 - Notification API: ügyfél értesítések (email/push).
3. Technológiák .NET 6/7, Docker, Kubernetes, RabbitMQ, gRPC, SQL Server és MongoDB, Azure Service Bus, Swagger/OpenAPI.
4. Egyes részek működése Minden API a saját domainjéhez tartozó funkciókat kezeli, saját adatbázissal.
5. DDD / Bounded Context Minden mikroszolgáltatás egy-egy bounded contexthez kapcsolódik, saját adatbázissal, elkerülve a közvetlen adatbázisközi kommunikációt.

6. Szeparálási módszer Vertikális szeparáció: minden mikroszolgáltatás önálló, teljes felelősségű vertikális szelet.
7. Lépések az eShopOnContainers elkészítéséhez
8. Projekt létrehozása és architektúra megtervezése (Bounded Context-ek).
9. Mikroszolgáltatások létrehozása (Catalog, Basket, Ordering stb.).
10. Konténerizálás Dockerrel (Dockerfile + docker-compose.yml).
11. Eseményalapú kommunikáció (RabbitMQ / Azure Service Bus).
12. Adatbázis kezelése (külön adatbázis szolgáltatásonként, EF Core migrációk).
13. Biztonság és hitelesítés (Identity Server 4, OAuth2, OpenID Connect).
14. Konténer menedzsment Kubernetes-szel.
15. CI/CD beállítása (GitHub Actions, Azure DevOps).
16. Monitoring és logolás (Prometheus, Grafana, ELK stack).

#12 .Net Reference Application Comparison

Steve Smith Clean Architecture (ardalis/CleanArchitecture)

- Architektúra: szorosan Clean Architecture, monolitikus, rétegek interfészekon/DI-n keresztül.
- Technológiák: ASP.NET Core, EF Core, MediatR, XUnit, Moq.
- Szeparálás: vertikális, de monolitikus futás.
- Bounded Context: DDD elemek jelen vannak, de nincsenek elkülönített bounded context-ek.

Milan Jovanovic Clean Architecture

- Architektúra: Clean Architecture finomítással, use case-alapú Application réteg.
- Technológiák: ASP.NET Core, EF Core, FluentValidation, Serilog, AutoMapper, MediatR (CQRS).
- Szeparálás: vertikális.
- Bounded Context: DDD elemek jelen, de nem teljesen elkülönített kontextusok.

Microsoft eShopOnContainers

- Architektúra: mikroszolgáltatás-alapú, elosztott, event-driven.
- Technológiák: .NET Core, Docker, Kubernetes, RabbitMQ, gRPC, SQL Server/MongoDB, Azure Service Bus, Identity Server 4.
- Szeparálás: vertikális, teljesen elkülönített szolgáltatások.
- Bounded Context: minden mikroszolgáltatás egy bounded context, szorosan DDD-alapú.

Működés Az első két referenciaalkalmazás monolitikusan fut, rétegek interfészeken kommunikálnak; az eShopOnContainers különálló mikroszolgáltatásokból áll, amelyek üzenetközvetítőkön és HTTP/gRPC-n kommunikálnak.

#13 CQRS Command and Query responsibility Segregation by Greg Young 2010

Eredeti forrás: [Greg Young – CQRS Documents \(PDF\)](#)

Command ≠ Query

- Command: állapotot módosító művelet, nem ad vissza adatot (legfeljebb státuszt/azonosítót). Pl. CreateOrder, UpdateUserEmail, DeleteAccount.
- Query: csak olvas, visszaadja a kért adatokat.

Külön modellek a commandhoz és a queryhez A parancs oldalon gazdag domain modell (DDD), a lekérdezési oldalon egyszerűbb, olvasásra optimalizált modell (DTO, projection).

Fejlett esetben külön adatforrások is lehetnek (pl. Event Sourcing + Read Model Projection).

Előnyök: egyszerűbb modellek, jobb skálázhatóság (olvasási oldal külön optimalizálható, pl. cache, NoSQL), tisztább felelősség (command: gazdag domain modell, query: DTO/LINQ/SQL View).

#14 MediatR

Verzió: MediatR: .NET Standard 2.0+; a 12.x óta változott licencmodell – alternatíva a Wolverine (lásd a vonatkozó fejezetet).

A MediatR a mediator pattern implementációja .NET-ben, elkerülve az erős függőségeket a komponensek között, közvetítőként a kérések (requests) és kezelőik (handlers) között. Különösen hasznos CQRS implementálásához.

Előnyei: laza csatolás, Single Responsibility Principle, egyszerűség.

Telepítés:

```
dotnet add package MediatR
# A MediatR.Extensions.Microsoft.DependencyInjection csomag a MediatR 12 óta megszűnt,
# a DI regisztráció a fő csomag része.
```

Regisztráció (Program.cs):

```

using MediatR;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
// MediatR 12+ szintaxis (a régi AddMediatR(Assembly) túlterhelés megszűnt):
builder.Services.AddMediatR(cfg => cfg.RegisterServicesFromAssemblyContaining<Program>());

var app = builder.Build();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

Query és handler:

```

public class GetUserByIdQuery : IRequest<User>
{
    public int Id { get; }
    public GetUserByIdQuery(int id) => Id = id;
}

public class GetUserByIdQueryHandler : IRequestHandler<GetUserByIdQuery, User>
{
    private readonly IUserRepository _userRepository;
    public GetUserByIdQueryHandler(IUserRepository userRepository) => _userRepository
    = userRepository;

    public async Task<User> Handle(GetUserByIdQuery request, CancellationToken
    cancellationToken) =>
        await _userRepository.GetByIdAsync(request.Id);
}

```

Command és handler:

```

public class CreateUserCommand : IRequest<int>
{
    public string Name { get; }
    public string Email { get; }
    public CreateUserCommand(string name, string email) { Name = name; Email = email; }
}

public class CreateUserCommandHandler : IRequestHandler<CreateUserCommand, int>
{
    private readonly IUserRepository _userRepository;
    public CreateUserCommandHandler(IUserRepository userRepository) => _userRepository
    = userRepository;

    public async Task<int> Handle(CreateUserCommand request, CancellationToken cancellationToken)
    {
        var user = new User { Name = request.Name, Email = request.Email };
        await _userRepository.AddAsync(user);
        return user.Id;
    }
}

```

Best Practices

1. Egyszerűen tartani a Handlereket – egy feladatra koncentrálnak.
2. Request-ek kis méretű osztályok legyenek, üzleti logika nélkül.
3. Separation of Concerns – illeszkedik a vertikális szeleteléshez és a CQRS-hez.
4. Middleware használata keresztirányú aggályokhoz (logging, tranzakciókezelés).

Tranzakciókezelés middleware a MediatR pipeline-ban:

```
public class TransactionBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
{
    private readonly DbContext _dbContext;
    public TransactionBehavior(DbContext dbContext) => _dbContext = dbContext;

    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
    CancellationToken cancellationToken)
    {
        using var transaction = await _dbContext.Database.BeginTransactionAsync();
        try
        {
            var response = await next();
            await _dbContext.SaveChangesAsync();
            await transaction.CommitAsync();
            return response;
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;
        }
    }
}
```

Regisztráció:

```
builder.Services.AddTransient(typeof(IPipelineBehavior<, >), typeof(TransactionBehavior<, >));
```

#15 SOLID

A SOLID elvek (Robert C. Martin) a kód olvashatóságát, karbantarthatóságát, rugalmasságát célozzák.

1. Single Responsibility Principle (SRP) – egy osztálynak egyetlen feladata legyen.


```
// SRP megsértése
public class UserService
{
    public void RegisterUser(string email, string password)
    {
        SendEmail(email);
        LogToFile("User registered: " + email);
    }
    private void SendEmail(string email) { }
    private void LogToFile(string message) { }
}

// SRP alkalmazása
public class UserService
{
    private readonly IEmailService _emailService;
    private readonly ILogger _logger;

    public UserService(IEmailService emailService, ILogger logger)
    {
        _emailService = emailService;
        _logger = logger;
    }

    public void RegisterUser(string email, string password)
    {
        _emailService.SendEmail(email);
        _logger.Log("User registered: " + email);
    }
}
```

2. Open/Closed Principle (OCP) – nyitott bővítésre, zárt módosításra.

```
public interface IDiscountStrategy { double GetDiscount(); }

public class RegularCustomerDiscount : IDiscountStrategy { public double GetDiscount() => 0.1; }
public class VIPCustomerDiscount : IDiscountStrategy { public double GetDiscount() => 0.2; }

public class DiscountService
{
    private readonly IDiscountStrategy _discountStrategy;
    public DiscountService(IDiscountStrategy discountStrategy) => _discountStrategy
    = discountStrategy;
    public double GetDiscount() => _discountStrategy.GetDiscount();
}
```

3. Liskov Substitution Principle (LSP) – leszármazottal helyettesíthető az őosztály a viselkedés megváltozása nélkül.

```

public abstract class Shape { public abstract int GetArea(); }

public class Rectangle : Shape
{
    public int Width { get; set; }
    public int Height { get; set; }
    public override int GetArea() ⇒ Width * Height;
}

public class Square : Shape
{
    public int Side { get; set; }
    public override int GetArea() ⇒ Side * Side;
}

```

4. Interface Segregation Principle (ISP) – kisebb, specializált interfészek nagy helyett.

```

public interface IWorkable { void Work(); }
public interface IFeedable { void Eat(); }

public class Robot : IWorkable { public void Work() { } }
public class Human : IWorkable, IFeedable { public void Work() { } public void Eat() { } }

```

5. Dependency Inversion Principle (DIP) – magas szintű modulok absztrakcióktól függjenek, ne konkrét implementációktól.

```

public interface ILogger { void Log(string message); }
public class FileLogger : ILogger { public void Log(string message) { } }

public class UserService
{
    private readonly ILogger _logger;
    public UserService(ILogger logger) ⇒ _logger = logger;
    public void RegisterUser(string email, string password) ⇒ _logger.Log("User registered: "
+ email);
}

```

Összefoglaló előnyök Karbantarthatóság, modularitás, tesztelhetőség — különösen a DIP és SRP alkalmazásával.

#16 Design Patterns

A design pattern-ek bevált tervezési megoldások ismétlődő problémákra. Három fő kategória: Kreációs, Szerkezeti, Viselkedési.

Kreációs minták

1. Singleton

```
// FIGYELEM: a "_instance ??= new Singleton()" NEM szálbiztos - két szál egyszerre is
// beléphet. Szálbiztos, lusta megvalósítás Lazy<T>-vel:
public sealed class Singleton
{
    private static readonly Lazy<Singleton> _instance =
        new(() => new Singleton(), LazyThreadSafetyMode.ExecutionAndPublication);

    private Singleton() { }
    public static Singleton Instance => _instance.Value;
}

// .NET-ben a leggyakoribb helyes megoldás azonban nem is ez a minta, hanem a DI konténer:
// builder.Services.AddSingleton<IMyService, MyService>();
```

2. Factory Method

```
public interface IProduct { void Operate(); }
public class ConcreteProductA : IProduct { public void Operate() => Console.WriteLine("Product A operation"); }
public class ConcreteProductB : IProduct { public void Operate() => Console.WriteLine("Product B operation"); }

public abstract class Creator { public abstract IProduct FactoryMethod(); }
public class ConcreteCreatorA : Creator { public override IProduct FactoryMethod() => new ConcreteProductA(); }
public class ConcreteCreatorB : Creator { public override IProduct FactoryMethod() => new ConcreteProductB(); }
```

3. Abstract Factory

```
public interface IGUIFactory { IButton CreateButton(); ICheckbox CreateCheckbox(); }
public class MacOSFactory : IGUIFactory { public IButton CreateButton() => new MacButton();
public ICheckbox CreateCheckbox() => new MacCheckbox(); }
public class WindowsFactory : IGUIFactory { public IButton CreateButton() => new WindowsButton();
public ICheckbox CreateCheckbox() => new WindowsCheckbox(); }
```

4. Builder

```
public class Car { public string Engine { get; set; } public int Seats { get; set; } }

public interface ICarBuilder { ICarBuilder SetEngine(string engine); ICarBuilder SetSeats(int seats); Car Build(); }

public class CarBuilder : ICarBuilder
{
    private Car _car = new Car();
    public ICarBuilder SetEngine(string engine) { _car.Engine = engine; return this; }
    public ICarBuilder SetSeats(int seats) { _car.Seats = seats; return this; }
    public Car Build() => _car;
}
```

5. Prototype

```

public class Prototype : ICloneable
{
    public string Name { get; set; }
    public object Clone() ⇒ MemberwiseClone();
}

```

Szerkezeti minták

6. Adapter

```

public interface ITarget { void Request(); }
public class Adaptee { public void SpecificRequest() ⇒ Console.WriteLine("Specific request"); }
public class Adapter : ITarget
{
    private readonly Adaptee _adaptee;
    public Adapter(Adaptee adaptee) ⇒ _adaptee = adaptee;
    public void Request() ⇒ _adaptee.SpecificRequest();
}

```

7. Bridge

```

public interface IImplementation { void Operation(); }
public class ConcreteImplementationA : IImplementation { public void Operation() ⇒ Console.WriteLine("Implementation A"); }

public class Abstraction
{
    protected IImplementation _implementation;
    public Abstraction(IImplementation implementation) ⇒ _implementation = implementation;
    public virtual void Operation() ⇒ _implementation.Operation();
}

public class RefinedAbstraction : Abstraction
{
    public RefinedAbstraction(IImplementation implementation) : base(implementation) { }
    public override void Operation()
    {
        _implementation.Operation();
        Console.WriteLine("Refined abstraction");
    }
}

```

8. Composite

```

public interface IComponent { void Operation(); }
public class Leaf : IComponent { public void Operation() ⇒ Console.WriteLine("Leaf"); }

public class Composite : IComponent
{
    private List<IComponent> _children = new List<IComponent>();
    public void Add(IComponent component) ⇒ _children.Add(component);
    public void Operation() { foreach (var child in _children) child.Operation(); }
}

```

9. Decorator

```
public interface IComponent { void Operation(); }
public class ConcreteComponent : IComponent { public void Operation() ⇒
Console.WriteLine("Concrete Component"); }

public class Decorator : IComponent
{
    protected IComponent _component;
    public Decorator(IComponent component) ⇒ _component = component;
    public virtual void Operation() ⇒ _component.Operation();
}

public class ConcreteDecorator : Decorator
{
    public ConcreteDecorator(IComponent component) : base(component) { }
    public override void Operation()
    {
        base.Operation();
        Console.WriteLine("Decorator Added Functionality");
    }
}
```

10. Facade

```
public class SubsystemA { public void OperationA() ⇒ Console.WriteLine("Subsystem A"); }
public class SubsystemB { public void OperationB() ⇒ Console.WriteLine("Subsystem B"); }

public class Facade
{
    private SubsystemA _subsystemA = new SubsystemA();
    private SubsystemB _subsystemB = new SubsystemB();
    public void Operation() { _subsystemA.OperationA(); _subsystemB.OperationB(); }
}
```

11. Flyweight

```
public class Flyweight
{
    public string SharedState { get; }
    public Flyweight(string sharedState) ⇒ SharedState = sharedState;
    public void Operation(string uniqueState) ⇒ Console.WriteLine($"SharedState: {SharedState},
UniqueState: {uniqueState}");
}

public class FlyweightFactory
{
    private Dictionary<string, Flyweight> _flyweights = new Dictionary<string, Flyweight>();
    public Flyweight GetFlyweight(string sharedState)
    {
        if (!_flyweights.ContainsKey(sharedState)) _flyweights[sharedState] =
new Flyweight(sharedState);
        return _flyweights[sharedState];
    }
}
```

12. Chain of Responsibility

```
public abstract class Handler
{
    protected Handler _nextHandler;
    public void SetNext(Handler handler) ⇒ _nextHandler = handler;
    public virtual void Handle(string request) ⇒ _nextHandler?.Handle(request);
}

public class ConcreteHandlerA : Handler
{
    public override void Handle(string request)
    {
        if (request == "A") Console.WriteLine("Handled by A");
        else base.Handle(request);
    }
}
```

13. Command

```
public interface ICommand { void Execute(); }
public class Light { public void TurnOn() ⇒ Console.WriteLine("Light is ON"); public void TurnOff() ⇒ Console.WriteLine("Light is OFF"); }

public class TurnOnCommand : ICommand
{
    private Light _light;
    public TurnOnCommand(Light light) ⇒ _light = light;
    public void Execute() ⇒ _light.TurnOn();
}

public class RemoteControl
{
    private ICommand _command;
    public void SetCommand(ICommand command) ⇒ _command = command;
    public void PressButton() ⇒ _command.Execute();
}
```

14. Interpreter

```

public interface IExpression { bool Interpret(string context); }

public class TerminalExpression : IExpression
{
    private string _data;
    public TerminalExpression(string data) ⇒ _data = data;
    public bool Interpret(string context) ⇒ context.Contains(_data);
}

public class OrExpression : IExpression
{
    private IExpression _expr1, _expr2;
    public OrExpression(IExpression expr1, IExpression expr2) { _expr1 = expr1; _expr2 = expr2; }
    public bool Interpret(string context) ⇒ _expr1.Interpret(context)
    || _expr2.Interpret(context);
}

```

15. Iterator

```

public interface IIterator { bool HasNext(); object Next(); }

public class ConcreteIterator : IIterator
{
    private List<object> _collection;
    private int _position = 0;
    public ConcreteIterator(List<object> collection) ⇒ _collection = collection;
    public bool HasNext() ⇒ _position < _collection.Count;
    public object Next() ⇒ _collection[_position++];
}

```

16. Mediator

```

public interface IMediator { void Notify(object sender, string ev); }

public class ConcreteMediator : IMediator
{
    private Component1 _component1;
    private Component2 _component2;
    public ConcreteMediator(Component1 component1, Component2 component2) { _component1 = component1; _component2 = component2; }

    public void Notify(object sender, string ev)
    {
        if (ev == "A") _component2.DoC();
        if (ev == "B") _component1.DoB();
    }
}

public class Component1
{
    private IMediator _mediator;
    public void SetMediator(IMediator mediator) ⇒ _mediator = mediator;
    public void DoA() { _mediator.Notify(this, "A"); }
    public void DoB() { }
}

public class Component2
{
    private IMediator _mediator;
    public void SetMediator(IMediator mediator) ⇒ _mediator = mediator;
    public void DoC() { }
}

```

17. Memento

```

public class Memento
{
    public string State { get; }
    public Memento(string state) ⇒ State = state;
}

public class Originator
{
    public string State { get; set; }
    public Memento SaveState() ⇒ new Memento(State);
    public void RestoreState(Memento memento) ⇒ State = memento.State;
}

public class Caretaker
{
    private List<Memento> _mementos = new List<Memento>();
    public void SaveMemento(Memento memento) ⇒ _mementos.Add(memento);
    public Memento GetMemento(int index) ⇒ _mementos[index];
}

```

18. Observer


```

public interface IObserver { void Update(string message); }
public class ConcreteObserver : IObserver { public void Update(string message) ⇒
Console.WriteLine("Observer notified: " + message); }

public class Subject
{
    private List<IObserver> _observers = new List<IObserver>();
    public void Attach(IObserver observer) ⇒ _observers.Add(observer);
    public void Detach(IObserver observer) ⇒ _observers.Remove(observer);
    public void Notify(string message) { foreach (var observer in _observers)
observer.Update(message); }
}

```

19. State

```

public interface IState { void Handle(Context context); }

public class ConcreteStateA : IState
{
    public void Handle(Context context)
    {
        Console.WriteLine("Handling State A");
        context.SetState(new ConcreteStateB());
    }
}

public class ConcreteStateB : IState { public void Handle(Context context) ⇒
Console.WriteLine("Handling State B"); }

public class Context
{
    private IState _state;
    public Context(IState state) ⇒ _state = state;
    public void SetState(IState state) ⇒ _state = state;
    public void Request() ⇒ _state.Handle(this);
}

```

20. Strategy

```

public interface IStrategy { void Execute(); }
public class ConcreteStrategyA : IStrategy { public void Execute() ⇒ Console.WriteLine("Strategy
A"); }
public class ConcreteStrategyB : IStrategy { public void Execute() ⇒ Console.WriteLine("Strategy
B"); }

public class Context
{
    private IStrategy _strategy;
    public void SetStrategy(IStrategy strategy) ⇒ _strategy = strategy;
    public void ExecuteStrategy() ⇒ _strategy.Execute();
}

```

21. Template Method

```

public abstract class AbstractClass
{
    public void TemplateMethod() { Step1(); Step2(); }
    protected abstract void Step1();
    protected abstract void Step2();
}

public class ConcreteClassA : AbstractClass
{
    protected override void Step1() => Console.WriteLine("Class A Step1");
    protected override void Step2() => Console.WriteLine("Class A Step2");
}

```

22. Visitor

```

public interface IElement { void Accept(IVisitor visitor); }
public class ConcreteElementA : IElement { public void Accept(IVisitor visitor) =>
visitor.VisitConcreteElementA(this); }
public class ConcreteElementB : IElement { public void Accept(IVisitor visitor) =>
visitor.VisitConcreteElementB(this); }

public interface IVisitor
{
    void VisitConcreteElementA(ConcreteElementA elementA);
    void VisitConcreteElementB(ConcreteElementB elementB);
}

public class ConcreteVisitor : IVisitor
{
    public void VisitConcreteElementA(ConcreteElementA elementA) => Console.WriteLine("Visitor
visits Element A");
    public void VisitConcreteElementB(ConcreteElementB elementB) => Console.WriteLine("Visitor
visits Element B");
}

```

Ezek a minták széles körben alkalmazhatók a .NET 8 projektekben, és segítenek a kód strukturáltabbá, rugalmasabbá és újrafelhasználhatóbbá tételében.

#17 UnitOfWork

A Unit of Work (UoW) mintázat egy tranzakciós menedzsment minta, amely biztosítja, hogy egy munkafolyamat során végrehajtott összes művelet egy tranzakcióban legyen kezelve.

Mikor használjuk?

- Több adatbázisműveletet kell egyszerre, egy tranzakcióban kezelni.
- Több repository használatakor biztosítani kell az egységes tranzakciót.
- Az adatbázis-kapcsolat optimális kezelése (minimalizált tranzakciók és hívások).

1. IUnitOfWork interfész:

```
public interface IUnitOfWork : IDisposable
{
    IUserRepository Users { get; }
    IProductRepository Products { get; }
    Task<int> SaveChangesAsync();
}
```

2. Repository interfészek:

```
public interface IUserRepository { Task<User> GetByIdAsync(int id); Task AddAsync(User user); }
public interface IProductRepository { Task<Product> GetByIdAsync(int id); Task AddAsync(Product product); }
```

3. EF Core alapú implementáció:

```
public class ApplicationDbContext : DbContext
{
    public DbSet<User> Users { get; set; }
    public DbSet<Product> Products { get; set; }
}

public class UnitOfWork : IUnitOfWork
{
    private readonly ApplicationDbContext _context;
    private IUserRepository _userRepository;
    private IProductRepository _productRepository;

    public UnitOfWork(ApplicationDbContext context) => _context = context;

    public IUserRepository Users => _userRepository ??= new UserRepository(_context);
    public IProductRepository Products => _productRepository ??= new ProductRepository(_context);

    public async Task<int> SaveChangesAsync() => await _context.SaveChangesAsync();
    public void Dispose() => _context.Dispose();
}
```

4. Repository-k implementálása:

```
public class UserRepository : IUserRepository
{
    private readonly ApplicationDbContext _context;
    public UserRepository(ApplicationDbContext context) => _context = context;
    public async Task<User> GetByIdAsync(int id) => await _context.Users.FindAsync(id);
    public async Task AddAsync(User user) => await _context.Users.AddAsync(user);
}

public class ProductRepository : IProductRepository
{
    private readonly ApplicationDbContext _context;
    public ProductRepository(ApplicationDbContext context) => _context = context;
    public async Task<Product> GetByIdAsync(int id) => await _context.Products.FindAsync(id);
    public async Task AddAsync(Product product) => await _context.Products.AddAsync(product);
}
```

5. Használat egy szolgáltatásban:

```
public class UserService
{
    private readonly IUnitOfWork _unitOfWork;
    public UserService(IUnitOfWork unitOfWork) => _unitOfWork = unitOfWork;

    public async Task RegisterUserAsync(User user, Product product)
    {
        await _unitOfWork.Users.AddAsync(user);
        await _unitOfWork.Products.AddAsync(product);
        await _unitOfWork.SaveChangesAsync();
    }
}
```

Előnyök

1. Tranzakciós integritás – minden adatbázisművelet egyetlen tranzakcióban.
2. Könnyebb menedzsment – egy helyen kezelhetők a repository műveletek.
3. Tesztelhetőség – könnyen mockolható.

#18 Yield

Verzió: C# 2.0-tól (`yield return` / `yield break`).

A `yield return` egy speciális kulcsszó, amellyel lusta kiértékelésű (lazy evaluation) iterátorokat hozhatunk létre. Az elemek egyesével kerülnek visszaadásra a teljes kollekció egyszerre történő visszaadása helyett, memóriát spórolva.

Mikor használjuk?

1. Lusta kiértékelés (Lazy Evaluation) – nagy kollekciók, adatbázis-lekérdezések esetén.
2. Több elem egyesével való visszaadása előre létrehozott lista nélkül.
3. Memóriahatékonyság nagy kollekcióknál.
4. Végtelen sorozatok kezelése (generátorok).

Egyszerű iterátor:

```
public static IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
    yield return 3;
}
```

Végtelen generátor:

```
public static IEnumerable<int> GenerateNumbers()
{
    int number = 0;
    while (true) yield return number++;
}
```

Összetett iteráció (bináris fa bejárás):

```
public class TreeNode
{
    public int Value { get; set; }
    public TreeNode Left { get; set; }
    public TreeNode Right { get; set; }
}

public static IEnumerable<int> Traverse(TreeNode node)
{
    if (node == null) yield break;
    foreach (var val in Traverse(node.Left)) yield return val;
    yield return node.Value;
    foreach (var val in Traverse(node.Right)) yield return val;
}
```

Előnyök: memóriahatékonyság, egyszerűség, lusta kiértékelés, ami javítja a teljesítményt nagy adathalmazoknál vagy végtelen sorozatoknál.

#19 Covariance Contravariance

Verzió: C# 4.0-tól (generikus `in` / `out` variancia).

A covariance és contravariance a típusparaméterek rugalmas kezelését teszik lehetővé generikus interfészek és delegáltak esetén öröklési hierarchiában.

1. Covariance – lehetővé teszi, hogy egy általános típusból származó típusparamétert egy származtatott típusú objektummal helyettesítsünk (kimeneti típusok, `out` kulcsszó).

```
public class Animal { public string Name { get; set; } }
public class Dog : Animal { public string Breed { get; set; } }

IEnumerable<Dog> dogs = new List<Dog> { new Dog { Name = "Rex" } };
IEnumerable<Animal> animals = dogs; // Covariance
```

2. Contravariance – bemeneti típusok (`in` kulcsszó); egy általánosabb típus helyett speciálisabb típust fogadhatunk el bemenetként.

```
Action<Animal> action = DoSomethingWithAnimal;
Action<Dog> dogAction = action; // Contravariance

void DoSomethingWithAnimal(Animal animal) => Console.WriteLine("Animal: " + animal.Name);
```

Covariant interfész (out):

```
public interface ICovariant<out T> { T GetItem(); }
```

Contravariant interfész (in):

```
public interface IContravariant<in T> { void SetItem(T item); }
```

Összefoglalás Covariance kimeneti típusokra (pl. IEnumerable), contravariance bemeneti típusokra (pl. Action, IComparer) használható, nagy rugalmasságot biztosítva öröklési hierarchiában.

#20 IEnumerable vs. List

Az IEnumerable gyakran előnyösebb a List-hez képest: lusta kiértékelés, teljesítményoptimalizálás, rugalmasság, memóriahasználat.

1. Lusta kiértékelés – az IEnumerable műveletei nem futnak azonnal, csak amikor ténylegesen iterálunk.

```
public static IEnumerable<int> GetNumbers()
{
    for (int i = 1; i ≤ 5; i++) yield return i;
}
```

2. Memóriahatékonyság – nagy adathalmazoknál az IEnumerable nem tölti be egyszerre az összes elemet, ellentétben a List-vel.

3. Teljesítményoptimalizálás – szűrés (Where) csak akkor fut, amikor iterálunk rajta:

```
var numbers = Enumerable.Range(1, 1000).Where(x ⇒ x % 2 == 0); // IEnumerable, lusta
var numbers2 = Enumerable.Range(1, 1000).ToList().Where(x ⇒ x % 2 == 0); // List, azonnali
```

4. Rugalmasság és absztrakció – nem kötődik konkrét kollekciós típushoz, adatbázisból, fájlból, bármilyen forrásból származhat.
5. Amikor nem szükséges azonnali hozzáférés – a List indexelést biztosít, az IEnumerable nem.

Mikor érdemes List-t használni?

- Közvetlen indexelésre van szükség.
- Azonnali műveletekre van szükség.
- Többszöri bejárás szükséges (List-t egyszer töltjük fel, majd újrahasználjuk).

Összefoglalás Az IEnumerable előnyösebb lusta kiértékeléshez és memóriahatékonysághoz; a List előnyösebb, ha indexelés vagy többszöri bejárás szükséges.

#21 Swagger és Kiota

Verzió: Swashbuckle: ASP.NET Core 1.0-tól; a .NET 9 sablonokban beépített `Microsoft.AspNetCore.OpenApi` váltja.

Swagger – API dokumentáció, tesztelés, OpenAPI specifikáció automatikus generálása.

Kiota – OpenAPI specifikációból automatikusan generál kliens kódot (típusbiztos SDK-t) az API-val való kommunikációhoz.

Mikor célszerű használni?

Swagger:

1. API dokumentáció.
2. Interaktív API tesztelés (Swagger UI).
3. Kommunikáció más fejlesztőkkel egy univerzális OpenAPI specifikáción keresztül.

Kiota:

1. Automatikus kliensgenerálás OpenAPI leírásból.
2. Kód újrafelhasználás több projektben.
3. Gyors fejlesztés, típusbiztos, automatikusan generált kód.

Swagger telepítése és konfigurálása:

```
dotnet add package Swashbuckle.AspNetCore
```

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(c => c.SwaggerEndpoint("/swagger/v1/swagger.json", "My API v1"));
}
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Elérés: <http://localhost:5000/swagger>

Kiota használata:

```
dotnet tool install --global Kiota
kiota generate -d https://example.com/swagger/v1/swagger.json -c MyApiClient -n MyNamespace
```

```
var apiClient = new MyApiClient(new HttpClient());  
var response = await apiClient.SomeEndpoint.GetAsync();
```

Összefoglalás Swagger: automatizált API dokumentáció és interaktív tesztelés. Kiota: automatikusan generált kliens SDK-k OpenAPI specifikációból, amelyek egyszerűsítik és gyorsítják az API-hívások implementációját.

#22 EF Core Interceptors

Verzió: EF Core 3.0-tól (DbCommandInterceptor); SaveChangesInterceptor : EF Core 5.0-tól.

Az EF Core Interceptor lehetővé teszi, hogy közvetlenül beavatkozzunk az EF Core folyamataiba (SQL parancsok, tranzakciók, kapcsolatnyitás/zárás, hibafigyelés).

Mikor célszerű használni?

1. SQL parancsok logolása/módosítása.
2. Globális auditálás.
3. Tranzakciók monitorozása.
4. Globális szabályok betartatása (pl. soft delete, multi-tenant).
5. Hibakezelés.
6. Query teljesítmény mérés.

Előnyök: központi vezérlés, hatékony auditálás, elválasztás az üzleti logikától, automatikus műveletek.

Hátrányok: teljesítmény overhead, komplexitás, potenciális visszaélések.

1. SQL parancsok logolása:

```
public class CommandInterceptor : DbCommandInterceptor  
{  
    public override InterceptionResult<int> NonQueryExecuting(DbCommand command, CommandEventData  
    eventData, InterceptionResult<int> result)  
    {  
        Console.WriteLine($"Executing command: {command.CommandText}");  
        return base.NonQueryExecuting(command, eventData, result);  
    }  
}
```

2. Globális auditálás (Soft Delete):


```
// FONTOS: a soft delete-et a mentés ELŐTT (SavingChanges) kell elvégezni - a SavedChanges
// már a törlés végrehajtása után fut, ott az állapot átírása hatástalan.
public class SoftDeleteInterceptor : SaveChangesInterceptor
{
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result, CancellationToken
        cancellationToken = default)
    {
        if (eventData.Context is null) return base.SavingChangesAsync(eventData,
            result, cancellationToken);

        foreach (var entry in eventData.Context.ChangeTracker.Entries<ISoftDelete>()
            .Where(e => e.State == EntityState.Deleted))
        {
            entry.State = EntityState.Modified;
            entry.Entity.IsDeleted = true;
        }

        return base.SavingChangesAsync(eventData, result, cancellationToken);
    }
}
```

3. Tranzakció naplózás:

```
public class TransactionInterceptor : DbTransactionInterceptor
{
    public override InterceptionResult TransactionStarting(DbTransaction transaction,
        TransactionEventData eventData, InterceptionResult result)
    {
        Console.WriteLine($"Transaction started: {eventData.TransactionId}");
        return base.TransactionStarting(transaction, eventData, result);
    }
}
```

Regisztrálás:

```
public class MyDbContext : DbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder
            .UseSqlServer("your_connection_string")
            .AddInterceptors(new CommandInterceptor(), new SoftDeleteInterceptor(),
            new TransactionInterceptor());
    }
}
```

Amire figyelni kell: teljesítmény, tranzakciós problémák, versenyhelyzetek több interceptor esetén.

#23 EF Core Migration Parancsok

1. dotnet ef migrations add – új migráció létrehozása.

2. dotnet ef database update – migrációk alkalmazása az adatbázisra.
3. dotnet ef migrations remove – legutóbbi migráció törlése (csak projektből, adatbázison nem vonja vissza).
4. dotnet ef migrations list – migrációk listázása.
5. dotnet ef database update – adatbázis egy adott migráció állapotára frissítése.
6. dotnet ef database drop --force – adatbázis eltávolítása.
7. dotnet ef dbcontext scaffold "connection_string" Microsoft.EntityFrameworkCore.SqlServer – DbContext és entitások generálása meglévő adatbázisból.

Folder struktúra: a Migrations mappában jönnek létre a `YYYYMMDDHHMMSS_<migration_name>.cs` (Up/Down metódusok) és a `.Designer.cs` fájlok.

Példa:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

```
dotnet ef migrations add AddProductTable
dotnet ef database update
```

Összefoglalás: a legfontosabb parancsok a projekt gyökérkönyvtárában futtatandók, ahol a .csproj fájl található.

#24 Keycloak

Keycloak – nyílt forráskódú identitás- és hozzáférés-kezelési rendszer (hitelesítés, engedélyezés, SSO, felhasználó- és csoportkezelés).

1. Keycloak futtatása Dockerben – docker-compose.yml Keycloak + PostgreSQL szolgáltatásokkal:

```

version: '3'
services:
  keycloak:
    image: quay.io/keycloak/keycloak:latest
    environment:
      # Keycloak 26-tól: KC_BOOTSTRAP_ADMIN_USERNAME / KC_BOOTSTRAP_ADMIN_PASSWORD
      - KC_BOOTSTRAP_ADMIN_USERNAME=admin
      - KC_BOOTSTRAP_ADMIN_PASSWORD=admin
      - KC_DB=postgres
      - KC_DB_URL=jdbc:postgresql://postgres:5432/keycloak
      - KC_DB_USERNAME=keycloak
      - KC_DB_PASSWORD=password
      - KC_HOSTNAME=localhost
    ports:
      - 8080:8080
    command: start-dev
    depends_on:
      - postgres
  postgres:
    image: postgres:14
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: password
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:

```

```
docker-compose up -d
```

2. Keycloak beállítása – Admin Console (<http://localhost:8080>), Realm és Client létrehozása (confidential típus), Redirect URI beállítása.

3. .NET 8 projekt beállítása

```

dotnet add package Microsoft.AspNetCore.Authentication.OpenIdConnect
dotnet add package Microsoft.IdentityModel.Protocols.OpenIdConnect
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer

```

appsettings.json:

```
{
  "Authentication": {
    "Keycloak": {
      "Authority": "http://localhost:8080/realms/MyAppRealm",
      "ClientId": "my-dotnet-client",
      "ClientSecret": "your-client-secret",
      "CallbackPath": "/signin-oidc",
      "ResponseType": "code",
      "SaveTokens": true,
      "UsePkce": true
    }
  }
}
```

Program.cs:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.Authority = builder.Configuration["Authentication:Keycloak:Authority"];
    options.Audience = builder.Configuration["Authentication:Keycloak:ClientId"];
    options.RequireHttpsMetadata = false;
    options.TokenValidationParameters = new
Microsoft.IdentityModel.Tokens.TokenValidationParameters
    {
        ValidateIssuerSigningKey = true,
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true
    }
});

builder.Services.AddAuthorization();

var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Védett API végpont:

```

[Authorize]
[ApiController]
[Route("api/[controller]")]
public class SecureController : ControllerBase
{
    [HttpGet]
    public IActionResult Get() => Ok(new { message = "This is a secured endpoint!" });
}

```

Összefoglalás: Keycloak Docker futtatás → Realm/Client konfigurálás → .NET projekt hitelesítés beállítása (OpenID Connect / JWT) → védett API végpontok.

#25 REST API használata .NET

A REST API verziózása kritikus a stabilitás és a kompatibilitás fenntartásához.

Projekt létrehozása:

```

dotnet new webapi -n MyVersionedApi
cd MyVersionedApi

```

API verziózási beállítások (Program.cs):

```

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();

builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
});

// Az Asp.Versioning.Mvc.ApiExplorer csomagban a különálló AddVersionedApiExplorer helyett
// ma az AddApiVersioning láncolt AddApiExplorer hívása a helyes:
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
})
.AddApiExplorer(options =>
{
    options.GroupNameFormat = "'v'VVV";
    options.SubstituteApiVersionInUrl = true;
});

var app = builder.Build();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

Controller v1:

```
[ApiController]
[Route("api/v{version:apiVersion}/products")]
[ApiVersion("1.0")]
public class ProductsController : ControllerBase
{
    private static List<Product> _products = new List<Product>
    {
        new Product { Id = 1, Name = "Product 1", Price = 10.0M },
        new Product { Id = 2, Name = "Product 2", Price = 20.0M },
    };

    [HttpGet]
    public IActionResult GetAll() => Ok(_products);

    [HttpGet("{id}")]
    public IActionResult GetById(int id)
    {
        var product = _products.FirstOrDefault(p => p.Id == id);
        return product == null ? NotFound() : Ok(product);
    }

    [HttpPost]
    public IActionResult Create(Product product)
    {
        product.Id = _products.Max(p => p.Id) + 1;
        _products.Add(product);
        return CreatedAtAction(nameof(GetById), new { id = product.Id }, product);
    }

    [HttpPut("{id}")]
    public IActionResult Update(int id, Product product)
    {
        var existingProduct = _products.FirstOrDefault(p => p.Id == id);
        if (existingProduct == null) return NotFound();
        existingProduct.Name = product.Name;
        existingProduct.Price = product.Price;
        return NoContent();
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        var product = _products.FirstOrDefault(p => p.Id == id);
        if (product == null) return NotFound();
        _products.Remove(product);
        return NoContent();
    }
}
```

Controller v2 (új mezővel):

```
[ApiController]
[Route("api/v{version:apiVersion}/products")]
[ApiVersion("2.0")]
public class ProductsV2Controller : ControllerBase
{
    private static List<Product> _products = new List<Product>
    {
        new Product { Id = 1, Name = "Product 1", Price = 10.0M },
    };

    [HttpGet]
    public IActionResult GetAll() => Ok(_products.Select(p => new { p.Id, p.Name, PriceInUSD = p.Price }));
}
```

Swagger integráció verziózással:

```
dotnet add package Swashbuckle.AspNetCore
```

```
builder.Services.AddSwaggerGen();
builder.Services.AddEndpointsApiExplorer();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint("/swagger/v1/swagger.json", "My API V1");
        options.SwaggerEndpoint("/swagger/v2/swagger.json", "My API V2");
    });
}
```

Összefoglalás: a verziózás biztosítja, hogy az API változtatásai ne törjék meg a régebbi kliensalkalmazásokat; a Swagger integráció megkönnyíti a verziók dokumentálását és tesztelését.

#26 JOIN left right inner outer in SQL and EF Core LINQ

1. INNER JOIN – csak a mindkét táblában megtalálható rekordokat adja vissza.

```
SELECT p.Name, c.Name
FROM Products p
INNER JOIN Categories c ON p.CategoryId = c.Id;
```

```
var result = from p in context.Products
              join c in context.Categories on p.CategoryId equals c.Id
              select new { ProductName = p.Name, CategoryName = c.Name };
```

2. LEFT JOIN (LEFT OUTER JOIN) – az összes bal oldali sort visszaadja, a jobb oldali hiányzó egyezéseket NULL-lal.

```
SELECT p.Name, c.Name
FROM Products p
LEFT JOIN Categories c ON p.CategoryId = c.Id;
```

```
var result = from p in context.Products
              join c in context.Categories on p.CategoryId equals c.Id into productCategories
              from pc in productCategories.DefaultIfEmpty()
              select new { ProductName = p.Name, CategoryName = pc != null ? pc.Name : "No
Category" };
```

3. RIGHT JOIN (RIGHT OUTER JOIN) – az összes jobb oldali sort visszaadja.

```
SELECT p.Name, c.Name
FROM Products p
RIGHT JOIN Categories c ON p.CategoryId = c.Id;
```

EF Core-ban nincs natív támogatás; LEFT JOIN-nal, megfordított táblasorrenddel érhető el:

```
var result = from c in context.Categories
              join p in context.Products on c.Id equals p.CategoryId into categoryProducts
              from cp in categoryProducts.DefaultIfEmpty()
              select new { ProductName = cp != null ? cp.Name : "No Product", CategoryName =
c.Name };
```

4. FULL OUTER JOIN – mindkét tábla összes sorát visszaadja.

```
SELECT p.Name, c.Name
FROM Products p
FULL OUTER JOIN Categories c ON p.CategoryId = c.Id;
```

EF Core-ban két LEFT JOIN Union-nal:

```
var leftJoin = from p in context.Products
                join c in context.Categories on p.CategoryId equals c.Id into productCategories
                from pc in productCategories.DefaultIfEmpty()
                select new { ProductName = p.Name, CategoryName = pc != null ? pc.Name : "No
Category" };

var rightJoin = from c in context.Categories
                 join p in context.Products on c.Id equals p.CategoryId into categoryProducts
                 from cp in categoryProducts.DefaultIfEmpty()
                 select new { ProductName = cp != null ? cp.Name : "No Product", CategoryName =
c.Name };

var fullOuterJoin = leftJoin.Union(rightJoin);
```

Mikor melyik JOIN típus használatos?

- INNER JOIN: csak azok az adatok érdekesek, amelyek mindkét táblában megtalálhatók.
- LEFT JOIN: minden sor kell a bal oldali táblából, akkor is, ha nincs egyezés.
- RIGHT JOIN: minden sor kell a jobb oldali táblából.

- FULL OUTER JOIN: minden sor kell mindkét táblából, függetlenül az egyezéstől.

#27 gRPC service – gRPC Service létrehozása .NET 8-ban

Verzió: gRPC for .NET: .NET Core 3.0-tól; gRPC JSON transcoding: .NET 7-től.

A gRPC egy nagy teljesítményű RPC keretrendszer (Google), natív .NET 8 támogatással, mikroservice architektúrákhoz és nagy teljesítményű hálózati kommunikációhoz.

Előnyei: nagy teljesítmény (Protobuf), kódgenerálás, streaming támogatás, platformfüggetlenség.

Hátrányai: nem támogatja natívan a böngészőket (gRPC-Web szükséges), komplexitás, kezdeti tanulási görbe.

1. Protobuf fájl (greet.proto):

```
syntax = "proto3";
option _namespace = "GrpcDemo";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest { string name = 1; }
message HelloReply { string message = 1; }
```

2. Szerver implementáció:

```
public class GreeterService : Greeter.GreeterBase
{
    public override Task<HelloReply> SayHello(HelloRequest request, ServerCallContext context) =>
        Task.FromResult(new HelloReply { Message = $"Hello, {request.Name}" });
}
```

Program.cs:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddGrpc();
var app = builder.Build();
app.MapGrpcService<GreeterService>();
app.MapGet("/", () => "gRPC endpoint is available");
app.Run();
```

3. Kliens létrehozása:

```
dotnet add package Grpc.Net.Client
dotnet add package Google.Protobuf
dotnet add package Grpc.Tools
```

```
var channel = GrpcChannel.ForAddress("https://localhost:5001");
var client = new Greeter.GreeterClient(channel);
var reply = await client.SayHelloAsync(new HelloRequest { Name = "World" });
Console.WriteLine(reply.Message);
```

4. Docker deploy:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 80
EXPOSE 443

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["GrpcDemo/GrpcDemo.csproj", "GrpcDemo/"]
RUN dotnet restore "GrpcDemo/GrpcDemo.csproj"
COPY . .
WORKDIR "/src/GrpcDemo"
RUN dotnet build "GrpcDemo.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "GrpcDemo.csproj" -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "GrpcDemo.dll"]
```

```
docker build -t grpcdemo .
docker run -d -p 5000:80 -p 5001:443 grpcdemo
```

Összefoglalás: projekt létrehozása → .proto definíció → szerver implementáció → kliens tesztelés → Docker release és deploy.

#28 xUnit Tesztesetek Példa

Lépésről lépésre útmutató xUnit negatív és pozitív tesztek létrehozásához FluentValidation-nal, egy User rekordhoz (Age: 1–150, Name: 4–255 karakter, Gender: Male/Female).

1. Model:

```
public record User(int Age, string Name, string Gender);
```

2. FluentValidation:

```
dotnet add package FluentValidation
```

```

using FluentValidation;

public class UserValidator : AbstractValidator<User>
{
    public UserValidator()
    {
        RuleFor(user => user.Age).InclusiveBetween(1, 150).WithMessage("Age must be between 1
and 150.");
        RuleFor(user => user.Name).NotEmpty().WithMessage("Name is required.")
.Length(4, 255).WithMessage("Name must be between 4 and 255 characters.");
        RuleFor(user => user.Gender).NotEmpty().WithMessage("Gender is required.")
.Must(gender => gender == "Male" || gender == "Female").WithMessage("Gender must be
'Male' or 'Female'.");
    }
}

```

3. xUnit + FluentValidation.TestHelper:

```

dotnet add package xunit
dotnet add package FluentValidation.TestHelper

```

4. Pozitív teszt:

```

using Xunit;
using FluentValidation.TestHelper;

public class UserValidatorTests
{
    private readonly UserValidator _validator = new();

    [Fact]
    public void Should_Validate_Valid_User()
    {
        var user = new User(30, "John Doe", "Male");
        var result = _validator.TestValidate(user);
        result.ShouldNotHaveAnyValidationErrors();
    }
}

```

Negatív tesztek:

```

[Fact]
public void Should_Have_Error_When_Age_Is_Too_Low()
{
    var user = new User(0, "John Doe", "Male");
    var result = _validator.TestValidate(user);
    result.ShouldHaveValidationErrorFor(u => u.Age).WithErrorMessage("Age must be between 1
and 150.");
}

[Fact]
public void Should_Have_Error_When_Name_Is_Empty()
{
    var user = new User(30, "", "Male");
    var result = _validator.TestValidate(user);
    result.ShouldHaveValidationErrorFor(u => u.Name).WithErrorMessage("Name is required.");
}

[Fact]
public void Should_Have_Error_When_Gender_Is_Invalid()
{
    var user = new User(30, "John Doe", "Unknown");
    var result = _validator.TestValidate(user);
    result.ShouldHaveValidationErrorFor(u => u.Gender).WithErrorMessage("Gender must be 'Male'
or 'Female'.");
}

```

Futtatás:

```
dotnet test
```

Összefoglalás: FluentValidation-t használtunk a User rekord validálására, xUnit-tal pozitív és negatív teszteket írtunk, biztosítva a hibás bemenetek megfelelő kezelését.

#29 IEntityConfiguration

Az IEntityConfiguration lehetővé teszi az entitások konfigurálását külön osztályban, ahelyett hogy a DbContext-ben adnánk meg a konfigurációkat — javítja a kód tisztaságát nagyobb projektekben.

```

public class UserConfiguration : IEntityConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> builder)
    {
        builder.ToTable("Users");
        builder.HasKey(u => u.Id);
    }
}

```

Alkalmazás a DbContext-ben:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfiguration(new UserConfiguration());
}
```

#30 EF Core Global Filter

Verzió: EF Core 2.0-tól (`HasQueryFilter`).

Globális szűrővel biztosítható, hogy az `ILogicalDelete` interfészt implementáló entitások lekérdezésekor csak a `DeletedAt == null` entitások jelenjenek meg.

```
public interface ILogicalDelete { DateTime? DeletedAt { get; set; } }

public class Resource : IEntity, ILogicalDelete
{
    public int Id { get; set; }
    public string Name { get; set; }
    public DateTime? DeletedAt { get; set; }
    public ICollection<Person> PersonResources { get; set; } = new List<Person>();
}
```

DbContext:

```
public class ApplicationDbContext : DbContext
{
    public DbSet<Resource> Resources { get; set; }
    public DbSet<Person> Persons { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<Resource>().HasQueryFilter(r => !r.DeletedAt.HasValue);
        modelBuilder.Entity<Person>()
            .HasMany(p => p.Resources)
            .WithOne()
            .OnDelete(DeleteBehavior.ClientCascade);
    }
}
```

#31 File-scoped namespace

Verzió: C# 10-től.

A file-scoped namespace (.NET 6) egyszerűsített szintaxist ad: egy namespace utasítással a teljes fájlra érvényes névtér definiálható, kevesebb behúzással.

```
namespace MyNamespace;

public class MyClass { }
```

Előnyök: egyszerűbb szintaxis, csökkentett kódmélység, jobb olvashatóság és karbantarthatóság nagyobb, moduláris projekteknél.

#32 EF Core Joins

Join típusok, példák és teljesítménybeli buktatók EF Core 8-ban.

1. Inner Join:

```
var query = from a in context.TableA
             join b in context.TableB on a.Id equals b.TableAId
             select new { a, b };
```

```
SELECT * FROM TableA a INNER JOIN TableB b ON a.Id = b.TableAId;
```

2. Left Join:

```
var query = from a in context.TableA
             join b in context.TableB on a.Id equals b.TableAId into ab
             from b in ab.DefaultIfEmpty()
             select new { a, b };
```

```
SELECT * FROM TableA a LEFT JOIN TableB b ON a.Id = b.TableAId;
```

3. Cross Join – Cartesian product, általában elkerülendő:

```
var query = from a in context.TableA
             from b in context.TableB
             select new { a, b };
```

```
SELECT * FROM TableA, TableB;
```

4. Right Join / Full Outer Join – LINQ-ban nincs natív támogatás; raw SQL vagy left/right kombinációk szükségesek.

Gyakori teljesítménybeli hibák

1. Több `.ToList()` hívás a lekérdezésben – idő előtti memóriába töltés.
2. Túlzott `.Include()` használat – felesleges adatbetöltés.
3. `AsNoTracking` elkerülése olvasás-csak lekérdezéseknél.
4. Felesleges beágyazott join-ok – növelik a lekérdezés komplexitását.

LINQ kulcsszavak

- Include – kapcsolódó adatok eager loading-ja.
- ThenInclude – beágyazott kapcsolódó adatok eager loading-ja.
- AsNoTracking – csak olvasás, nem módosító lekérdezésekhez.
- Select – projekciók a betöltött adatmennyiség csökkentésére.

#33 Controller return values

1. GET – 200 OK ha van eredmény, 404 Not Found ha nincs.

```
[HttpGet("{id}")]
public async Task<IActionResult> GetResource(int id)
{
    var resource = await _context.Resources.FindAsync(id);
    return resource == null ? NotFound() : Ok(resource);
}
```

2. POST – 201 Created új erőforrás létrehozásakor, 400 Bad Request érvénytelen adatnál.

```
[HttpPost]
public async Task<IActionResult> CreateResource([FromBody] Resource resource)
{
    if (!ModelState.IsValid) return BadRequest(ModelState);
    _context.Resources.Add(resource);
    await _context.SaveChangesAsync();
    return CreatedAtAction(nameof(GetResource), new { id = resource.Id }, resource);
}
```

3. PUT – 204 No Content sikeres frissítésnél, 404 Not Found ha nem létezik.

```
[HttpPut("{id}")]
public async Task<IActionResult> UpdateResource(int id, [FromBody] Resource resource)
{
    if (id != resource.Id) return BadRequest();
    _context.Entry(resource).State = EntityState.Modified;
    await _context.SaveChangesAsync();
    return NoContent();
}
```

4. DELETE – 204 No Content sikeres törlésnél, 404 Not Found ha nem létezik.

```
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteResource(int id)
{
    var resource = await _context.Resources.FindAsync(id);
    if (resource == null) return NotFound();
    _context.Resources.Remove(resource);
    await _context.SaveChangesAsync();
    return NoContent();
}
```

Összefoglalás GET: 200 OK / 404 Not Found · POST: 201 Created / 400 Bad Request · PUT: 204 No Content / 404 Not Found · DELETE: 204 No Content / 404 Not Found.

#34 Get Set Init

Verzió: `init` accessor: C# 9-től; `required` tagok: C# 11-től.

1. `get` / `set` – alapértelmezett hozzáférésmódosítók, változó értékekhez.

```
public int Age { get; set; }
```

2. `init` – csak inicializáláskor beállítható, immutable objektumokhoz.

```
public string Name { get; init; }
```

3. `private set` – csak az osztályon belül módosítható.

```
public DateTime CreatedAt { get; private set; }
```

4. `readonly` (mezőkhöz) – csak inicializáláskor vagy konstruktorban állítható be.

#35 Record vs Struct vs Class

Verzió: `record`: C# 9-től, `record struct`: C# 10-től.

C#-ban négy alap "kategória" közül lehet választani egy típus deklarálásakor: `class`, `record class` (röviden `record`), `struct`, `record struct`. A választás két fő tengelyen dől el: **érték- vagy referenciatípus**, illetve **mutable vagy immutable**.

Típus	Alapértelmezett szemantika	Tárolás	Alapértelmezett immutabilitás	Equals/GetHashCode
<code>class</code>	referencia	heap	mutable (kivéve ha <code>init / readonly</code> tagokkal explicit immutable-re építjük)	referencia szerinti (identity)
<code>record / record class</code>	referencia	heap	immutable by default , ha a primary constructoros szintaxist és <code>init</code> property-ket használjuk (de simán lehet <code>set</code> -tel mutable-re is írni)	érték szerinti (member-wise), fordító generálja
<code>struct</code>	érték	stack / inline (kivéve ha boxolják, vagy osztály tagja)	mutable alapból (kivéve <code>readonly struct</code>)	érték szerinti, de alapból reflection-alapú (lassabb), érdemes felülírni
<code>record struct</code>	érték	stack / inline	mutable alapból (C# 10-ben <code>record struct</code> nem immutable, csak a <code>readonly record struct</code> az)	érték szerinti , fordító generálja, optimalizált

Kulcs különbségek:

- **class**: referenciatípus, identitás alapú egyenlőség (= alapból referenciát hasonlít, hacsak nincs felülírva), önmagában semmilyen immutabilitást nem garantál – a fejlesztő felelőssége, hogy `init / readonly` mezőkkel azzá tegye.
- **record**: referenciatípus, de a fordító automatikusan generál `Equals`, `GetHashCode`, `ToString`, valamint `with` kifejezést támogató "clone" konstruktort. Elsődleges konstruktoros (`public record User(int Age, string Name);`) formában a property-k `init`-tel jönnek létre, tehát a példány létrehozás után nem módosítható – ez teszi "immutable by default"-tá, de ha explicit `{ get; set; }`-et írunk a record property-ire, az már mutable lesz.
- **struct**: értéktípus, minden hozzárendelés/paraméterátadás másolatot készít (kivéve `ref / in / out` esetén). Alapból mutable, ami hibaforrás lehet (pl. gyűjteményben tárolt struct módosítása másolaton történik). Ha immutable-nek szánjuk, célszerű `readonly struct`-ként deklarálni, és minden tagot `readonly / get -only`-ra tenni – ekkor a fordító ki is kényszeríti.
- **record struct**: a `struct` érték-szemantikáját ötvözi a `record` generált `Equals/GetHashCode/ToString/with` funkcióival. **Fontos**: önmagában NEM immutable (a property-k alapból `{ get; set; }`), csak akkor immutable, ha `readonly record struct`-ként és `init` property-kkel deklaráljuk.

Immutabilitás összefoglalva (ki mikor immutable ténylegesen):

- `class` → csak akkor immutable, ha minden tag `readonly` mező vagy `init`-only property, és nincs setter sehol.

- `record` (primary konstruktorral, `init` property-kkel) → **immutable by default**.
- `struct` → **mutable by default**, immutable csak explicit `readonly struct`-tal.
- `record struct` → **mutable by default**, immutable csak explicit `readonly record struct`-tal.

Mikor melyiket válasszuk:

- **class**: viselkedés-központú, identitással bíró entitások (pl. EF Core entity-k, service-ek, aggregate root-ok DDD-ben) – itt az identitás (adatbázis kulcs) számít, nem az érték.
- **record**: adatátviteli/érték-objektumok (DTO-k, CQRS command/query-k, value object-ek), ahol két példány akkor egyenlő, ha a tartalmuk egyezik, és a `with` kifejezéssel kényelmesen lehet módosított másolatot készíteni (`var updated = user with { Name = "Kovács" };`).
- **struct**: kis méretű (általánosan < 16 byte ökölszabály), gyakran másolt, rövid életű értékek (pl. `Point`, `Money`), ahol a heap-allokáció elkerülése teljesítmény szempontból számít.
- **record struct**: amikor struct-szerű teljesítményt szeretnénk, de value-equality-vel és `with` szintaxissal – tipikusan kis, immutable value object-ekhez (pl. koordináták, pénzösszeg + devizanem pár).

#36 Immutability C#-ban

Verzió: `init / record`: C# 9, `readonly struct`: C# 7.2, `required`: C# 11.

Az immutabilitás azt jelenti, hogy egy objektum állapota a létrehozás után nem változtatható meg. Ennek előnyei: szálbiztonság (thread-safety) extra lock nélkül, egyszerűbb reasoning a kódról (nincs váratlan mellékhatás), biztonságos megosztás több komponens/thread között, könnyebb cache-elhetőség és equality-kezelés.

Immutabilitás elérésének eszközei C#-ban:

1. `readonly` mező – csak a deklarációnál vagy a konstruktorban állítható be, utána nem.

```
public class Money
{
    public readonly decimal Amount;
    public Money(decimal amount) => Amount = amount;
}
```

2. `init` accessor – property, ami csak objektum-inicializálóban vagy konstruktorban állítható.
3. `record / with` kifejezés – immutable objektum "módosítása" új példány létrehozásával (nem a régi mutálásával):

```
public record Address(string City, string ZipCode);
var a1 = new Address("Budapest", "1011");
var a2 = a1 with { ZipCode = "1012" }; // a1 változatlan marad
```

4. `readonly struct` – a fordító kikényszeríti, hogy egyetlen tag se legyen módosítható a struct létrehozása után.

5. Immutable gyűjtemények (`System.Collections.Immutable` – `ImmutableList<T>` , `ImmutableDictionary<TKey,TValue>` , `ImmutableArray<T>` stb.) – minden módosító művelet (`Add` , `Remove`) új gyűjteményt ad vissza, az eredeti változatlan marad. Ez különbözik a `ReadOnlyCollection<T>` -től, ami csak egy "csak olvasható nézet" a mögöttes (esetleg továbbra is mutálható) gyűjteményre.

Mutable vs immutable – gyakorlati szempont:

- Mutable objektum megosztása több thread között race condition-höz vezethet, ha nincs szinkronizáció.
- Immutable objektumnál a `GetHashCode()` biztonságosan cache-elhető és stabil marad, ezért ideális `Dictionary` / `HashSet` kulcsként (lásd következő szekció) – mutable objektum hash kódjának változása "elveszett" bejegyzéshez vezethet a hash alapú gyűjteményekben.
- DDD (Domain-Driven Design) terminológiában a **Value Object**-eket célszerű immutable-nek tervezni (pl. `record` / `readonly struct`), az **Entity**-ket viszont gyakran mutable class-ként modellezzük, mert az állapotuk időben változik, de az identitásuk (pl. `Id`) állandó.

#37 Collection Framework – hash alapú gyűjtemények

A .NET gyűjtemény-keretrendszer (`System.Collections.Generic`) fő kategóriái és melyik épül hash-elésre:

Gyűjtemény	Hash alapú?	Belső struktúra	Rendezett-e	Jellemző komplexitás (átlag)
Dictionary<TKey, TValue>	igen	hash tábla (bucket-ek + GetHashCode())	nem (sorrend nem garantált)	Add/Contains/Remove: O(1)
HashSet<T>	igen	hash tábla	nem	Add/Contains/Remove: O(1)
SortedDictionary<TKey, TValue>	nem	önkiegyensúlyozó bináris fa (piros-fekete fa)	igen (kulcs szerint)	O(log n)
SortedSet<T>	nem	önkiegyensúlyozó bináris fa	igen	O(log n)
SortedList<TKey, TValue>	nem	rendezett tömb (binary search)	igen	keresés O(log n), beszúrás/törlés O(n)
List<T>	nem	dinamikus tömb	beszúrási sorrend	Contains: O(n), index elérés: O(1)
LinkedList<T>	nem	kétirányú láncolt lista	beszúrási sorrend	Contains: O(n), beszúrás/törlés csomópontnál: O(1)
Queue<T> / Stack<T>	nem	körkörös tömb	FIFO / LIFO	Enqueue/Dequeue, Push/Pop: O(1)
ConcurrentDictionary<TKey, TValue>	igen	hash tábla, lock-mentes/finomszemzés zárolás	nem	Add/Contains/Remove: átlag O(1)

Miért fontos, hogy melyik hash alapú:

- A hash alapú gyűjtemények (Dictionary , HashSet , ConcurrentDictionary) az O(1) átlagos műveleti idejüket a kulcs/elem GetHashCode() értékéből számított bucket-index alapján érik el, majd Equals() -szel ütközés (collision) esetén ellenőriznek egyezést.
- Ha a kulcsként/elemként használt típus **rosszul vagy egyáltalán nem implementálja** a GetHashCode() / Equals() párost (pl. sima class , felülírás nélkül), akkor referencia szerinti azonosság alapján kerül be a gyűjteménybe – két, tartalmilag azonos objektum két különböző bucket-be kerülhet, és Contains / TryGetValue nem fogja megtalálni a "logikailag egyenlő" elemet.
- Ha egy hash alapú gyűjteményben lévő kulcs **mutable**, és a benne tárolt objektum olyan mezője változik meg, ami a GetHashCode() számításába belép, a gyűjtemény "elveszítheti" az elemet (rossz bucket-ben keresi tovább) – ezért ajánlott immutable típusokat (pl. record , readonly struct) használni hash alapú gyűjtemény kulcsaként (lásd előző szekció).
- Egyedi IEqualityComparer<T> átadható a Dictionary / HashSet konstruktorának, ha a típus Equals / GetHashCode felülírása nem lehetséges vagy nem kívánt (pl. harmadik féltől származó típus, vagy több különböző egyenlőség-definícióra van szükség kontextusonként).

#38 Mit implementáljunk egy új C# class-ban (Equals, GetHashCode, ToString stb.)

Amikor egy új, érték-szemantikájú (nem entity-jellegű) `class`-t vagy `struct`-ot hozunk létre, célszerű átgondolni és implementálni az alábbi tagokat:

1. `Equals(object? obj)` és `Equals(T other)` (`IEquatable<T>` **implementálásával**) – így elkerülhető a boxolás/reflection-alapú alapértelmezett összehasonlítás, és explicit definiáljuk, mi számít "egyenlőnek" (pl. minden mező egyezik, vagy csak egy üzleti kulcs).

```
public class Money : IEquatable<Money>
{
    public decimal Amount { get; }
    public string Currency { get; }

    public bool Equals(Money? other) =>
        other is not null && Amount == other.Amount && Currency == other.Currency;

    public override bool Equals(object? obj) => Equals(obj as Money);

    public override int GetHashCode() => HashCode.Combine(Amount, Currency);
}
```

2. `GetHashCode()` – mindig együtt írjuk felül az `Equals`-szal (ha két objektum `Equals` szerint egyenlő, a `GetHashCode()`-juknak is egyeznie kell, különben hash alapú gyűjteményekben hibásan viselkednek – lásd előző szekció). A `System.HashCode.Combine(...)` a javasolt, beépített módja a hash-generálásnak C# 8-tól, elkerülve a kézzel írt (és gyakran hibás) prímszám-kombinálást.
3. `ToString()` – felülírása jelentősen segíti a debuggolást, logolást (pl. `($"{Amount} {Currency}"`), enélkül a típus teljes neve jelenik meg (`Namespace.Money`).
4. `=` és `≠` **operátorok felülírása** – ha az `Equals`-t érték szerintire írjuk, érdemes az operátorokat is konzisztensre írni, különben `a.Equals(b)` és `a == b` eltérő eredményt adhat (ez különösen `class`-nál gyakori hiba, mert alapból az operátor referenciát hasonlít).
5. `IComparable<T>` / `IComparable` – ha a típus sorbarendeázhető (pl. `Money`, `Priority`), érdemes implementálni, hogy `List<T>.Sort()`, `OrderBy`, `SortedSet<T>` stb. natívan tudja kezelni komparátor nélkül is.
6. `IEqualityComparer<T>` (külön osztályként) – amikor a típus alapértelmezett egyenlőségén kívül más kontextus-függő egyenlőség-definícióra is szükség van (pl. case-insensitive stringösszehasonlítás egy adott `Dictionary`-ben).

Megjegyzés: mindezt a `record` / `record struct` a fordító automatikusan legenerálja (member-wise `Equals`, `GetHashCode`, `ToString`, `=` / `≠` operátorok) – ezért érdemes ilyen érték-objektumoknál `record`-ot használni sima `class` helyett, hogy ne kelljen ezt kézzel karbantartani. `class`-nál (identitás alapú entitásoknál, pl. EF Core entity-k) viszont gyakran **szándékosan** nem írjuk felül az `Equals` / `GetHashCode`-ot érték szerintire, mert az entitás azonosságát az adatbázis-kulcs (pl. `Id`) adja, nem a mezők tartalma – ilyenkor, ha mégis egyenlőséget definiálunk, célszerű kizárólag az `Id`-

ra alapozni, és figyelni arra, hogy egy még nem mentett (Id = 0/default) entitás `GetHashCode()` - ja ne változzon a mentés után (ez klasszikus buktató EF Core-ban hash alapú gyűjteményben tárolt új entitásoknál).

#39 EF Core relationship with Fluent API

1. Egy az egyhez (One-to-One):

```
modelBuilder.Entity<User>()
    .HasOne(u => u.Profile)
    .WithOne(p => p.User)
    .HasForeignKey<Profile>(p => p.UserId);
```

2. Egy a többhöz (One-to-Many):

```
modelBuilder.Entity<User>()
    .HasMany(u => u.Posts)
    .WithOne(p => p.User)
    .HasForeignKey(p => p.UserId);
```

3. Több a többhöz (Many-to-Many):

```
modelBuilder.Entity<Course>()
    .HasMany(c => c.Students)
    .WithMany(s => s.Courses);
```

#40 EF Core enable Lazy Loading

1. Csomag telepítése:

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies
```

2. DbContext konfigurálása:

```
optionsBuilder.UseLazyLoadingProxies();
```

3. Navigációs tulajdonságok virtual-ként:

```
public class Blog
{
    public int Id { get; set; }
    public virtual ICollection<Post> Posts { get; set; }
}
```

Ez lehetővé teszi a kapcsolódó adatok automatikus betöltését, amikor szükség van rájuk.

#41 Entity States

1. Unchanged – az entitás már az adatbázisban van, nem módosult.
2. Added – új entitás, amely a következő mentéskor kerül be az adatbázisba.
3. Modified – az entitás értékei módosultak, frissíteni kell.
4. Deleted – az entitást törölni kell.
5. Detached – az entitás nincs nyomon követve az aktuális DbContext által.

Ezek az állapotok segítenek az EF Core-nak optimalizálni az adatbázis műveleteket és biztosítani az adatkonzisztenciát.

#42 Add Attach Update

1. Add() – Added állapotba helyezi, a következő mentésnél beszúrja az adatbázisba.

```
var newEntity = new Entity { Name = "New" };
context.Add(newEntity);
context.SaveChanges();
```

2. Attach() – Unchanged állapotba helyezi, nem módosítja az adatbázist, kivéve ha manuálisan állítjuk a mezőket.

```
var existingEntity = new Entity { Id = 1, Name = "Existing" };
context.Attach(existingEntity);
```

3. Update() – Modified állapotba helyezi, frissíti az adatbázist mentéskor, még változatlan értékek esetén is.

```
var entityToUpdate = new Entity { Id = 1, Name = "Updated" };
context.Update(entityToUpdate);
context.SaveChanges();
```

Összegzés: Add() új beszúráshoz, Attach() nyomon követéshez módosítás nélkül, Update() módosítások érvényesítéséhez.

#43 EF Core composite key

A kompozit kulcs több oszlop kombinációja, amelyek együtt alkotnak egyedi azonosítót.

```
public class Order
{
    public int CustomerId { get; set; }
    public int OrderId { get; set; }
    public string ProductName { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>().HasKey(o => new { o.CustomerId, o.OrderId });
}
```

Használat: ha nincs egyedi elsődleges kulcs, de két vagy több oszlop kombinációja egyediséget biztosít.

#44 EF Core Concurrency control

Verzió: EF Core 1.0-tól (`IsConcurrencyToken` , `IsRowVersion`).

A konkurenciakezelés (concurrency control) arra a problémára ad választ, hogy mi történjen, ha ugyanazt a sort/rekordot két (vagy több) tranzakció egyidejűleg próbálja olvasni és módosítani. Két alapstratégia van: **optimista** (feltételezzük, hogy ritka az ütközés, csak commit-kor ellenőrizzük) és **pesszimista** (előre zárolunk, hogy más ne is tudja módosítani, amíg mi dolgozunk rajta).

Optimista konkurenciakezelés EF Core-ban

Feltételezi, hogy nincs konfliktus; egy verziószám/timestamp jelzi az ütközést. Két gyakori megoldás:

1. `[Timestamp]` attribútummal jelölt `byte[]` oszlop (SQL Server `rowversion` / `timestamp` típus), amit a motor automatikusan léptet minden UPDATE-nél:

```
public class Resource
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Score { get; set; }
    [Timestamp]
    public byte[] RowVersion { get; set; }
}
```

2. Fluent API-val bármely mező kijelölhető konkurencia-tokenként (`IsConcurrencyToken()`), akár egy sima `int Version` / `DateTime LastModified` oszlop is, ami nem SQL Server-specifikus:

```
modelBuilder.Entity<Resource>()
    .Property(r => r.Version)
    .IsConcurrencyToken();
// vagy: modelBuilder.Entity<Resource>().UseXminAsConcurrencyToken(); // PostgreSQL xmin oszlop
```


Az EF Core a generált `UPDATE / DELETE` utasítás `WHERE` feltételébe beleteszi a konkurencia-token eredeti (betöltéskori) értékét is (`WHERE Id = @id AND RowVersion = @originalRowVersion`). Ha közben más módosította a sort, 0 sor érintődik, és az EF Core `DbUpdateConcurrencyException`-t dob:

```
try
{
    await context.SaveChangesAsync();
}
catch (DbUpdateConcurrencyException ex)
{
    var entry = ex.Entries.Single();
    var databaseValues = await entry.GetDatabaseValuesAsync();

    if (databaseValues == null)
    {
        // a sort közben törölték
    }
    else
    {
        // ütközés-feloldási stratégiák:
        // 1) Store Wins: eldobjuk a saját módosításunkat, DB értékét vesszük át
        entry.OriginalValues.SetValues(databaseValues);
        // 2) Client Wins: felülírjuk a DB-t a saját (aktuális) értékeinkkel
        entry.OriginalValues.SetValues(databaseValues); // csak a token frissül, a CurrentValues
        marad a miénk
        // 3) Merge: mezőnkénti egyeztetés, majd újra SaveChangesAsync() a frissített
        RowVersion-nel
    }
}
```

Használat: webes/API alkalmazásoknál tipikus, ahol a felhasználó "gondolkodik" a rekord felett (pl. form szerkesztése), és ritka az egyidejű módosítás – így elkerülhető a felesleges zárolási overhead.

Pesszimista konkurenciakezelés EF Core-ban

Zárolja az erőforrást olvasáskor, hogy más tranzakció ne tudja módosítani/olvasni, amíg a miénk le nem zárul. EF Core-ban ehhez nyers SQL hint vagy explicit tranzakciós izolációs szint szükséges, natív API nincs rá:

```
using var transaction = await
context.Database.BeginTransactionAsync(IsolationLevel.Serializable);

var resource = await context.Resources
    .FromSqlRaw("SELECT * FROM Resources WITH (UPDLOCK, ROWLOCK) WHERE Id = {0}", id)
    .FirstOrDefaultAsync();

resource.Score += 10;
await context.SaveChangesAsync();
await transaction.CommitAsync();
```

Használat: gyakori párhuzamos módosítási kísérletek esetén (pl. készletcsökkentés, számlaegyenleg módosítása), amikor elengedhetetlen, hogy egy tranzakció alatt mások ne érhék el/módosítsák a rekordot. Hátránya: hosszabb zárolási idő → nagyobb blocking/deadlock kockázat, rosszabb

átbocsátóképesség magas konkurrencia mellett.

Konkurenciakezelés sima SQL script esetén

ORM nélkül, közvetlen SQL-ben ugyanez a két stratégia jelenik meg, csak kézzel implementálva:

1. Optimista – verziószám/rowversion + feltételes UPDATE A `WHERE` záradékba belefoglaljuk az eredetileg beolvasott verzió/timestamp értéket, és az érintett sorok számából (`@@ROWCOUNT` / `ROW_COUNT()`) derül ki, hogy volt-e ütközés:

```
-- SQL Server
UPDATE Resources
SET Score = @newScore, RowVersion = @newRowVersion
WHERE Id = @id AND RowVersion = @originalRowVersion;

IF @@ROWCOUNT = 0
    THROW 51000, 'Concurrency conflict: a rekordot időközben más módosította.', 1;
```

```
-- PostgreSQL (kézi verzió oszloppal, mivel nincs beépített rowversion)
UPDATE resources
SET score = :new_score, version = version + 1
WHERE id = :id AND version = :original_version;
-- az alkalmazás ellenőrzi a visszaadott/érintett sorok számát (pl. RETURNING id, vagy execute
utáni rowcount)
```

Ez lényegében ugyanaz, mint amit az EF Core automatikusan generál a konkurencia-token mögött – a hívó alkalmazás felelőssége az ütközés detektálása (0 érintett sor) és a feloldási stratégia (retry, store/client wins, hibaüzenet a usernek).

2. Pesszimista – explicit sorzárolás egy tranzakción belül

```
-- SQL Server: UPDLOCK a rendszeres UPDATE-versenyhelyzet, ROWLOCK finomítja sor szintre,
HOLDLOCK ~ SERIALIZABLE-hez hasonló tartás
BEGIN TRAN;
SELECT Score FROM Resources WITH (UPDLOCK, ROWLOCK) WHERE Id = @id;
UPDATE Resources SET Score = Score + 10 WHERE Id = @id;
COMMIT;
```

```
-- PostgreSQL / Oracle: SELECT ... FOR UPDATE zárolja a kijelölt sorokat a tranzakció végéig
BEGIN;
SELECT score FROM resources WHERE id = :id FOR UPDATE;
UPDATE resources SET score = score + 10 WHERE id = :id;
COMMIT;
```

`FOR UPDATE NOWAIT` / `FOR UPDATE SKIP LOCKED` variánsokkal elkerülhető a várakozás: az előbbi azonnal hibát dob, ha a sor már zárolt, az utóbbi egyszerűen kihagyja a zárolt sorokat (tipikus felhasználás: worker/queue feldolgozó rendszerekben, ahol több worker versenyez ugyanazon feladatsorokért).

3. Izolációs szintek (isolation level) hatása A tranzakciós izolációs szint globálisan befolyásolja, mennyire "látják" egymást a párhuzamos tranzakciók, és mekkora a zárolási/ütközési kockázat:

- `READ UNCOMMITTED` – nincs zárolás olvasáskor, "dirty read" lehetséges; gyakorlatban ritkán javasolt.
- `READ COMMITTED` – az alapértelmezett a legtöbb motorban; csak commitolt adatot olvasunk, de két egymást követő olvasás eltérhet (non-repeatable read).
- `REPEATABLE READ` – egy tranzakción belül ugyanaz a lekérdezés mindig ugyanazt adja, de "phantom row" (új sorok megjelenése) még előfordulhat (MySQL InnoDB-nél ez már phantom-mentes is).
- `SERIALIZABLE` – legszigorúbb, teljes izoláció, mintha a tranzakciók sorban futnának; jelentős zárolási/blocking overhead, gyakori deadlock-kockázat magas konkurrencia mellett.
- `SNAPSHOT` (SQL Server) / MVCC alapú `READ COMMITTED` (PostgreSQL alapból ilyen) – olvasáskor nem zárol, egy konzisztens "pillanatkép" alapján dolgozik, író tranzakciók nem blokkolják az olvasókat; ütközés csak írás-írás esetén (`SNAPSHOT` esetén `update conflict` hibával).

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRAN;
-- ...
COMMIT;
```

EF Core-ban ugyanez: `context.Database.BeginTransactionAsync(IsolationLevel.Serializable)`.

4. Deadlock kezelés és retry Pesszimista zárolásnál (és magasabb izolációs szinteknél)

elkerülhetetlen a deadlock lehetősége, ha két tranzakció fordított sorrendben zárol ugyanazon erőforrásokra. Az adatbázismotor az egyik tranzakciót automatikusan megszakítja (SQL Server: hiba 1205), a hívó oldalon ezért érdemes **retry logikát** (pl. exponenciális backoff-fal, 2-3 újrapróbálkozással) beépíteni mind nyers SQL, mind EF Core (`EnableRetryOnFailure` execution strategy) esetén.

Összegzés – melyiket mikor:

- Optimista (verzió/timestamp + feltételes UPDATE) → alapértelmezett választás, ha az ütközés ritka, és elfogadható, hogy néha újra kell próbálni/értesíteni a felhasználót; jó skálázhatóság, nincs hosszú zárolás.
- Pesszimista (`UPDLOCK` / `SELECT ... FOR UPDATE`) → ha az ütközés gyakori, és kritikus, hogy egyszerre csak egy tranzakció dolgozhasson az adott soron (pl. pénzügyi egyenleg, készletfoglalás); ára a rosszabb konkurens átbocsátóképesség és a deadlock-kockázat.
- Magasabb izolációs szint (`SERIALIZABLE`) csak akkor indokolt, ha a `READ COMMITTED` /optimista megoldás mellett bizonyítottan előfordulnak konzisztencia-problémák (pl. phantom read érzékeny riportok, összegzések) – alapból a legtöbb OLTP rendszer `READ COMMITTED` -del (vagy `SNAPSHOT` -tal) fut jól.

#45 EF Core TPH, TPT, TPC

Verzió: TPH: EF Core 1.0, TPT: EF Core 5.0, TPC: EF Core 7.0.

1. TPH (Table Per Hierarchy) – minden leszármazott egy táblában, diszkriminátor oszloppal.

```
modelBuilder.Entity<Vehicle>().HasDiscriminator<string>("VehicleType");
```

Használat: kevesebb tábla, hasonló tulajdonságú típusok esetén.

2. TPT (Table Per Type) – minden entitás saját táblát kap, relációval.

```
modelBuilder.Entity<Car>().ToTable("Cars");  
modelBuilder.Entity<Bike>().ToTable("Bikes");
```

Használat: összetettebb adatstruktúrák, felesleges oszlopok elkerülése.

3. TPC (Table Per Concrete Type) – minden konkrét típus saját, teljes táblát kap, öröklés nélküli relációval. Használat: ha nincs szükség relációra a leszármazottak között, és el akarjuk kerülni a diszkriminátoroszlopot.

Összegzés: TPH hatékonyabb hasonló mezőknél, TPT részletesebb szétválasztott adatoknál, TPC ha nincs szükség kapcsolatra az entitások között.

#46 EF Core transaction handling (Unit of Work variáció)

Unit of Work nélkül – közvetlen tranzakciókezelés a DbContext-en keresztül:

```
using var transaction = await context.Database.BeginTransactionAsync();  
try  
{  
    var entity = new Entity { Name = "Test" };  
    context.Add(entity);  
    await context.SaveChangesAsync();  
  
    var otherEntity = new OtherEntity { Name = "Other Test" };  
    context.Add(otherEntity);  
    await context.SaveChangesAsync();  
  
    await transaction.CommitAsync();  
}  
catch (Exception ex)  
{  
    await transaction.RollbackAsync();  
}
```

Unit of Work mintával – tranzakciók kezelése egy UnitOfWork osztályon keresztül, ha több repository-t kezelünk egy tranzakción belül:

```

public class UnitOfWork : IUnitOfWork
{
    private readonly ApplicationDbContext _context;
    public UnitOfWork(ApplicationDbContext context) => _context = context;

    public async Task CompleteAsync()
    {
        using var transaction = await _context.Database.BeginTransactionAsync();
        try
        {
            await _context.SaveChangesAsync();
            await transaction.CommitAsync();
        }
        catch
        {
            await transaction.RollbackAsync();
            throw;
        }
    }
}

```

AutoMapper a tranzakción belül:

```

try
{
    var entity = await _context.Entities.FirstOrDefaultAsync(e => e.Id == id);
    await _context.SaveChangesAsync();
    var mappedEntity = _mapper.Map<EntityDto>(entity);
    return mappedEntity;
}
catch (Exception ex)
{
    throw;
}

```

Mikor melyik? Unit of Work nélkül egyszerűbb tranzakciókhoz elegendő; Unit of Work mintával nagyobb alkalmazásoknál, több repository-ból álló tranzakciókhoz.

#47 EF Core performance optimization strategies

Verzió: AsNoTracking : EF Core 1.0, compiled query: EF Core 2.0, AsSplitQuery : EF Core 5.0, ExecuteUpdate / ExecuteDelete : EF Core 7.0, complex types: EF Core 8.0.

1. Eager Loading – ha előre tudjuk, hogy szükség lesz kapcsolódó adatokra.

```
var orders = context.Orders.Include(o => o.Customer).ToList();
```

2. Projection – csak a szükséges mezők lekérése.

```
var customerNames = context.Customers.Select(c => new { c.Name }).ToList();
```

3. Batching – több művelet egyidejű végrehajtása.

```
context.BulkInsert(entities);
```

4. AsNoTracking – csak olvasási műveleteknél.

```
var customers = context.Customers.AsNoTracking().ToList();
```

5. Indexing – gyakran szűrt oszlopokon.

```
modelBuilder.Entity<Customer>().HasIndex(c => c.Email);
```

6. Pooling – nagy terhelésű alkalmazásoknál, csökkentve a DbContext példányosítási időt.

```
services.AddDbContextPool<ApplicationDbContext>(options =>  
options.UseSqlServer(connectionString));
```

#48 EF Core In-Memory database

Az In-Memory Database Provider elsősorban tesztelésre szolgál, valódi adatbázis-kapcsolat nélkül.

```
var options = new DbContextOptionsBuilder<ApplicationDbContext>()  
    .UseInMemoryDatabase(databaseName: "TestDb")  
    .Options;  
  
using var context = new ApplicationDbContext(options);  
context.Add(new User { Name = "Test User" });  
context.SaveChanges();  
  
var user = context.Users.FirstOrDefault();  
Console.WriteLine(user?.Name); // Test User
```

Használati esetek: egységtesztek, gyors tesztek (memóriában fut, gyorsabb a valódi adatbázisnál).

#49 Middleware

Verzió: ASP.NET Core 1.0-tól.

A middleware olyan komponens, amely az HTTP kérések és válaszok feldolgozása közben fut.

1. Hitelesítés és Jogosultságkezelés:

```
app.Use(async (context, next) =>
{
    if (!context.User.Identity.IsAuthenticated)
    {
        context.Response.StatusCode = 401;
        return;
    }
    await next();
});
```

2. Kivételkezelés:

```
app.UseExceptionHandler("/error");
```

3. Követési Naplózás:

```
app.Use(async (context, next) =>
{
    Console.WriteLine($"Request: {context.Request.Path}");
    await next();
});
```

4. Caching:

```
app.UseResponseCaching();
```

5. CORS:

```
app.UseCors(builder => builder.AllowAnyOrigin().AllowAnyMethod());
```

Működés: a middleware-ek sorrendben hívódnak meg minden kérés/válasz feldolgozásakor; a hozzáadás sorrendje határozza meg a végrehajtási sorrendet.

A teljes javasolt pipeline-sorrend, a saját middleware három megvalósítási formája, az elágazás (`UseWhen` / `Map`) és a middleware/filter/behavior elhatárolás a "Middleware részletesen" fejezetben.

#50 JWT, OAuth, OpenId Connect

JWT (JSON Web Token) – aláírt, információt hordozó token; API-khoz alkalmas azonosításra és autorizációra.

```
var tokenHandler = new JwtSecurityTokenHandler();
var token = tokenHandler.CreateToken(new SecurityTokenDescriptor
{
    Subject = new ClaimsIdentity(new[] { new Claim("sub", "user1") })
});
```

OAuth – hozzáférés-kezelési protokoll harmadik fél erőforrásaihoz (pl. Google, Facebook).

```
services.AddAuthentication().AddOAuth("provider", options =>
{
    options.ClientId = "id";
    options.ClientSecret = "secret";
    options.CallbackPath = "/signin";
});
```

OpenID Connect – az OAuth-ra épülő hitelesítési protokoll, felhasználó-azonosítási adatokkal (pl. SSO).

```
services.AddAuthentication().AddOpenIdConnect("provider", options =>
{
    options.ClientId = "id";
    options.ClientSecret = "secret";
});
```

Melyiket mikor? JWT: API hozzáférés, külső autentikáció nélkül. OAuth: harmadik fél erőforrásaihoz. OpenID Connect: felhasználó hitelesítéséhez, SSO-hoz.

#51 ASP.NET core Caching strategies

Verzió: IMemoryCache / IDistributedCache : ASP.NET Core 1.0-tól; Output Caching: .NET 7; HybridCache : .NET 9.

1. In-Memory Cache – egy szerveren futó alkalmazás, gyors lokális elérés.

```
var cacheOptions = new MemoryCacheEntryOptions().SetSlidingExpiration(TimeSpan.FromMinutes(5));
_cache.Set("key", data, cacheOptions);
var cachedData = _cache.Get("key");
```

2. Distributed Cache (Redis, SQL Server) – több szerveren futó/skálázott környezet.

```
await _distributedCache.SetStringAsync("key", data);
var cachedData = await _distributedCache.GetStringAsync("key");
```

3. Response Caching – statikus oldalak/API válaszok gyorsítótárazása.

```
[ResponseCache(Duration = 60)]
public IActionResult GetData() => Ok(data);
```

4. Cache Tag Helper – nézetkomponens gyorsítótárazása.

```
<cache expires-after="00:05:00"><p>@DateTime.Now</p></cache>
```

Összefoglalás: In-Memory gyors lokális tároláshoz, Distributed több szerveres környezethez, Response Caching statikus API válaszokhoz, Cache Tag Helper dinamikus nézetekhez.

#52 Extension method – klasszikus alap

Verzió: C# 3.0-tól; kiterjesztett tagok (extension members, property/statikus): C# 14 / .NET 10.

Egy ImageFilter osztály és egy hozzá tartozó extension method szűréshez: az ApplyFilter kiterjesztési metódus opcionálisan alkalmazza a szűrést.

```
public class ImageFilter
{
    public Guid PersonId { get; set; }
    public string? ImageType { get; set; }
}

public static class ImageExtensions
{
    public static IQueryable<Image> ApplyFilter(this IQueryable<Image> query,
        ImageFilter? filter)
    {
        if (filter == null) return query;
        query = query.Where(i => i.PersonId == filter.PersonId);
        if (!string.IsNullOrEmpty(filter.ImageType))
            query = query.Where(i => i.ImageType == filter.ImageType);
        return query;
    }
}

public class ImageRepository
{
    private readonly YourDbContext _context;
    public ImageRepository(YourDbContext context) => _context = context;

    public async Task<List<Image>> GetImagesByPersonId(ImageFilter? filter) =>
        await _context.Images.ApplyFilter(filter).ToListAsync();
}
```

#53 .NET 7 Újdonságai C# 11

- required property Name → kötelező Name property
- Generic Math → Add(T a, T b)
- ExecuteUpdateAsync (EF Core) → módosítás betöltés nélkül
- Route Grouping → Minimal API /users csoport
- TryGetNonEnumeratedCount() → hatékony elemszámlálás
- Span switch → switch "hello" span-ra
- await foreach cancellation → WithCancellation(ct)

#54 Extension method – minden, amit tudni érdemes

Verzió: C# 3.0-tól; C# 14-ben `extension` blokk (extension property, operátor, statikus tag).

Mi az extension method? Az extension method (kiterjesztési metódus) lehetővé teszi, hogy egy meglévő típushoz (akár olyanhoz is, amelynek nincs hozzáférésünk a forráskódjához, pl. `string`, `int`, `IEnumerable`) új metódust adjunk hozzá anélkül, hogy módosítanánk a típus definícióját, öröklődnénk belőle, vagy újrafordítanánk. C# 3.0 óta (2007) létezik, és a LINQ (`Where`, `Select`, `OrderBy` stb.) is extension methodokra épül.

Szintaxis és szabályok

- Csak static osztályban, static metódusként definiálható.
- Az első paraméter előtt a `this` kulcsszó jelzi, hogy melyik típust bővítjük.
- A metódus úgy hívható meg, mintha a kiterjesztett típus tagja lenne (instance method szintaxissal).

```
public static class StringExtensions
{
    public static bool IsNullOrEmptyCustom(this string value)
    {
        return string.IsNullOrEmpty(value);
    }

    public static string Truncate(this string value, int maxLength)
    {
        if (string.IsNullOrEmpty(value) || value.Length ≤ maxLength) return value;
        return value.Substring(0, maxLength) + "...";
    }
}
```

Használat:

```
string name = "Hello World";
bool isEmpty = name.IsNullOrEmptyCustom(); // false
string shortName = name.Truncate(5);      // "Hello..."
```

Generikus extension method

```
public static class EnumerableExtensions
{
    public static bool IsNullOrEmpty<T>(this IEnumerable<T>? source)
    {
        return source == null || !source.Any();
    }

    public static IEnumerable<T> WhereNotNull<T>(this IEnumerable<T?> source) where T : class
    {
        return source.Where(x ⇒ x ≠ null)!;
    }
}
```

Extension method interfészen (nagyon gyakori minta)

```
public interface IRepository<T> { IQueryable<T> GetAll(); }

public static class RepositoryExtensions
{
    public static IQueryable<T> Paginate<T>(this IQueryable<T> query, int page, int pageSize)
    {
        return query.Skip((page - 1) * pageSize).Take(pageSize);
    }
}
```

Method resolution order (fontos szabály) Ha egy típusnak van egy azonos szignatúrájú instance metódusa, akkor a fordító MINDIG az instance metódust részesíti előnyben az extension methoddal szemben. Az extension method csak "utolsó lehetőségként", overload resolution után kerül szóba, ha nincs jobban illeszkedő instance tag. Emiatt egy extension method soha nem tudja "felülírni" egy típus saját metódusát, csak akkor hívódik meg, ha az adott metódus a típuson (vagy az öröklési láncban) nem létezik.

Namespace és using Az extension methodokat tartalmazó static osztály namespace-ét be kell importálni (`using Namespace;`), különben a metódus nem lesz látható IntelliSense-ben és fordítási hibát kapunk. Ez az egyik oka annak, hogy nagyobb projekteknél célszerű az extension methodokat logikusan csoportosítva, jól elnevezett namespace-ekbe/mappákba (pl. `Extensions/StringExtensions.cs`) szervezni.

Mikor érdemes extension methodot használni?

1. Ha egy külső/lezárt (sealed vagy harmadik féltől származó, pl. `string`, `DateTime`, `IEnumerable`) típushoz szeretnénk segédfunkciót adni.
2. Fluent API kialakításához (pl. `IServiceCollection.AddXyz()` minták ASP.NET Core-ban: `AddControllers()`, `AddSwaggerGen()`).
3. LINQ-stílusú, láncolható lekérdezés-építő metódusokhoz (pl. `query.ApplyFilter(filter).Paginate(page, size)`).
4. Ha a metódus logikailag egy típushoz kapcsolódik, de nem indokolt vagy nem lehetséges azt a típus definíciójába felvenni (pl. DTO-k, generált kód, harmadik féltől jövő NuGet típusok).
5. Tesztelhetőség és Separation of Concerns javítására – a segédlogika kikerül a domain/entitás osztályból.

Mikor NE használjunk extension methodot?

- Ha a logika szorosan a típus belső állapotához (privát mezőikhez) kapcsolódik – akkor inkább instance metódus vagy a típus saját tagja legyen.
- Ha a metódus üzleti/domain logikát valósít meg, amelynek a domain modellen (Rich Domain Model) lenne a helye – ne "extension methodosítsunk ki" mindent a tesztelhetőség ürügyén, mert szétesik a domain viselkedése.
- Ha state-et (állapotot) kellene tárolnia – extension method nem tárolhat állapotot (mivel static metódus), csak a paraméterként kapott objektumon dolgozhat.

- Túlzott, indokolatlan használat esetén nehezebben követhető, "hova lett implementálva ez a metódus" jellegű kód keletkezhet.

Extension method vs. statikus segédmetódus (helper class) Egy hagyományos statikus segédmetódus (pl. `StringHelper.Truncate(name, 5)`) funkcionálisan ekvivalens, de az extension method szintaktikailag olvashatóbb, láncolható (fluent), és jobban illeszkedik az IntelliSense-be (a `.` után megjelenik a lehetséges metódusok között), miközben a fordított kód mögöttesen ugyanaz: egy static metódushívás.

Extension property, extension static tagok – C# 14 / .NET 10 újdonság A hagyományos extension method (this paraméteres static metódus) 2007 óta csak metódusokat tett lehetővé – property-t, operátort vagy static tagot nem lehetett hozzáadni egy típushoz. A C# 14 (amely a .NET 10-zel érkezett, 2025 november) bevezette az "extension block" (extension members) szintaxist, amely mindezt lehetővé teszi:

```
public static class StringExtensions
{
    extension(string str)
    {
        // Extension property
        public bool IsPalindrome =>
            str.SequenceEqual(str.Reverse());

        // Extension instance method (új szintaxis)
        public string Shout() => str.ToUpper() + "!";
    }

    extension(string) // típusra vonatkozó (static) extension tagok
    {
        public static string Empty2 => string.Empty;
    }
}

// Használat:
bool isPalindrome = "racecar".IsPalindrome;
```

Ez a régi `this string str` szintaxist egy strukturáltabb, felfedezhetőbb modellre cseréli, és property-eket, indexereket, operátorokat és static tagokat is lehetővé tesz — nem csak metódusokat.

Összefoglalás Az extension method a C# egyik legfontosabb eszköze a meglévő típusok (főleg lezárt/harmadik féltől származó típusok) API-jának bővítésére újrafordítás vagy öröklés nélkül. Alapja a LINQ-nak és számos ASP.NET Core fluent regisztrációs API-nak. Használatakor figyelni kell a namespace importra, a method resolution sorrendre (instance metódus mindig előnyt élvez), és arra, hogy ne kerüljön bele domain-logika, amelynek valójában a típus saját viselkedéseként kellene léteznie.

#55 Mikor használjunk sync és mikor async API metódusokat

Az ASP.NET Core (és általában a .NET) API-k tervezésekor alapvető döntés, hogy egy adott végpont/metódus szinkron vagy aszinkron (async/await, Task-alapú) legyen-e. A helyes választás jelentősen befolyásolja a skálázhatóságot és a szálkezelés hatékonyságát.

Miért fontos ez? Az ASP.NET Core kérelmfeldolgozás a Thread Pool szárait használja. Ha egy szál egy I/O-műveletre (adatbázis, hálózat, fájlrendszer) szinkron módon vár, az a szál blokkolva marad, és nem tud másik bejövő kérést kiszolgálni, amíg a művelet be nem fejeződik. Async/await esetén a szál a várakozás idejére visszakerül a Thread Poolba, és más kéréseket szolgálhat ki, majd a művelet befejeztével egy (esetleg másik) szál folytatja a végrehajtást. Ez nagy terhelés (sok egyidejű kérés) esetén drasztikusan jobb átbocsátóképességet (throughput) és skálázhatóságot eredményez.

Mikor használjunk async metódust (ez legyen az alapértelmezett)

1. I/O-kötött (I/O-bound) műveleteknél mindig: adatbázis-lekérdezés (EF Core: `ToListAsync()`, `SaveChangesAsync()`, `FirstOrDefaultAsync()`), külső API hívás (`HttpClient.GetAsync()`), fájl olvasás/írás (`File.ReadAllTextAsync()`), üzenetsor (RabbitMQ, Service Bus) kommunikáció, gRPC hívások.
2. Webes API kontrollerekben és minimal API handlerekben szinte mindig, mert a HTTP kérés kiszolgálása jellemzően valamilyen I/O-t (adatbázis, külső szolgáltatás) tartalmaz, és a szerver sok egyidejű kérést fogad.
3. Skálázandó, nagy terhelésű (magas konkurencia) szolgáltatásoknál, ahol a Thread Pool szálak felszabadítása kritikus a válaszidő és a kiszolgálható kérésszám szempontjából.
4. Ha egy metódus több async műveletet hív meg és ésszerű őket párhuzamosan futtatni (`Task.WhenAll`), mert így az összesített várakozási idő minimalizálható.

```
// Helyes: I/O-kötött EF Core lekérdezés async-ként
[HttpGet("{id}")]
public async Task<IActionResult> GetOrder(int id)
{
    var order = await _context.Orders.FirstOrDefaultAsync(o => o.Id == id);
    return order == null ? NotFound() : Ok(order);
}

// Több független I/O művelet párhuzamosítása
var ordersTask = _context.Orders.ToListAsync();
var customersTask = _context.Customers.ToListAsync();
await Task.WhenAll(ordersTask, customersTask);
```

Mikor jó / elegendő a sync (szinkron) metódus

1. Tisztán CPU-kötött (CPU-bound), rövid, memóriában végzett számításoknál, ahol nincs I/O-várakozás (pl. egy összeg kiszámítása egy már betöltött listán, string formázás, egyszerű validáció). Az async/await ilyenkor csak felesleges overheadet (state machine generálás, context switching) ad hozzá, valós előnyt nem.

2. Rövid életű konzol-/parancssori eszközöknél vagy egyszerű, alacsony konkurenciájú belső szkripteknél, ahol a skálázhatóság nem szempont.
3. Ha egy szinkron API-t (pl. régi, csak sync metódusokat kínáló könyvtár) kell hívni, és nincs async megfelelője – ilyenkor nem érdemes mesterségesen `Task.Run` -nal "aszinkronizálni", mert az csak egy plusz Thread Pool szálát foglal le a blokkoló hívás idejére, valódi skálázási előny nélkül (sőt, szerver oldalon inkább árt).
4. Egyszerű, tisztán szinkron property-lekérdezéseknél, konstruktorokban (konstruktor soha nem lehet async).

Amit mindenképp kerülni kell

- `.Result` vagy `.Wait()` hívása egy async metóduson szinkron kontextusból – ez könnyen deadlockhoz vezethet (klasszikus ASP.NET-ben a `SynchronizationContext` miatt, de érdemes mindenhol elkerülni) és blokkolja a hívó szálát, elveszve az async minden előnyét.
- "async void" metódusok írása (kivéve event handlerok) – kivételkezelés és tesztelhetőség szempontjából problémás, mivel a hívó nem tudja megvárni a befejezését, és a kivétel nem propagálódik normál módon.
- Vegyes (szinkron blokkoló hívás async metóduson belül) kód, amely blokkolja a szálát, miközben a metódus "async"-nek van jelölve.
- Async metódus írása, ha nincs benne semmilyen valódi await – ilyenkor felesleges a `Task` becsomagolás, elég a sync verzió (vagy ha az interfész megköveteli az async szignatúrát, `Task.FromResult(...)` -tal lehet megoldani await nélkül).

Ökölszabály

- Adatbázis-, hálózat-, fájl- vagy egyéb I/O-műveletet végző metódus → async, végig az egész hívási láncon ("async all the way").
- Tiszta, memóriában futó, gyors számítás → sync, kivéve ha a hívó felület (pl. egy már meglévő async interfész) mást kényszerít ki.
- Könyvtár (library) fejlesztésénél célszerű mindkét verziót (sync és async) felkínálni, ha van értelme mindkettőnek (ahogy pl. az EF Core is teszi: `SaveChanges()` és `SaveChangesAsync()`), hogy a fogyasztó eldönthesse, melyikre van szüksége.

#56 SQL lassú query – mit kell ellenőrizni

Ha egy SQL lekérdezés (vagy egy EF Core LINQ query mögötti generált SQL) lassú, az alábbi szempontokat érdemes sorban végignézni.

1. Végrehajtási terv (Execution Plan) elemzése A legelső lépés mindig az execution plan (SQL Server: "Actual Execution Plan", PostgreSQL: `EXPLAIN ANALYZE`) megnézése. Ez megmutatja, hogy az adatbázismotor hogyan hajtja végre a lekérdezést, és felfedi a szűk keresztmetszeteket.

```
-- SQL Server
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
-- majd a lekérdezés futtatása, vagy Include Actual Execution Plan bekapcsolása

-- PostgreSQL
EXPLAIN ANALYZE SELECT * FROM Orders WHERE CustomerId = 123;
```

Kulcsfontosságú jelek a tervben: Table Scan / Seq Scan (index helyett teljes táblabejárás), Key Lookup / Bookmark Lookup nagy számban, magas becsült vs. tényleges sorok közti eltérés (rossz statisztika jele), Sort vagy Hash Match magas költséggel.

2. Hiányzó vagy nem megfelelő indexek A leggyakoribb ok. Ellenőrizendő: a WHERE, JOIN ON, ORDER BY és GROUP BY oszlopokon van-e index; összetett (composite) index esetén a szűrési sorrend megfelel-e az index oszlopsorrendjének; nincs-e az indexelt oszlopon függvényhívás vagy típuskonverzió a WHERE feltételben (ez "SARGability"-t tör, azaz az index nem használható: pl.

WHERE YEAR(OrderDate) = 2024 helyett WHERE OrderDate ≥ '2024-01-01' AND OrderDate < '2025-01-01'); covering index (olyan index, amely tartalmazza a SELECT-ben szereplő összes oszlopot is, elkerülve a Key Lookup-ot) hasznos lehet gyakran futó lekérdezéseknél.

```
CREATE INDEX IX_Orders_CustomerId_OrderDate ON Orders (CustomerId, OrderDate)
INCLUDE (TotalAmount);
```

3. N+1 lekérdezés probléma (különösen EF Core-nál) Ha egy lekérdezés helyett ciklusban sok apró lekérdezés fut (pl. lazy loading miatt minden entitáshoz külön adatbázis-hívás), az drasztikusan lassítja a rendszert. Ellenőrizzük: van-e felesleges lazy loading; szükséges-e Include() / ThenInclude() a kapcsolódó adatokhoz; naplózzuk-e (pl. EF Core logging vagy SQL Profiler/Extended Events segítségével), hogy ténylegesen hány és milyen SQL parancs fut le egyetlen kérés kiszolgálása során.
4. Statisztikák frissessége Az adatbázismotor a tábla statisztikái (oszlopeloszlás, sorok száma) alapján dönt a végrehajtási tervről. Elavult statisztikák rossz tervet (pl. rossz join sorrendet vagy indexválasztást) eredményezhetnek nagy adatmennyiség-változás után.

```
-- SQL Server
UPDATE STATISTICS Orders;
-- PostgreSQL
ANALYZE Orders;
```

5. Fragmentált indexek Sok írási/módosítási művelet után az indexek fragmentálódhatnak, ami lassítja az olvasást. Rendszeres index karbantartás (REBUILD/REORGANIZE) szükséges lehet.

6. Túl sok visszaadott/lekért adat (over-fetching)

- SELECT * helyett csak a ténylegesen szükséges oszlopok lekérése (projekció).
- Lapozás (paging: OFFSET / FETCH , illetve EF Core-ban Skip().Take()) nagy eredményhalmazoknál.

- EF Core-nál `AsNoTracking()` használata olvasás-csak lekérdezéseknél, hogy elkerüljük a felesleges change tracking overheadet.

7. Nem megfelelő JOIN-ok és felesleges adatduplikáció

- Cartesian product (véletlen cross join) hiányzó JOIN feltétel miatt.
- Túl sok egymásba ágyazott JOIN vagy subquery, amely CTE-vel vagy indexelt nézettel kiváltható lenne.
- Több egymást követő `Include()` EF Core-ban, amely "cartesian explosion"-t okozhat (a fő entitás duplikálódik minden kapcsolódó rekordhoz) – ilyenkor érdemes split query-t használni (`.AsSplitQuery()`).

8. Paraméterezés és plan cache (parameter sniffing) Nem paraméterezett (dinamikusan összefűzött) SQL minden hívásnál újrafordíttatja a tervet (plusz overhead, plán cache "szemetelés"), míg rosszul paraméterezett lekérdezéseknél előfordulhat, hogy egy nem tipikus paraméterértékre optimalizált (cache-elt) terv fut le minden további, eltérő eloszlású paraméterre is (parameter sniffing probléma).

9. Zárolások és blokkolás (locking, blocking) Egyidejű tranzakciók zárolásai (lock, blocking, deadlock) is okozhatnak "lassúságot", ami valójában várakozás egy másik tranzakcióra. Ellenőrizendő: hosszú tranzakciók, túl magas izolációs szint (pl. Serializable feleslegesen), hiányzó `READ COMMITTED SNAPSHOT` / megfelelő konkurenciakezelés (lásd EF Core Concurrency control szekció).

10. Hardver- és konfigurációs szűk keresztmetszetek

- Elégtelen memória (buffer pool / shared_buffers), ami sok lemez I/O-t okoz.
- Lassú lemez alrendszer.
- Rossz connection pool méretezés (túl sok/kevés kapcsolat az adatbázis szerver felé).

11. Alkalmazásoldali (EF Core) tipikus hibák ellenőrzőlistája

- A generált SQL lekérdezhető: `context.Orders.Where(...).ToQueryString()` (EF Core 5+) vagy logolással (`.LogTo(Console.WriteLine, LogLevel.Information)`).
- Kliens oldali kiértékelés (client-side evaluation) helyett minden szűrés/rendezés kerüljön az adatbázis oldalára (ne `ToList()` után `Where()`).
- Split query vs. single query megfelelő megválasztása több `Include()` esetén.
- Compiled query (`EF.CompileAsyncQuery`) használata gyakran ismétlődő, statikus lekérdezéseknél extrém teljesítménykritikus helyeken.

Összefoglaló ellenőrzőlista

1. Execution plan megnézése.
2. Indexek megléte és megfelelősége (SARGable feltételek, composite index sorrend, covering index).
3. N+1 probléma kizárása.
4. Statisztikák és index-fragmentáció frissessége.
5. Csak a szükséges oszlopok/sorok lekérése (projekció, lapozás, `AsNoTracking`).
6. JOIN-ok és Include-ok ésszerűsége (cartesian explosion elkerülése, split query).

7. Parameter sniffing és plan cache viselkedés.
8. Zárolások/blokkolások vizsgálata egyidejű tranzakcióknál.
9. Infrastruktúra (memória, lemez, connection pool) ellenőrzése.
10. A generált SQL tényleges megtekintése EF Core esetén (ToQueryString / logging).

#57 .NET 8 Újdonságai C# 12

- Primary constructors osztályokon és struct-okon → `public class Order(int id, Customer customer) { ... }` – a konstruktor paraméterei közvetlenül elérhetők az osztály törzsében, kevesebb boilerplate (backing mező és külön konstruktor nélkül).
- Collection expressions → egységes szintaxis kollekciók létrehozására: `int[] numbers = [1, 2, 3];`, `List<int> list = [1, 2, 3];`, spread operátorral: `int[] combined = [..array1, ..array2];`
- Default lambda paraméterek → `var greet = (string name = "World") => $"Hello, {name}!";`
- Alias any type → `using` direktívával bármilyen típusra (tuple-re, array-re, generikusra is) alias lehet készíteni: `using Point = (int X, int Y);`
- ref readonly paraméterek metódusokban.
- Interceptors (előzetes/experimental funkció) forráskód-generátoroknak.
- .NET 8 runtime: Native AOT (Ahead-of-Time) compilation kiterjesztett támogatása ASP.NET Core-hoz (gyorsabb indulás, kisebb memóriaigény, konténeres/serverless workloadokhoz ideális).
- Time abstraction: `TimeProvider` és `ITimer` – tesztelhető, injektálható idő-absztrakció (nem kell többé `DateTime.Now`-t mockolni).
- `System.Text.Json`: jobb teljesítmény, `JsonSerializerOptions.Web` preset, forrás-generátor fejlesztések.
- Blazor: renderelési módok (Static SSR, Interactive Server, Interactive WebAssembly, Interactive Auto) egy projekten belül kombinálhatók.
- ASP.NET Core Identity API endpoints (`MapIdentityApi<TUser>()`) – kész, azonnal használható regisztráció/bejelentkezés végpontok.
- Keyed services a beépített DI konténerben:

```
builder.Services.AddKeyedScoped<IMessageService, EmailMessageService>("email");
builder.Services.AddKeyedScoped<IMessageService, SmsMessageService>("sms");
// feloldás:
public MyService([FromKeyedServices("email")] IMessageService service) { ... }
```

#58 .NET 9 Újdonságai C# 13

- Params collections → a `params` kulcsszó eddig csak tömböt engedett, C# 13-tól bármilyen gyűjtemény típus (`List<T>`, `Span<T>`, `ReadOnlySpan<T>`, `IEnumerable<T>` stb.) is használható:

```
public void PrintNumbers(params IEnumerable<int> numbers) { ... }
```

- `System.Threading.Lock` → új, dedikált szinkronizációs típus a hagyományos `lock` (object) helyett, gyorsabb és biztonságosabb (a fordító figyelmeztet, ha `lock` példányt hagyományos objektumként próbálunk zárolni).

```
private readonly Lock _lock = new();
lock (_lock) { /* kritikus szakasz */ }
```

- Partial properties és partial indexerek → a partial method mintájára property/indexer deklarációja és implementációja szétválasztható (source generatoroknak hasznos, pl. `[GeneratedRegex]` attribútum property-n).
- Új index a kollekció végétől (`Index from end`) objektum-inicializálóban is használható.
- `ref struct` típusok engedélyezettek generikus paraméterként (`allows ref struct` megszorítással) – pl. `Span<T>` generikusokban is használható lett.
- Escape sequence `\e` (ESC karakter) stringekben.
- Method group konverziók egyszerűsítése, natural type inferencia javítás.
- .NET 9 runtime: jelentős GC- és JIT-teljesítményjavítások (Dynamic PGO alapértelmezetten bekapcsolva, Adaptive Server GC), több mint 1000 teljesítmény-jellegű változtatás a runtime-ban és a könyvtárakban.
- Új LINQ metódusok: `CountBy` , `AggregateBy` , `Index()` .
- `Tensor<T>` típus AI/ML integrációhoz (ML.NET, TorchSharp, ONNX Runtime).
- ASP.NET Core 9: beépített OpenAPI dokumentum-generálás (`AddOpenApi()`), `HybridCache` (in-memory + distributed cache egységes API-val), optimalizált statikus asset kiszolgálás.
- EF Core 9: `ExecuteUpdateAsync` / `ExecuteDeleteAsync` finomítások, `LeftJoin/RightJoin` LINQ operátorok natív támogatása, jobb JSON oszloptámogatás.
- Megjegyzés: a .NET 9 Standard-Term Support (STS) kiadás, 18 hónapos támogatással; az azt követő .NET 10 lett a hosszú távú (LTS) kiadás.

#59 .NET 10 Újdonságai C# 14

A .NET 10 2025 novemberében jelent meg, mint LTS (Long-Term Support) kiadás, 2028 novemberéig biztosított támogatással.

- Extension members (extension block szintaxis) → a klasszikus `this` paraméteres extension methodok mellett/helyett C# 14 lehetővé teszi extension property-k, indexerek, operátorok és static tagok definiálását is egy típushoz `extension(TypeName) { ... }` blokkban (lásd részletesen az "Extension method – minden" fejezetben).
- `field` kulcsszó → közvetlen hozzáférés az auto-implementált property mögöttes (backing) mezőjéhez, kézzel írt backing mező nélkül, egyedi validációs/logika hozzáadásához:

```
public string Name
{
    get => field;
    set => field = value?.Trim() ?? throw new ArgumentNullException(nameof(value));
}
```

- Null-conditional assignment → `person?.Name = "John";` (csak akkor történik értékadás, ha `person` nem null).
- `params ReadOnlySpan<T>` szélesebb körű használata a standard könyvtárban (pl. `Console.WriteLine` túlterhelések) – zero-allokációs hívások tömbfoglalás nélkül.
- Implicit span konverziók bővítése (`Span<T>` / `ReadOnlySpan<T>` és tömbök között természetesebb konverzió).
- File-based apps (fájlalapú, "single-file" C# programok) → egyetlen `.cs` fájl közvetlenül futtatható `dotnet run app.cs` paranccsal, projektfájl (.csproj) nélkül; ha a szkript növekszik, `dotnet project convert app.cs` paranccsal teljes projektté alakítható. Kifejezetten szkriptekhez, CLI-eszközökhöz hasznos.
- .NET 10 runtime: a JIT fordító jelentősen kihasználja a modern vektor-utasításkészleteket (AVX-512, AVX10.2, ARM SVE/SVE2) — sok esetben kódmódosítás nélkül is 15-30%-os teljesítményjavulás érhető el.
- ASP.NET Core 10: Minimal API beépített validáció-támogatás, OpenAPI 3.1 dokumentum-generálás, beépített rate limiting fejlesztések.
- EF Core 10: complex types (értéktípusok) és JSON-oszlopok kezelésének további finomítása.
- .NET Aspire és WebAssembly (Blazor WASM) integráció további érése a felhő-natív/böngésző-natív fejlesztéshez.

Összefoglaló táblázat a legfontosabb nyelvi újonságokról

Verzió	Legfontosabb C# nyelvi újonságok
.NET 6 / C# 10	file-scoped namespace, global using, record struct
.NET 7 / C# 11	required property, generic math, list pattern, raw string literal
.NET 8 / C# 12	primary constructors, collection expressions, alias any type, default lambda paraméterek
.NET 9 / C# 13	params collections, System.Threading.Lock, partial properties/indexerek, ref struct generikusban
.NET 10 / C# 14	extension members (property/operator/static), field keyword, null-conditional assignment, file-based apps

#60 Dependency Injection Lifetime-ok (Transient, Scoped, Singleton)

Verzió: ASP.NET Core 1.0-tól; kulcsolt (keyed) szolgáltatások: .NET 8-tól.

Az ASP.NET Core beépített DI konténere három élettartamot ismer. A helyes választás elmulasztása az egyik leggyakoribb produkciós hibaforrás (pl. memóriaszivárgás, "captive dependency", szálbiztonsági problémák).

1. Transient – minden feloldáskor új példány jön létre. Könnyű, állapotmentes szolgáltatásokhoz ideális.

```
builder.Services.AddTransient<IEmailSender, EmailSender>();
```

2. Scoped – kérezenként (HTTP request) egy példány. EF Core `DbContext` tipikusan scoped, mert nem szálbiztos, és egy kérezen belül konzisztens change tracker szükséges.

```
builder.Services.AddScoped<IOrderService, OrderService>();
```

3. Singleton – az alkalmazás teljes élettartama alatt egyetlen példány. Konfigurációs, cache-, vagy stateless segédszolgáltatásokhoz.

```
builder.Services.AddSingleton<ICacheService, MemoryCacheService>();
```

Captive Dependency probléma: ha egy Singleton szolgáltatás konstruktorában Scoped (pl. `DbContext`) függőséget kér, az a Scoped szolgáltatást "befogja" (captive), és az az egész alkalmazás élettartama alatt ugyanaz a példány marad – ez race conditiont és stale adatokat okozhat. A .NET DI konténer fejlesztői módban ezt kivétellel jelzi (`ValidateScopes = true` , alapértelmezett Development környezetben).

Megoldás Singletonban Scoped függőség eléréséhez: `IServiceScopeFactory` injektálása, és explicit scope nyitása:

```
public class MySingletonService
{
    private readonly IServiceScopeFactory _scopeFactory;
    public MySingletonService(IServiceScopeFactory scopeFactory) => _scopeFactory = scopeFactory;

    public async Task DoWorkAsync()
    {
        using var scope = _scopeFactory.CreateScope();
        var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
        // ...
    }
}
```

Ökölszabály: Singleton csak Singletont vagy stateless szolgáltatást injektálhat biztonságosan; Scoped injektálhat Scoped-et és Transientet; Transient bármit injektálhat, de maga rövid életű marad.

#61 Options Pattern (IOptions, IOptionsSnapshot, IOptionsMonitor)

Verzió: ASP.NET Core 1.0-tól; `ValidateOnStart()` : .NET 6-tól; `[OptionsValidator]` forráskód-generátor: .NET 8-től.

Konfiguráció (`appsettings.json`) erősen típusos, injektálható eléréséhez az Options Pattern a javasolt megoldás a `IConfiguration["Key"]` közvetlen (string alapú, hibalehetőséggel teli) használata helyett.

```
public class SmtpOptions
{
    public const string SectionName = "Smtp";
    public string Host { get; set; } = string.Empty;
    public int Port { get; set; }
}
```

Regisztráció, validációval együtt:

```
builder.Services
    .AddOptions<SmtpOptions>()
    .Bind(builder.Configuration.GetSection(SmtpOptions.SectionName))
    .ValidateDataAnnotations()
    .ValidateOnStart(); // már induláskor buktassa el a hibás konfigurációt
```

A három interfész közötti különbség:

- `IOptions<T>` – Singleton, egyszer olvassa be az értéket az induláskor, futásidőben nem frissül.
- `IOptionsSnapshot<T>` – Scoped, minden kérésnél újraolvassa a konfigurációt (hasznos, ha a konfiguráció fájl változhat futás közben, pl. `reloadOnChange: true`).
- `IOptionsMonitor<T>` – Singleton, de tud élő változás-értesítést küldeni (`OnChange` callback), így Singleton szolgáltatásokban is biztonságosan követhető a konfigurációváltozás.

```
public class EmailService
{
    private readonly SmtpOptions _options;
    public EmailService(IOptionsMonitor<SmtpOptions> optionsMonitor)
    {
        _options = optionsMonitor.CurrentValue;
        optionsMonitor.OnChange(newOptions => _options = newOptions);
    }
}
```

#62 Globális kivételkezelés és ProblemDetails

Verzió: `ProblemDetails` : ASP.NET Core 2.1; `ExceptionHandler` és `AddProblemDetails()` : .NET 8-től.

A publikus API-knak konzisztens, gépileg feldolgozható hibaválaszt kell adniuk. Az RFC 7807 (Problem Details for HTTP APIs) szabvány erre ad formátumot, amit az ASP.NET Core natívan támogat.

Globális kivételkezelő middleware (`IExceptionHandler` , .NET 8-tól):

```
public class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;
    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger) => _logger = logger;

    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext, Exception exception, CancellationToken cancellationToken)
    {
        _logger.LogError(exception, "Kezeletlen kivétel történt");

        var (statusCode, title) = exception switch
        {
            NotFoundException => (StatusCodes.Status404NotFound, "Erőforrás nem található"),
            ValidationException => (StatusCodes.Status400BadRequest, "Validációs hiba"),
            _ => (StatusCodes.Status500InternalServerError, "Váratlan szerverhiba")
        };

        httpContext.Response.StatusCode = statusCode;
        await httpContext.Response.WriteAsJsonAsync(new ProblemDetails
        {
            Status = statusCode,
            Title = title,
            Detail = exception.Message,
            Instance = httpContext.Request.Path
        }, cancellationToken);

        return true;
    }
}
```

Regisztráció:

```
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();
// ...
app.UseExceptionHandler();
```

A teljes, éles használatra szánt felállítás (kivétel-hierarchia hibakódokkal, láncolt kezelők, `traceId` , log-szintek, riasztás) az "Enterprise szintű globális hibakezelés" fejezetben található.

Fontos: a globális kivételkezelő nem helyettesíti a domain-szintű validációt (pl. `FluentValidation`), hanem az "utolsó védvonal" – minden nem várt kivételt egységes, kliensbarát formába alakít, és garantálja, hogy stack trace ne szivároгjon ki produkciós környezetben.

#63 Strukturált logolás – `ILogger` és Serilog

A `Console.WriteLine` produkciós logolásra alkalmatlan: nincs szintezés, szűrés, strukturáltság, célrendszer (fájl, Seq, ElasticSearch, Application Insights). A beépített `ILogger<T>` absztrakció és a Serilog kombinációja a de facto standard.

Beépített `ILogger` használata – mindig strukturált (template) paraméterekkel, nem string interpolációval:

```
public class OrderService
{
    private readonly ILogger<OrderService> _logger;
    public OrderService(ILogger<OrderService> logger) => _logger = logger;

    public void PlaceOrder(int orderId, decimal amount)
    {
        // JÓ: strukturált – az orderId és amount kereshető property-ként kerül be a logba
        _logger.LogInformation("Rendelés létrehozva: {OrderId}, összeg: {Amount}",
            orderId, amount);

        // ROSSZ: string interpoláció, elveszik a strukturáltság
        // _logger.LogInformation($"Rendelés létrehozva: {orderId}, összeg: {amount}");
    }
}
```

Log szintek (növekvő súlyosság): `Trace` < `Debug` < `Information` < `Warning` < `Error` < `Critical`.
Éles környezetben tipikusan `Information` vagy magasabb szint aktív.

Serilog beállítása (fájl + konzol + Seq sink):

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .Enrich.FromLogContext()
    .Enrich.WithMachineName()
    .WriteTo.Console()
    .WriteTo.File("logs/log-.txt", rollingInterval: RollingInterval.Day)
    .WriteTo.Seq("http://localhost:5341")
    .CreateLogger();

builder.Host.UseSerilog();
```

Korrelációs azonosító (Correlation Id / Traceld): elosztott rendszerekben elengedhetetlen, hogy egy adott kérés összes logsora egy közös azonosítóval legyen összekapcsolható a szolgáltatások között (pl. `Activity.Current?.Id`, vagy egyedi middleware, amely a `X-Correlation-Id` HTTP fejléct olvassa/generálja, és `LogContext.PushProperty` -vel minden logsorhoz hozzáadja).

#64 HttpClientFactory és Polly (resilience)

Verzió: `IHttpClientFactory` : ASP.NET Core 2.1; Polly v8 `ResiliencePipeline` és `AddStandardResilienceHandler` : .NET 8-tól.

Közvetlenül `new HttpClient()` -tel dolgozni veszélyes: socket exhaustion léphet fel (a DNS-változásokat nem követi le, mert a kapcsolat sokáig nyitva marad). A megoldás az `IHttpClientFactory`, amely kezeli a `HttpMessageHandler` -ek életciklusát és pool-ozását.

```
builder.Services.AddHttpClient<IWeatherApiClient, WeatherApiClient>(client =>
{
    client.BaseAddress = new Uri("https://api.weather.example.com/");
    client.Timeout = TimeSpan.FromSeconds(10);
});
```

Resilience (újrapróbálkozás, circuit breaker) hozzáadása Polly-val (.NET 8-tól a `Microsoft.Extensions.Http.Resilience` csomag natívan integrálja):

```
builder.Services.AddHttpClient<IWeatherApiClient, WeatherApiClient>(client =>
{
    client.BaseAddress = new Uri("https://api.weather.example.com/");
})
.AddResilienceHandler("weather-pipeline", pipeline =>
{
    pipeline.AddRetry(new HttpRetryStrategyOptions
    {
        MaxRetryAttempts = 3,
        BackoffType = DelayBackoffType.Exponential
    });
    pipeline.AddCircuitBreaker(new HttpCircuitBreakerStrategyOptions());
    pipeline.AddTimeout(TimeSpan.FromSeconds(5));
});
```

Miért fontos a Circuit Breaker: ha egy külső szolgáltatás tartósan elérhetetlen, a folyamatos újrapróbálkozás csak terheli és lassítja a rendszert ("cascading failure"). A circuit breaker rövid időre teljesen leállítja a hívásokat (Open állapot), majd óvatosan próbálkozik újra (Half-Open), mielőtt visszaáll normál (Closed) működésre.

#65 Háttérfolyamatok – IHostedService és BackgroundService

Verzió: `IHostedService` : ASP.NET Core 2.0, `BackgroundService` : 2.1; `PeriodicTimer` : .NET 6;
`IHostedLifecycleService` : .NET 8.

Hosszan futó, az alkalmazás életciklusához kötött háttérfeladatokhoz (időzített job, üzenetsor feldolgozó, cache melegítő) a `BackgroundService` absztrakt osztály a javasolt kiindulópont.


```

public class QueueProcessorService : BackgroundService
{
    private readonly ILogger<QueueProcessorService> _logger;
    private readonly IServiceScopeFactory _scopeFactory;

    public QueueProcessorService(ILogger<QueueProcessorService> logger,
    IServiceScopeFactory scopeFactory)
    {
        _logger = logger;
        _scopeFactory = scopeFactory;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using var scope = _scopeFactory.CreateScope();
            var queueService = scope.ServiceProvider.GetRequiredService<IQueueService>();
            await queueService.ProcessNextAsync(stoppingToken);

            await Task.Delay(TimeSpan.FromSeconds(5), stoppingToken);
        }
    }
}

// Regisztráció:
builder.Services.AddHostedService<QueueProcessorService>();

```

Fontos szempontok:

- A `BackgroundService` maga mindig Singletonként regisztrálódik, ezért a `Scoped` függőségeket (pl. `DbContext`) minden iterációban külön scope-ból kell feloldani (lásd fent, `IServiceScopeFactory`).
- A `stoppingToken`-t mindig figyelembe kell venni (`while (!stoppingToken.IsCancellationRequested)` és minden `async` hívásnak át kell adni), hogy az alkalmazás graceful shutdown-ja (`IHostApplicationLifetime`) ne akadjon el.
- Időzített (cron-szerű) feladatokhoz gyakran a `Quartz.NET` vagy a `Hangfire` könyvtárat érdemes bevonni a pusztá `Task.Delay` ciklus helyett, mert azok perzisztens ütemezést, `retry`-t és monitorozó felületet is adnak.

#66 Nullable Reference Types

Variáció: C# 8.0-tól (projekt szinten `<Nullable>enable</Nullable>`).

A C# 8-tól elérhető Nullable Reference Types (NRT) funkció fordítási időben figyelmeztet a potenciális `NullReferenceException` -okra – ez nem futásidejű ellenőrzés, hanem statikus elemzés.

Bekapcsolás projekt szinten (`.csproj`):

```
<PropertyGroup>
  <Nullable>enable</Nullable>
</PropertyGroup>
```

Bekapcsolás után minden referencia típus alapértelmezetten non-nullable; a nullázhatóságot explicit `?` jelöli:

```
public class Customer
{
    public string Name { get; set; } = string.Empty; // soha nem lehet null
    public string? MiddleName { get; set; }          // lehet null
}

public string GetGreeting(Customer customer)
{
    // figyelmeztetés fordításkor, ha customer.MiddleName-t null-check nélkül használjuk
    if (customer.MiddleName is not null)
        return $"{customer.Name} {customer.MiddleName}";
    return customer.Name;
}
```

Hasznos operátorok és attribútumok:

- `!` (null-forgiving operátor) – kikapcsolja a figyelmeztetést egy konkrét helyen, ha a fejlesztő biztos benne, hogy az érték nem null (óvatosan használandó, mert futásidőben nem véd).
- `[NotNullWhen(true)]`, `[MaybeNullWhen(false)]` – TryGet mintájú metódusok annotálására, hogy a statikus elemző kövesse a null-áramlást.
- `ArgumentNullException.ThrowIfNull(parameter)` (.NET 6+) – tömör guard clause metódus paraméterek null-ellenőrzésére.

Gyakorlati tanács: az NRT bekapcsolása egy meglévő, nagy kódbázison sok warningot generálhat egyszerre – érdemes projektenként, fokozatosan bevezetni, és a CI-ban `TreatWarningsAsErrors` opcióval kikényszeríteni, hogy új nullázhatósági hiba ne kerülhessen be.

#67 Async/await Best Practices és CancellationToken

Verzió: `async / await`: C# 5.0; `IAsyncDisposable / await foreach`: C# 8.0.

Néhány gyakori hiba, ami produkciós deadlockhoz vagy erőforrás-pazarláshoz vezet:

1. `async void` kerülése – kivétel esetén a hívó nem tudja elkapni a kivételt (a folyamat összeomolhat). Kivétel: event handlerok, ahol a szignatúra kötött. Mindenhol máshol `async Task`.
2. `.Result` / `.Wait()` kerülése `async` metóduson – ez szinkron blokkolást okoz egy `async` hívásra, ami UI vagy ASP.NET klasszikus (nem Core) szinkronizációs kontextusban könnyen deadlockhoz vezet. ASP.NET Core-ban nincs `SynchronizationContext`, így kevésbé veszélyes, de best practice akkor is a végig-`async` lánc ("async all the way").

```
// KERÜLENDŐ
var result = GetDataAsync().Result;

// HELYES
var result = await GetDataAsync();
```

3. `ConfigureAwait(false)` – könyvtárkódban (nem ASP.NET Core web alkalmazás végpontjaiban) érdemes használni, hogy az `await` utáni folytatás ne térjen vissza az eredeti szinkronizációs kontextusra, ha az nem szükséges. ASP.NET Core-ban nincs ilyen kontextus, így a hatása minimális, de class library-kban (NuGet csomagokban) továbbra is ajánlott.
4. `CancellationToken` következetes továbbadása – minden async metódusnak, amely I/O-t végez, el kell fogadnia és tovább kell adnia egy `CancellationToken`-t, hogy a hívás megszakítható legyen (pl. a kliens lecsatlakozik, vagy timeout történik).

```
public async Task<Order> GetOrderAsync(int id, CancellationToken cancellationToken)
{
    return await _dbContext.Orders
        .FirstOrDefaultAsync(o => o.Id == id, cancellationToken);
}
```

ASP.NET Core kontrollerekben/minimal API végpontokban a keretrendszer automatikusan injektál egy, a kérés élettartamához kötött `CancellationToken`-t, ha azt paraméterként felvesszük.

Haladó témák (`ValueTask`, thread pool starvation, timeout, fire-and-forget, kivételkezelés `Task.WhenAll` esetén) az "Async/await haladó szint" fejezetben.

5. `Task.WhenAll` párhuzamos I/O művelethez, ne szekvenciális `await`-ek egymás után, ha a műveletek függetlenek:

```
// LASSABB - szekvenciális
var user = await GetUserAsync(id);
var orders = await GetOrdersAsync(id);

// GYORSABB - párhuzamos
var userTask = GetUserAsync(id);
var ordersTask = GetOrdersAsync(id);
await Task.WhenAll(userTask, ordersTask);
var user = await userTask;
var orders = await ordersTask;
```

#68 Health Checks

Verzió: ASP.NET Core 2.2-től.

A Health Check végpontok lehetővé teszik, hogy egy orchestrator (Kubernetes, Docker Swarm, load balancer) vagy monitoring rendszer megállapítsa, egy alkalmazáspéldány üzemképes-e (liveness) és fogadhat-e forgalmat (readiness).

Alap regisztráció:

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<AppDbContext>()
    .AddSqlServer(connectionString, name: "sql-server")
    .AddRedis(redisConnectionString, name: "redis")
    .AddUrlGroup(new Uri("https://external-api.example.com/health"), name: "external-api");

app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("live")
});

app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready")
});
```

Liveness vs. Readiness különbség Kubernetes környezetben:

- Liveness probe – ha sikertelen, a Kubernetes újraindítja a podot (az alkalmazás "beragadt", nem válaszol).
- Readiness probe – ha sikertelen, a pod kikerül a load balancer mögül, de nem indul újra (pl. az adatbázis-kapcsolat még épp inicializálódik, vagy egy külső függőség átmenetileg nem elérhető).

Egyedi health check implementálása:

```
public class DiskSpaceHealthCheck : IHealthCheck
{
    public Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context, CancellationToken cancellationToken = default)
    {
        var freeSpace = new DriveInfo("C").AvailableFreeSpace;
        return Task.FromResult(freeSpace > 1_000_000_000
            ? HealthCheckResult.Healthy("Elegendő szabad lemezterület")
            : HealthCheckResult.Degraded("Kevés a szabad lemezterület"));
    }
}
```

#69 API Versioning

Amint egy publikus vagy több kliens által használt API-t módosítunk törő módon (breaking change), verziózásra van szükség, hogy a régi kliensek ne törjenek el. A `Asp.Versioning.Http` (korábban `Microsoft.AspNetCore.Mvc.Versioning`) csomag ad erre kész megoldást.

```
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    options.ApiVersionReader = ApiVersionReader.Combine(
        new UrlSegmentApiVersionReader(),
        new HeaderApiVersionReader("X-API-Version"));
})
.AddApiExplorer(options =>
{
    options.GroupNameFormat = "'v'VVV";
    options.SubstituteApiVersionInUrl = true;
});
```

Verziózási stratégiák összehasonlítása:

1. URL alapú (/api/v1/orders , /api/v2/orders) – legegyszerűbb, jól cache-elhető, könnyen dokumentálható, de "szennyezi" az URL struktúrát.
2. Query string alapú (/api/orders?api-version=1.0) – nem szennyezi az útvonalat, de kevésbé nyilvánvaló, könnyen elfelejthető.
3. Header alapú (X-API-Version: 1.0) – tiszta URL-eket eredményez, de nehezebben tesztelhető böngészőből/linkből.
4. Media type alapú (Accept: application/json;v=1.0) – RESTful szempontból "legtisztább", de a gyakorlatban a legritkábban használt, mert bonyolultabb kliensoldali kezelést igényel.

Minimal API-ban verziózott végpont csoport:

```
var versionSet = app.NewApiVersionSet()
    .HasApiVersion(new ApiVersion(1, 0))
    .HasApiVersion(new ApiVersion(2, 0))
    .ReportApiVersions()
    .Build();

app.MapGroup("/api/v{version:apiVersion}/orders")
    .WithApiVersionSet(versionSet)
    .MapGet("/", GetOrdersV1).HasApiVersion(new ApiVersion(1, 0));
```

#70 Unit Testing – Mocking Moq-kal

Verzió: xUnit v3: 2024-től; TimeProvider (tesztelhető idő): .NET 8-tól.

Unit tesztekben a tesztelt egység (SUT – System Under Test) függőségeit izolálni kell, hogy a teszt gyors, determinisztikus és a valós külső rendszerektől (adatbázis, HTTP hívás) független maradjon. Erre a legelterjedtebb .NET könyvtár a Moq.

```

public class OrderServiceTests
{
    [Fact]
    public async Task PlaceOrder_ValidOrder_SavesToRepository()
    {
        // Arrange
        var repositoryMock = new Mock<IOrderRepository>();
        repositoryMock
            .Setup(r => r.AddAsync(It.IsAny<Order>(), It.IsAny<CancellationToken>()))
            .Returns(Task.CompletedTask);

        var loggerMock = new Mock<ILogger<OrderService>>();
        var sut = new OrderService(repositoryMock.Object, loggerMock.Object);

        var order = new Order { Id = 1, Amount = 100M };

        // Act
        await sut.PlaceOrderAsync(order, CancellationToken.None);

        // Assert
        repositoryMock.Verify(
            r => r.AddAsync(It.Is<Order>(o => o.Id == 1), It.IsAny<CancellationToken>()),
            Times.Once);
    }
}

```

Fontos fogalmak:

- `Mock<T>.Setup` – meghatározza, hogy egy adott hívásra mit adjon vissza a mock (stub viselkedés).
- `Mock<T>.Verify` – ellenőrzi, hogy egy adott metódus meghívásra került-e (adott számú alkalommal) – ez a "viselkedés-alapú" tesztelés lényege.
- `It.IsAny<T>()`, `It.Is<T>(predicate)` – rugalmas paraméter-illesztők.
- Mock vs. Stub vs. Fake: a Mock a hívásokat is ellenőrzi (Verify), a Stub csak visszaad előre definiált adatot, a Fake egy egyszerűsített, de működő implementáció (pl. in-memory repository).

Gyakorlati tanács: ne mockoljunk mindent – ha egy osztálynak sok (5+) mockolt függősége van egy teszthez, az gyakran jelzi, hogy az osztály túl sok felelősséget hordoz (SRP sérülés), és érdemesebb lehet felbontani vagy integrációs teszttel (pl. EF Core In-Memory database, lásd korábbi fejezet) lefedni.

#71 Alkalmazásszintű (in-memory) Concurrency Handling – szálbiztos programozás

Verzió: Lock : C# 1.0; System.Threading.Lock típus: .NET 9 / C# 13.

Fontos elhatárolni az adatbázis-szintű konkurenciakezelést (lásd "EF Core Concurrency control" fejezet – optimista/pesszimista stratégiák a perzisztens adatokra) az alkalmazáson belüli, memóriában tartott megosztott állapot (shared state) szálbiztos kezelésétől. Az utóbbi akkor kerül elő, amikor egy

Singleton szolgáltatás, statikus mező, in-memory cache vagy háttérfolyamat egyszerre több szálról/kérésből érhető el.

A race condition problémája

```
public class CounterService
{
    private int _count = 0;

    public void Increment() => _count++; // NEM szálbiztos! a ++ nem atomi művelet (olvasás-
    módosítás-írás)

    public int Count => _count;
}
```

Ha ezt a szolgáltatást Singletonként regisztráljuk, és sok párhuzamos kérés hívja az `Increment()` -et, elveszett módosítások (lost update) történhetnek, mert két szál egyszerre olvashatja ki ugyanazt a régi értéket.

1. `lock` / `System.Threading.Lock` – kritikus szakasz védelme

```
public class CounterService
{
    private readonly Lock _lock = new(); // .NET 9+, C# 13-tól; korábban: private readonly object
    _lock = new();
    private int _count = 0;

    public void Increment()
    {
        lock (_lock)
        {
            _count++;
        }
    }
}
```

A `lock` szinkron blokkolást végez – **soha ne használjunk** `await` -et egy `lock` **blokkon belül**, mert az fordítási hibát ad (a `lock` nem tartható meg async folytatáson át), és logikailag is hibás lenne (a szál elengedhetné a lock-ot a folytatás előtt).

2. `SemaphoreSlim` – async-kompatibilis zárolás

Ha a kritikus szakaszban async hívás (pl. adatbázis vagy HTTP hívás) is történik, a `lock` helyett `SemaphoreSlim` -et kell használni, mert az `WaitAsync()` -et biztosít:

```

public class CacheWarmupService
{
    private readonly SemaphoreSlim _semaphore = new(1, 1); // maxCount = 1 → mutex-
    szerű viselkedés
    private List<Product>? _cachedProducts;

    public async Task<List<Product>> GetProductsAsync(CancellationTok
    en cancellationToken)
    {
        if (_cachedProducts is not null)
            return _cachedProducts;

        await _semaphore.WaitAsync(cancellationToken);
        try
        {
            // double-check locking: mire megkaptuk a semaphore-t, lehet, hogy már betöltötte egy
            másik szál
            _cachedProducts ??= await LoadProductsFromDatabaseAsync(cancellationToken);
            return _cachedProducts;
        }
        finally
        {
            _semaphore.Release();
        }
    }
}

```

A `try/finally` elengedhetetlen, hogy kivétel esetén se maradjon "beragadva" a semaphore.

3. Interlocked – lock nélküli atomi műveletek egyszerű számlálókhoz

Ha csak egyetlen primitív érték (int, long) atomi növelése/cseréje a cél, a `lock` -nál olcsóbb és egyszerűbb megoldás az `Interlocked` osztály:

```

private long _requestCount = 0;
public void TrackRequest() ⇒ Interlocked.Increment(ref _requestCount);
public long RequestCount ⇒ Interlocked.Read(ref _requestCount);

```

4. Szálbiztos gyűjtemények (System.Collections.Concurrent)

Ahelyett, hogy egy sima `Dictionary<TKey, TValue>` -t manuálisan `lock` -kal védenénk, a `System.Collections.Concurrent` névtér finomszemcsés zárolással (vagy lock-mentesen) megvalósított, kifejezetten többszálú használatra tervezett gyűjteményeket ad: `ConcurrentDictionary<TKey, TValue>`, `ConcurrentQueue<T>`, `ConcurrentBag<T>`, `BlockingCollection<T>`.

```

private readonly ConcurrentDictionary<string, CachedItem> _cache = new();

public CachedItem GetOrAdd(string key, Func<string, CachedItem> factory)
    ⇒ _cache.GetOrAdd(key, factory); // atomi "ha nincs, hozd létre" művelet

```


5. Channel<T> – producer/consumer minta async kontextusban

Ha egy háttérfolyamat (lásd "Háttérfolyamatok – IHostedService és BackgroundService" fejezet) aszinkron módon fogyaszt egy sorba beérkező munkát, a `System.Threading.Channels` a javasolt, lock-mentes, backpressure-t is támogató megoldás sima `Queue<T>` + `lock` kombináció helyett:

```
var channel = Channel.CreateBounded<WorkItem>(capacity: 100);

// Producer
await channel.Writer.WriteAsync(workItem, cancellationToken);

// Consumer (pl. egy BackgroundService ExecuteAsync-jában)
await foreach (var item in channel.Reader.ReadAllAsync(cancellationToken))
{
    await ProcessAsync(item, cancellationToken);
}
```

Adatbázis-szintű, több példányra és elosztott folyamatokra kiterjedő konkurenciáról lásd a "Konkurenciakezelés haladó szinten" fejezetet.

Ökölszabály: egyszerű számláló → `Interlocked`; összetett, de tisztán szinkron kritikus szakasz → `lock`; async kód a kritikus szakaszban → `SemaphoreSlim`; megosztott kulcs-érték tárr → `ConcurrentDictionary`; producer/consumer feldolgozási lánc → `Channel<T>`.

#72 xUnit Tesztelés Alapjai – napi szinten szükséges patternek

Fact vs. Theory

- `[Fact]` – egyetlen, paraméter nélküli tesztesetet jelöl.
- `[Theory]` – paraméterezett teszt, amit több bemeneti adatkészlettel is lefuttat a keretrendszer; ezzel elkerülhető a szinte azonos `[Fact]`-ek másolgatása.

```

public class DiscountCalculatorTests
{
    [Fact]
    public void CalculateDiscount_NoDiscountCode_ReturnsFullPrice()
    {
        var sut = new DiscountCalculator();

        var result = sut.Calculate(price: 100M, discountCode: null);

        Assert.Equal(100M, result);
    }

    [Theory]
    [InlineData(100, "SAVE10", 90)]
    [InlineData(100, "SAVE50", 50)]
    [InlineData(0, "SAVE10", 0)]
    public void CalculateDiscount_ValidCode_AppliesCorrectDiscount(decimal price, string code,
        decimal expected)
    {
        var sut = new DiscountCalculator();

        var result = sut.Calculate(price, code);

        Assert.Equal(expected, result);
    }
}

```

Ha az adatok nem primitívek, vagy máshonnan (fájl, adatbázis) származnak, `[MemberData]` vagy `[ClassData]` használható `[InlineData]` helyett:

```

public static IEnumerable<object[]> DiscountCases =>
    new List<object[]>
    {
        new object[] { 100M, "SAVE10", 90M },
        new object[] { 100M, "SAVE50", 50M },
    };

[Theory]
[MemberData(nameof(DiscountCases))]
public void CalculateDiscount_MemberData_AppliesCorrectDiscount(decimal price, string code,
    decimal expected)
{
    var sut = new DiscountCalculator();
    Assert.Equal(expected, sut.Calculate(price, code));
}

```

Arrange-Act-Assert (AAA) és elnevezési konvenció

Minden teszt három, vizuálisan is elkülönített részből álljon: **Arrange** (előkészítés – SUT és függőségek létrehozása, bemeneti adat), **Act** (a tesztelt művelet meghívása), **Assert** (az eredmény ellenőrzése). Egy tesztben lehetőleg egyetlen logikai állítást ellenőrizzünk (nem feltétlenül egyetlen `Assert` hívást, de egyetlen viselkedést).

Elterjedt elnevezési minta: `MetódusNév_Szenárió_VártEredmény`, pl.

`CalculateDiscount_ValidCode_AppliesCorrectDiscount` – így a teszt neve önmagában dokumentációként szolgál, és bukás esetén azonnal érthető, mi romlott el.

Setup és teardown – konstruktor, `IDisposable`, `fixture`-ök

xUnit-ban nincs külön `[SetUp]` / `[TearDown]` attribútum (szemben pl. NUnit-tal): minden

`[Fact]` / `[Theory]` metódushoz **új példány** jön létre a teszt osztályból, így maga a konstruktor a setup, az `IDisposable.Dispose()` pedig a teardown:

```
public class OrderServiceTests : IDisposable
{
    private readonly AppDbContext _dbContext;
    private readonly OrderService _sut;

    public OrderServiceTests() // minden teszt előtt lefut
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;
        _dbContext = new AppDbContext(options);
        _sut = new OrderService(_dbContext);
    }

    public void Dispose() => _dbContext.Dispose(); // minden teszt után lefut
}
```

Ha a setup **költséges** (pl. adatbázis-konténer indítása Testcontainers-szel) és tesztek között megosztható, `IClassFixture<T>` használandó – ekkor a fixture egyszer jön létre az adott teszt osztály összes tesztje számára:

```
public class DatabaseFixture : IAsyncLifetime
{
    public AppDbContext DbContext { get; private set; } = null!;

    public async Task InitializeAsync()
    {
        // pl. Testcontainers SQL Server konténer indítása, migráció futtatása
        DbContext = await CreateAndMigrateDbContextAsync();
    }

    public Task DisposeAsync() => DbContext.DisposeAsync().AsTask();
}

public class OrderServiceTests : IClassFixture<DatabaseFixture>
{
    private readonly DatabaseFixture _fixture;
    public OrderServiceTests(DatabaseFixture fixture) => _fixture = fixture;
}
```

Ha több teszt osztály között kell megosztani egy fixture-t (pl. egyetlen konténer az egész tesztfutáshoz), `ICollectionFixture<T>` és `[Collection("...")]` szükséges. Fontos: az azonos collection-be tartozó teszt osztályok xUnit-ban **nem párhuzamosan** futnak egymással, mert megosztott állapotot használnak.

Egyéb napi szinten hasznos elemek

- `Assert.Throws<TException>(() => ...) / await Assert.ThrowsAsync<TException>(async () => ...)` – kivétel-alapú viselkedés tesztelése.
- `[Trait("Category", "Integration")]` – tesztek kategorizálása, hogy CI-ban szűrhetők legyenek (pl. csak unit tesztek fussanak minden commit-nál, integrációs tesztek csak nightly buildben).
- FluentAssertions (opcionális NuGet csomag) olvashatóbb assert szintaxist ad:
`result.Should().Be(90M); , action.Should().Throw<ValidationException>();`
- Tesztizoláció: egy teszt sikere/bukása soha ne függjön egy másik teszt futási sorrendjétől vagy mellékhatásától (pl. megosztott statikus mező) – ez a leggyakoribb oka a "flaky" (véletlenszerűen hol sikeres, hol bukó) teszteknek.

#73 Teszt adat-előállítási patternek – Object Mother, Test Data Builder

Összetett, unit és perzisztált (integrációs/funkcionális) tesztekben egyaránt használható megvalósítás: lásd az "Object Mother és Test Data Builder – haladó, kettős célú megvalósítás" fejezetet.

Ahogy egy tesztkészlet nő, a tesztadatok (entitások, DTO-k) manuális, ismételt felépítése minden tesztben sok boilerplate-et és karbantartási terhet jelent (ha egy kötelező mező bekerül az entitásba, minden teszt törlik). Erre két elterjedt minta ad megoldást.

Object Mother pattern

Egy statikus (vagy Singleton) "gyár" osztály, amely előre elkészített, névvel ellátott, tipikus tesztpéldányokat ad vissza. Előnye az egyszerűség és az olvashatóság, hátránya, hogy sok variáció esetén sok metódus/kombináció halmozódik fel.

```

public static class OrderMother
{
    public static Order ValidOrder() => new()
    {
        Id = 1,
        CustomerName = "Teszt Elek",
        Items = [new OrderItem { ProductId = 1, Quantity = 2, UnitPrice = 10M }],
        Status = OrderStatus.Pending
    };

    public static Order EmptyOrder() => new()
    {
        Id = 2,
        CustomerName = "Teszt Elek",
        Items = [],
        Status = OrderStatus.Pending
    };

    public static Order CancelledOrder() => ValidOrder() with { Status = OrderStatus.Cancelled };
}

// Használat:
var order = OrderMother.ValidOrder();

```

Test Data Builder (Builder) pattern

Fluent, láncolható API-val engedi finomhangolni a tesztadatot, alapértelmezett ("happy path") értékekkel, amelyeket csak a teszt szempontjából releváns mezőknél kell felülírni. Jobban skálázódik sok variáció esetén, mint az Object Mother, mert nem kell külön metódus minden kombinációhoz.

```

public class OrderBuilder
{
    private int _id = 1;
    private string _customerName = "Teszt Elek";
    private OrderStatus _status = OrderStatus.Pending;
    private readonly List<OrderItem> _items = [new() { ProductId = 1, Quantity = 1, UnitPrice = 10M }];

    public OrderBuilder WithId(int id) { _id = id; return this; }
    public OrderBuilder WithCustomerName(string name) { _customerName = name; return this; }
    public OrderBuilder WithStatus(OrderStatus status) { _status = status; return this; }
    public OrderBuilder WithNoItems() { _items.Clear(); return this; }

    public Order Build() => new()
    {
        Id = _id,
        CustomerName = _customerName,
        Status = _status,
        Items = _items
    };
}

// Használat - csak a tesztben releváns eltérés kerül explicit megadásra:
var cancelledOrder = new OrderBuilder()
    .WithStatus(OrderStatus.Cancelled)
    .Build();

var emptyOrder = new OrderBuilder()
    .WithNoItems()
    .Build();

```

Automatizált adatgenerálás – AutoFixture és Bogus

Nagyobb projekteknél, ahol a tesztadat konkrét értéke lényegtelen (csak az számít, hogy "van egy érvényes objektum"), érdemes megnézni az `AutoFixture` (automatikusan generál véletlenszerű, de érvényes property-értékeket bármely típushoz reflection alapján) és a `Bogus` (realisztikus, "fake" adatok generálására: nevek, címek, e-mail címek) NuGet csomagokat is – ezek kombinálhatók a fenti Builder mintával (pl. Bogus-szal feltöltött Builder alapértékek).

Mikor melyiket használjuk: Object Mother kevés, jól elnevezhető, gyakran ismétlődő szcenárióhoz (pl. "érvényes rendelés", "lejárt előfizetés"); Builder sok apró variációhoz, ahol a teszt olvashatóságát az számít, mi tér el az alapesettől; AutoFixture/Bogus tömeges, "bármilyen érvényes adat jó" jellegű tesztekhez (pl. teljesítménytesztek, property-based tesztelés).

#74 CancellationToken használata API-kban – gyakorlati példa

A `CancellationToken` célja, hogy egy hosszan futó (jellemzően I/O-kötött: adatbázis-hívás, külső HTTP hívás) műveletet kooperatív módon meg lehessen szakítani, mielőtt az természetes úton befejeződne – ezzel erőforrást (adatbázis-kapcsolat, szál, memória) szabadítva fel, ha az eredményre már nincs szükség.

Miért fontos ez kifejezetten API-knál?

1. **Kliens lecsatlakozása** – ha egy HTTP kliens (böngésző bezárása, timeout a kliens oldalán, mobilhálózat megszakadása) lezárja a kapcsolatot, mielőtt a szerver válaszolna, az ASP.NET Core automatikusan jelzi ezt egy `CancellationToken`-en keresztül. Ha a backend kód nem figyeli ezt a tokenet, a művelet (pl. egy drága riport lekérdezés) feleslegesen tovább fut, pazarolva az adatbázis- és CPU-erőforrást egy olyan válaszáért, amit senki nem fog megkapni.
2. **Timeout kikényszerítése** – egy külső szolgáltatás hívásának felső időkorlátot lehet szabni, hogy egy lassan válaszoló függőség ne blokkolja/lassítsa be a teljes kérést a végtelenségig.
3. **Erőforrás-fel szabadítás terhelés alatt** – magas terhelésnél különösen fontos, hogy a már "értelmetlenné vált" munkát minél hamarabb elengedjük, hogy a szerver kapacitása a még hasznos kérésekre jusson.

Végponttól az adatbázis-hívásig – a token továbbadása ("token propagation")

Minimal API-ban és kontrollerekben egyaránt egyszerűen felvehető paraméterként; a keretrendszer automatikusan a kérés élettartamához (`HttpContext.RequestAborted`) kötött tokenet adja át:

```
// Minimal API
app.MapGet("/orders/{id:int}", async (int id, AppDbContext db, CancellationToken
cancellationToken) =>
{
    var order = await db.Orders
        .FirstOrDefaultAsync(o => o.Id == id, cancellationToken);

    return order is not null ? Results.Ok(order) : Results.NotFound();
});

// Kontroller
[HttpGet("{id:int}")]
public async Task<IActionResult> GetOrder(int id, CancellationToken cancellationToken)
{
    var order = await _orderService.GetByIdAsync(id, cancellationToken);
    return order is not null ? Ok(order) : NotFound();
}
```

A kulcsfontosságú szabály: **a tokenet végig kell vinni a teljes hívási láncban** – minden async metódusnak, ami I/O-t végez (EF Core lekérdezés, `HttpClient` hívás, fájl művelet), át kell vennie és tovább kell adnia a kapott tokenet, különben a lánc egy pontján "megszakad" a megszakíthatóság.

```
public async Task<Order?> GetByIdAsync(int id, CancellationToken cancellationToken)
{
    // a service réteg továbbadja a repository-nak
    return await _repository.GetByIdAsync(id, cancellationToken);
}
```

Timeout és külső token összekapcsolása – `CancellationTokenSource.CreateLinkedTokenSource`

Ha egy külső hívásra saját timeout-ot is akarunk szabni a kliens megszakítása mellett, egy "linked" tokenet kell létrehozni, amely akkor jelez megszakítást, ha **bármelyik** forrás (kliens megszakítás VAGY timeout) bekövetkezik:

```
public async Task<WeatherData> GetWeatherAsync(string city, CancellationToken requestAborted)
{
    using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(3));
    using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(requestAborted,
        timeoutCts.Token);

    try
    {
        return await _httpClient.GetFromJsonAsync<WeatherData>
            ($"/weather/{city}", linkedCts.Token)
            ?? throw new WeatherServiceException("Üres válasz érkezett.");
    }
    catch (OperationCanceledException) when (timeoutCts.IsCancellationRequested)
    {
        // explicit módon megkülönböztethető: mi okozta a megszakítást
        throw new TimeoutException($"A időjárás-szolgáltatás nem válaszolt 3 másodpercen
            belül ({city}).");
    }
}
```

Mikor NE szakítsuk meg a műveletet a token ellenére

Vannak esetek, amikor a kérés megszakadása után is be kell fejezni a munkát – pl. egy pénzügyi tranzakció, ami már elindult az adatbázisban, vagy egy audit log bejegyzés. Ilyenkor tudatosan `CancellationToken.None`-t kell átadni az adott (kritikus) résznek, hogy az ne szakadjon meg a kliens lecsatlakozása esetén sem – ezt mindig kommentben érdemes indokolni, mert a kód olvasói alapból elvárják a token továbbadását.

```
// A fizetés state-jét mindenképp frissítjük, akkor is, ha a kliens időközben lecsatlakozott
await _paymentRepository.MarkAsProcessedAsync(paymentId, CancellationToken.None);
```

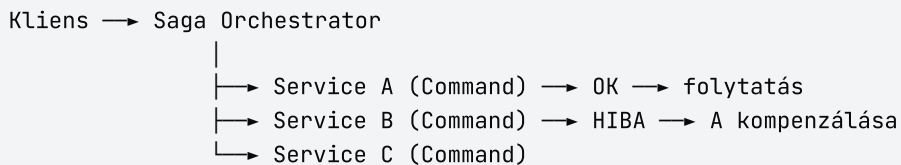
Összefoglalva: a `CancellationToken` nem "nice to have", hanem az API-k skálázhatóságának és erőforrás-hatékonyságának alapvető eszköze – minden I/O-kötött láncszemben tovább kell adni, és tudatosan kell döntenie arról, hol indokolt (vagy éppen tilos) a megszakíthatóság.

#75 Saga Orchestration

A **Saga** minta hosszú lefutású, elosztott tranzakciókat old meg, amelyek több microservice-t érintenek. Mivel a klasszikus ACID tranzakció nem alkalmazható több független szolgáltatás között, a Saga sorozatos lokális tranzakciókat hajt végre, és ha valamelyik lépés hibára fut, **kompenzáló tranzakciókat** indít el a korábbi változások visszavonásához.

Két fő megközelítés:

- **Choreography (koreográfia)** – minden service eseményre reagál, és maga dönti el, mi a következő lépés; nincs központi koordinátor.
- **Orchestration (orchestráció)** – egy dedikált Saga Orchestrator (vagy State Machine) irányítja a folyamatot, ismeri az állapotát, és parancsokat küld az egyes service-eknek.



Mikor érdemes használni: amikor egy üzleti folyamat (pl. rendelés leadása: készlet, fizetés, szállítás) több microservice-t érint, és az atomicitást kell valamilyen szinten garantálni; ha a folyamat állapotát naplózni/visszakövetni kell; komplex elágazások esetén az orchestration jobban követhető, mint a choreography.

Legjobb gyakorlatok:

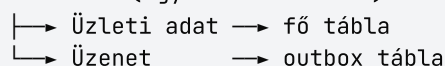
- Minden lépéshez tervezz előre kompenzáló műveletet (compensating transaction).
- Az orchestrator állapotát perzisztensen tárold (adatbázisban), hogy újraindítás után is folytatható legyen.
- A saga lépései legyenek idempotensek – ha egy üzenet kétszer érkezik, ne hajtódjon végre kétszer.
- State machine könyvtár használata az állapotkezeléshez (pl. MassTransit Saga, Temporal).
- Minden külső híváshoz timeout és retry mechanizmus.

Mit kerülj el: ne keverd a Choreography-t és az Orchestration-t ugyanabban a folyamatban; ne tételezd fel, hogy a kompenzáció mindig sikeres (tervezz Dead Letter Queue-ra/manuális beavatkozásra is); ne alkalmazz Saga-t egyszerű, kétlépéses folyamatokra; kerülj a nagyon sok lépéses (~10+) saga-kat egyetlen orchestratorban.

#76 Outbox Pattern

Az **Outbox (kimenő levelesláda) minta** garantálja, hogy az adatbázisba írás és az üzenetküldés (pl. Kafka, RabbitMQ felé) atomikusan történjen. Az üzenetet a tranzakció részeként nem közvetlenül a brokerre küldjük, hanem egy **outbox táblába** mentjük; egy háttérfolyamat (poller vagy CDC – Change Data Capture) olvassa ki és továbbítja ténylegesen.

Alkalmazás (egy tranzakcióban):



Háttérfolyamat: outbox tábla → Message Broker → fogyasztó service-ek

```

public async Task PlaceOrderAsync(Order order, CancellationToken cancellationToken)
{
    using var transaction = await _dbContext.Database.BeginTransactionAsync(cancellationToken);

    _dbContext.Orders.Add(order);
    _dbContext.OutboxMessages.Add(new OutboxMessage
    {
        Id = Guid.NewGuid(),
        Type = nameof(OrderPlacedEvent),
        Payload = JsonSerializer.Serialize(new OrderPlacedEvent(order.Id, order.CustomerId)),
        OccurredAt = DateTimeOffset.UtcNow
    });

    await _dbContext.SaveChangesAsync(cancellationToken); // ugyanaz a tranzakció
    menti mindkettőt
    await transaction.CommitAsync(cancellationToken);
}

```

Mikor érdemes használni: ha garantálni kell, hogy egy domain event biztosan eljut a többi service-hez akkor is, ha a broker átmenetileg elérhetetlen; microservice kommunikációban, ahol az at-least-once szemantika elfogadható/szükséges; CQRS read model frissítésénél.

Legjobb gyakorlatok: az outbox rekord és az üzleti adat mindig ugyanabba a DB tranzakcióba kerüljön; a fogyasztók legyenek idempotensek (egy üzenet többször is megérkezhet); a feldolgozott rekordokat jelöld meg (`ProcessedAt`) és időnként takarítsd; nagy volumennél CDC eszköz (pl. Debezium) hatékonyabb, mint a polling; figyelj a feldolgozási sorrendre, ha az fontos.

Mit kerülj el: ne küldj üzenetet közvetlenül a brokernek a tranzakción kívül; az outbox olvasása és törlése/jelölése is legyen atomi; ne pollozz feleslegesen agresszívan; ne tárold az outbox táblában a teljes entitást, csak a szükséges payloadot.

#77 Circuit Breaker Pattern

A **Circuit Breaker (megszakító)** minta megakadályozza, hogy egy hibás vagy lassan válaszoló külső függőség (service, API, adatbázis) kaszkádszerű hibát okozzon az egész rendszerben – ez a gyakorlati implementáció (Polly, `AddResilienceHandler`) már szerepel a "HttpClientFactory és Polly" fejezetben, itt a minta önmagában, technológiafüggetlenül kerül összefoglalásra.

Állapot	Leírás
Closed (zárt)	Normál működés, a hívások átmennek
Open (nyitott)	A megszakító nyitva, a hívások azonnal hibát adnak vissza (a hívott fél felé el sem indulnak)
Half-Open (félig nyitott)	Próbahívás: ha sikeres, visszazár (Closed), ha nem, újra kinyílik (Open)

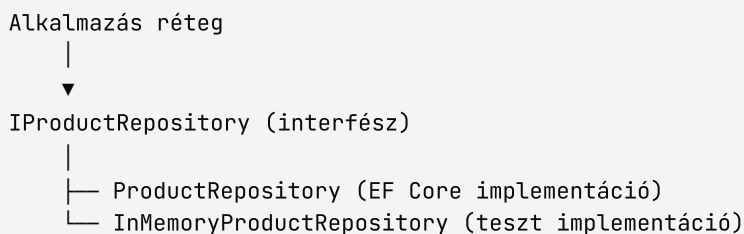
Mikor érdemes használni: microservices architektúrában, ahol egy szolgáltatás külső service-t hív (HTTP, gRPC, adatbázis); ha el akarod kerülni, hogy egy lassuló függőség kimerítse a thread pool-t; ha gyors fallback választ szeretnél adni (cache-ből, default értékből); bármikor, ha a hívott fél nem garantáltan elérhető.

Legjobb gyakorlatok: mindig legyen fallback stratégia arra az esetre, ha a megszakító nyitva van; kombináld Retry mintával; a küszöbértékeket (hibaszám/időablak, Half-Open próbaidő) valós mérések alapján hangold; tedd monitorozhatóvá az állapotátmeneteket (metrika, logolás); ne csak HTTP hívásra alkalmazd, DB/broker hívásokra is hasznos lehet.

Mit kerülsz el: ne állíts be túl alacsony hibaszám-küszöböt (1 hiba = nyitás), mert forgalmas rendszerben állandóan nyitva lesz; a Half-Open állapotot ne hagyd konfigurálatlanul, különben a megszakító sosem zár vissza; ne keverd össze a Retry-jal (a Retry egyedi hívásra próbálkozik újra, a Circuit Breaker az összesített, idő szerinti állapotot figyeli); ne alkalmazd megbízható, belső (in-process) hívásokra, mert felesleges overhead.

#78 Repository Pattern

A **Repository (tároló) minta** az adatelérési logikát absztrahálja egy interfész mögé: az üzleti logika nem adatbázis-specifikus kódot hív közvetlenül, hanem egy repository interfészt, amelynek implementációja mögött bármilyen adatforrás állhat (SQL, NoSQL, in-memory, fájl). Ez elősegíti a tesztelhetőséget és a rétegek szétválasztását.



```

public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id, CancellationToken cancellationToken);
    Task<IReadOnlyList<Order>> GetPendingOrdersAsync(CancellationToken cancellationToken);
    void Add(Order order);
}

public class OrderRepository(AppDbContext dbContext) : IOrderRepository
{
    public Task<Order?> GetByIdAsync(int id, CancellationToken cancellationToken) =>
        dbContext.Orders.FirstOrDefaultAsync(o => o.Id == id, cancellationToken);

    public Task<IReadOnlyList<Order>> GetPendingOrdersAsync(CancellationToken
cancellationToken) =>
        dbContext.Orders.Where(o => o.Status == OrderStatus.Pending)
            .ToListAsync(cancellationToken)
            .ContinueWith(t => (IReadOnlyList<Order>)t.Result, cancellationToken);

    public void Add(Order order) => dbContext.Orders.Add(order);
}

```

Mikor érdemes használni: ha a tesztelhetőség kiemelt fontosságú (a repository mockolható, lásd "Unit Testing – Mocking Moq-kal" fejezet); ha az adatforrás elvileg cserélhető; Domain-Driven Design alkalmazásakor, ahol a repository az aggregátum gyökerek mentén szervezi az elérést; ha az adatelérési logikát egy helyen szeretnéd tartani.

Legjobb gyakorlatok: definiálj interfészt minden repository-hoz; szervezd aggregátum-gyökök szerint (egy aggregátumhoz egy repository, nem egy tábla = egy repository); ne szivárogtass ki adatbázis-specifikus fogalmat (pl. `IQueryable<T>` ne legyen publikus visszatérési típus); kombináld Unit of Work mintával, ha több repository közös tranzakciót igényel.

Mit kerülj el: ne tegyél üzleti logikát a repository-ba, csak adatelérést; ne adj vissza `IQueryable<T>`-t, mert azzal az üzleti réteg tovább komponálhat LINQ kifejezéseket, és elmosódik az absztrakció határa; ne hozz létre repository-t reflexszerűen minden táblához, csak aggregátum-gyökökhez; ha sok helyen ismétlődik ugyanaz a lekérdezési logika, adj neki nevet (lásd "Specification Pattern" fejezet), ne duplikáld.

#79 Service Collection Extension Pattern

A .NET DI konténer (`IServiceCollection`) konfigurálásának szervezett módja: Extension Method-ok segítségével a regisztrációs logika modulárisan, rétegenként/feature-önként kerül szervezésre, ahelyett hogy a `Program.cs` egyetlen, több száz soros fájlja dagadna.

```
// Program.cs - tiszta és olvasható
builder.Services.AddApplicationServices();
builder.Services.AddInfrastructureServices(builder.Configuration);
builder.Services.AddPersistenceServices(builder.Configuration);

// Külön fájlban (Application réteg):
public static class ApplicationServiceExtensions
{
    public static IServiceCollection AddApplicationServices(this IServiceCollection services)
    {
        services.AddMediatR(cfg =>
        cfg.RegisterServicesFromAssembly(typeof(ApplicationServiceExtensions).Assembly));
        services.AddValidatorsFromAssembly(typeof(ApplicationServiceExtensions).Assembly);
        return services;
    }
}
```

Mikor érdemes használni: minden közepes/nagy méretű .NET alkalmazásban; ha a DI regisztrációkat rétegek szerint (Application, Infrastructure, Persistence) szeretnéd szeparálni; könyvtárfejlesztésnél, hogy a fogyasztó egyetlen hívással (`services.AddMyLibrary()`) regisztrálhassa az összes szükséges szolgáltatást; csapatmunkánál, hogy a rétegek fejlesztői önállóan kezelhessék a saját regisztrációikat.

Legjobb gyakorlatok: névkonvenció `Add[ModulNév]` (pl. `AddAuthentication`, `AddCaching`); mindig add vissza az `IServiceCollection`-t a láncolhatóság (fluent API) miatt; fogadd el paraméterként az `IConfiguration`-t, ha konfigurációból kell olvasni; helyezd az extension class-t a megfelelő projektrétegbe.

Mit kerülj el: ne regisztrálj mindent egyetlen óriási extension method-ban, bontsd logikai egységekre; ne végezz adatbázis-kapcsolatot vagy I/O műveletet a regisztráció közben, az extension csak leírja, mit kell létrehozni; ne keverd a rétegek felelősségét (pl. Infrastructure extension ne hivatkozzon Application belső osztályaira).

#80 REPR Pattern – Request-Endpoint-Response

A **REPR (Request-Endpoint-Response)** minta alternatíva a hagyományos, sok akciómetódust tartalmazó MVC Controller-ekkel szemben. Minden HTTP végpont egy önálló osztályban kap helyet, amely tartalmazza a Request modellt (bemeneti DTO), az Endpoint osztályt (a feldolgozó logika, jellemzően csak delegálás) és a Response modellt (kimeneti DTO). Steve Smith (Ardalis) népszerűsítette; .NET-ben tipikusan Minimal API-val vagy a FastEndpoints könyvtárral valósul meg.

```

Endpoints/
  Orders/
    CreateOrder/
      CreateOrderRequest.cs
      CreateOrderEndpoint.cs
      CreateOrderResponse.cs
    GetOrderById/
      GetOrderByIdRequest.cs
      GetOrderByIdEndpoint.cs
      GetOrderByIdResponse.cs

```

```

// Minimal API + MediatR kombinációval megvalósított REPR endpoint
public record CreateOrderRequest(int CustomerId, List<OrderItemDto> Items);
public record CreateOrderResponse(int OrderId);

public static class CreateOrderEndpoint
{
    public static void MapCreateOrderEndpoint(this IEndpointRouteBuilder app)
    {
        app.MapPost("/orders", async (CreateOrderRequest request, ISender mediator,
CancellationTokens ct) =>
        {
            var orderId = await mediator.Send(new CreateOrderCommand(request.CustomerId,
request.Items), ct);
            return Results.Ok(new CreateOrderResponse(orderId));
        });
    }
}

```

Mikor érdemes használni: ha a Controller osztályok túl nagyra nőttek ("God Controller" antipattern); ha szeretnéd egy helyen tartani az összetartozó kérés/feldolgozás/válasz kódot; Vertical Slice Architecture alkalmazásakor (lásd "Vertical sliced code Organisation pattern" fejezet); kis/közepes microservice-eknél, ahol a Minimal API + REPR áttekinthető; ha a csapat feature-k szerint (nem rétegek szerint) szeretné szervezni a kódot.

Legjobb gyakorlatok: minden endpointnak egy felelőssége legyen; FastEndpoints vagy Minimal API + MediatR kombináció az implementációhoz; a Request/Response modellek legyenek DTO-k, ne doménentitások; szervezd az endpointokat feature mappák szerint; adj minden endpointnak egyértelmű nevet.

Mit kerülj el: ne tegyél üzleti logikát az Endpoint osztályba, delegálj handler/use case/service felé; ne csinálj közös base endpointból öröklési hierarchiát; ne keverd következtlenül a REPR-t és a hagyományos Controllert ugyanabban a projektben.

#81 Domain Events

Verzió: A példa C# 12 elsődleges konstruktort és gyűjtemény-kifejezést (`[]`) használ (.NET 8).

A **Domain Events** minta lehetővé teszi, hogy az aggregátumok belső eseményeket definiáljanak, amelyek az üzleti folyamat során keletkeznek (pl. `OrderPlaced`, `PaymentReceived`). Ezeket az eseményeket az alkalmazásréteg vagy az infrastruktúra kezeli le, így maga az aggregátum nem függ közvetlenül semmilyen külső rendszertől (email küldés, üzenetsor, audit log).

```
public sealed record OrderPlacedDomainEvent(int OrderId, int CustomerId, decimal TotalAmount)
: INotification;

public class Order
{
    private readonly List<INotification> _domainEvents = [];
    public IReadOnlyList<INotification> DomainEvents => _domainEvents.AsReadOnly();

    public void Place()
    {
        Status = OrderStatus.Placed;
        _domainEvents.Add(new OrderPlacedDomainEvent(Id, CustomerId, TotalAmount));
    }

    public void ClearDomainEvents() => _domainEvents.Clear();
}

// EF Core SaveChangesAsync interceptorban (lásd "EF Core Interceptors" fejezet) publikálva a
mentés után:
public class DomainEventDispatchInterceptor(IPublisher publisher) : SaveChangesInterceptor
{
    public override async ValueTask<int> SavedChangesAsync(
        SaveChangesCompletedEventData eventData, int result, CancellationToken cancellationToken
        = default)
    {
        var context = eventData.Context!;
        var entitiesWithEvents = context.ChangeTracker.Entries<Order>()
            .Select(e => e.Entity)
            .Where(o => o.DomainEvents.Count != 0)
            .ToList();

        foreach (var entity in entitiesWithEvents)
        {
            var events = entity.DomainEvents.ToList();
            entity.ClearDomainEvents();
            foreach (var domainEvent in events)
                await publisher.Publish(domainEvent, cancellationToken);
        }

        return result;
    }
}
```

Mikor érdemes használni: ha az aggregátumon belüli változásokra más részeknek is reagálniuk kell (pl. saga indítása, outbox feltöltése, audit log írása); laza csatolású, event-driven architektúrában.

Legjobb gyakorlatok: a domain eventek legyenek immutable record-ok; nevezd őket mindig múlt időben (`OrderPlaced`, nem `PlaceOrder`); az eseményt csak akkor publikáld, ha a tranzakció (`SaveChanges`) sikeresen lezárult, különben a fogyasztók olyan állapotra reagálnak, ami esetleg nem is

került mentésre.

Mit kerülj el: ne tegyél navigációs property-t vagy EF entitást a domain eventbe, csak primitív értékeket/azonosítókat; ne publikálj domain eventet a `SaveChanges` előtt (rollback esetén "fantom" eseményt kapnának a fogyasztók).

#82 Decorator Pattern (Pipeline)

A **Decorator** minta egy meglévő objektum köré csomagol egy újabbat, ezzel hozzáadva vagy módosítva a viselkedést anélkül, hogy az eredeti osztály kódját módosítanánk. ASP.NET Core-ban a Middleware pipeline (lásd "Middleware" fejezet) ennek rendszer szintű megvalósítása; MediatR-rel a Pipeline Behavior (lásd "MediatR" fejezet) az alkalmazás szintű megfelelője; klasszikus OOP szinten pedig DI-vel regisztrálható decorator-lánc:

```
public interface IOrderService
{
    Task PlaceOrderAsync(Order order, CancellationToken cancellationToken);
}

public class OrderService : IOrderService
{
    public Task PlaceOrderAsync(Order order, CancellationToken cancellationToken)
    => Task.CompletedTask;
}

// Decorator: logolást ad a meglévő implementációhoz, anélkül hogy azt módosítaná
public class LoggingOrderServiceDecorator(IOrderService inner,
ILogger<LoggingOrderServiceDecorator> logger) : IOrderService
{
    public async Task PlaceOrderAsync(Order order, CancellationToken cancellationToken)
    {
        logger.LogInformation("Rendelés feldolgozása kezdődik: {OrderId}", order.Id);
        await inner.PlaceOrderAsync(order, cancellationToken);
        logger.LogInformation("Rendelés feldolgozása befejeződött: {OrderId}", order.Id);
    }
}

// Regisztráció (pl. Scrutor csomaggal egyszerűsíthető: services.Decorate<IOrderService,
LoggingOrderServiceDecorator>())
builder.Services.AddScoped<OrderService>();
builder.Services.AddScoped<IOrderService>(sp =>
    new LoggingOrderServiceDecorator(sp.GetRequiredService<OrderService>(),
sp.GetRequiredService<ILogger<LoggingOrderServiceDecorator>>()));
```

Mikor érdemes használni: cross-cutting concern-ök kezelésére (logging, caching, retry, validáció, tranzakció); ha öröklés helyett kompozícióval szeretnéd bővíteni a viselkedést.

Legjobb gyakorlatok: tartsd a dekorátor felelősségét minimálisra (Single Responsibility); definiálj pontos sorrendet a dekorátoroknak/pipeline lépéseknek; nagyobb projektben a Scrutor NuGet csomag automatizálja a decorator regisztrációt.

Mit kerülj el: kerüld a mélyen egymásba ágyazott dekorátorok láncát, mert nehéz debuggolni és követni a végrehajtási sorrendet.

#83 Specification Pattern

A **Specification** minta egy üzleti szabályt vagy lekérdezési feltételt önálló, újrahasználható és tesztelhető objektumba zár, ahelyett hogy a szűrési logika (LINQ `Where` kifejezés) szétszóródna a repository-k és service-ek között. Különösen jól kombinálható a "Repository Pattern" fejezetben említett problémával, hogy a repository ne adjon vissza `IQueryable<T>`-t, mégis rugalmasan lekérdezhető maradjon.

```
public abstract class Specification<T>
{
    public abstract Expression<Func<T, bool>> ToExpression();
    public bool IsSatisfiedBy(T entity) => ToExpression().Compile()(entity);
}

public class PendingHighValueOrdersSpecification(decimal minimumAmount) : Specification<Order>
{
    public override Expression<Func<Order, bool>> ToExpression() =>
        order => order.Status == OrderStatus.Pending && order.TotalAmount >= minimumAmount;
}

// Repository-ban:
public async Task<IReadOnlyList<Order>> FindAsync(Specification<Order> specification,
CancellationTokens cancellationTokens) =>
    await _dbContext.Orders.Where(specification.ToExpression()).ToListAsync(cancellationTokens);

// Használat:
var orders = await _orderRepository.FindAsync(new PendingHighValueOrdersSpecification(10_000M),
cancellationTokens);
```

Mikor érdemes használni: ha ugyanaz a szűrési logika több helyen (repository, validáció, in-memory ellenőrzés) is előfordul; ha a lekérdezési feltételeket kombinálni szeretnéd (`And`, `Or`, `Not` operátorokkal); komplex domain szabályok esetén, ahol a feltétel önmagában is dokumentáló értékű elnevezést érdemel.

Legjobb gyakorlatok: adj a specifikációnak beszédes, üzleti nyelvet tükröző nevet (`PendingHighValueOrdersSpecification`, ne `OrderFilter1`); tedd kombinálhatóvá (`AndSpecification`, `OrSpecification` wrapperekkel, vagy NuGet csomaggal, pl. `Ardalis.Specification`); a specifikáció maradjon deklaratív, ne tartalmazzon I/O-t.

Mit kerülj el: ne hozz létre specifikációt minden apró, egyszeri lekérdezéshez – csak akkor éri meg, ha a feltétel ismétlődik vagy önmagában is domain fogalom; ne keverd bele a lapozási/rendezési logikát a specifikációba, azok külön paraméterek maradjanak.

#84 Result Pattern / Railway-Oriented Programming

A **Result minta** (más néven Railway-Oriented Programming) a kivételek (exception) helyett explicit, a visszatérési típusban kifejezett hibakezelést valósít meg – a hívó a fordító kényszerítésével (nem elfelejthető `try/catch` -cse) szembesül azzal, hogy a művelet sikertelen is lehet. Ez különösen a **várható** (üzleti) hibáknál hasznos (pl. validációs hiba, "nincs elég készlet"), szemben a **kivételes**, valóban váratlan hibákkal (pl. adatbázis-kapcsolat megszakadása), amelyekre továbbra is kivétel és a globális kivételkezelő (lásd "Globális kivételkezelés és ProblemDetails" fejezet) a megfelelő eszköz.

```
public class Result<T>
{
    public bool IsSuccess { get; }
    public T? Value { get; }
    public string? Error { get; }

    private Result(bool isSuccess, T? value, string? error) => (IsSuccess, Value, Error) =
(isSuccess, value, error);

    public static Result<T> Success(T value) => new(true, value, null);
    public static Result<T> Failure(string error) => new(false, default, error);
}

public async Task<Result<Order>> PlaceOrderAsync(CreateOrderRequest request,
CancellationTokens cancellationTokens)
{
    if (request.Items.Count == 0)
        return Result<Order>.Failure("A rendelésnek legalább egy tételt tartalmaznia kell.");

    var stockAvailable = await _inventoryService.CheckStockAsync(request.Items,
cancellationTokens);
    if (!stockAvailable)
        return Result<Order>.Failure("Nincs elegendő készlet a kért tételekhez.");

    var order = Order.Create(request);
    return Result<Order>.Success(order);
}

// Minimal API végpontban a Result egyszerűen HTTP válasszá alakítható:
app.MapPost("/orders", async (CreateOrderRequest request, OrderService service, CancellationTokens
ct) =>
{
    var result = await service.PlaceOrderAsync(request, ct);
    return result.IsSuccess ? Results.Ok(result.Value) : Results.BadRequest(result.Error);
});
```

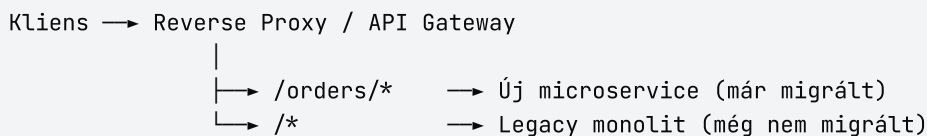
Mikor érdemes használni: gyakran előforduló, "elvárt" üzleti hibáknál, ahol a kivételdobás/elkapás overhead-je és a stack unwinding felesleges; olyan pipeline-oknál (pl. validáció → üzleti szabály → mentés), ahol a hibát a lánc korai szakaszában kell "rövidre zárni".

Legjobb gyakorlatok: gyakran használt könyvtár erre a `FluentResults` vagy az `ErrorOr` NuGet csomag, amelyek generikus `Result<T>` / `ErrorOr<T>` típust és kombinálható (`Bind`, `Map`) műveleteket adnak; legyen egyértelmű konvenció a csapatban, mikor `Result` és mikor `exception` a helyes eszköz.

Mit kerülj el: ne keverd össze a "várható" üzleti hibákat a valóban kivételes, programozási hibát jelző esetekkel (pl. `NullPointerException`) – az utóbbiakra a `Result` minta nem helyettesíti a megfelelő hibakezelést; ne alkalmazd a mintát következetlenül (egy rétegben `Result`, egy másikban `exception` ugyanarra a hibatípusra), mert az összezavarja a hívókat.

#85 Strangler Fig Pattern

A **Strangler Fig** minta egy monolitikus vagy legacy rendszer fokozatos, kockázatmentes lecserélésére szolgáló migrációs stratégia: az új funkciókat (vagy a régiéket újraírt verzióit) egy köztes réteg (pl. reverse proxy, API Gateway) mögé helyezve, fokozatosan "átirányítjuk" a forgalmat a régi rendszerről az újra, amíg a régi teljesen feleslegessé nem válik (a névadó "füge" analógia szerint az új rendszer lassan "körbenövi és kiszorítja" a régit).



Mikor érdemes használni: nagy, régóta futó monolit rendszerek fokozatos, "big bang rewrite" nélküli modernizálásakor; ha a teljes rendszer egyszerre történő leállítása/cseréje üzletileg elfogadhatatlan kockázatot jelentene; ha a csapat kapacitása csak részleges, iteratív migrációt tesz lehetővé.

Legjobb gyakorlatok: azonosíts jól elhatárolható, önállóan migrálható funkcionális szeleteket (bounded context-eket, lásd "Bounded Context" fejezet); a routing réteget (proxy/gateway) tedd konfigurálhatóvá, hogy a forgalom aránya/célja gyorsan állítható legyen; minden migrált szeletet alaposan tesztelj és monitorozz éles forgalom mellett is (pl. shadow traffic vagy canary release formájában), mielőtt a régi kódot töröld.

Mit kerülj el: ne hagyd a legacy és az új rendszert évekig párhuzamosan élni terv és véghatáridő nélkül – a strangler fig egy átmeneti állapot, nem végállapot; ne migrálj olyan szeletet, amelynek határai (adatmodell, tranzakciók) nem tisztázottak – ez inkonzisztens állapotokhoz vezethet a két rendszer között.

#86 Backend for Frontend (BFF) és API Gateway

Az **API Gateway** egyetlen belépési pontot biztosít a kliensek számára egy microservices rendszer elé, amely elrejtja a belső szolgáltatások topológiáját, és olyan cross-cutting funkciókat lát el, mint az autentikáció, rate limiting, routing, aggregáció. A **Backend for Frontend (BFF)** ennek egy specializált változata: minden kliens-típushoz (webes SPA, mobil app, harmadik fél integráció) **külön, az adott kliens igényeire szabott** gateway/backend réteg tartozik, ahelyett hogy egyetlen, mindenkinek megfelelő, kompromisszumos API-t tartanánk fenn.



Mikor érdemes használni: ha különböző kliens-típusoknak jelentősen eltérő adatformára/aggregációra van szükségük (pl. a mobil kliens kevesebb mezőt és kisebb payloadot igényel, mint a webes admin felület); ha egy közös, "univerzális" API már túl sok kompromisszumot és felesleges komplexitást hordoz; microservices architektúrában, ahol a kliens ne ismerje a belső szolgáltatások számát és címét.

Legjobb gyakorlatok: a BFF réteg maradjon vékony – aggregáció, adatformázás, kliens-specifikus optimalizálás, de üzleti logika nélkül (az a mögöttes service-ekben él); az API Gateway/BFF szintjén valósítsd meg a keresztirányú aggályokat (authenticáció, rate limiting, logolás, circuit breaker a belső hívásokhoz); külön csapat/felelős tartozzon minden BFF-hez, igazodva a fogyasztó csathoz.

Mit kerülj el: ne csempéssz üzleti logikát a BFF-be, mert az duplikációhoz és inkonzisztenciához vezet a mögöttes szolgáltatásokkal; ne hozz létre BFF-et minden egyes klienshez automatikusan – csak akkor éri meg, ha az igények valóban jelentősen eltérnek; egyetlen, minden klienst kiszolgáló API Gateway is elegendő lehet, ha a kliensek igényei hasonlóak.

#87 Anti-Corruption Layer

Az **Anti-Corruption Layer (ACL)**, Domain-Driven Design fogalom, egy védőréteg, amely elszigeteli a saját (tiszt) domain modellünket egy külső rendszer (legacy alkalmazás, harmadik fél API, más bounded context) eltérő, "szennyező" modelljétől és fogalmi world-view-jától. A réteg feladata a fordítás (mapping/translation) a két modell között, hogy a külső rendszer struktúrája, hibái vagy hiányosságai ne "szivárognak be" a saját domain modellünkbe.

```
// Külső, legacy rendszer modellje (amit nem kontrollálunk, esetleg csúnya/inkonzisztens)
public class LegacyCustomerRecord
{
    public string CUST_NM { get; set; } = string.Empty;
    public string ADDR_LN1 { get; set; } = string.Empty;
    public int STAT_CD { get; set; } // 0/1/2 kód, aminek jelentését csak a legacy rendszer
    dokumentációja ismeri
}

// Anti-Corruption Layer: fordítás a saját, tiszta domain modellünkre
public class LegacyCustomerAdapter
{
    public Customer ToDomainCustomer(LegacyCustomerRecord legacy) => new()
    {
        Name = legacy.CUST_NM.Trim(),
        Address = new Address(legacy.ADDR_LN1),
        Status = legacy.STAT_CD switch
        {
            0 => CustomerStatus.Active,
            1 => CustomerStatus.Suspended,
            2 => CustomerStatus.Closed,
            _ => throw new InvalidOperationException($"Ismeretlen legacy
státuszkód: {legacy.STAT_CD}")
        }
    };
}
```

Mikor érdemes használni: ha egy legacy rendszerrel vagy harmadik fél API-val kell integrálódni, amelynek modellje eltér a saját domain modellünktől; microservices/DDD környezetben, ha két bounded context között a fogalmak (pl. "Customer" jelentése) eltérnek, és nem szabad, hogy az egyik modell közvetlenül "átszivárogon" a másikba; ha a külső rendszer instabil vagy gyakran változik, és a saját domain modellt védeni kell ettől a változékonyságtól.

Legjobb gyakorlatok: az ACL-t helyezd az Infrastructure rétegbe (lásd Clean Architecture fejezetek), a Domain réteg ne tudjon a külső rendszer létezéséről; a fordítási logikát (mapping) egy helyen, jól tesztelve tartsd; ha a külső rendszer hibás/hiányos adatot ad, az ACL-ben dönts el, hogyan kezeled (alapértelmezett érték, kivétel, naplózás).

Mit kerülj el: ne engedd, hogy a külső rendszer DTO-i vagy enumjai közvetlenül megjelenjenek a domain modellben vagy az alkalmazás rétegben; ne tedd az ACL-t "vékonyabbá" idővel azzal, hogy egyre több helyen közvetlenül a legacy modellt használod – ez visszavezet az eredeti, elkerülni kívánt csatoláshoz.

#88 Idempotency Pattern API-kban

Az **idempotencia** azt jelenti, hogy egy művelet többszöri, azonos bemenettel történő végrehajtása ugyanazt az eredményt adja, mint az egyszeri végrehajtás – ez elosztott rendszerekben (hálózati újrapróbálkozás, at-least-once üzenetkezelés, lásd "Outbox Pattern" és "Saga

Orchestration" fejezetek) kritikus, hogy elkerüljük a duplikált mellékhatásokat (pl. kétszeres terhelés egy bankkártyán).

HTTP szemantika szerint a `GET`, `PUT`, `DELETE` metódusok definíció szerint idempotensek kellene legyenek, a `POST` viszont nem – ezért a `POST` végpontoknál (pl. fizetés indítása, rendelés létrehozása) explicit idempotencia-kezelés szükséges, tipikusan egy kliens által generált **Idempotency Key** HTTP fejléc segítségével:

```
app.MapPost("/payments", async (
    [FromHeader(Name = "Idempotency-Key")] string idempotencyKey,
    CreatePaymentRequest request,
    IIdempotencyStore idempotencyStore,
    IPaymentService paymentService,
    CancellationToken cancellationToken) =>
{
    var existingResult = await idempotencyStore.TryGetAsync(idempotencyKey, cancellationToken);
    if (existingResult is not null)
        return Results.Ok(existingResult); // korábbi válasz visszaadása, a művelet nem fut
        le újra

    var result = await paymentService.ProcessPaymentAsync(request, cancellationToken);
    await idempotencyStore.SaveAsync(idempotencyKey, result, cancellationToken);

    return Results.Ok(result);
});
```

Mikor érdemes használni: pénzügyi/tranzakciós jellegű `POST` / `PATCH` végpontoknál, ahol egy véletlen duplikált kérés (kliens újrapróbálkozása timeout után, mobil hálózati instabilitás) súlyos üzleti következménnyel járna; message consumer-eknél, amelyek at-least-once szemantikájú brokerből (pl. az Outbox mintából) olvasnak.

Legjobb gyakorlatok: az Idempotency Key-t a kliens generálja (jellemzően GUID) és minden újrapróbálkozásnál ugyanazt küldi; a kulcs-eredmény párokat egy gyors, megosztott tárban (Redis, adatbázis tábla) tárol, TTL-lel (nem kell örökké megőrizni); a tárolt válasz legyen a **teljes** HTTP válasz (státuszkód + body), hogy az ismételt kérés valóban ugyanazt kapja vissza.

Mit kerülj el: ne csak a "sikeres" eredményt cache-eld – ha egy hibás válasz idempotencia-kulcs nélkül duplikálódhat, az önmagában nem probléma, de dönteni kell, hogy a hibás válaszokat is idempotensen kezeled-e; ne tévedsz össze az idempotenciát a HTTP metódus szemantikájával – önmagában attól, hogy egy végpont `PUT`, még lehet nem-idempotens a mögöttes implementáció, ha pl. side effectet (email küldés) is kivált minden hívásnál.

#89 SignalR

Verzió: ASP.NET Core 2.1-től (withAutomaticReconnect : ASP.NET Core 3.0).

A **SignalR** az ASP.NET Core beépített valós idejű kommunikációs könyvtára, amely lehetővé teszi, hogy a szerver **kezdemenyezze** az adatküldést a klienseknek (böngésző, mobil app, asztali app), nem csak a hagyományos request-response modell szerint (kliens kér, szerver válaszol). A háttérben automatikusan a legjobban elérhető szállítási módot (transport) választja: **WebSockets** (ha támogatott) → **Server-Sent Events** → **Long Polling**, így a fejlesztőnek nem kell ezekkel manuálisan foglalkoznia.

Mire használjuk – tipikus esetek

1. **Élő értesítések, dashboard-ok** – pl. egy admin felület, ahol új rendelés érkezésekor azonnal frissül a lista, anélkül hogy a felhasználó frissítené az oldalt.
2. **Chat alkalmazások** – klasszikus, tankönyvi SignalR use case: több felhasználó közötti üzenetváltás valós időben.
3. **Kollaboratív szerkesztés** – pl. egy Trello-szerű tábla, ahol több felhasználó egyszerre látja a másik módosításait (kártya mozgatus, státuszváltás).
4. **Élő folyamat-visszajelzés (progress reporting)** – hosszan futó háttér folyamat (fájl feldolgozás, riport generálás) állapotának valós idejű visszajelzése a felhasználó felé, ahelyett hogy pollozná az API-t.
5. **Élő adatfolyamok** – tőzsdei árfolyamok, IoT szenzor adatok, sportmérkőzés élő eredményei.
6. **Szerver-oldali push más microservice-ek felől** – pl. egy háttér folyamat (BackgroundService) végzett egy műveletet, és erről valós időben értesíteni kell a böngészőben nyitva lévő UI-t.

Alapfogalmak: Hub, Connection, Group

A SignalR középpontjában a **Hub** áll – ez egy magas szintű absztrakció, amely metódusokat exponál a kliens felé (amit a kliens hívhat), és a szerver oldalról is hívhat metódusokat a csatlakozott kliense(ke)n.

```
public class NotificationHub : Hub
{
    // Kliens hívhatja ezt a metódust
    public async Task JoinOrderGroup(int orderId)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId, $"order-{orderId}");
    }

    public override async Task OnConnectedAsync()
    {
        // pl. autentikált felhasználó automatikus csoportba sorolása
        var userId = Context.UserIdentifier;
        if (userId is not null)
            await Groups.AddToGroupAsync(Context.ConnectionId, $"user-{userId}");

        await base.OnConnectedAsync();
    }
}
```

Regisztráció és végpont kijelölés:


```
builder.Services.AddSignalR();
// ...
app.MapHub<NotificationHub>("/hubs/notifications");
```

Szerver oldalról, egy másik szolgáltatásból (pl. egy Command Handlerből vagy BackgroundService-ből) a `IHubContext<T>` segítségével küldhetünk üzenetet a csatlakozott klienseknek, közvetlenül a Hub példányosítása nélkül:

```
public class OrderStatusNotifier(IHubContext<NotificationHub> hubContext)
{
    public async Task NotifyOrderStatusChangedAsync(int orderId, string newStatus,
        CancellationToken cancellationToken)
    {
        // csak az adott rendelést figyelő kliensek kapják meg
        await hubContext.Clients.Group($"order-{orderId}")
            .SendAsync("OrderStatusChanged", new { orderId, newStatus }, cancellationToken);
    }
}
```

Kliens oldal (JavaScript, `@microsoft/signalr` csomaggal):

```
const connection = new signalR.HubConnectionBuilder()
    .withUrl("/hubs/notifications")
    .withAutomaticReconnect()
    .build();

connection.on("OrderStatusChanged", (data) => {
    console.log(`Rendelés ${data.orderId} új státusza: ${data.newStatus}`);
});

await connection.start();
await connection.invoke("JoinOrderGroup", 42);
```

Fontos gyakorlati szempontok

- **Skálázás több szerverpéldány között** – ha az alkalmazás több példányban fut (load balancer mögött), egy SignalR üzenetnek el kell jutnia minden példányhoz kapcsolódó kliensekhez is. Erre szolgál a **backplane** (pl. `AddStackExchangeRedis()` – Redis backplane, vagy Azure SignalR Service), amely a szerverpéldányok közötti üzenettovábbítást oldja meg.

```
builder.Services.AddSignalR().AddStackExchangeRedis(redisConnectionString);
```

- **Autentikáció** – a SignalR Hub végpontok is védhetők `[Authorize]` attribútummal; JWT token-nel való autentikáció esetén a tokent a WebSocket kapcsolat felépítésekor query string paraméterként kell átadni (mert a WebSocket protokoll nem támogat egyedi HTTP fejléceket kézbesítéskor), ezt a keretrendszer az `access_token` query paraméterből automatikusan felismeri.
- **Csoportkezelés** – ne küldj minden üzenetet minden kliensnek (`Clients.All`), ha csak egy szűkebb kör érintett; a `Groups` API-val (`AddToGroupAsync` / `RemoveFromGroupAsync`) célzottan érdemes küldeni, ez jelentősen csökkenti a hálózati és szerver terhelést.

- **Erőforrás-kezelés** – minden nyitott SignalR kapcsolat egy szálát/erőforrást köt le a szerveren; nagy számú egyidejű kapcsolatnál (10 000+) érdemes megfontolni a felhő-alapú Azure SignalR Service-t, amely a kapcsolatkezelést kiszervezi a saját szerverfolyamatból.
- **Mikor NE használjuk** – ha csak egyszerű, ritka (percenkénti/óránkénti) frissítésről van szó, egy egyszerű pollozó `GET` végpont (esetleg `HybridCache` -el kombinálva, lásd "ASP.NET core Caching strategies" fejezet) gyakran egyszerűbb és olcsóbb megoldás, mint egy állandóan nyitva tartott kapcsolat fenntartása.

#90 Quartz.NET – Háttérfolyamat ütemezés

Verzió: Quartz.NET 3.x: .NET Standard 2.0+ / .NET 6+ projektekben.

A **Quartz.NET** egy nyílt forráskódú, gazdag funkcionalitású job-ütemező könyvtár .NET-hez, amely a pusztán `BackgroundService` + `Task.Delay` ciklusnál (lásd "Háttérfolyamatok – IHostedService és BackgroundService" fejezet) jóval kifinomultabb ütemezést tesz lehetővé: cron-szerű időzítés, perzisztens állapot (túlélve az alkalmazás újraindítását), retry, misfire-kezelés (mi történjen, ha a job éppen nem futtatott le a tervezett időpontban), és cluster-támogatás több szerverpéldány esetén.

Mikor érdemes Quartz.NET-et használni a sima `BackgroundService` helyett

- Ha **konkrét, cron kifejezéssel leírható** időpontokban kell futtatni egy feladatot (pl. "minden hétköznap 2:00-kor", "minden hónap első napján"), nem csak fix időközönként.
- Ha a job-oknak **túl kell élniük** egy alkalmazás-újraindítást, és tudniuk kell, hogy egy adott futtatás megtörtént-e már (persistent job store).
- Ha **több szerverpéldány** (skálázott, klaszterezett környezet) fut egyszerre, és biztosítani kell, hogy egy adott job-ot **csak egyszer**, csak az egyik példány futtassa (elosztott lock).
- Ha a job-oknak **prioritást, függőségeket, vagy misfire szabályokat** kell kezelniük (pl. "ha a szerver 10 percig le volt állva, a kimaradt futtatást pótolja vagy hagyja ki").
- Ha dinamikusan, futásidőben kell job-okat létrehozni/törölni/módosítani (nem csak statikusan, induláskor regisztrált feladatokról van szó).

Ha ezek egyike sem áll fenn, és elég egy egyszerű, fix időközönként ismétlődő művelet, a sima `BackgroundService` + `PeriodicTimer` kombináció is elégséges és egyszerűbb.

Alapfogalmak: Job, Trigger, Scheduler

- **Job** – maga a végrehajtandó munka (egy osztály, amely az `IJob` interfészt implementálja).
- **Trigger** – meghatározza, *mikor* fusson a job (pl. cron kifejezés, egyszeri időpont, ismétlődő intervallum).
- **Scheduler** – a Quartz futásidejű motorja, amely a Job-okat a hozzájuk rendelt Trigger-ek szerint futtatja.

Telepítés és regisztráció (Quartz.Extensions.Hosting csomaggal)

```
dotnet add package Quartz.Extensions.Hosting
```

```
public class DailyReportJob(ILogger<DailyReportJob> logger, IServiceScopeFactory scopeFactory)
: IJob
{
    public async Task Execute(IJobExecutionContext context)
    {
        logger.LogInformation("Napi riport generálás elindult: {Time}", DateTimeOffset.Now);

        using var scope = scopeFactory.CreateScope();
        var reportService = scope.ServiceProvider.GetRequiredService<IReportService>();

        // context.CancellationToken automatikusan elérhető, ha a Scheduler leáll
        await reportService.GenerateDailyReportAsync(context.CancellationToken);
    }
}
```

Regisztráció a Program.cs -ben:

```
builder.Services.AddQuartz(q =>
{
    var jobKey = new JobKey(nameof(DailyReportJob));

    q.AddJob<DailyReportJob>(opts => opts.WithIdentity(jobKey));

    q.AddTrigger(opts => opts
        .ForJob(jobKey)
        .WithIdentity($"{nameof(DailyReportJob)}-trigger")
        .WithCronSchedule("0 0 2 * * ?")); // minden nap hajnali 2:00-kor (Quartz cron formátum,
6-7 mezős)
});

// A Scheduler-t hosted service-ként futtatja, és graceful shutdown-nál megvárja a futó job-okat
builder.Services.AddQuartzHostedService(options =>
{
    options.WaitForJobsToComplete = true;
});
```

A Quartz cron formátuma **hét mezős** (a Unix cronnal ellentétben van egy extra "másodperc" mező elöl, és a "nap-hónap"/"nap-hét" mezők közül csak az egyik lehet konkrét érték, a másiknak ? -nek kell lennie):

másodperc	perc	óra	nap(hónap)	hónap	nap(hét)	[év]
0	0	2	*	*	?	

Egyszerű időzített (ismétlődő intervallumú) trigger cron helyett

```
q.AddTrigger(opts => opts
    .ForJob(jobKey)
    .WithSimpleSchedule(s => s
        .WithIntervalInMinutes(15)
        .RepeatForever()));
```

Dependency Injection job-okon belül

A Quartz.Extensions.Hosting csomag automatikusan integrálódik az ASP.NET Core DI konténerével, így a `IJob` implementációk konstruktorán keresztül kaphatnak függőségeket (a job maga tipikusan Transient élettartammal regisztrálódik, ezért a Scoped függőségeket – pl. `DbContext` – a job `Execute` metódusán belül, `IServiceScopeFactory` -val kell feloldani, ugyanúgy, mint a sima `BackgroundService` -nél, lásd a korábbi fejezetet).

Perzisztens Job Store és klaszterezés

Alapértelmezetten a Quartz a job-okat és trigger-eket memóriában tárolja (`RAMJobStore`) – ez elveszik újraindításkor. Produkciós, több szerverpéldányos környezetben `AdoJobStore` -t érdemes beállítani (adatbázisban, pl. SQL Server-ben tárolt job-definíciók), ezzel a Quartz automatikusan gondoskodik arról, hogy egy adott job-ot egyidejűleg csak az egyik klaszter-tag futtassa (elosztott lock a job store-on keresztül):

```
builder.Services.AddQuartz(q =>
{
    q.UsePersistentStore(s =>
    {
        s.UseProperties = true;
        s.UseSqlServer(connectionString);
        s.UseNewtonsoftJsonSerializer();
        s.UseClustering(); // több szerverpéldány esetén
    });
});
```

Misfire kezelés

Ha a szerver nem futott (leállt, túlterhelt volt) a tervezett futtatási időpontban, a Quartz "misfire"-nek tekinti az elmaradt futtatást, és a trigger típusától függő stratégia szerint dönt: azonnal pótolja (`WithMisfireHandlingInstructionFireNow`), teljesen kihagyja (`WithMisfireHandlingInstructionDoNothing`), vagy az alapértelmezett viselkedést alkalmazza. Ezt explicit be kell állítani olyan job-oknál, ahol fontos, hogy az elmaradt futtatás pótlódjon-e vagy sem (pl. egy napi riport pótlása értelmes lehet, egy "percenkénti health check" pótlása viszont felesleges).

Mit kerülj el

- Ne tárolj a Job osztályban mutable állapotot mezőként, ha a job Singleton-ként van regisztrálva – párhuzamos futtatásoknál (pl. két egymást átfedő trigger) race condition alakulhat ki; alapértelmezetten a Quartz minden végrehajtáshoz új Job példányt hoz létre, ezt ne írd felül

ok nélkül.

- Ne felejtse el a `context.CancellationToken`-t átadni a job belsejében hívott async metódusoknak, különben a Scheduler graceful leállítása (`WaitForJobsToComplete`) nem tudja időben megszakítani a futó munkát.
- Ne keverje a Quartz-ot és a sima `BackgroundService`-t ugyanarra a feladatra – válasszon egyet a feladat jellege alapján (lásd fenti "mikor melyiket" szempontokat), és legyen konzisztens a csapaton belül.
- Ha egyszerű, alacsony komplexitású időzítésről van szó, és a projektben már úgyis van Hangfire (webes dashboard-dal rendelkező alternatíva), fontold meg, hogy szükség van-e mindkét könyvtárra egyszerre – a párhuzamos ütemező könyvtárak használata felesleges komplexitást és karbantartási terhet ad.

#91 Apache Kafka .NET-ben – alapfogalmak és első producer/consumer

Verzió: Confluent.Kafka 2.x: .NET Standard 2.0+ / .NET 8+ ajánlott; a példa C# 12 gyűjtemény-kifejezést és UTF-8 literált (" ... "u8 , C# 11) használ.

A **Kafka** egy elosztott, perzisztens **commit log**, nem klasszikus üzenetsor. A legfontosabb különbség a RabbitMQ-hoz képest: az üzenet nem tűnik el, amikor egy fogyasztó elolvassa – a broker egy megadott ideig (retention) megőrzi, és minden fogyasztói csoport saját pozíciót (**offset**) tart nyilván benne. Ez teszi lehetővé az újrajátszást (replay), az event sourcing-ot és azt, hogy több, egymástól független szolgáltatás dolgozza fel ugyanazt az eseményfolyamot.

Alapfogalmak:

- **Topic** – nevesített eseményfolyam (pl. `orders.placed.v1`).
- **Partition** – a topic fizikai felosztása; a sorrend **csak partíción belül** garantált, a topic egészére nem.
- **Key** – az üzenet kulcsa dönti el, melyik partícióba kerül (`hash(key) % partitionCount`). Azonos kulcsú üzenetek mindig azonos partícióba, tehát sorrendhelyesen érkeznek.
- **Offset** – az üzenet sorszáma a partíción belül; a fogyasztó ezt commitolja.
- **Consumer group** – egy csoporton belül minden partíciót pontosan egy fogyasztó kap meg; így a párhuzamosság felső korlátja a partíciók száma.

Telepítés és producer (Confluent.Kafka):

```
dotnet add package Confluent.Kafka
```

```

public sealed class OrderEventProducer : IAsyncDisposable
{
    private readonly IProducer<string, string> _producer;

    public OrderEventProducer(IConfiguration configuration)
    {
        var config = new ProducerConfig
        {
            BootstrapServers = configuration["Kafka:BootstrapServers"],
            Acks = Acks.All, // minden in-sync replikának meg kell erősítenie
            EnableIdempotence = true, // duplikátumok elkerülése újraküldéskor
            MessageSendMaxRetries = 5,
            CompressionType = CompressionType.Snappy,
            LingerMs = 20 // batchelés: kis késleltetés, nagyobb átbocsátás
        };
        _producer = new ProducerBuilder<string, string>(config).Build();
    }

    public async Task PublishAsync(OrderPlacedIntegrationEvent @event,
        CancellationToken cancellationToken)
    {
        var message = new Message<string, string>
        {
            Key = @event.OrderId.ToString(), // partíciókulcs:
rendelésenként sorrendhelyes
            Value = JsonSerializer.Serialize(@event),
            Headers =
            [
                new Header("event-type", "OrderPlaced"u8.ToArray()),
                new Header("correlation-id",
Encoding.UTF8.GetBytes(Activity.Current?.TraceId.ToString() ?? ""))
            ]
        };

        var result = await _producer.ProduceAsync("orders.placed.v1",
message, cancellationToken);
        // result.Offset, result.Partition - logolásra, diagnosztikára
    }

    public async ValueTask DisposeAsync()
    {
        _producer.Flush(TimeSpan.FromSeconds(10)); // a bufferelt üzenetek kiküldése
leállítás előtt
        _producer.Dispose();
        await ValueTask.CompletedTask;
    }
}

```

Consumer `BackgroundService` -ként (lásd "Háttérfolyamatok – `IHostedService` és `BackgroundService`" fejezet):

```

public class OrderPlacedConsumer(IServiceScopeFactory scopeFactory, IConfiguration configuration,
    ILogger<OrderPlacedConsumer> logger) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        var config = new ConsumerConfig
        {
            BootstrapServers = configuration["Kafka:BootstrapServers"],
            GroupId = "inventory-service",
            AutoOffsetReset = AutoOffsetReset.Earliest,
            EnableAutoCommit = false // manuális commit: csak sikeres feldolgozás után
        };

        using var consumer = new ConsumerBuilder<string, string>(config).Build();
        consumer.Subscribe("orders.placed.v1");

        // a blokkoló Consume miatt saját szálon futtatjuk, nehogy a host indulását akadályozza
        await Task.Run(async () =>
        {
            while (!stoppingToken.IsCancellationRequested)
            {
                try
                {
                    var result = consumer.Consume(stoppingToken);
                    var @event = JsonSerializer.Deserialize<OrderPlacedIntegrationEvent>
(resultt.Message.Value)!;

                    using var scope = scopeFactory.CreateScope();
                    var handler = scope.ServiceProvider.GetRequiredService<IOrderPlacedHandler>
();

                    await handler.HandleAsync(@event, stoppingToken);

                    consumer.Commit(result); // at-least-once szemantika
                }
                catch (OperationCanceledException) { break; }
                catch (Exception ex)
                {
                    logger.LogError(ex, "Hiba az üzenet feldolgozásakor");
                    // poison message kezelés: DLQ-ba írás, majd commit - lásd a
következő fejezetet
                }
            }
            consumer.Close(); // rendezett kilépés a consumer groupból, azonnali rebalance
        }, stoppingToken);
    }
}

```

Mikor érdemes használni: nagy átbocsátású eseményfolyamoknál (több tízezer üzenet/másodperc), ha az eseményeket több, független fogyasztónak is fel kell dolgoznia, ha az újrajátszás (replay) vagy az eseménytörténet megőrzése üzleti követelmény, illetve stream feldolgozásnál.

Legjobb gyakorlatok: mindig adj meg partíciókulcsot, ha a sorrend számít (tipikusan az aggregátum azonosítója); kapcsold be az `EnableIdempotence` -t; a topic nevébe tedd bele a verziót (`orders.placed.v1`); a fogyasztó mindig legyen idempotens (lásd "Kézbesítési szemantika és idempotens consumer" fejezet).

Mit kerülj el: ne használj auto-commitot, ha a feldolgozás elvesztése nem megengedett – az auto-commit a *beolvasás* és nem a *sikeres feldolgozás* után történik; ne hozz létre partícióként több fogyasztót ugyanabban a csoportban (feleslegesen üresjáratban álló fogyasztókat kapsz); ne használd a Kafkát klasszikus RPC/kérés-válasz csatornaként.

#92 Kafka gyakorlati minták – rebalance, offset, DLQ, kulcsválasztás

Partíciószám és párhuzamosság. A partíciók száma határozza meg, hány fogyasztó dolgozhat párhuzamosan egy consumer groupban. Ezt előre, kapacitástervezéssel érdemes megválasztani (jellemzően a várható csúcsterhelés / egy fogyasztó átbocsátása × biztonsági tényező), mert a partíciószám növelése utólag **megváltoztatja a kulcs** → **partíció leképezést**, tehát az addigi sorrendgaranciát megtöri.

Rebalance. Ha egy fogyasztó csatlakozik vagy kiesik, a csoport újraosztja a partíciókat. Ez alatt a feldolgozás szünetel ("stop-the-world"), ezért:

```
var config = new ConsumerConfig
{
    // ...
    SessionTimeoutMs = 45_000,          // ennyi idő után tekinti a broker halottnak a fogyasztót
    MaxPollIntervalMs = 300_000,        // két Consume hívás között ennyi idő telhet el
    PartitionAssignmentStrategy = PartitionAssignmentStrategy.CooperativeSticky // fokozatos, nem
    "stop-the-world" rebalance
};
```

Ha a feldolgozás hosszabb ideig tart, mint a `MaxPollIntervalMs`, a broker kidobja a fogyasztót a csoportból, és az üzenetet más újra megkapja – ez a leggyakoribb oka a "miért dolgozza fel kétszer" jellegű hibáknak.

Dead Letter Queue (poison message). A nem feldolgozható üzenet nem blokkolhatja a partíciót:

```
catch (Exception ex) when (ex is not OperationCanceledException)
{
    await _dlqProducer.ProduceAsync("orders.placed.v1.dlq", new Message<string, string>
    {
        Key = result.Message.Key,
        Value = result.Message.Value,
        Headers =
        [
            new Header("x-exception", Encoding.UTF8.GetBytes(ex.Message)),
            new Header("x-original-topic", Encoding.UTF8.GetBytes(result.Topic)),
            new Header("x-failed-at",
            Encoding.UTF8.GetBytes(DateTimeOffset.UtcNow.ToString("0")))
        ]
    }, cancellationToken);

    consumer.Commit(result); // továbblépünk, a partíció nem akad meg
}
```

Retry topic minta. Átmeneti hiba (pl. egy külső szolgáltatás nem elérhető) esetén nem érdemes azonnal DLQ-ba tenni: gyakori megoldás egy vagy több késleltetett retry topic (`orders.placed.v1.retry-5m` , `...retry-1h`), amelyekből egy külön fogyasztó a késleltetés letelte után visszateszi az üzenetet.

Kulcsválasztás. Túl kevés különböző kulcs → "forró partíció" (az egyik partíció túlterhelt, a többi üres). Túl finom szemcsés kulcs (pl. véletlen GUID) → nincs sorrendgarancia semmire. A jó kulcs jellemzően az aggregátum azonosítója (`orderId` , `customerId`).

Tömörítés és batchelés. A `LingerMs` + `CompressionType` páros drámaian javítja az átbocsátást a hálózati kihasználtság rovására – érdemes terhelés alatt mérni.

Mit kerülj el: ne commitolj offsetet feldolgozás előtt; ne dolgozz fel üzenetet "tűzz és felejtse" módon, kivétel-nyeléssel; ne feltételezz globális sorrendet a topicon; ne állítsd a partíciószámot élesben menet közben átgondolás nélkül.

#93 RabbitMQ alapfogalmak – exchange, queue, binding, routing

A **RabbitMQ** klasszikus üzenetsor (broker), amely az AMQP protokollra épül. Itt az üzenet **elfogyasztás után eltűnik** a sorból (nyugtázás – ack – után), és a broker feladata az útválasztás.

Fő elemek:

- **Producer** → **Exchange** → (binding + routing key) → **Queue** → **Consumer**
- **Exchange típusok:**
 - `direct` – pontos routing key egyezés (pl. `order.created`).
 - `topic` – mintaillesztés wildcardval (`order.*` , `order.#`).
 - `fanout` – minden bekötött sorba másolja (broadcast).
 - `headers` – fejlécek alapján irányít.
- **Queue** – ténylegesen itt tárolódnak az üzenetek; lehet durable (újraindítás után is megmarad), exclusive, auto-delete.
- **Ack / Nack** – a fogyasztó nyugtázza a sikeres feldolgozást; nyugtázatlan üzenetet a broker újraküld.
- **Prefetch (QoS)** – hány nyugtázatlan üzenetet kaphat egy fogyasztó egyszerre.


```

var factory = new ConnectionFactory { HostName = "localhost", UserName = "guest", Password =
"guest" };
await using var connection = await factory.CreateConnectionAsync();
await using var channel = await connection.CreateChannelAsync();

await channel.ExchangeDeclareAsync("orders", ExchangeType.Topic, durable: true);
await channel.QueueDeclareAsync("inventory.order-placed", durable: true, exclusive: false,
autoDelete: false,
arguments: new Dictionary<string, object?>
{
    ["x-dead-letter-exchange"] = "orders.dlx", // hibás üzenetek ide kerülnek
    ["x-message-ttl"] = 86_400_000
});
await channel.QueueBindAsync("inventory.order-placed", "orders", routingKey: "order.placed");

await channel.BasicQosAsync(prefetchSize: 0, prefetchCount: 10, global: false);

var consumer = new AsyncEventingBasicConsumer(channel);
consumer.ReceivedAsync += async (_, ea) =>
{
    try
    {
        var json = Encoding.UTF8.GetString(ea.Body.ToArray());
        await HandleAsync(json);
        await channel.BasicAckAsync(ea.DeliveryTag, multiple: false);
    }
    catch
    {
        // requeue: false → a dead-letter exchange-be kerül, nem pörög végtelen ciklusban
        await channel.BasicNackAsync(ea.DeliveryTag, multiple: false, requeue: false);
    }
};
await channel.BasicConsumeAsync("inventory.order-placed", autoAck: false, consumer);

```

Legjobb gyakorlatok: mindig `durable` exchange/queue és `persistent` üzenet, ha a tartósság számít; állíts be dead-letter exchange-et minden fontos sorhoz; a `prefetchCount` legyen véges (különben egy fogyasztó magához ránthatja az összes üzenetet); a csatorna (`IChannel`) **nem szálbiztos**, fogyasztónként/szálanként külön csatornát használj, a kapcsolatot viszont oszd meg.

Mit kerülj el: ne használj `autoAck: true` -t, ha az üzenetvesztés nem elfogadható; ne `requeue: true` -val nack-elj végtelen ciklusban egy hibás üzenetet; ne nyiss kapcsolatot üzenetenként.

#94 Kafka vs. RabbitMQ – mikor melyiket

Szempont	Kafka	RabbitMQ
Modell	elosztott, perzisztens log	üzenetsor / broker routinggal
Üzenet élettartama	retention alapján megmarad, újrajátszható	ack után eltűnik
Sorrend	partíciók belül garantált	soron belül (egy fogyasztóval)
Átbocsátás	nagyon magas (százezres nagyságrend/mp)	magas, de alacsonyabb
Útválasztás	egyszerű (topic + kulcs), a logika a fogyasztónál	gazdag (exchange típusok, routing key, header)
Késleltetett / ütemezett üzenet	natívan nincs	plugin, TTL + DLX trükk
Tipikus felhasználás	event streaming, audit, analitika, event sourcing	feladatsor, munkamegosztás, RPC-szerű integráció
Skálázás	partíciók számával	fogyasztók számával (competing consumers)

Gyakorlati döntési szempontok: ha **eseményfolyamot** publikálsz, amit több szolgáltatás is felhasznál, és/vagy szükség lehet visszamenőleges újrafeldolgozásra → Kafka. Ha **munkát osztasz szét** fogyasztók között, összetett útválasztással, késleltetéssel, prioritással → RabbitMQ. Nagy rendszerekben gyakran mindkettő jelen van, eltérő szerepben.

Fontos: a választás nem szabad, hogy átszivárogon az alkalmazás rétegbe. Az üzenetküldés mögé érdemes egy vékony absztrakciót (`IEventPublisher`) vagy egy keretrendszert (MassTransit, Wolverine) tenni, amely elrejt a broker-specifikus részleteket.

#95 MassTransit – alapok, konfiguráció, consumer

Verzió: MassTransit 8.x: .NET 6+; a v9 kereskedelmi licenc alá kerül – ez tervezési szempont új projekteknél.

A **MassTransit** egy .NET üzenetküldési keretrendszer, amely absztrakciót ad a broker (RabbitMQ, Azure Service Bus, Amazon SQS, Kafka rider) fölé, és beépítve hozza a szerializációt, a retry/redelivery logikát, a sagákat, az outboxot és a telemetriát.

```
dotnet add package MassTransit
dotnet add package MassTransit.RabbitMQ
```

Üzenet-szerződések (jellemzően külön, megosztott contracts projektben, `record` -ként):

```
public record OrderPlaced(Guid OrderId, Guid CustomerId, decimal TotalAmount,
    DateTimeOffset PlacedAt);
public record ReserveInventory(Guid OrderId, IReadOnlyList<OrderLine> Lines);
public record InventoryReserved(Guid OrderId);
```

Consumer:

```
public class OrderPlacedConsumer(IInventoryService inventory, ILogger<OrderPlacedConsumer>
logger)
    : IConsumer<OrderPlaced>
{
    public async Task Consume(ConsumeContext<OrderPlaced> context)
    {
        logger.LogInformation("Rendelés érkezett: {OrderId}", context.Message.OrderId);

        await inventory.ReserveAsync(context.Message.OrderId, context.CancellationToken);

        // válasz vagy további esemény publikálása ugyanabban a kontextusban
        (correlation megmarad)
        await context.Publish(new InventoryReserved(context.Message.OrderId));
    }
}
```

Regisztráció:

```
builder.Services.AddMassTransit(x =>
{
    x.SetKebabCaseEndpointNameFormatter(); // order-placed néven jön létre a queue
    x.AddConsumers(typeof(Program).Assembly);

    // Tranzakciós outbox EF Core-ral: az üzenet és az adatbázis-írás egy tranzakcióban
    x.AddEntityFrameworkOutbox<AppDbContext>(o =>
    {
        o.UseSqlServer();
        o.UseBusOutbox();
        o.QueryDelay = TimeSpan.FromSeconds(1);
    });

    x.UsingRabbitMq((context, cfg) =>
    {
        cfg.Host(builder.Configuration["RabbitMq:Host"], h =>
        {
            h.Username(builder.Configuration["RabbitMq:Username"]!);
            h.Password(builder.Configuration["RabbitMq:Password"]!);
        });

        cfg.UseMessageRetry(r => r.Exponential(5,
            TimeSpan.FromSeconds(1), TimeSpan.FromSeconds(30), TimeSpan.FromSeconds(2)));

        cfg.ConfigureEndpoints(context);
    });
});
```

Publikálás és küldés közti különbség: a `Publish` **eseményt** szór szét (minden érdeklődő fogyasztó megkapja, publish-subscribe), a `Send` egy konkrét végpontnak küldött **parancs** (pontosan egy fogyasztó dolgozza fel).

```

public class PlaceOrderHandler(IPublishEndpoint publishEndpoint,
ISendEndpointProvider sendEndpointProvider)
{
    public async Task HandleAsync(Order order, CancellationToken ct)
    {
        await publishEndpoint.Publish(new OrderPlaced(order.Id, order.CustomerId, order.Total,
DateTimeOffset.UtcNow), ct);

        var endpoint = await sendEndpointProvider.GetSendEndpoint(new
Uri("queue:reserve-inventory"));
        await endpoint.Send(new ReserveInventory(order.Id, order.Lines), ct);
    }
}

```

Legjobb gyakorlatok: az üzenet-szerződések legyenek immutable `record`-ok, és éljenek külön, verziózott csomagban; parancs = felszólító mód (`ReserveInventory`), esemény = múlt idő (`InventoryReserved`); mindig kapcsolod be a tranzakciós outboxot, ha az üzenetküldés adatbázis-írással jár együtt (lásd "Outbox Pattern" fejezet).

Mit kerülj el: ne tegyél domain entitást az üzenetbe – csak primitív mezőket és azonosítókat; ne `Publish` -olj parancsot és ne `Send` -elj eseményt; ne hívj szinkron HTTP API-t egy consumerből `retry/timeout` védelem nélkül (lásd "Polly v8" fejezet).

#96 MassTransit Saga State Machine

A MassTransit **Automatonymous**-alapú state machine-je a "Saga Orchestration" fejezetben leírt minta gyakorlati, éles környezetben is használható megvalósítása: az állapot perzisztens (EF Core, MongoDB, Redis), a korreláció (melyik üzenet melyik saga-példányhoz tartozik) pedig deklaratív.

```

public class OrderState : SagaStateMachineInstance
{
    public Guid CorrelationId { get; set; } // = OrderId
    public string CurrentState { get; set; } = null!;
    public decimal TotalAmount { get; set; }
    public DateTimeOffset PlacedAt { get; set; }
    public Guid? PaymentTimeoutTokenId { get; set; }
    public byte[] RowVersion { get; set; } = []; // optimista konkurenciakezelés
}

public class OrderStateMachine : MassTransitStateMachine<OrderState>
{
    public State AwaitingPayment { get; private set; } = null!;
    public State AwaitingShipment { get; private set; } = null!;

    public Event<OrderPlaced> OrderPlaced { get; private set; } = null!;
    public Event<PaymentReceived> PaymentReceived { get; private set; } = null!;
    public Event<PaymentFailed> PaymentFailed { get; private set; } = null!;
    public Schedule<OrderState, PaymentTimeoutExpired> PaymentTimeout { get; private set; }
    = null!;

    public OrderStateMachine()
    {
        InstanceState(x => x.CurrentState);

        Event(() => OrderPlaced, e => e.CorrelateById(m => m.Message.OrderId));
        Event(() => PaymentReceived, e => e.CorrelateById(m => m.Message.OrderId));
        Event(() => PaymentFailed, e => e.CorrelateById(m => m.Message.OrderId));

        Schedule(() => PaymentTimeout, x => x.PaymentTimeoutTokenId,
            s => s.Delay = TimeSpan.FromMinutes(15));

        Initially(
            When(OrderPlaced)
                .Then(ctx =>
                {
                    ctx.Saga.TotalAmount = ctx.Message.TotalAmount;
                    ctx.Saga.PlacedAt = ctx.Message.PlacedAt;
                })
                .Schedule(PaymentTimeout, ctx => new
PaymentTimeoutExpired(ctx.Saga.CorrelationId))
                .Send(new Uri("queue:process-payment"),
                    ctx => new ProcessPayment(ctx.Saga.CorrelationId, ctx.Saga.TotalAmount))
                .TransitionTo(AwaitingPayment));

        During(AwaitingPayment,
            When(PaymentReceived)
                .Unschedule(PaymentTimeout)
                .Send(new Uri("queue:ship-order"), ctx => new ShipOrder(ctx.Saga.CorrelationId))
                .TransitionTo(AwaitingShipment),

            // kompenzáció: időtúllépés vagy sikertelen fizetés esetén a foglalás feloldása
            When(PaymentFailed).Or(When(PaymentTimeout.Received))
                .Send(new Uri("queue:release-inventory"), ctx => new
ReleaseInventory(ctx.Saga.CorrelationId))
                .Finalize());

        SetCompletedWhenFinalized(); // a lezárt saga sora törlődik az adatbázisból
    }
}

```

```
}  
}
```

Regisztráció EF Core perzisztenciával:

```
x.AddSagaStateMachine<OrderStateMachine, OrderState>()  
    .EntityFrameworkRepository(r =>  
    {  
        r.ConcurrencyMode = ConcurrencyMode.Optimistic;  
        r.ExistingDbContext<SagaDbContext>();  
    });
```

Legjobb gyakorlatok: a `CorrelationId` legyen az üzleti folyamat természetes azonosítója (pl. rendelésazonosító); minden várakozó állapothoz tartozzon **timeout** (scheduled message), különben a saga örökre beragadhat; minden előrehaladó lépéshez tervezd meg a **kompenzáló** lépést; használj optimista konkurenciakezelést, mert több üzenet érkezhethet egyszerre ugyanahhoz a saga-példányhoz.

Mit kerülj el: ne tegyél üzleti logikát a state machine-be – az csak koordinál, a munkát a consumerek végzik; ne tárolj nagy adatszerkezetet a saga állapotában (csak azonosítókat és a döntéshez szükséges minimumot); ne használj sagát ott, ahol egy egyszerű lokális tranzakció is elég lenne.

#97 MassTransit hibakezelés – retry, redelivery, error queue, circuit breaker

A MassTransit háromszintű hibakezelést kínál, és fontos érteni a különbséget:

1. **Retry** – *azonnali*, memóriában történő újrapróbálkozás, a fogyasztó nem engedi el az üzenetet. Rövid, átmeneti hibákra (deadlock, pillanatnyi hálózati hiba).
2. **Redelivery (delayed redelivery)** – az üzenet visszakerül a brokerhez, és késleltetve érkezik újra. Hosszabb kiesésre (egy külső szolgáltatás percekig nem elérhető).
3. **Error queue** – ha minden próbálkozás elfogyott, az üzenet a `<queue>_error` sorba kerül, a kivétel részleteivel a fejlécekben. Innen manuálisan vagy eszközzel visszajátszható.

```

cfg.ReceiveEndpoint("order-placed", e =>
{
    // 1. azonnali retry - csak átmeneti hibákra
    e.UseMessageRetry(r =>
    {
        r.Exponential(3, TimeSpan.FromMilliseconds(200), TimeSpan.FromSeconds(5),
        TimeSpan.FromMilliseconds(500));
        r.Ignore<ValidationException>(); // értelmetlen újrapróbálni: azonnal error queue-ba
        r.Handle<DbUpdateConcurrencyException>();
    });

    // 2. késleltetett újrakézbekérés - hosszabb kiesésre
    e.UseDelayedRedelivery(r => r.Intervals(
        TimeSpan.FromMinutes(1), TimeSpan.FromMinutes(5), TimeSpan.FromMinutes(15)));

    // 3. circuit breaker - ha a hibaarány tartósan magas, szünetelteti a fogyasztást
    e.UseCircuitBreaker(cb =>
    {
        cb.TrackingPeriod = TimeSpan.FromMinutes(1);
        cb.TripThreshold = 15; // %
        cb.ActiveThreshold = 10; // legalább ennyi üzenet után értékel
        cb.ResetInterval = TimeSpan.FromMinutes(5);
    });

    e.ConfigureConsumer<OrderPlacedConsumer>(context);
});

```

Kill switch – tartós hiba esetén automatikusan leállítja a végpontot, hogy ne pörgesse végig az egész sort hibás üzenetekké:

```

cfg.UseKillSwitch(k => k.SetActivationThreshold(10).SetTripThreshold(0.15).SetRestartTimeout(m:
1));

```

Legjobb gyakorlatok: különböztess meg az **átmeneti** (retry-olható) és a **végleges** (üzleti/validációs) hibát – az utóbbit `r.Ignore<T>()` -vel azonnal engedd az error queue-ba; figyeld monitorozással az error queue-k méretét (ez az egyik legfontosabb riasztás egy event-driven rendszerben); a fogyasztó legyen idempotens, mert a retry duplikált feldolgozást jelenthet.

Mit kerülj el: ne állíts be végtelen vagy nagyon sok azonnali retry-t (ez blokkolja a fogyasztót és a sort); ne nyeld el a kivételt a consumerben, mert akkor a MassTransit sikeresnek látja a feldolgozást, és az üzenet elveszik.

#98 Wolverine – kódközpontú messaging és mediátor

Verzió: Wolverine 3.x: .NET 8+ (a .NET 9/10 támogatott).

A **Wolverine** (a JasperFx/Marten család tagja) egyszerre tölti be a MediatR (in-process mediátor) és a MassTransit (broker-alapú üzenetküldés) szerepét, de radikálisan más filozófiával: **nincs kötelező interfész és attribútum**, a handlereket konvenció alapján találja meg, és a pipeline-t futásidőben

generált kóddal állítja elő (nincs runtime reflexió-overhead, a generált kód meg is nézhető).

```
dotnet add package WolverineFx
dotnet add package WolverineFx.RabbitMQ      # vagy WolverineFx.Kafka
dotnet add package WolverineFx.EntityFrameworkCore
```

Handler – sima osztály, sima metódus, interfész nélkül:

```
public record PlaceOrder(Guid CustomerId, IReadOnlyList<OrderLine> Lines);
public record OrderPlaced(Guid OrderId, decimal TotalAmount);

public static class PlaceOrderHandler
{
    // A metódus paraméterei DI-ből érkeznek (method injection), a visszatérési érték
    // "cascading message": a Wolverine automatikusan publikálja.
    public static async Task<OrderPlaced> Handle(PlaceOrder command, AppDbContext dbContext,
        CancellationToken ct)
    {
        var order = Order.Create(command.CustomerId, command.Lines);
        dbContext.Orders.Add(order);
        await dbContext.SaveChangesAsync(ct);

        return new OrderPlaced(order.Id, order.Total);
    }
}
```

Konfiguráció:

```
builder.Host.UseWolverine(opts =>
{
    opts.UseRabbitMq(new Uri(builder.Configuration["RabbitMq:Uri"]!)).AutoProvision();

    opts.PublishMessage<OrderPlaced>().ToRabbitExchange("orders");
    opts.ListenToRabbitQueue("order-commands");

    // Tranzakciós middleware: automatikus SaveChanges + outbox a handler körül
    opts.Policies.AutoApplyTransactions();
    opts.Policies.UseDurableLocalQueues();
    opts.Policies.UseDurableOutboxOnAllSendingEndpoints();

    // Beépített resilience
    opts.OnException<DbUpdateConcurrencyException>().RetryTimes(3);
    opts.OnException<HttpRequestException>()
        .RetryWithCooldown(TimeSpan.FromSeconds(1), TimeSpan.FromSeconds(5),
            TimeSpan.FromSeconds(15));
});
```

Használat végpontból (mediátor módban vagy üzenetküldésként):


```

app.MapPost("/orders", async (PlaceOrder command, IMessageBus bus) =>
{
    var result = await bus.InvokeAsync<OrderPlaced>(command);    // in-process, mint a
MediatR Send
    return Results.Ok(result);
});

app.MapPost("/orders/async", async (PlaceOrder command, IMessageBus bus) =>
{
    await bus.PublishAsync(command);    // sorba téve, aszinkron feldolgozás
    return Results.Accepted();
});

```

A Wolverine önmagában megvalósítja a **tranzakciós outboxot** (EF Core vagy Marten mellett), és a `MapWolverineEndpoints()` segítségével a handler-ekből közvetlenül HTTP végpontokat is tud generálni – ez a "REPR Pattern" fejezetben leírt megközelítés még kevesebb boilerplate-tel.

Mikor érdemes választani: ha zavar a MediatR + MassTransit kettőssége és a sok interfész/boilerplate; ha fontos a teljesítmény (generált kód, nincs reflexiós pipeline); ha Marten/PostgreSQL stackben dolgozol (a kettő szorosan integrált). A MediatR licenccmodell-változása óta több csapat is emiatt vált.

Mit kerülj el: ne keverd a Wolverine-t és a MediatR-t ugyanabban a szolgáltatásban – a két pipeline párhuzamos működtetése zavaró; a konvenció-alapú felderítés miatt tartsd következetesen a handler-elvevézéseket (`Handle / Consume`), különben nehéz megtalálni, mi hol fut le; éles bevezetés előtt nézd meg a generált kódot (`opts.CodeGeneration.TypeLoadMode`), és futtass előre generálást (`dotnet run - codegen write`) a hidegindítás gyorsítására.

#99 Kézbesítési szemantika és idempotens consumer (Inbox Pattern)

Elosztott rendszerben három kézbesítési szemantika létezik:

- **At-most-once** – legfeljebb egyszer; lehet üzenetvesztés. (Auto-commit feldolgozás előtt.)
- **At-least-once** – legalább egyszer; lehet duplikáció. **A gyakorlatban ez az alapértelmezés.**
- **Exactly-once** – pontosan egyszer; broker szinten csak korlátozottan létezik (Kafka tranzakciók, csak Kafka→Kafka útvonalon). Adatbázissal vagy külső API-val kombinálva **gyakorlatilag elérhetetlen** – helyette at-least-once + idempotens fogyasztó a helyes megközelítés ("effectively-once").

Az **Inbox Pattern** az Outbox párja: a fogyasztó oldalon egy táblában rögzíti a már feldolgozott üzenetazonosítókat, ugyanabban a tranzakcióban, mint az üzleti hatást.

```

public class InboxMessage
{
    public Guid MessageId { get; set; } // elsődleges kulcs
    public string MessageType { get; set; } = null!;
    public DateTimeOffset ProcessedAt { get; set; }
}

public class IdempotentOrderPlacedConsumer(AppDbContext dbContext, IInventoryService inventory)
    : IConsumer<OrderPlaced>
{
    public async Task Consume(ConsumeContext<OrderPlaced> context)
    {
        var messageId = context.MessageId ?? throw new InvalidOperationException("Hiányzó MessageId");

        // 1. már feldolgoztuk? → csendben eldobjuk
        if (await dbContext.InboxMessages.AnyAsync(m => m.MessageId == messageId, context.CancellationToken))
            return;

        // 2. üzleti hatás és az inbox bejegyzés EGY tranzakcióban
        await using var transaction = await
            dbContext.Database.BeginTransactionAsync(context.CancellationToken);

        await inventory.ReserveAsync(context.Message.OrderId, context.CancellationToken);

        dbContext.InboxMessages.Add(new InboxMessage
        {
            MessageId = messageId,
            MessageType = nameof(OrderPlaced),
            ProcessedAt = DateTimeOffset.UtcNow
        });

        await dbContext.SaveChangesAsync(context.CancellationToken);
        await transaction.CommitAsync(context.CancellationToken);
    }
}

```

Az `InboxMessage.MessageId`-n lévő **egyedi index** a végső védelem: ha két példány párhuzamosan dolgozza fel ugyanazt az üzenetet, a második `DbUpdateException`-t kap, amit duplikátumként lehet kezelni.

Alternatív idempotencia-stratégiák, amikor nincs saját adatbázis-tranzakció:

- **Természetes idempotencia** – a művelet önmagában ismételhető (`SET status = 'Paid'` ugyanazzal az értékkel).
- **Verzió/sorszám ellenőrzés** – az üzenet hordoz egy verziószámot, és csak akkor alkalmazod, ha nagyobb a tároltnál (ez a "későn érkező, régi üzenet" problémára is megoldás).
- **Elosztott kulcstár** – Redis `SETNX` TTL-lel (gyors, de nem tranzakcionális az üzleti írással; kis kockázat marad).

Legjobb gyakorlatok: az inbox táblát rendszeresen takarítsd (régi bejegyzések törlése, pl. 30 nap után, ütemezett job-bal – lásd "Quartz.NET" fejezet); a `MessageId` a producer által generált, stabil azonosító legyen, ne a broker generálta kézbesítési azonosító; a kettőt (Outbox + Inbox) együtt használva kapod meg az end-to-end "effectively-once" garanciát.

Mit kerülj el: ne higgy a "exactly-once" marketingnek; ne az üzenet tartalmának hash-elésével azonosíts (két legitim, azonos tartalmú üzenet is érkezhetsz); ne tedd az inbox-ellenőrzést és az üzleti írást külön tranzakcióba.

#100 Üzenet-szerződések verziózása és séma-evolúció

Elosztott rendszerben a producer és a consumer **külön deployol**, ezért soha nem feltételezheted, hogy egyszerre frissülnek. Az üzenet-szerződés (contract) éppolyan publikus API, mint egy REST végpont – visszafelé és előre kompatibilis módon kell fejleszteni.

Biztonságos (kompatibilis) változtatások:

- Új, **opcionális** mező hozzáadása alapértelmezett értékkel.
- Mező hozzáadása, amit a régi fogyasztó egyszerűen figyelmen kívül hagy.

Törő (breaking) változtatások:

- Mező törlése vagy átnevezése.
- Mező típusának megváltoztatása.
- Enum érték jelentésének megváltoztatása.
- Kötelezővé tétel.

Törő változás esetén a helyes eljárás a **párhuzamos verziózás** (expand–contract / parallel change):

```
// 1. lépés: új verzió bevezetése, a régi is publikálódik egy ideig
public record OrderPlacedV1(Guid OrderId, decimal TotalAmount);
public record OrderPlacedV2(Guid OrderId, Money Total, Guid CustomerId);

// 2. lépés: minden fogyasztó átáll a V2-re (a V1-et még kezeli)
// 3. lépés: a V1 publikálása megszűnik, majd a V1 handler törölhető
```

Kafkánál a topic nevébe (`orders.placed.v2`), MassTransitnél a message type névterébe érdemes verziót tenni. Fejlécben mindig utazzon a szerződés típusa és verziója:

```
new Header("content-type", "application/json"u8.ToArray()),
new Header("message-type", "OrderPlaced"u8.ToArray()),
new Header("message-version", "2"u8.ToArray())
```

Schema Registry (Kafka). Confluent Schema Registry-vel a séma (Avro, Protobuf vagy JSON Schema) központilag regisztrált, és a registry **kikényszeríti** a kompatibilitási szabályt (`BACKWARD` , `FORWARD` , `FULL`) – inkompatibilis séma publikálása már fejlesztéskor elbukik, nem éles üzemben.

```
var schemaRegistry = new CachedSchemaRegistryClient(  
    new SchemaRegistryConfig { Url = configuration["Kafka:SchemaRegistryUrl"] });  
  
using var producer = new ProducerBuilder<string, OrderPlaced>(producerConfig)  
    .SetValueSerializer(new JsonSerializer<OrderPlaced>(schemaRegistry))  
    .Build();
```

Legjobb gyakorlatok: a szerződések külön, verziózott NuGet csomagban éljenek, amit a producer és a consumer is hivatkozik; **tolerant reader** elv – a fogyasztó hagyja figyelmen kívül az ismeretlen mezőket (a `System.Text.Json` alaptól ezt teszi); dokumentáld, meddig él egy régi verzió (deprecation policy).

Mit kerülj el: ne használd újra egy mező nevét más jelentéssel; ne feltételezd, hogy "úgyis mindenki egyszerre frissül"; ne serializálj domain entitást közvetlenül üzenetbe – a domain modell változása így azonnal törő API-változássá válik.

#101 Event-Driven integrációs stílusok

Nem minden "esemény" egyforma. Négy, gyakran összekevert stílus létezik (Martin Fowler taxonómiája alapján), és nagyon fontos, hogy a csapat tudatosan válasszon:

1. Event Notification – az esemény csak jelzés, minimális adattal ("történt valami, kérdezz rá, ha érdekel").

```
public record OrderPlaced(Guid OrderId);    // a fogyasztó visszahív az Orders API-ra  
a részletekért
```

Előny: kicsi üzenet, laza csatolás a modell szintjén. Hátrány: visszahívás (chatty), futásidejű csatolás – ha a forrás szolgáltatás nem elérhető, a fogyasztó sem tud dolgozni.

2. Event-Carried State Transfer (ECST) – az esemény hordozza a fogyasztóhoz szükséges teljes állapotot.

```
public record OrderPlaced(  
    Guid OrderId, Guid CustomerId, string CustomerEmail,  
    IReadOnlyList<OrderLineDto> Lines, decimal TotalAmount, DateTimeOffset PlacedAt);
```

Előny: a fogyasztó autonóm, nincs szinkron hívás, saját olvasási modellt építhet. Hátrány: nagyobb üzenet, adatduplikáció, eventual consistency, és az üzenet-szerződés gazdagabb (nehezebb változtatni).

3. Event Sourcing – az események az igazság forrása, az állapot belőlük származtatott (lásd a következő fejezetet).

4. CQRS-integráció – az események olvasási modellt (projekciót) frissítenek egy másik szolgáltatásban.

Gyakorlati ajánlás mikroszolgáltatásoknál: az **ECST** a leggyakoribb és általában a legjobb választás integrációs eseményekhez, mert megszünteti a futásidejű függést a forrás szolgáltatástól. Fontos megkülönböztetni:

- **Domain event** – a bounded contexten *belül*, in-process, gazdag, a domain nyelvén ("Domain Events" fejezet).
- **Integration event** – a contexten *kívülre*, brokerre, publikus, verziózott szerződés, csak primitívekkel.

A kettő között az alkalmazásréteg fordít – a domain event nem szivároghat ki a brokerre.

Mit kerülj el: ne publikálj "CRUD eseményeket" (`CustomerUpdated` az összes mezővel) – ezek semmit nem mondanak az üzleti szándékról, és minden fogyasztót arra kényszerítenek, hogy kitalálja, mi is történt; nevezd az eseményt üzleti eseményként (`CustomerMovedToNewAddress` , `SubscriptionCancelled`).

#102 Event Sourcing .NET-ben (Marten)

Verzió: Marten 7.x: .NET 8+ és PostgreSQL 12+.

Event Sourcing esetén nem az aktuális állapotot tároljuk, hanem az összes állapotváltozást okozó eseményt, időrendben. Az aktuális állapot ezek visszajátszásából (fold/replay) áll elő. Előnye a teljes audit trail, a "hogyan jutottunk ide" kérdés megválaszolhatósága és a visszamenőleg építhető új olvasási modellek.

A .NET ökoszisztémában a **Marten** (PostgreSQL fölött) és az **EventStoreDB** a két legelterjedtebb megoldás.

```
dotnet add package Marten
```

```
// Események - múlt idejű, immutable record-ok
public record OrderStarted(Guid OrderId, Guid CustomerId);
public record ItemAdded(Guid ProductId, int Quantity, decimal UnitPrice);
public record OrderConfirmed(DateTimeOffset ConfirmedAt);

// Aggregátum: az események alkalmazásából áll elő (Apply konvenció)
public class Order
{
    public Guid Id { get; private set; }
    public OrderStatus Status { get; private set; }
    public decimal Total { get; private set; }

    public void Apply(OrderStarted e) { Id = e.OrderId; Status = OrderStatus.Draft; }
    public void Apply(ItemAdded e) => Total += e.Quantity * e.UnitPrice;
    public void Apply(OrderConfirmed e) => Status = OrderStatus.Confirmed;
}
```

```
builder.Services.AddMarten(options =>
{
    options.Connection(builder.Configuration.GetConnectionString("Postgres")!);
    options.Projections.Snapshot<Order>(SnapshotLifecycle.Inline); // aktuális állapot
    // azonnal lekérdezhető
    options.Projections.Add<OrderSummaryProjection>(ProjectionLifecycle.Async); // olvasási
    // modell háttérben
}).UseLightweightSessions().AddAsyncDaemon(DaemonMode.HotCold);
```

```
// Új stream indítása
var orderId = Guid.NewGuid();
session.Events.StartStream<Order>(orderId, new OrderStarted(orderId, customerId));
await session.SaveChangesAsync(ct);

// Esemény hozzáfűzése, optimista konkurenciakezeléssel (várt verzió)
await session.Events.AppendExclusive(orderId, ct);
session.Events.Append(orderId, new ItemAdded(productId, 2, 4990M));
await session.SaveChangesAsync(ct);

// Aktuális állapot
var order = await session.Events.AggregateStreamAsync<Order>(orderId, token: ct);
```

Mikor érdemes használni: ha az audit trail vagy a jogszabályi megfelelés megköveteli a teljes változástörténetet (pénzügy, egészségügy, biztosítás); ha az üzleti folyamat maga is eseményekben gondolkodik; ha utólag is szeretnél új nézeteket építeni a történeti adatokból.

Mit kerülsz el: **ne alkalmazd az egész rendszerre** – jellemzően egy-két aggregátum indokolja, a többi maradhat klasszikus CRUD; ne feledkezz meg az események verziózásáról (az eseményeket soha nem lehet megváltoztatni, csak upcaster-rel átalakítani olvasáskor); ne várj azonnali konzisztenciát az aszinkron projekcióktól; ne keverd össze a CQRS-t és az event sourcing-ot – az egyik nem feltétele a másiknak.

#103 Eventual consistency és az elosztott tranzakciók valósága

Mikroszolgáltatásoknál minden szolgáltatásnak saját adatbázisa van ("database per service"), így egy üzleti folyamat több adatbázist érint. A klasszikus, ACID elosztott tranzakció (**2PC / XA**) technikailag létezik, de a gyakorlatban kerülendő: szinkron blokkolást, rendelkezésre állási csatolást és rossz skálázhatóságot okoz (a CAP-tétel értelmében a konzisztenciáért a rendelkezésre állással fizetsz).

A gyakorlati recept:

1. **Egy lokális ACID tranzakció** szolgáltatásonként (EF Core, `SaveChangesAsync`).
2. **Outbox** az üzenet és az adatváltozás atomikus összekötésére (lásd "Outbox Pattern" fejezet).
3. **Saga** a szolgáltatásokon átívelő folyamat koordinálására, kompenzáló műveletekkel (lásd "Saga Orchestration" és "MassTransit Saga State Machine" fejezetek).
4. **Idempotens fogasztó** a duplikátumok ellen (Inbox).
5. **Eventual consistency** tudatos elfogadása és **kezelése a felhasználói felületen is**.

Az utolsó pont a leggyakrabban elfelejtett rész. Ha egy folyamat aszinkron, a UI-nak ezt kommunikálnia kell:

```
// Nem 200 OK a "kész" jelentéssel, hanem 202 Accepted + státusz-végpont
app.MapPost("/orders", async (PlaceOrder command, IMessageBus bus) =>
{
    var orderId = Guid.NewGuid();
    await bus.PublishAsync(command with { OrderId = orderId });
    return Results.Accepted($"/orders/{orderId}/status", new { orderId, status = "Processing" });
});

app.MapGet("/orders/{id:guid}/status", async (Guid id, IOrderStatusReader reader,
Cancellation token ct) =>
    await reader.GetStatusAsync(id, ct) is { } status ? Results.Ok(status) : Results.NotFound());
```

Konzisztenciahatár tervezése. Az aggregátum a tranzakció határ: *egy tranzakcióban egy aggregátumot módosíts*. Ha rendszeresen két aggregátumot kell egyszerre, konzisztensen módosítani, az jel: vagy rosszul vannak meghúzva a határok, vagy a folyamat valójában eventually consistent lehet.

Read-your-own-writes probléma. Aszinkron projekcióknál a felhasználó közvetlenül a mentés után nem látja a saját módosítását. Megoldások: a UI optimista frissítése, a parancs válaszában visszaadott friss adat használata, vagy verzió-token ("olvasd, amíg a projekció el nem éri ezt a verziót").

Mit kerülj el: ne próbálj 2PC-t bevezetni mikroszolgáltatások közé; ne hívj szinkron láncban 4-5 szolgáltatást egy kérés kiszolgálásához (a rendelkezésre állások szorzódnak: $5 \times 99,9\% \approx 99,5\%$); ne ígérj a felhasználói felületen azonnali konzisztenciát, ha a háttérben aszinkron folyamat fut.

#104 Vertical Slice Architecture a gyakorlatban

A "Vertical sliced code Organisation pattern" fejezet a koncepciót írja le; itt a napi gyakorlat következik: hogyan néz ki egy szelet, mi maradjon közös, és hol a határ a Clean Architecture felé.

Mappastruktúra – feature-enként, nem rétegenként:

```
src/
  Orders.Api/
    Features/
      PlaceOrder/
        PlaceOrder.cs           // Command + Validator + Handler + Endpoint + Response
EGY fájlban
    PlaceOrderTests.cs
    CancelOrder/
      CancelOrder.cs
    GetOrderDetails/
      GetOrderDetails.cs        // Query + Handler + Endpoint, közvetlen SQL/projekció
Domain/                          // csak az, ami valóban több szeletben közös: aggregátumok, VO-k
  Order.cs
  OrderStatus.cs
Infrastructure/
  AppDbContext.cs
  Migrations/
Common/
  Results/
  Behaviors/
Program.cs
```

Egy teljes szelet, egyetlen fájlban:


```

public static class PlaceOrder
{
    public record Command(Guid CustomerId, IReadOnlyList<Line> Lines) : IRequest<Result<Guid>>;
    public record Line(Guid ProductId, int Quantity);
    public record Response(Guid OrderId, decimal Total);

    public class Validator : AbstractValidator<Command>
    {
        public Validator()
        {
            RuleFor(x => x.CustomerId).NotEmpty();
            RuleFor(x => x.Lines).NotEmpty().WithMessage("A rendelésnek legalább egy tételt tartalmaznia kell.");
            RuleForEach(x => x.Lines).ChildRules(l => l.RuleFor(x => x.Quantity).GreaterThan(0));
        }
    }

    public class Handler(AppDbContext db, IPublishEndpoint publisher) :
    IRequestHandler<Command, Result<Guid>>
    {
        public async Task<Result<Guid>> Handle(Command command, CancellationToken ct)
        {
            var products = await db.Products
                .Where(p => command.Lines.Select(l => l.ProductId).Contains(p.Id))
                .ToDictionaryAsync(p => p.Id, ct);

            var order = Order.Create(command.CustomerId);
            foreach (var line in command.Lines)
            {
                if (!products.TryGetValue(line.ProductId, out var product))
                    return Result<Guid>.Failure($"Ismeretlen termék: {line.ProductId}");
                order.AddItem(product, line.Quantity);
            }

            db.Orders.Add(order);
            await db.SaveChangesAsync(ct);
            await publisher.Publish(new OrderPlaced(order.Id, order.Total), ct);

            return Result<Guid>.Success(order.Id);
        }
    }

    public class Endpoint : IEndpoint
    {
        public void Map(IEndpointRouteBuilder app) => app
            .MapPost("/orders", async (Command command, ISender sender, CancellationToken ct) =>
            {
                var result = await sender.Send(command, ct);
                return result.IsSuccess
                    ? TypedResults.Created($" /orders/{result.Value}", result.Value)
                    : TypedResults.BadRequest(result.Error);
            })
            .WithTags("Orders")
            .RequireAuthorization("orders:write");
    }
}

```

Az `IEndpoint` interfészt egy egyszerű assembly-scan regisztrálja, így nem kell minden szeletet kézzel bekötni a `Program.cs`-be:

```
public interface IEndpoint { void Map(IEndpointRouteBuilder app); }

public static class EndpointExtensions
{
    public static IServiceCollection AddEndpoints(this IServiceCollection services,
        Assembly assembly)
    {
        var descriptors = assembly.DefinedTypes
            .Where(t => t is { IsAbstract: false, IsInterface: false } &&
                t.IsAssignableTo(typeof(IEndpoint)))
            .Select(t => ServiceDescriptor.Transient(typeof(IEndpoint), t));
        services.TryAddEnumerable(descriptors);
        return services;
    }

    public static IApplicationBuilder MapEndpoints(this WebApplication app)
    {
        foreach (var endpoint in app.Services.GetRequiredService<IEnumerable<IEndpoint>>())
            endpoint.Map(app);
        return app;
    }
}
```

Vezérelvek:

- **A csatolás iránya számít.** Szeleten belül nyugodtan lehet "rendetlenség" (a handler közvetlenül a `DbContext`-et használja); szeletek között viszont szigorú a határ – egy szelet ne hívja közvetlenül egy másik szelet handlerét.
- **A megosztás legyen szándékos.** Egy absztrakciót akkor emelj ki a `Common / Domain` mappába, amikor **harmadszor** ismétlődik, nem elsőre. A vertical slice tudatosan vállal némi duplikációt a laza csatolásért cserébe.
- **Query ≠ Command.** A lekérdező szeletekben nem kell repository, aggregátum, domain modell – nyugodtan lehet közvetlen EF projekció vagy Dapper SQL. A parancs oldalon jöjjön a gazdag domain modell.
- **Együtt él a Clean Architecture-rel.** Gyakori hibrid: a *domain* réteg klasszikusan izolált, de az *application* rétegen belül feature-mappák vannak, use-case-enként.

Mit kerülj el: ne hozz létre "Common" szemetesládát, ahová minden bekerül; ne tarts fenn üres, mindent átvezető réteget (`IOrderService` , ami csak továbbhív) csak azért, mert "így szokás"; ne csússzon vissza a struktúra rétegekbe (`Handlers/` , `Validators/` , `Dtos/` mappák) – az pontosan az, amit a minta el akart kerülni.

#105 Modular Monolith – mikor NE microservice

A mikroszolgáltatás nem cél, hanem eszköz, aminek jelentős ára van: elosztott tranzakciók, hálózati hibák, verziózott szerződések, observability infrastruktúra, deployment komplexitás, nehezebb lokális fejlesztés. A **modular monolith** ugyanazt a moduláris gondolkodást (bounded contextek, tiszta határok) adja, de **egy** deployolható egységben.

```
src/
  App.Host/                // egyetlen ASP.NET Core alkalmazás
  Modules/
    Orders/
      Orders.Public/       // AMIT más modul láthat: szerződések, integration eventek
      Orders.Internal/     // domain, EF Core, handlerok - más modul NEM hívható
    Billing/
      Billing.Public/
      Billing.Internal/
  Shared/
    Shared.Kernel/        // valóban közös primitívek (Result, Money, Entity ősosztály)
```

A modulhatár kikényszerítése a kulcs, különben csak egy jól mappázott gombóc lesz belőle:

- Külön projekt (assembly) modulonként, és csak a `*.Public` projektre hivatkozhatnak mások.
- Külön **séma** az adatbázisban modulonként (`orders.Orders` , `billing.Invoices`), tiltott a séma közötti JOIN és a másik modul tábláinak írása.
- Modulok közötti kommunikáció csak in-process üzenetküldéssel (MediatR/Wolverine) vagy publikus interfészen keresztül.
- **Architektúra teszt**, ami elbukik, ha valaki átlép egy határt (lásd "Architektúra tesztek" fejezet).

Előnyök: egyetlen tranzakció, egyszerű refaktorálás, egyszerű debug és deploy, nincs hálózati hibamód – miközben a modulhatárok készen állnak arra, hogy egy modul később **kiemelhető legyen** külön szolgáltatásba (a Strangler Fig fejezet szerint).

Mikor indokolt mégis mikroszolgáltatás: külön skálázási igény (egy komponens 100×-os terhelést kap), külön technológiai stack, külön csapatok független release-ciklussal, eltérő rendelkezésre állási/megfelelőségi követelmény, vagy a szervezet mérete (Conway-törvény).

Mit kerülj el: ne kezdj greenfield projektet mikroszolgáltatásokkal, ha a domain határai még nem tisztázottak – rossz határokat sokkal olcsóbb monolitban javítani; ne hozz létre "elosztott monolitot" (mikroszolgáltatások, amelyek szinkron láncban hívogatják egymást és együtt kell deployolni őket) – ez mindkét világ hátrányait egyesíti.

#106 .NET Aspire – helyi orchestration, service discovery, telemetria

Verzió: .NET 8-tól (Aspire 8.0); önálló, .NET verziótól függetlenített kiadás: Aspire 9.x.

A **.NET Aspire** egy opinionated stack több projektből álló (jellemzően microservices) alkalmazások fejlesztéséhez: kódban leírt helyi orchestration, automatikus service discovery, előre bekötött OpenTelemetry, health check-ek, resilience, valamint egy dashboard, amin a logok, trace-ek és metrikák egy helyen látszanak. A `docker-compose.yml` kézi karbantartása nagyrészt kiváltható vele.

AppHost – az egész rendszer topológiája C#-ban:

```
var builder = DistributedApplication.CreateBuilder(args);

var postgres = builder.AddPostgres("postgres")
    .WithDataVolume()
    .WithPgAdmin()
    .AddDatabase("ordersdb");

var kafka = builder.AddKafka("kafka").WithKafkaUI();
var redis = builder.AddRedis("cache");

var ordersApi = builder.AddProject<Projects.Orders_Api>("orders-api")
    .WithReference(postgres)
    .WithReference(kafka)
    .WithReference(redis)
    .WaitFor(postgres);

var inventoryWorker = builder.AddProject<Projects.Inventory_Worker>("inventory-worker")
    .WithReference(kafka)
    .WaitFor(kafka);

builder.AddNpmApp("frontend", "../frontend", "dev")
    .WithReference(ordersApi)
    .WithHttpEndpoint(env: "PORT")
    .WithExternalHttpEndpoints();

builder.Build().Run();
```

ServiceDefaults – minden szolgáltatásban közös keresztirányú beállítások:

```
public static IHostApplicationBuilder AddServiceDefaults(this IHostApplicationBuilder builder)
{
    builder.ConfigureOpenTelemetry();
    builder.AddDefaultHealthChecks();
    builder.Services.AddServiceDiscovery();

    builder.Services.ConfigureHttpClientDefaults(http =>
    {
        http.AddStandardResilienceHandler(); // retry, circuit breaker, timeout - lásd
        Polly fejezet
        http.AddServiceDiscovery();
    });

    return builder;
}
```

Ezután a szolgáltatások **logikai néven** hívják egymást, nem URL-en:

```
builder.Services.AddHttpClient<InventoryClient>(c => c.BaseAddress = new  
Uri("https+http://inventory-api"));
```

A connection stringeket, portokat, környezeti változókat az AppHost injektálja – lokálisan a konténerekhez, éles környezetben (Azure Container Apps, Kubernetes az Aspire manifest/ `aspire` eszközzel) a valós erőforrásokhoz.

Mikor érdemes használni: 3+ projektből álló megoldásoknál, ahol a lokális futtatáshoz több infrastruktúra-elem (adatbázis, broker, cache) kell; ha az onboarding ideje ("F5 és fut az egész rendszer") fontos; ha egységes observability alapot szeretnél mindenhol.

Mit kerülj el: ne tekintsd teljes értékű éles orchestrátornak – a produkciós futtatás továbbra is Kubernetes/Container Apps; ne tegyél üzleti logikát az AppHost projektbe; ne kösd magad hozzá szorosan a domain kódból (az Aspire infrastruktúra-szintű eszköz).

#107 OpenTelemetry – distributed tracing, metrikák, korreláció

Elosztott rendszerben a logfájl önmagában kevés: egy felhasználói kérés 5 szolgáltatáson és 2 üzenetsoron megy át. Az **OpenTelemetry** (OTel) egységes, gyártófüggetlen szabvány a három jelre: **trace, metric, log**. A .NET natívan támogatja (`System.Diagnostics.Activity` = OTel span, `System.Diagnostics.Metrics.Meter` = OTel metrika).

```
dotnet add package OpenTelemetry.Extensions.Hosting  
dotnet add package OpenTelemetry.Instrumentation.AspNetCore  
dotnet add package OpenTelemetry.Instrumentation.Http  
dotnet add package OpenTelemetry.Instrumentation.EntityFrameworkCore  
dotnet add package OpenTelemetry.Exporter.OpenTelemetryProtocol
```

```

builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService(
        serviceName: builder.Environment.ApplicationName,
        serviceVersion: typeof(Program).Assembly.GetName().Version?.ToString()))
    .WithTracing(tracing => tracing
        .AddAspNetCoreInstrumentation(o => o.RecordException = true)
        .AddHttpClientInstrumentation()
        .AddEntityFrameworkCoreInstrumentation(o => o.SetDbStatementForText = true)
        .AddSource("MassTransit") // vagy "Confluent.Kafka", saját
ActivitySource-ok
        .AddSource(DiagnosticsConfig.ActivitySource.Name)
        .AddOtlpExporter())
    .WithMetrics(metrics => metrics
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddRuntimeInstrumentation() // GC, thread pool, exception ráta
        .AddMeter(DiagnosticsConfig.Meter.Name)
        .AddOtlpExporter());

builder.Logging.AddOpenTelemetry(o =>
{
    o.IncludeScopes = true;
    o.IncludeFormattedMessage = true;
    o.AddOtlpExporter();
});

```

Saját span és üzleti metrika:

```

public static class DiagnosticsConfig
{
    public const string ServiceName = "orders-api";
    public static readonly ActivitySource ActivitySource = new(ServiceName);
    public static readonly Meter Meter = new(ServiceName);

    public static readonly Counter<long> OrdersPlaced =
        Meter.CreateCounter<long>("orders.placed", unit: "{order}", description: "Leadott
rendelések száma");

    public static readonly Histogram<double> OrderValue =
        Meter.CreateHistogram<double>("orders.value", unit: "HUF");
}

public async Task<Guid> PlaceOrderAsync(PlaceOrder command, CancellationToken ct)
{
    using var activity = DiagnosticsConfig.ActivitySource.StartActivity("PlaceOrder");
    activity?.SetTag("customer.id", command.CustomerId);
    activity?.SetTag("order.line_count", command.Lines.Count);

    try
    {
        var order = await _handler.HandleAsync(command, ct);

        DiagnosticsConfig.OrdersPlaced.Add(1, new KeyValuePair<string, object?>
("channel", command.Channel));
        DiagnosticsConfig.OrderValue.Record((double)order.Total);
        activity?.SetStatus(ActivityStatusCode.Ok);

        return order.Id;
    }
    catch (Exception ex)
    {
        activity?.SetStatus(ActivityStatusCode.Error, ex.Message);
        activity?.AddException(ex);
        throw;
    }
}

```

Korreláció üzenetsoron át. A trace context (W3C `traceparent`) HTTP-n automatikusan utazik; brokeren keresztül az üzenet **fejlécében** kell átvinni. A MassTransit és a Wolverine ezt magától megteszi; nyers Confluent.Kafka használatakor manuálisan kell propagálni (`Propagators.DefaultTextMapPropagator.Inject/Extract`), különben a trace elszakad a fogyasztónál.

Log és trace összekötése. Serilog esetén az enricher beemeli a `TraceId` / `SpanId` mezőket, így a log és a trace kereshetően összekapcsolódik:

```

.Enrich.WithSpan()
.Enrich.FromLogContext()

```

Legjobb gyakorlatok: a metrika-címkék (tag) **kis kardinalitásúak** legyenek (státusz, csatorna, végpont neve – **soha nem** `userId` vagy `orderId`, mert az felrobbantja a metrikátárát); éles környezetben állíts be **sampling**-et (pl. `ParentBasedSampler` + arányos mintavétel), a hibás kéréseket viszont mindig mintázd;

a `RecordException` és a strukturált log (lásd "Strukturált logolás" fejezet) együtt adja a teljes képet.

Mit kerülj el: ne logolj személyes adatot vagy titkot `span` attribútumba; ne írd saját, kézi korrelációs ID mechanizmust, ha az `Activity.Current.TraceId` már megvan; ne kapcsolod be a teljes SQL statement rögzítést éles környezetben átgondolás nélkül (érzékeny adat + adatmennyiség).

#108 Polly v8 és Microsoft.Extensions.Resilience

Verzió: Polly v8 / `Microsoft.Extensions.Http.Resilience` : .NET 8-tól.

A "HttpClientFactory és Polly (resilience)" fejezet a klasszikus Polly v7 API-t mutatja. A **Polly v8** teljesen új, allokációmentesebb API-t hozott (`ResiliencePipeline`), a **Microsoft.Extensions.Resilience** csomag pedig ezt integrálja a .NET hosttal, telemetriával együtt.

```
dotnet add package Microsoft.Extensions.Http.Resilience
```

A "standard" pipeline – ez az esetek 90%-ára elég:

```
builder.Services.AddHttpClient<InventoryClient>(c => c.BaseAddress =
new Uri("https://inventory"))
    .AddStandardResilienceHandler(options =>
    {
        options.Retry.MaxRetryAttempts = 3;
        options.Retry.BackoffType = DelayBackoffType.Exponential;
        options.Retry.UseJitter = true; // thundering
    herd elkerülése
        options.AttemptTimeout.Timeout = TimeSpan.FromSeconds(5); // egy próbálkozás
        options.TotalRequestTimeout.Timeout = TimeSpan.FromSeconds(30); // az egész művelet
        options.CircuitBreaker.FailureRatio = 0.3;
        options.CircuitBreaker.SamplingDuration = TimeSpan.FromSeconds(30);
        options.CircuitBreaker.BreakDuration = TimeSpan.FromSeconds(15);
    });
```

A standard handler sorrendje fixen: **rate limiter** → **total timeout** → **retry** → **circuit breaker** → **attempt timeout**. Ez a sorrend nem véletlen: a **retry** a **circuit breaker** *főltt* van, így a megszakított áramkör nem generál felesleges próbálkozásokat.

Saját, névvel ellátott pipeline (nem csak HTTP-hez):


```

builder.Services.AddResiliencePipeline("kafka-publish", pipeline =>
{
    pipeline
        .AddRetry(new RetryStrategyOptions
        {
            ShouldHandle = new PredicateBuilder().Handle<KafkaException>(),
            MaxRetryAttempts = 5,
            Delay = TimeSpan.FromMilliseconds(200),
            BackoffType = DelayBackoffType.Exponential,
            UseJitter = true,
            OnRetry = args =>
            {
                args.Context.Properties.TryGetValue(new ResiliencePropertyKey<ILogger>("logger"),
out var log);
                log?.LogWarning(args.Outcome.Exception, "Újrapróbálkozás #
{Attempt}", args.AttemptNumber);
                return default;
            }
        })
        .AddTimeout(TimeSpan.FromSeconds(10));
});

// Használat
public class KafkaPublisher(ResiliencePipelineProvider<string> provider)
{
    public async Task PublishAsync(string topic, string payload, CancellationToken ct)
    {
        var pipeline = provider.GetPipeline("kafka-publish");
        await pipeline.ExecuteAsync(async token => await _producer.ProduceAsync(topic, payload,
token), ct);
    }
}

```

Hedging – ha egy hívás lassú, párhuzamosan indít egy másodikat, és az elsőként megérkező választ használja. Farok-latencia (p99) csökkentésére való, **kizárólag idempotens** műveleteknél:

```

.AddHedging(new HttpHedgingStrategyOptions { MaxHedgedAttempts = 2, Delay =
TimeSpan.FromSeconds(1) })

```

Legjobb gyakorlatok: mindig legyen **timeout** – retry timeout nélkül csak sokszorozza a beragadást; **jitter** nélkül a retry-ok szinkronizálódnak és lavinát okoznak; a circuit breaker per-endpoint (nem globálisan) figyeljen; a resilience pipeline telemetriája automatikusan OTel-be kerül, használd riasztáshoz.

Mit kerülj el: ne retry-olj nem idempotens műveletet (POST fizetés) idempotencia-kulcs nélkül (lásd "Idempotency Pattern" fejezet); ne retry-olj 4xx üzleti hibán (400, 404, 422) – az nem lesz sikeresebb ötödjére sem; ne építs több egymásba ágyazott retry-réteget (kliens + gateway + service), mert a próbálkozások száma szorozódik.

#109 HybridCache és elosztott gyorsítótár

Verzió: .NET 9-től (Microsoft.Extensions.Caching.Hybrid).

A .NET 9-ben megjelent **HybridCache** egyesíti az in-memory (L1) és az elosztott (L2, tipikusan Redis) cache-t, és megoldja a klasszikus `IDistributedCache` két nagy gyengeségét: a **stampede** (cache miss esetén egyszerre több szál számolja ki ugyanazt) problémát és a manuális szerializációt.

```
dotnet add package Microsoft.Extensions.Caching.Hybrid
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

```
builder.Services.AddStackExchangeRedisCache(o => o.Configuration =
builder.Configuration["Redis:ConnectionString"]);

builder.Services.AddHybridCache(options =>
{
    options.DefaultEntryOptions = new HybridCacheEntryOptions
    {
        Expiration = TimeSpan.FromMinutes(10),           // L2 (elosztott)
        LocalCacheExpiration = TimeSpan.FromMinutes(2) // L1 (memória)
    };
    options.MaximumPayloadBytes = 1024 * 1024;
});
```

```
public class ProductReader(HybridCache cache, AppDbContext db)
{
    public async Task<ProductDto?> GetAsync(Guid id, CancellationToken ct) =>
        await cache.GetOrCreateAsync(
            $"product:{id}",
            factory: async token => await db.Products
                .Where(p => p.Id == id)
                .Select(p => new ProductDto(p.Id, p.Name, p.Price))
                .FirstOrDefaultAsync(token),
            tags: ["products", $"product:{id}"],
            cancellationToken: ct);

    public async Task InvalidateAsync(Guid id, CancellationToken ct) =>
        await cache.RemoveByTagAsync($"product:{id}", ct);
}
```

Cache-stratégiák és mikor melyik:

Stratégia	Leírás	Mikor
Cache-aside (lazy)	olvasáskor tölt, íráskor invalidál	általános eset, HybridCache alapértelmezés
Write-through	íráskor a cache is frissül	ha a friss adat azonnal kell
Read-through	a cache maga tölt a forrásból	ha a hozzáférés egységesíthető
Refresh-ahead	lejárat előtt a háttérben frissít	drága, gyakran olvasott adat

Invalidáció elosztott környezetben. Több példány esetén a lokális L1 cache-ek nem tudnak egymás invalidálásáról. Megoldás: rövid L1 lejárati idő (percek), vagy Redis pub/sub alapú invalidációs üzenet, amit minden példány feldolgoz.

Mit érdemes cache-elni: ritkán változó, gyakran olvasott, drágán előállítható adatot (törzsadatok, konfiguráció, jogosultsági szabályok, külső API válaszai). **Mit nem:** felhasználó-specifikus, gyorsan változó, vagy jogi szempontból mindig friss adatot igénylő tartalmat.

Mit kerülj el: ne cache-elj kulcs nélkül tenant/felhasználó-specifikus adatot (adatszívargás más tenant felé – a cache kulcsba mindig menjen bele a tenant/user azonosító, ha releváns); ne tegyél a cache-be nagy objektumgráfot; ne feledd, hogy a cache **nem** adatbázis – legyen a rendszer működőképese akkor is, ha a cache üres vagy nem elérhető.

#110 Rate limiting és Output Caching

Verzió: Rate limiting middleware: .NET 7; Output Caching: .NET 7 (tag alapú invalidálás: .NET 8).

Rate limiting (.NET 7+ beépített middleware) – védelem a túlterhelés és a visszaélés ellen. Négy algoritmus közül lehet választani:

```

builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    // Fix ablak: egyszerű, de ablakhatáron duplázódhat a terhelés
    options.AddFixedWindowLimiter("fixed", o =>
    {
        o.PermitLimit = 100;
        o.Window = TimeSpan.FromMinutes(1);
        o.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
        o.QueueLimit = 10;
    });

    // Token bucket: megengedi a rövid csúcsokat (burst)
    options.AddTokenBucketLimiter("burst", o =>
    {
        o.TokenLimit = 100;
        o.TokensPerPeriod = 20;
        o.ReplenishmentPeriod = TimeSpan.FromSeconds(10);
    });

    // Egyidejű kérések korlátozása (drága végpontokra)
    options.AddConcurrencyLimiter("expensive", o => { o.PermitLimit = 5; o.QueueLimit = 20; });

    // Particionálás felhasználónként / API kulcsonként
    options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(context =>
        RateLimitPartition.GetSlidingWindowLimiter(
            partitionKey: context.User.FindFirst("sub")?.Value
                ?? context.Connection.RemoteIpAddress?.ToString()
                ?? "anonymous",
            factory: _ => new SlidingWindowRateLimiterOptions
            {
                PermitLimit = 300, Window = TimeSpan.FromMinutes(1), SegmentsPerWindow = 6
            }));

    options.OnRejected = async (context, ct) =>
    {
        if (context.Lease.TryGetMetadata(MetadataName.RetryAfter, out var retryAfter))
            context.HttpContext.Response.Headers.RetryAfter =
                ((int)retryAfter.TotalSeconds).ToString();
        await context.HttpContext.Response.WriteAsync("Túl sok kérés. Próbálja újra később.", ct);
    };
});

app.UseRateLimiter();
app.MapGet("/reports/heavy", GenerateReport).RequireRateLimiting("expensive");

```

Output Caching (.NET 7+) – a *válasz* gyorsítótárazása szerveroldalon, a `ResponseCaching` -gel ellentétben szerver által vezérelt, taggelhető és programozottan invalidálható:

```

builder.Services.AddOutputCache(options =>
{
    options.AddBasePolicy(b => b.Expire(TimeSpan.FromSeconds(30)));
    options.AddPolicy("products", b => b
        .Expire(TimeSpan.FromMinutes(5))
        .SetVaryByQuery("page", "pageSize", "category")
        .Tag("products"));
});

app.UseOutputCache();

app.MapGet("/products", GetProducts).CacheOutput("products");

// Célzott invalidálás íráskor
app.MapPost("/products", async (CreateProduct cmd, ISender sender, IOutputCacheStore cacheStore,
CancellationTokens ct) =>
{
    var id = await sender.Send(cmd, ct);
    await cacheStore.EvictByTagAsync("products", ct);
    return TypedResults.Created($"products/{id}", id);
});

```

Legjobb gyakorlatok: a rate limitet felhasználó/API-kulcs szerint particionáld, ne globálisan; küldj `Retry-After` fejléceket; a limitet a gateway és az alkalmazás szintjén is érdemes megfontolni; az output cache **csak** anonim vagy explicit módon `VaryBy` -jal elkülönített tartalomra való.

Mit kerülj el: ne cache-elj kimenetet autentikált, felhasználó-specifikus végponton `VaryBy` nélkül; ne állíts be olyan szigorú limitet, ami a saját belső szolgáltatásaidat is kizárja (a belső hívóknak külön policy vagy kizárás kell).

#111 Keyed Services és haladó DI (.NET 8+)

Verzió: .NET 8-tól (`AddKeyedScoped` , `[FromKeyedServices]`).

A .NET 8 óta a beépített DI konténer is támogatja a **kulcsolt regisztrációt** – az a funkció, amiért korábban gyakran Autofacra kellett váltani (lásd "Autofac Benefits" fejezet).

```

builder.Services.AddKeyedScoped<IPaymentProvider, StripePaymentProvider>("stripe");
builder.Services.AddKeyedScoped<IPaymentProvider, BarionPaymentProvider>("barion");
builder.Services.AddKeyedSingleton<INotificationSender, EmailSender>("email");

// Konstruktor injektálás attribútummal
public class CheckoutService([FromKeyedServices("stripe")] IPaymentProvider payment)
{
    public Task PayAsync(Order order, CancellationToken ct) => payment.ChargeAsync(order, ct);
}

// Futásidejű feloldás (amikor a kulcs csak futásidőben derül ki)
public class PaymentRouter(IServiceProvider serviceProvider)
{
    public IPaymentProvider Resolve(string providerKey) =>
        serviceProvider.GetRequiredKeyedService<IPaymentProvider>(providerKey);
}

// Minimal API paraméterben is működik
app.MapPost("/pay/{provider}", async (string provider,
    [FromKeyedServices("stripe")] IPaymentProvider stripe, PaymentRequest req) => { /* ... */ });

```

További, napi szinten hasznos DI technikák:

```

// Több implementáció egyszerre (pipeline, plugin, szabálylanc)
builder.Services.AddScoped<IValidationRule, StockRule>();
builder.Services.AddScoped<IValidationRule, CreditLimitRule>();
public class Validator(IEnumerable<IValidationRule> rules) { /* mindegyiket lefuttatja */ }

// TryAdd: csak akkor regisztrál, ha még nincs (könyvtárak alapértelmezéseihez)
services.TryAddScoped<IClock, SystemClock>();

// Dekorálás Scrutorral (lásd "Decorator Pattern" fejezet)
services.Decorate<IOrderService, CachingOrderServiceDecorator>();

// Factory delegate, ha futásidejű paraméter kell
services.AddScoped<Func<string, IReportGenerator>>(sp => format => format switch
{
    "pdf" => sp.GetRequiredService<PdfReportGenerator>(),
    "xlsx" => sp.GetRequiredService<ExcelReportGenerator>(),
    _ => throw new NotSupportedException(format)
});

```

Élettartam-hibák felismerése. A leggyakoribb hiba a **captive dependency**: egy Singleton szolgáltatás Scoped függőséget kap, ami így örökre "bennragad". Fejlesztéskor kapcsold be az érvényesítést:

```

builder.Host.UseDefaultServiceProvider((context, options) =>
{
    options.ValidateScopes = true; // futásidejű ellenőrzés
    options.ValidateOnBuild = context.HostingEnvironment.IsDevelopment(); // induláskor bukik,
    nem első kérésnél
});

```

Mit kerülni el: ne használd a `IServiceProvider`-t service locatorként az üzleti kódban (a konstruktor injektálás olvashatóbb és tesztelhetőbb); Singletonból Scoped függőséget csak `IServiceScopeFactory`-val oldj fel; ne regisztráld `DbContext`-et Singletonként.

#112 FluentValidation és validáció a pipeline-ban

A validációt érdemes rétegenként elkülöníteni: **input validáció** (formátum, kötelezőség, tartomány) a kérés belépésekor, **üzleti szabály** (invariáns) a domainben.

```
dotnet add package FluentValidation.DependencyInjectionExtensions
```

```
public class CreateOrderValidator : AbstractValidator<CreateOrderCommand>
{
    public CreateOrderValidator(ICustomerRepository customers)
    {
        RuleFor(x => x.CustomerId)
            .NotEmpty()
            .MustAsync(async (id, ct) => await customers.ExistsAsync(id, ct))
            .WithMessage("A megadott ügyfél nem létezik.");

        RuleFor(x => x.Email).NotEmpty().EmailAddress();
        RuleFor(x => x.Lines).NotEmpty();
        RuleForEach(x => x.Lines).SetValidator(new OrderLineValidator());

        // Feltételes szabály
        When(x => x.PaymentMethod == PaymentMethod.CreditCard, () =>
            RuleFor(x => x.CardToken).NotEmpty().WithMessage("Kártyás fizetéshez
token szükséges."));
    }
}
```

MediatR pipeline behavior, hogy ne kelljen minden handlerben kézzel validálni:

```

public class ValidationBehavior<TRequest, TResponse>(IEnumerable<IValidator<TRequest>>
validators)
    : IPipelineBehavior<TRequest, TResponse> where TRequest : notnull
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
        CancellationToken cancellationToken)
    {
        if (!validators.Any()) return await next();

        var context = new ValidationContext<TRequest>(request);
        var results = await Task.WhenAll(validators.Select(v =>
v.ValidateAsync(context, cancellationToken)));
        var failures = results.SelectMany(r => r.Errors).Where(f => f is not null).ToList();

        if (failures.Count != 0) throw new ValidationException(failures);

        return await next();
    }
}

builder.Services.AddValidatorsFromAssemblyContaining<CreateOrderValidator>();
builder.Services.AddMediatR(c => c.RegisterServicesFromAssemblyContaining<Program>()
.AddOpenBehavior(typeof(ValidationBehavior<,>)));

```

A `ValidationException`-t a globális kivételkezelő fordítja `ValidationProblemDetails`-re (lásd "Globális kivételkezelés és ProblemDetails" fejezet), így az API konzisztens, RFC 9457 szerinti hibaválaszt ad:

```

public class ValidationExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext context, Exception exception,
CancellationToken ct)
    {
        if (exception is not ValidationException validationException) return false;

        var errors = validationException.Errors
            .GroupBy(e => e.PropertyName)
            .ToDictionary(g => g.Key, g => g.Select(e => e.ErrorMessage).ToArray());

        await Results.ValidationProblem(errors, title:
"Érvénytelen kérés").ExecuteAsync(context);
        return true;
    }
}

```

Minimal API-nál a `MinimalApis.Extensions` vagy egy egyszerű endpoint filter (lásd "Minimal API haladó szint" fejezet) ugyanezt adja pipeline behavior nélkül.

Legjobb gyakorlatok: egy validátor egy kéréshez; az **aszinkron, adatbázist érintő** szabályokat óvatosan használd (ez plusz kör az adatbázis felé, és race conditionre nyitott – a végső ellenőrzés az adatbázis constraintje); a hibaüzenetek legyenek felhasználónak szólóak és lokalizálhatóak.

Mit kerülj el: ne duplikáld a domain invariánsokat a validátorban (a domain őrizze a saját szabályait, a validátor a bemenet formáját); ne végezz validációt a controllerben kézzel, ha van pipeline; ne szivárogtass ki belső mezőneveket a hibaválaszban.

#113 Objektum-mapping: Mapperly vs. AutoMapper

Verzió: Mapperly (forráskód-generátor): .NET 6+ / C# 9+ projektekbén.

A DTO ↔ entitás leképezés napi feladat. Az **AutoMapper** reflexió-alapú (futásidejű konfiguráció, csendben elrontható), a **Mapperly** ezzel szemben **source generator**: fordításkor generál sima C# hozzárendelő kódot, tehát nulla futásidejű overhead, teljes IntelliSense és **fordítási idejű hibajelzés**, ha egy mező nem képezhető le. Az AutoMapper licenccmodell-változása óta a Mapperly a legnépesebb alternatíva.

```
dotnet add package Riok.Mapperly
```

```
[Mapper]
public partial class OrderMapper
{
    public partial OrderDto ToDto(Order order);

    [MapProperty(nameof(Order.Customer.Name), nameof(OrderDto.CustomerName))]
    [MapperIgnoreSource(nameof(Order.DomainEvents))]
    public partial OrderDetailsDto ToDetailsDto(Order order);

    // Egyedi leképezés: a generátor automatikusan felhasználja a megfelelő típusoknál
    private static string MapMoney(Money money) => $"{{money.Amount:N0}} {{money.Currency}}";
}
```

A generált kód olvasható és debuggolható (`obj/generated` alatt megnézhető), lényegében az, amit kézzel írnál:

```
public partial OrderDto ToDto(Order order)
{
    return new OrderDto
    {
        Id = order.Id,
        Total = order.Total,
        Status = (OrderStatusDto)order.Status
    };
}
```

Mikor ne mappelj egyáltalán. Lekérdezéseknél a leghatékonyabb, ha az EF Core **közvetlenül a DTO-ra projektál** – így csak a szükséges oszlopok jönnek le az adatbázisból, nincs sem entitás-materializáció, sem mapping:

```
var orders = await db.Orders
    .Where(o => o.CustomerId == customerId)
    .Select(o => new OrderListItemDto(o.Id, o.PlacedAt, o.Total, o.Status)) // SQL-be fordul
    .AsNoTracking()
    .ToListAsync(ct);
```

Legjobb gyakorlatok: a **command/write** oldalon inkább explicit, kézzel írt factory metódus vagy konstruktor (`Order.Create(...)`) – a domain objektum létrehozása üzleti művelet, nem mechanikus másolás; a **query/read** oldalon EF projekció; a maradék mechanikus leképezésre Mapperly.

Mit kerülj el: ne mappelj entitást DTO-ra memóriában, ha projekcióval is megoldható (felesleges oszlopok és tracking); ne rejsz üzleti logikát mapping-profilba; ne használj mappert domain modell **létrehozására** külső bemenetből (kikerüli az invariánsokat).

#114 Dapper és hibrid perzisztencia

Az EF Core kiváló a **write** oldalon (change tracking, unit of work, domain modell), de összetett riportlekérdezéseknél gyakran előnyösebb a nyers SQL. A **Dapper** egy vékony micro-ORM: te írod az SQL-t, ő elvégzi a materializációt.

```
public class OrderReportReader(IDbConnectionFactory connectionFactory) : IOrderReportReader
{
    public async Task<IReadOnlyList<MonthlyRevenueDto>> GetMonthlyRevenueAsync(
        int year, CancellationToken cancellationToken)
    {
        const string sql = """
            SELECT    DATEPART(MONTH, o.PlacedAt) AS Month,
                     SUM(o.TotalAmount)         AS Revenue,
                     COUNT(*)                    AS OrderCount
            FROM      Orders o
            WHERE     DATEPART(YEAR, o.PlacedAt) = @Year
                     AND    o.Status = @Status
            GROUP BY  DATEPART(MONTH, o.PlacedAt)
            ORDER BY  Month;
            """;

        using var connection = connectionFactory.Create();
        return (await connection.QueryAsync<MonthlyRevenueDto>(
            new CommandDefinition(sql, new { Year = year, Status = (int)OrderStatus.Completed },
            cancellationToken: cancellationToken))).AsList();
    }
}
```

Több eredményhalmaz egy körben (N+1 helyett):

```
using var multi = await connection.QueryMultipleAsync(
    "SELECT * FROM Orders WHERE Id = @Id; SELECT * FROM OrderItems WHERE OrderId = @Id;", new {
        Id = orderId });

var order = await multi.ReadSingleAsync<OrderDto>();
order.Items = (await multi.ReadAsync<OrderItemDto>()).AsList();
```

Hibrid megközelítés CQRS-sel: a parancs oldal EF Core + domain modell, a lekérdező oldal Dapper + kézi SQL. Ugyanaz az adatbázis, két különböző hozzáférési stílus, mindegyik ott, ahol erős. Fontos, hogy ugyanabban a tranzakcióban a két technológia is együtt tudjon működni – ehhez ossz meg kapcsolatot és tranzakciót:

```
var connection = dbContext.Database.GetDbConnection();
var transaction = dbContext.Database.CurrentTransaction?.GetDbTransaction();
await connection.ExecuteAsync(sql, parameters, transaction);
```

Legjobb gyakorlatok: **mindig paraméterezett** lekérdezés (SQL injection ellen); a hosszú SQL-eket raw string literálban (`"""`) tartsd olvashatóan; a lekérdezéseket a szelet (feature) mappájában tárold, ne egy központi "Queries" osztályban; a Dapper-es olvasókat integrációs teszttel fedd le (Testcontainers, lásd a következő fejezetet), mert a fordító nem ellenőrzi az SQL-t.

Mit kerülj el: ne írd Dapperrel írási műveleteket, ha a domain invariánsokat az EF Core-os aggregátum őrzi (kikerülsz a domain logikát és a change trackinget); ne fűzz össze SQL-t stringkonkatenációval felhasználói bemenetből; ne másold ugyanazt a lekérdezést öt helyre.

#115 Testcontainers – integrációs tesztek valós infrastruktúrával

Verzió: Testcontainers for .NET: .NET Standard 2.0+ / .NET 6+ projektekből.

Az EF Core In-Memory provider (lásd az azonos című fejezetet) nem relációs adatbázis: nem ismer tranzakciót, constraintet, raw SQL-t, és másképp fordítja a lekérdezéseket – ezért a vele írt teszt hamis biztonságérzetet ad. A **Testcontainers** valódi adatbázist, brokert, Rediszt indít Docker konténerben a teszt idejére, majd eldobja.

```
dotnet add package Testcontainers.MsSql
dotnet add package Testcontainers.Kafka
dotnet add package Testcontainers.Redis
```

```

public class IntegrationTestFixture : IAsyncLifetime
{
    private readonly MsSqlContainer _sqlContainer = new MsSqlBuilder()
        .WithImage("mcr.microsoft.com/mssql/server:2022-latest")
        .WithPassword("Str0ng!Passw0rd")
        .Build();

    private readonly RedisContainer _redisContainer = new RedisBuilder().Build();

    private readonly KafkaContainer _kafkaContainer = new KafkaBuilder()
        .WithImage("confluentinc/cfk-kafka:latest")
        .Build();

    public string SqlConnectionString => _sqlContainer.GetConnectionString();
    public string RedisConnectionString => _redisContainer.GetConnectionString();
    public string KafkaBootstrapServers => _kafkaContainer.GetBootstrapAddress();

    public async Task InitializeAsync()
    {
        await Task.WhenAll(_sqlContainer.StartAsync(), _redisContainer.StartAsync(),
            _kafkaContainer.StartAsync());

        // séma felhúzása a valódi migrációkkal - ez maga is teszteli a migrációkat
        var options = new DbContextOptionsBuilder<AppDbContext>
            ().UseSqlServer(SqlConnectionString).Options;
        await using var db = new AppDbContext(options);
        await db.Database.MigrateAsync();
    }

    public async Task DisposeAsync() =>
        await Task.WhenAll(_sqlContainer.DisposeAsync().AsTask(),
            _redisContainer.DisposeAsync().AsTask(),
            _kafkaContainer.DisposeAsync().AsTask());
}

[CollectionDefinition(nameof(IntegrationTestCollection))]
public class IntegrationTestCollection : ICollectionFixture<IntegrationTestFixture>;

```

A `CollectionDefinition` biztosítja, hogy a konténerek **egyszer** induljanak el a teljes tesztosztály-csoportra, ne tesztenként.

Teszt-elszigetelés. Minden teszt tiszta állapotból induljon. Két bevált módszer:

```

// 1. Respawn: a táblák ürítése tesztek között (gyors, megtartja a sémát)
_respawner = await Respawner.CreateAsync(connectionString, new RespawnerOptions
{
    TablesToIgnore = ["__EFMigrationsHistory"]
});
await _respawner.ResetAsync(connectionString);

// 2. Tranzakció, amit sosem commitolunk (gyors, de nem működik, ha a kód maga is
tranzakciót nyit)

```

Legjobb gyakorlatok: a CI pipeline-ban is fusson (a GitHub Actions/Azure DevOps ubuntu runner tud Docker-t); rögzítsd a konténer-image verzióját (ne `latest`), különben a build reprodukálhatatlan; a lassú integrációs teszteket külön kategóriába tedd (`[Trait("Category", "Integration")]`), hogy a gyors unit tesztek külön futhassanak.

Mit kerülj el: ne írd minden logikára integrációs tesztet – a piramis alja unit teszt legyen; ne oszd meg az állapotot tesztek között (sorrendfüggő, "néha piros" tesztek); ne használd az In-Memory providert olyan viselkedés tesztelésére, ami adatbázis-specifikus (tranzakció, konkurencia, constraint, SQL függvények).

#116 WebApplicationFactory – API-szintű integrációs tesztek

Verzió: ASP.NET Core 2.1-től (`Microsoft.AspNetCore.Mvc.Testing`).

A `WebApplicationFactory<TEntryPoint>` a teljes ASP.NET Core alkalmazást elindítja memóriában (hálózati port nélkül), így a middleware pipeline, a routing, a modellkötés, a szűrők, az autentikáció és a DI is a valóságnak megfelelően fut.

```
dotnet add package Microsoft.AspNetCore.Mvc.Testing
```

```

public class OrdersApiFactory(IntegrationTestFixture fixture) : WebApplicationFactory<Program>
{
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");

        builder.ConfigureTestServices(services =>
        {
            // valós adatbázis (Testcontainers) bekötése
            services.RemoveAll<DbContextOptions<AppDbContext>>();
            services.AddDbContext<AppDbContext>(o =>
o.UseSqlServer(fixture.SqlConnectionString));

            // külső, harmadik fél API kicserélése teszt duplára
            services.RemoveAll<IPaymentGateway>();
            services.AddSingleton<IPaymentGateway, FakePaymentGateway>();

            // autentikáció megkerülése teszt sémával
            services.AddAuthentication(defaultScheme: "Test")
                .AddScheme<AuthenticationSchemeOptions, TestAuthHandler>("Test", _ => { });
        });
    }
}

public class TestAuthHandler(IOptionsMonitor<AuthenticationSchemeOptions> options,
ILoggerFactory logger,
    UrlEncoder encoder) : AuthenticationHandler<AuthenticationSchemeOptions>(options,
logger, encoder)
{
    protected override Task<AuthenticateResult> HandleAuthenticateAsync()
    {
        var claims = new[] { new Claim(ClaimTypes.NameIdentifier, "test-user"), new
Claim("scope", "orders:write") };
        var identity = new ClaimsIdentity(claims, "Test");
        var ticket = new AuthenticationTicket(new ClaimsPrincipal(identity), "Test");
        return Task.FromResult(AuthenticateResult.Success(ticket));
    }
}

```

```

[Collection(nameof(IntegrationTestCollection))]
public class PlaceOrderEndpointTests(IntegrationTestFixture fixture) : IAsyncLifetime
{
    private HttpClient _client = null!;
    private OrdersApiFactory _factory = null!;

    public Task InitializeAsync()
    {
        _factory = new OrdersApiFactory(fixture);
        _client = _factory.CreateClient();
        return Task.CompletedTask;
    }

    [Fact]
    public async Task PlaceOrder_WithValidRequest_Returns201AndPersistsOrder()
    {
        var request = new { customerId = Guid.NewGuid(), lines = new[] { new { productId =
KnownIds.Product, quantity = 2 } } };

        var response = await _client.PostAsJsonAsync("/orders", request);

        response.StatusCode.Should().Be(HttpStatusCode.Created);
        var orderId = await response.Content.ReadFromJsonAsync<Guid>();

        using var scope = _factory.Services.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
        (await db.Orders.FindAsync(orderId)).Should().NotBeNull();
    }

    [Fact]
    public async Task PlaceOrder_WithEmptyLines_Returns400WithProblemDetails()
    {
        var response = await _client.PostAsJsonAsync("/orders", new { customerId =
Guid.NewGuid(), lines = Array.Empty<object>() });

        response.StatusCode.Should().Be(HttpStatusCode.BadRequest);
        var problem = await response.Content.ReadFromJsonAsync<ValidationProblemDetails>();
        problem!.Errors.Should().ContainKey("Lines");
    }

    public Task DisposeAsync() { _factory.Dispose(); return Task.CompletedTask; }
}

```

Ahhoz, hogy a `WebApplicationFactory<Program>` működjön top-level statementekkel írt `Program.cs` esetén, a teszt projektnek látnia kell a `Program` osztályt:

```

// a Program.cs végén
public partial class Program;
// vagy a csproj-ban: <InternalsVisibleTo Include="Orders.Api.Tests" />

```

Legjobb gyakorlatok: teszteld azt, ami a végponton keresztül **megfigyelhető** (státusz kód, válasz body, adatbázis állapota, kiküldött üzenet), ne az implementációt; a külső integrációkat (fizetés, email) mindig cseréld le teszt duplára; a kiküldött üzeneteket a MassTransit `ITestHarness` -ével ellenőrizheted (`harness.Published.Any<OrderPlaced>()`).

Mit kerülj el: ne indíts új factoryt minden tesztmetódushoz (lassú); ne teszteld a keretrendszert (hogyan működik-e); ne hagyd benne valós külső hálózati hívást a tesztben.

#117 Architektúra tesztek – a szabályok kikényszerítése

Egy Clean Architecture vagy modular monolith akkor marad tiszta, ha a szabályokat **a build kényszeríti ki**, nem a code review jó szándéka. Az architektúra teszt olyan unit teszt, amely az assembly-k és típusok kapcsolatait ellenőrzi.

```
dotnet add package NetArchTest.Rules
```



```

public class ArchitectureTests
{
    private const string Domain = "MyApp.Domain";
    private const string Application = "MyApp.Application";
    private const string Infrastructure = "MyApp.Infrastructure";
    private const string Api = "MyApp.Api";

    [Fact]
    public void Domain_Should_Not_DependOn_AnyOtherLayer()
    {
        var result = Types.InAssembly(typeof(Order).Assembly)
            .ShouldNot()
            .HaveDependencyOnAny(Application, Infrastructure,
Api, "Microsoft.EntityFrameworkCore")
            .GetResult();

        result.IsSuccessful.Should().BeTrue(
            $"a Domain réteg nem függhet másról. Sértő típusok: {string.Join(", ",
result.FailingTypeNames ?? [])}");
    }

    [Fact]
    public void Application_Should_Not_DependOn_Infrastructure()
        => Types.InAssembly(typeof(PlaceOrderHandler).Assembly)
            .ShouldNot().HaveDependencyOn(Infrastructure)
            .GetResult().IsSuccessful.Should().BeTrue();

    [Fact]
    public void Handlers_Should_Be_Sealed_And_Named_Correctly()
        => Types.InAssembly(typeof(PlaceOrderHandler).Assembly)
            .That().ImplementInterface(typeof(IRequestHandler<,>))
            .Should().BeSealed().And().HaveNameEndingWith("Handler")
            .GetResult().IsSuccessful.Should().BeTrue();

    [Fact]
    public void Modules_Should_Not_Reference_Each_Others_Internals()
        => Types.InAssembly(typeof(OrdersModule).Assembly)
            .ShouldNot().HaveDependencyOn("MyApp.Modules.Billing.Internal")
            .GetResult().IsSuccessful.Should().BeTrue();

    [Fact]
    public void Entities_Should_Have_Private_Parameterless_Constructor_For_EfCore()
        => Types.InAssembly(typeof(Order).Assembly)
            .That().Inherit(typeof(Entity))
            .Should().MeetCustomRule(new HasPrivateParameterlessConstructorRule())
            .GetResult().IsSuccessful.Should().BeTrue();
}

```

Tipikus, érdemben hasznos szabályok: a Domain nem hivatkozhat EF Core-ra vagy ASP.NET Core-ra; a Controller/Endpoint nem hivatkozhat közvetlenül `DbContext` -re; minden `record` üzenet-szerződés a `Contracts` névtérben legyen; nincs `DateTime.Now` a domainben (helyette injektált `TimeProvider`); minden publikus API-metódus fogadjon `CancellationToken` -t.

Legjobb gyakorlatok: az architektúra teszt legyen része a CI-nak, és **bukjon a build**, ha sérül a szabály; az üzenet a hibában mondja meg, *miért* van a szabály, ne csak azt, hogy sérült; kevés, de valóban fontos szabályt tarts fenn.

Mit kerülj el: ne írd tucatnyi triviális elnevezési szabályt, amit senki nem tart be – az elnémított teszt rosszabb, mint a nem létező; ne kényszeríts ki olyan szabályt, amit a csapat nem fogadott el közösen.

#118 Contract testing – szerződéses tesztek szolgáltatások között

Mikroszolgáltatásoknál a legdrágább hiba az, amikor A szolgáltatás megváltoztatja a válasza szerkezetét, és B csak élesben derül ki, hogy elromlott. Az end-to-end teszt drága, lassú és törékeny; a **consumer-driven contract testing** olcsóbb megoldás: a **fogyasztó** írja le, mit vár el, a **szolgáltató** pedig ez ellen ellenőrzi magát – külön-külön, a saját pipeline-jában.

```
dotnet add package PactNet
```

Fogyasztó oldal – elvárás rögzítése:

```
public class InventoryClientPactTests
{
    private readonly IPactBuilderV4 _pact = Pact.V4("orders-api", "inventory-api",
        new PactConfig { PactDir = "../..../pacts" }).WithHttpInteractions();

    [Fact]
    public async Task GetStock_WhenProductExists_ReturnsAvailableQuantity()
    {
        _pact
            .UponReceiving("egy létező termék készletének lekérdezése")
                .Given("a P-123 termék létezik 5 db készlettel")
                .WithRequest(HttpMethod.Get, "/stock/P-123")
                .WithHeader("Accept", "application/json")
                .WillRespond()
                    .WithStatus(HttpStatusCode.OK)
                    .WithJsonBody(new { productId = "P-123", available = 5 });

        await _pact.VerifyAsync(async ctx =>
        {
            var client = new InventoryClient(new HttpClient { BaseAddress = ctx.MockServerUri });
            var stock = await client.GetStockAsync("P-123", CancellationToken.None);
            stock.Available.Should().Be(5);
        });
    }
}
```

Szolgáltató oldal – a rögzített szerződés ellenőrzése:

```
[Fact]
public void EnsureOrdersApiExpectationsAreMet()
{
    using var verifier = new PactVerifier("inventory-api", new PactVerifierConfig());
    verifier
        .WithHttpEndpoint(_serverUri)
        .WithFileSource(new FileInfo("../..../pacts/orders-api-inventory-api.json"))
        .WithProviderStateUrl(new Uri(_serverUri, "/provider-states")) // állapot beállítás a
"Given"-hez
        .Verify();
}
```

Aszinkron üzenetekre ugyanez működik (message pact): a fogyasztó leírja, milyen üzenetstruktúrát tud feldolgozni, a publikáló pedig ellenőrzi, hogy az általa kibocsátott üzenet ennek megfelel – ez a Kafka/RabbitMQ integrációknál különösen értékes (lásd "Üzenet-szerződések verziózása" fejezet).

Legjobb gyakorlatok: a pact fájlokat töltsd fel egy Pact Broker-be, és a **can-i-deploy** ellenőrzést tedd a deploy pipeline kapujává; a szerződés a *fogyasztó tényleges igényét* írja le, ne a teljes API felületet; provider state-eket tarts kevés számban és jól elnevezve.

Mit kerülj el: ne helyettesítsd vele a funkcionális tesztelést (a contract teszt a *formát* ellenőrzi, nem az üzleti helyességet); ne írd olyan szerződést, ami minden mezőre pontos értéket vár (használd típus-matchereket), különben minden apró változás elbuktatja.

#119 System.Text.Json haladó szint

Verzió: System.Text.Json : .NET Core 3.0; forráskód-generátor: .NET 6; polimorfikus szerializáció: .NET 7.

A `System.Text.Json` (STJ) a .NET alapértelmezett szerializálója – gyors, allokációtakarékos, de a Newtonsoft.Json-nál szigorúbb. A napi munkában a következők a leggyakoribb kérdések.

Globális beállítások ASP.NET Core-ban:

```
builder.Services.ConfigureHttpJsonOptions(options => // Minimal API
{
    options.SerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.CamelCase;
    options.SerializerOptions.DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull;
    options.SerializerOptions.Converters.Add(new JsonStringEnumConverter());
    options.SerializerOptions.NumberHandling = JsonNumberHandling.AllowReadingFromString;
});

builder.Services.AddControllers().AddJsonOptions(o => { /* ugyanez MVC-hez */ });
```

Source generator (JsonSerializerContext) – reflexió helyett fordításkor generált szerializáló: gyorsabb hidegindítás, kisebb allokáció, és **Native AOT / trimming kompatibilis**:

```
[JsonSourceGenerationOptions(PropertyNamingPolicy = JsonKnownNamingPolicy.CamelCase,
WriteIndented = false)]
[JsonSerializable(typeof(OrderDto))]
[JsonSerializable(typeof(IReadOnlyList<OrderDto>))]
[JsonSerializable(typeof(ProblemDetails))]
public partial class AppJsonContext : JsonSerializerContext;

// használat
var json = JsonSerializer.Serialize(order, AppJsonContext.Default.OrderDto);
builder.Services.ConfigureHttpJsonOptions(o =>
o.SerializerOptions.TypeInfoResolverChain.Insert(0, AppJsonContext.Default));
```

Polimorfikus szerializáció (.NET 7+) – öröklődő típusok diszkriminátorral, ami üzenet-szerződéseknel és event sourcingnál elengedhetetlen:

```
[JsonPolymorphic(TypeDiscriminatorPropertyName = "$type")]
[JsonDerivedType(typeof(CardPayment), "card")]
[JsonDerivedType(typeof(TransferPayment), "transfer")]
public abstract record Payment(decimal Amount);

public record CardPayment(decimal Amount, string Last4) : Payment(Amount);
public record TransferPayment(decimal Amount, string Iban) : Payment(Amount);
```

Egyedi converter (pl. saját value object):

```
public class MoneyJsonConverter : JsonConverter<Money>
{
    public override Money Read(ref Utf8JsonReader reader, Type type,
JsonSerializerOptions options)
        => Money.Parse(reader.GetString()!);

    public override void Write(Utf8JsonWriter writer, Money value, JsonSerializerOptions options)
        => writer.WriteStringValue($"{value.Amount}:{value.Currency}");
}
```

Teljesítmény. Nagy payloadnál ne stringen keresztül dolgozz: `JsonSerializer.DeserializeAsync<T>(stream)` és `SerializeAsync(stream, value)` közvetlenül a HTTP streamre; a `JsonSerializerOptions` példányt **egyszer hozd létre és cache-eld** (a beállítások első használatkor "befagynak", és minden új példány újra felépíti a metaadat-gyorsítótárat – ez az egyik leggyakoribb rejtett teljesítményhiba).

Mit kerülj el: ne szerializálj domain entitást közvetlenül (körkörös referenciák, felesleges mezők, szivárgó belső szerkezet); ne kapcsolj be a `ReferenceHandler.Preserve`-t publikus API-n (`$id` / `$ref` mezők jelennek meg a válaszban, amit a kliensek nem várnak); ne feltételezd, hogy a `System.Text.Json` és a `Newtonsoft.Json` viselkedése azonos (nagybetű-érzékenység, konstruktor-kötés, `TimeSpan` kezelés eltér).

#120 Minimal API haladó szint – TypedResults, endpoint filter, route group

Verzió: TypedResults , endpoint filter, route group (MapGroup), [AsParameters] : .NET 7-től; beépített OpenAPI (AddOpenApi) : .NET 9-től.

A "Minimal API" fejezet az alapokat mutatja; éles projektben a következő eszközök teszik karbantarthatóvá.

TypedResults a **Results** helyett – erősen típusos válasz, ami egyszerre dokumentálja is az OpenAPI sémát, és tesztelhető visszatérési értéket ad:

```
static async Task<Results<Ok<OrderDto>, NotFound, ProblemHttpResult>> GetOrder(
    Guid id, IOrderReader reader, CancellationToken ct)
{
    var order = await reader.GetAsync(id, ct);
    return order is null
        ? TypedResults.NotFound()
        : TypedResults.Ok(order);
}
```

Route group – közös prefix, autentikáció, filter, OpenAPI metaadat egy helyen:

```
var orders = app.MapGroup("/api/v1/orders")
    .WithTags("Orders")
    .RequireAuthorization()
    .AddEndpointFilter<ValidationFilter>()
    .WithOpenApi();

orders.MapGet("/{id:guid}", GetOrder).AllowAnonymous();
orders.MapPost("/", PlaceOrder).RequireAuthorization("orders:write");
orders.MapDelete("/{id:guid}", CancelOrder).RequireRateLimiting("expensive");
```

Endpoint filter – a Minimal API "action filter" megfelelője, láncolható:

```
public class ValidationFilter<T>(IValidator<T> validator) : IEndpointFilter where T : class
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var argument = context.Arguments.OfType<T>().FirstOrDefault();
        if (argument is null) return await next(context);

        var result = await validator.ValidateAsync(argument, context.HttpContext.RequestAborted);
        if (!result.IsValid)
            return TypedResults.ValidationProblem(result.ToDictionary());

        return await next(context);
    }
}

orders.MapPost("/", PlaceOrder).AddEndpointFilter<ValidationFilter<PlaceOrderRequest>>();
```

Paraméterkötés – összetett esetekre [AsParameters] és IParsable :

```
public record OrderQuery([FromQuery] int Page = 1, [FromQuery] int PageSize = 20,
                        [FromQuery] OrderStatus? Status = null, [FromHeader(Name = "X-Tenant")]
                        string? Tenant = null);

orders.MapGet("/", async ([AsParameters] OrderQuery query, IOrderReader reader,
CancellationToken ct)
    => TypedResults.Ok(await reader.SearchAsync(query, ct)));
```

A .NET 9+ beépített OpenAPI támogatása kiváltja a Swashbuckle-t a séma generálásra
(`builder.Services.AddOpenApi()` + `app.MapOpenApi()`); a UI-hoz Scalar vagy Swagger UI köthető be.

Legjobb gyakorlatok: a végpont-delegate legyen **vékony** – egy statikus metódus, ami a handler/sender felé továbbít (így tesztelhető és olvasható marad); minden végpont kapjon `CancellationToken`-t és `WithName / WithSummary` metaadatot; a végpontokat szeletenként regisztráld (lásd "Vertical Slice Architecture" fejezet), ne egyetlen ezersoros `Program.cs`-ben.

Mit kerülj el: ne írd lambda-testbe üzleti logikát; ne használj `Results.Ok(object)`-et típusos válasz helyett, ha az OpenAPI dokumentáció fontos; ne kapcsold ki a nullable ellenőrzéseket, mert a paraméterkötés nullázhatósága fontos jelzés a klienseknek.

#121 Authorization – policy, requirement, claim, multi-tenant hozzáférés

Verzió: Policy-alapú engedélyezés: ASP.NET Core 1.0-tól; `FallbackPolicy` : ASP.NET Core 3.0-tól.

A "JWT, OAuth, OpenId Connect" fejezet a **hitelesítést** (ki vagy?) írja le; itt az **engedélyezés** (mit tehetsz?) következik. A szerep-alapú (`[Authorize(Roles = "Admin")]`) megközelítés gyorsan skálázhatatlanná válik – helyette **policy-alapú** engedélyezés a javasolt.

```
builder.Services.AddAuthorization(options =>
{
    // egyszerű claim-alapú policy
    options.AddPolicy("orders:write", p => p.RequireClaim("scope", "orders:write"));

    // összetett feltétel
    options.AddPolicy("SeniorApprover", p => p
        .RequireAuthenticatedUser()
        .RequireRole("Approver")
        .RequireAssertion(ctx => ctx.User.HasClaim(c =>
            c.Type == "approval_limit" && decimal.Parse(c.Value) >= 1_000_000M)));

    // saját requirement
    options.AddPolicy("SameTenant", p => p.AddRequirements(new SameTenantRequirement()));

    options.FallbackPolicy = new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build();
});
```

Requirement + handler – ott, ahol a döntéshez adatbázis vagy kontextus kell:

```

public class SameTenantRequirement : IAuthorizationRequirement;

public class SameTenantHandler(ITenantContext tenantContext)
    : AuthorizationHandler<SameTenantRequirement, IHasTenant>
{
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
        SameTenantRequirement requirement, IHasTenant resource)
    {
        var userTenant = context.User.FindFirst("tenant_id")?.Value;
        if (userTenant is not null && userTenant == resource.TenantId)
            context.Succeed(requirement);

        return Task.CompletedTask;
    }
}

```

Erőforrás-alapú engedélyezés – amikor a döntés a konkrét entitástól függ ("csak a saját rendelését módosíthatja"):

```

app.MapPut("/orders/{id:guid}", async (Guid id, UpdateOrder cmd, IAuthorizationService auth,
    ClaimsPrincipal user, IOrderRepository repo, CancellationToken ct) =>
{
    var order = await repo.GetAsync(id, ct);
    if (order is null) return Results.NotFound();

    var result = await auth.AuthorizeAsync(user, order, "SameTenant");
    if (!result.Succeeded) return Results.Forbid();

    // ...
});

```

Rétegzett védelem. Az engedélyezés nem csak a végponton él: az adatréteg szintjén az EF Core **global query filter** (lásd az azonos című fejezetet) automatikusan kiszűri a más tenantokhoz tartozó sorokat, tehát még egy elfelejtett `WHERE` sem okoz adatszivárgást.

Legjobb gyakorlatok: a policy nevei legyenek konstansok (elgépelés elleni védelem – az elgépelt policy név futásidejű hibát ad); a jogosultságokat finom szemcsés **permission** claim-ekkel modellezd, a szerepeket csak permission-halmazok nevéként használd; teszteld az engedélyezést integrációs teszttel (403 vs. 200), ne csak manuálisan.

Mit kerülj el: ne bízd az engedélyezést a frontendre; ne tegyél nagy jogosultsági listát a JWT tokenbe (mérethatár, és a visszavonás lehetetlen lesz) – helyette permission lekérdezés + cache; ne felejtse el, hogy az `[Authorize]` hiánya alapból **nyitva hagyja** a végpontot – ezért érdemes `FallbackPolicy`-t beállítani.

#122 Konfiguráció, titokkezelés és Options-validáció

Verzió: User Secrets: .NET Core 2.0; ValidateOnStart : .NET 6; [OptionsValidator] : .NET 8.

Az "Options Pattern" fejezet az `IOptions` családot mutatja; itt a konfiguráció **forrásai**, a titkok kezelése és a hibás konfiguráció **korai** felismerése következik.

Forrás-hierarchia (később regisztrált felülírja a korábbi): `appsettings.json` → `appsettings.{Environment}.json` → User Secrets (csak Development) → környezeti változók → parancssori argumentumok.

```
# lokális fejlesztés - a titok NEM kerül a repóba
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:Default" "Server=...;Password=..."
```

Éles környezetben Azure Key Vault / AWS Secrets Manager / HashiCorp Vault:

```
if (!builder.Environment.IsDevelopment())
{
    builder.Configuration.AddAzureKeyVault(
        new Uri($"https://{builder.Configuration["KeyVault:Name"]}.vault.azure.net/"),
        new DefaultAzureCredential()); // managed identity - nincs jelszó a konfigurációban
}
```

Options-validáció induláskor – a hibás konfiguráció ne az első kérésnél, hanem indításkor bukjon:

```
public class KafkaOptions
{
    public const string SectionName = "Kafka";

    [Required, Url] public string BootstrapServers { get; init; } = null!;
    [Required] public string ConsumerGroup { get; init; } = null!;
    [Range(1, 100)] public int MaxConcurrency { get; init; } = 4;
}

builder.Services.AddOptions<KafkaOptions>()
    .Bind(builder.Configuration.GetSection(KafkaOptions.SectionName))
    .ValidateDataAnnotations()
    .Validate(o => o.MaxConcurrency <= Environment.ProcessorCount * 2,
        "A MaxConcurrency nem lehet nagyobb a processzorszám kétszeresénél.")
    .ValidateOnStart(); // fail-fast: az alkalmazás el sem indul rossz konfigurációval
```

Forráskód-generált, AOT-barát validáció:

```
[OptionsValidator]
public partial class KafkaOptionsValidator : IValidateOptions<KafkaOptions>;

builder.Services.AddSingleton<IValidateOptions<KafkaOptions>, KafkaOptionsValidator>();
```

Legjobb gyakorlatok: **soha ne** kerüljön titok a forráskódba vagy az `appsettings.json`-ba (a `.gitignore` nem védelem – a repo történetében ott marad); a konfigurációt tipizált Options osztályba kösd, ne `IConfiguration["kulcs"]` szórt hivatkozásokkal; a titkok rotálhatók legyenek (`IOptionsMonitor` + Key Vault refresh); a konténerben környezeti változó a szokásos átadási mód (`Kafka__BootstrapServers` – a dupla aláhúzás a szintlépés).

Mit kerülj el: ne logold ki a teljes konfigurációt indításkor (titkok szivárognak a logba); ne használj `IOptionsSnapshot` -ot Singleton szolgáltatásban (captive dependency); ne tarts éles connection stringet fejlesztői gépen.

#123 Feature flags – funkciókapcsolók

A feature flag elválasztja a **deployt** a **release**-től: a kód kikerül élesbe, de a funkció csak akkor él, amikor bekapcsolod. Ez teszi lehetővé a trunk-based fejlesztést, a fokozatos kivezetést (canary), az A/B tesztet és a gyors visszaállítást deploy nélkül.

```
dotnet add package Microsoft.FeatureManagement.AspNetCore
```

```
{
  "FeatureManagement": {
    "NewCheckoutFlow": false,
    "BetaReporting": {
      "EnabledFor": [
        { "Name": "Percentage", "Parameters": { "Value": 10 } }
      ]
    },
    "MaintenanceMode": {
      "EnabledFor": [
        { "Name": "TimeWindow", "Parameters": { "Start": "2026-08-10T22:00:00Z", "End": "2026-08-11T02:00:00Z" } }
      ]
    }
  }
}
```

```
builder.Services.AddFeatureManagement().AddFeatureFilter<PercentageFilter>()
.AddFeatureFilter<TimeWindowFilter>();

// kódban
public class CheckoutService(IFeatureManager features)
{
    public async Task<CheckoutResult> CheckoutAsync(Cart cart, CancellationToken ct) =>
        await features.IsEnabledAsync("NewCheckoutFlow")
            ? await _newFlow.RunAsync(cart, ct)
            : await _legacyFlow.RunAsync(cart, ct);
}

// végponton
app.MapGet("/reports/beta", GetBetaReport).WithMetadata(new
FeatureGateAttribute("BetaReporting"));
```

Központi kezeléshez (több szolgáltatás, futásidejű kapcsolás újradeploy nélkül) Azure App Configuration, LaunchDarkly, Unleash vagy saját adatbázis-alapú provider köthető be – az `IFeatureManager` absztrakció változatlan marad.

Legjobb gyakorlatok: minden flaghez tartozzon **gazda és lejárat dátum** – a flag ideiglenes; a kapcsoló legyen bináris és jól nevezett (`NewCheckoutFlow` , nem `Flag17`); a flag állapotát logold és tedd rá span attribútumként a trace-re, hogy egy incidensnél látszódjon, melyik ágon futott a kérés; teszteld **mindkét** ágat.

Mit kerülj el: ne halmozz fel több tucat régi flaget (a kombinatorikus robbanás tesztelhetetlen kódot ad – a kivezetett flaget és a hozzá tartozó holt ágat töröld); ne használj feature flaget hosszú távú konfigurációként (arra Options való); ne ágazz el flagre a domain modell mélyén, csak jól látható, magas szintű döntési pontokon.

#124 Multi-tenancy .NET-ben

Ha egy alkalmazás több ügyfelet (tenantot) szolgál ki, három fő izolációs modell közül lehet választani:

Modell	Izoláció	Költség	Mikor
Közös adatbázis, <code>TenantId</code> oszlop	logikai	legolcsóbb	sok, kis tenant, SaaS
Közös szerver, tenantonként séma/adatbázis	erősebb	közepes	közepes számú tenant, megfeleléségi igény
Tenantonként külön infrastruktúra	teljes	legdrágább	kevés, nagy, szabályozott ügyfél

Tenant felismerése (subdomain, fejléc, JWT claim) middleware-ben:

```
public class TenantResolutionMiddleware(RequestDelegate next)
{
    public async Task InvokeAsync(HttpContext context, ITenantContext tenantContext,
ITenantStore store)
    {
        var tenantId = context.User.FindFirst("tenant_id")?.Value
            ?? context.Request.Headers["X-Tenant-Id"].FirstOrDefault()
            ?? context.Request.Host.Host.Split('.')[0];

        var tenant = await store.FindAsync(tenantId, context.RequestAborted);
        if (tenant is null)
        {
            context.Response.StatusCode = StatusCodes.Status400BadRequest;
            return;
        }

        tenantContext.SetTenant(tenant);
        using (Serilog.Context.LogContext.PushProperty("TenantId", tenant.Id)) // minden
log tartalmazza
            await next(context);
    }
}
```

Automatikus szűrés az adatrétegben – a legfontosabb védelem, mert nem lehet elfelejteni:

```

public class AppDbContext(DbContextOptions<AppDbContext> options, ITenantContext tenant)
: DbContext(options)
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Order>().HasQueryFilter(o => o.TenantId == tenant.CurrentTenantId);
        modelBuilder.Entity<Product>().HasQueryFilter(p => p.TenantId == tenant.CurrentTenantId);
    }

    public override Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
    {
        foreach (var entry in ChangeTracker.Entries<ITenantOwned>().Where(e => e.State
= EntityState.Added))
            entry.Entity.TenantId = tenant.CurrentTenantId;    // íráskor automatikus kitöltés

        return base.SaveChangesAsync(cancellationToken);
    }
}

```

Tenantonkénti adatbázis esetén a connection stringet a tenant kontextusból kell feloldani:

```

builder.Services.AddDbContext<AppDbContext>((sp, options) =>
    options.UseSqlServer(sp.GetRequiredService<ITenantContext>().ConnectionString));

```

Figyelendő pontok: a **cache kulcsba** mindig menjen bele a tenant azonosító; a háttérfolyamatok (`BackgroundService` , üzenet-fogyasztó) nem rendelkeznek HTTP kontextussal, ezért a tenantot az üzenet fejlécéből/tartalmából kell visszaállítani, mielőtt bármit tennének; a migrációkat tenantonként (adatbázisonként) kell futtatni, ütemezetten és követhetően; a "zajos szomszéd" ellen tenantonkénti rate limit indokolt.

Mit kerülj el: ne hagyatkozz kizárólag a kódban kézzel írt `WHERE TenantId = ...` feltételekre (egyetlen felejtés adatszivárgás); ne engeddd, hogy adminisztrátori funkció (cross-tenant riport) ugyanazon a `DbContext` -en fusson szűrő nélkül, kifejezett, auditált útvonal nélkül (`IgnoreQueryFilters()` csak szigorúan ellenőrzött helyen).

#125 Elosztott lock és háttérfolyamatok több példányban

Ha egy alkalmazás több példányban fut (Kubernetes replicák), minden példányban elindul az összes `BackgroundService` – így egy napi feladat háromszor fut le. A memóriabeli `lock` (lásd "Alkalmazásszintű Concurrency Handling" fejezet) itt semmit nem ér, mert csak a saját processzen belül érvényes.

Megoldások:

1. Adatbázis-alapú lock (nincs új infrastruktúra):

```
// SQL Server alkalmazás-lock: a tranzakció végéig tart, automatikusan
felszabadul kapcsolatvesztésnél
public async Task<bool> TryAcquireAsync(string resource, TimeSpan timeout, CancellationToken ct)
{
    var result = await connection.ExecuteScalarAsync<int>(
        "EXEC @Result = sp_getapplock @Resource, 'Exclusive', 'Transaction', @Timeout",
        new { Resource = resource, Timeout = (int)timeout.TotalMilliseconds }, transaction);
    return result ≥ 0;
}
```

PostgreSQL esetén ennek megfelelője a `pg_try_advisory_lock`.

2. Redis alapú lock (`RedLock.net` vagy `SETNX` + TTL):

```
await using var redLock = await _redLockFactory.CreateLockAsync(
    resource: "daily-report", expiryTime: TimeSpan.FromMinutes(5),
    waitTime: TimeSpan.FromSeconds(3), retryTime: TimeSpan.FromMilliseconds(200));

if (!redLock.IsAcquired) return; // másik példány már dolgozik rajta
await GenerateReportAsync(ct);
```

3. Beépített megoldás használata – gyakran ez a legjobb: a Quartz.NET klaszter-módja (lásd "Quartz.NET" fejezet), a Hangfire, vagy Kubernetesben egy külön `CronJob`, illetve egyetlen replikára állított worker deployment.

Fontos csapdák:

- A lock TTL-je **rövidebb** lehet, mint a munka futásideje – ilyenkor két példány dolgozhat egyszerre. Vagy hosszabbítsd meg a lockot periodikusan (lease renewal), vagy tedd a műveletet idempotenssé.
- A lock **nem** helyettesíti az idempotenciát: hálózati particionálás esetén nincs tökéletes elosztott kölcsönös kizárás.
- Mindig legyen lejárati idő: lejárat nélküli lock egy összeomló példánynál örökre blokkolja a rendszert.

Mit kerülsz el: ne építs saját elosztott lock implementációt, ha van bevált könyvtár vagy platformszolgáltatás; ne használj elosztott lockot nagy gyakoriságú (másodpercenkénti) műveletre – az szűk keresztmetszet lesz; ne tegyél lockot HTTP kérés kritikus útvonalába, ha az késleltetést okoz.

#126 Channels és IAsyncEnumerable – streaming és belső producer-consumer

Verzió: `System.Threading.Channels` : .NET Core 3.0; `IAsyncEnumerable<T>` és `await foreach` : C# 8.0 / .NET Core 3.0.

`System.Threading.Channels` egy magas teljesítményű, allokációtakarékos, aszinkron producer-consumer sor **processzen belül** – ott hasznos, ahol nem kell broker (pl. bejövő kérések háttérfeldolgozásba tolása, batch-elés, terhelés-kegyenlítés).

```

public class BackgroundTaskQueue
{
    private readonly Channel<Func<CancellationToken, ValueTask>> _channel =
        Channel.CreateBounded<Func<CancellationToken, ValueTask>>(new BoundedChannelOptions(1000)
        {
            FullMode = BoundedChannelFullMode.Wait,    // backpressure: a termelő vár, ha tele van
            SingleReader = false,
            SingleWriter = false
        });

    public ValueTask EnqueueAsync(Func<CancellationToken, ValueTask> workItem, CancellationToken
ct) =>
        _channel.Writer.WriteAsync(workItem, ct);

    public IEnumerable<Func<CancellationToken, ValueTask>> ReadAllAsync(CancellationToken
ct) =>
        _channel.Reader.ReadAllAsync(ct);
}

public class QueuedHostedService(BackgroundTaskQueue queue, ILogger<QueuedHostedService> logger)
: BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        await Parallel.ForEachAsync(queue.ReadAllAsync(stoppingToken),
            new ParallelOptions { MaxDegreeOfParallelism = 4, CancellationToken =
stoppingToken },
            async (workItem, ct) =>
            {
                try { await workItem(ct); }
                catch (Exception ex) { logger.LogError(ex, "Háttér munka hibára futott"); }
            });
    }
}

```

A **bounded** (korlátos) channel a lényeg: korlát nélküli sor memóriaszivárgáshoz vezet terhelés alatt. A `FullMode` dönti el, mi történjen telítettségnél: várakozás (backpressure), a legrégebbi eldobása, vagy az új eldobása.

`IAsyncEnumerable<T>` és **streaming API** – nagy eredményhalmaz átadása anélkül, hogy a teljes lista memóriába kerülne:

```

public async IEnumerable<OrderDto> StreamOrdersAsync(
    [EnumeratorCancellation] CancellationToken ct = default)
{
    await foreach (var order in
        _db.Orders.AsNoTracking().AsAsyncEnumerable().WithCancellation(ct))
        yield return new OrderDto(order.Id, order.Total);
}

// Minimal API: a válasz JSON tömbként, folyamatosan íródik ki
app.MapGet("/orders/stream", (IOrderReader reader, CancellationToken ct)
=> reader.StreamOrdersAsync(ct));

```

Az `[EnumeratorCancellation]` attribútum elhagyása gyakori hiba: nélküle a `WithCancellation()` -nel átadott token nem jut el a metódus törzsébe.

Mikor melyiket: **Channel** – processzen belüli munkaátadás, batchelés, terheléscsillapítás. **IAsyncEnumerable** – lusta, aszinkron adatfolyam olvasása (adatbázis kurzor, fájl, hálózati stream). **Üzenetsor (Kafka/RabbitMQ)** – ha a munkának túl kell élnie a processzt, vagy más szolgáltatásnak szól.

Mit kerülj el: ne használj Channelt tartós munkasorként – **az alkalmazás leállásakor a tartalma elveszik** (erre való az Outbox + broker); ne felejtse el a `Writer.Complete()` -et, ha a fogyasztó a channel lezárására vár; ne streamelj `IAsyncEnumerable` -t olyan tranzakció fölött, amit a HTTP válasz írása közben nyitva kell tartani.

#127 Teljesítmény: Native AOT, trimming, benchmarkolás

Verzió: Native AOT: .NET 7 (konzol), ASP.NET Core Minimal API AOT támogatás: .NET 8-tól.

Native AOT – az alkalmazás előre, gépi kódra fordul: néhány milliszekundumos indulási idő, kisebb memória-lábnyom, önálló natív bináris (nincs JIT, nincs runtime telepítés). Kubernetes/serverless környezetben ahol a skálázás gyakori és a hidegindítás számít, jelentős előny.

```
<PropertyGroup>
  <PublishAot>true</PublishAot>
  <InvariantGlobalization>true</InvariantGlobalization>
  <StripSymbols>true</StripSymbols>
  <TrimMode>full</TrimMode>
</PropertyGroup>
```

```
dotnet publish -c Release -r linux-x64
```

Korlátok, amikkel számolni kell: nincs futásidejű kódgenerálás és korlátozott a reflexió, ezért az EF Core (jelenleg korlátozottan), az AutoMapper, a MediatR reflexiók felderítése, a `System.Text.Json` reflexiók módja problémás – ezek helyett source generator alapú megoldásokra van szükség (`JsonSerializerContext`, `Mapperly`, EF Core compiled models, `Dapper AOT`). A Minimal API támogatott, az MVC controller alapú stack nem teljesen.

```
// AOT-barát Minimal API belépési pont
var builder = WebApplication.CreateSlimBuilder(args);
builder.Services.ConfigureHttpJsonOptions(o =>
    o.SerializerOptions.TypeInfoResolverChain.Insert(0, AppJsonContext.Default));
```

Mérj, ne tippelj – BenchmarkDotNet:

```
dotnet add package BenchmarkDotNet
```

```

[MemoryDiagnoser] // allokáció is látszik, nem csak idő
[SimpleJob(RuntimeMoniker.Net10_0)]
public class SerializationBenchmarks
{
    private readonly OrderDto _order = TestData.CreateOrder();

    [Benchmark(Baseline = true)]
    public string Reflection() => JsonSerializer.Serialize(_order);

    [Benchmark]
    public string SourceGenerated() => JsonSerializer.Serialize(_order,
AppJsonContext.Default.OrderDto);
}

// Program.cs: BenchmarkRunner.Run<SerializationBenchmarks>(); (Release
konfigurációban futtatva!)

```

A gyakorlatban legtöbbet hozó optimalizációk – sorrendben, mielőtt bárki a mikro-optimalizáláshoz nyúlna:

1. **Adatbázis:** hiányzó index, N+1 lekérdezés, felesleges tracking, `SELECT *` projekció helyett (lásd "EF Core performance optimization strategies" és "SQL lassú query" fejezetek).
2. **Hálózat:** felesleges szinkron szolgáltatásláncok, tömörítés hiánya, túl nagy payload.
3. **Cache:** hiányzó gyorsítótárazás gyakran olvasott adatra.
4. **Aszinkronitás:** blokkoló hívás (`.Result` , `.Wait()`) thread pool éheztetéssel.
5. Csak ezután: allokációcsökkentés (`Span<T>` , `ArrayPool` , `struct`), source generatorok.

Mit kerülj el: ne optimalizálj mérés nélkül; ne futtass benchmarkot Debug buildben vagy a fejlesztői gépen futó egyéb terhelés mellett; ne válts Native AOT-ra csak mert új – mérd meg, hogy a te terhelési profilod (hosszan futó szolgáltatás vs. gyakran skálázódó) egyáltalán profitál-e belőle.

#128 Docker multi-stage build és Kubernetes-alapok

Verzió: Chiseled konténer képek: .NET 8-tól; `$APP_UID` nem-root felhasználó: .NET 8-tól.

Multi-stage Dockerfile – a build eszközök nem kerülnek bele a futtatókörnyezetbe, így a kép kicsi és biztonságosabb:

```

FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

# külön réteg a csproj-oknak: a restore csak akkor fut újra, ha a függőségek változtak
COPY ["src/Orders.Api/Orders.Api.csproj", "src/Orders.Api/"]
COPY ["src/Orders.Domain/Orders.Domain.csproj", "src/Orders.Domain/"]
RUN dotnet restore "src/Orders.Api/Orders.Api.csproj"

COPY . .
RUN dotnet publish "src/Orders.Api/Orders.Api.csproj" -c Release -o
/app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:10.0-noble-chiseled AS final
WORKDIR /app
COPY --from=build /app/publish .
USER $APP_UID # nem root felhasználó
ENV ASPNETCORE_HTTP_PORTS=8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "Orders.Api.dll"]

```

A `chiseled` képek shell és csomagkezelő nélküli, minimális Ubuntu alapok – lényegesen kisebb támadási felület és képméret.

Kubernetes deployment – a .NET szempontjából fontos részek:

```

spec:
  containers:
    - name: orders-api
      image: registry.example.com/orders-api:1.4.2 # soha ne :latest
      ports: [{ containerPort: 8080 }]
      env:
        - name: ConnectionStrings__Default
          valueFrom: { secretKeyRef: { name: orders-secrets, key: connection-string } }
      resources:
        requests: { cpu: "100m", memory: "256Mi" }
        limits: { memory: "512Mi" } # CPU limitet gyakran nem érdemes
megadni (throttling)
      startupProbe: # lassú induláshoz (migráció, cache melegítés)
        httpGet: { path: /health/startup, port: 8080 }
        failureThreshold: 30
        periodSeconds: 5
      livenessProbe: # "él-e a folyamat?" - újraindítást vált ki
        httpGet: { path: /health/live, port: 8080 }
      readinessProbe: # "fogadhat-e forgalmat?" - kiveszi a
load balancerből
        httpGet: { path: /health/ready, port: 8080 }
      terminationGracePeriodSeconds: 60

```

A három probe elkülönítése a health checkek (lásd "Health Checks" fejezet) címkézésével:

```

app.MapHealthChecks("/health/live", new HealthCheckOptions { Predicate = _ => false }); // csak
a folyamat
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = c =>
c.Tags.Contains("ready") });

```


Graceful shutdown. Kubernetes `SIGTERM`-et küld, majd a grace period után `SIGKILL`-t. A .NET host ezt `IHostApplicationLifetime.ApplicationStopping`-ként jelzi; állítsd be az elegendő időt a folyamatban lévő kérések és üzenetfeldolgozások befejezéséhez:

```
builder.Services.Configure<HostOptions>(o => o.ShutdownTimeout = TimeSpan.FromSeconds(45));
```

A .NET a konténer memórialimitjét figyelembe veszi a GC beállításánál; nagy memóriaigényű szolgáltatásoknál érdemes a Server GC-t explicit konfigurálni (`<ServerGarbageCollection>`), kis konténereknél viszont a Workstation GC takarékosabb.

Mit kerülj el: ne futtass rootként; ne tedd a titkokat a képbe vagy környezeti változóként a manifestbe (Secret vagy külső titokkezelő); ne indíts adatbázis-migrációt az alkalmazás indulásakor több replikánál (versenyhelyzet) – erre külön init container vagy Job való.

#129 CI/CD és adatbázis-migráció éles környezetben

Egy tipikus .NET pipeline lépései:

```
# GitHub Actions vázlat
- run: dotnet restore
- run: dotnet build -c Release --no-restore
- run: dotnet format --verify-no-changes           # stílus ellenőrzés
- run: dotnet test -c Release --no-build --collect:"XPlat Code Coverage"
- run: dotnet list package --vulnerable --include-transitive # ismert sérülékenységek
- run: dotnet publish -c Release -o out
- run: docker build -t $IMAGE:$GITHUB_SHA .
- run: docker push $IMAGE:$GITHUB_SHA
```

Fontos, hogy a build **egyszer** történjen, és ugyanaz az artefaktum (image) menjen végig minden környezeten – a környezeti különbség csak konfiguráció legyen.

Adatbázis-migráció. Az `app.Database.Migrate()` indításkori hívása fejlesztésre jó, éles környezetben veszélyes (több replika, visszaállíthatatlan lépés, hosszú lock). Éles gyakorlat:

```
# 1. SQL szkript generálása kódfelülvizsgálatra és DBA jóváhagyásra
dotnet ef migrations script --idempotent --output migrations.sql

# 2. Alkalmazás a deploy előtti, külön lépésben (Kubernetes Job / pipeline stage)
```

Expand–contract migráció – nulla leállású séma-változtatás. A séma és a kód nem változhat egyszerre, mert a deploy alatt a régi és az új verzió **egyszerre** fut:

1. **Expand:** új oszlop hozzáadása nullable-ként (a régi kód ezt figyelmen kívül hagyja).
2. **Migrate:** az új kód mindkét oszlopba ír, olvasáskor az újat preferálja; háttérben adatfeltöltés.
3. **Contract:** a régi oszlop törlése, miután minden példány az új verziót futtatja.

Ugyanez oszlop átnevezésénél és típusváltásnál is – a "gyors átnevezés" migráció a legbiztosabb módja egy éles leállítás előidézésének.

Deployment stratégiák: rolling update (alapértelmezés Kubernetesben), blue-green (két teljes környezet, azonnali váltás és visszaállítás), canary (a forgalom kis százaléka az új verzióra, metrikák figyelése mellett). Mindhárom feltételezi, hogy az API és az adatbázis **visszafelé kompatibilis** – ezért elválaszthatatlan az expand–contract gyakorlattól és a feature flagektől.

Mit kerülj el: ne futtass migrációt automatikusan több replikából; ne engedj adatvesztéssel járó migrációt felülvizsgálat nélkül élesbe; ne kösd össze a séma- és kódváltozást egyetlen, atomikusnak feltételezett lépésbe; ne tarts környezetenként eltérő buildet.

#130 Biztonsági alapok API-knál – gyakorlati ellenőrzőlista

Bemenet és adatkezelés

- Paraméterezett lekérdezés mindenhol (EF Core alapból, Dapper esetén figyelj rá); `FromSqlRaw` helyett `FromSqlInterpolated`.
- Túlkötés (mass assignment) elkerülése: a végpont **dedikált request DTO-t** fogadjon, soha ne közvetlenül az entitást.
- Fájlfeltöltésnél: méretkorlát (`RequestSizeLimit`), MIME/kiterjesztés ellenőrzés tartalom alapján, a fájl tárolása a webgyökéren kívül, generált fájlnev.
- Válaszban csak az szerepeljen, amit a hívó láthat – ne szerializálj entitást belső mezőkkel.

Fejlécek és transzport

```
app.UseHsts();
app.UseHttpsRedirection();
app.Use(async (context, next) =>
{
    context.Response.Headers.XContentTypeOptions = "nosniff";
    context.Response.Headers.XFrameOptions = "DENY";
    context.Response.Headers["Referrer-Policy"] = "no-referrer";
    context.Response.Headers["Content-Security-Policy"] = "default-src 'none'; frame-ancestors 'none'";
    await next();
});
```

CORS – konkrét originek, soha nem `AllowAnyOrigin()` + `AllowCredentials()` együtt:

```
builder.Services.AddCors(o => o.AddPolicy("spa", p => p
    .WithOrigins("https://app.example.com")
    .AllowAnyHeader().AllowAnyMethod().AllowCredentials()));
```

Titkosítás és titkok

- `IDataProtectionProvider` a rövid életű, alkalmazáson belüli titkosításhoz; több példány esetén a kulcsgyűrűt közös tárolóba (Redis, blob) kell tenni, különben a példányok nem tudják egymás sütijeit/tokenjeit feldolgozni.
- Jelszó tárolása kizárólag lassú, szózott hash-sel (`PasswordHasher<T>` , Argon2, bcrypt) – soha nem SHA-256-tal.
- Titkok: Key Vault / managed identity (lásd "Konfiguráció és titokkezelés" fejezet).

Tokenek és munkamenet

- Rövid élettartamú access token + refresh token rotációval; a refresh token visszavonható legyen.
- Token-validáció: `ValidateIssuer` , `ValidateAudience` , `ValidateLifetime` , `ClockSkew` szűkítése.
- Ne tarts jogosultsági listát kizárólag a tokenben, ha az azonnali visszavonás követelmény.

Naplózás és megfigyelhetőség

- Soha ne logolj jelszót, tokenet, kártyaszámot, személyes adatot – Serilog destructuring policy vagy explicit maszkolás.
- Naplózd a biztonsági eseményeket (sikertelen bejelentkezés, jogosultság-megtagadás, adminisztratív művelet) auditálhatóan.
- A hibaválasz ne szivárogtasson belső részletet: éles környezetben `ProblemDetails` stack trace nélkül (lásd "Globális kivételkezelés és ProblemDetails" fejezet).

Függőségek és ellátási lánc

```
dotnet list package --vulnerable --include-transitive
dotnet list package --deprecated
```

Ezt tedd a CI részévé, és kapcsold be a Dependabot/Renovate automatikus frissítéseket. Éles buildhez érdemes `Directory.Packages.props` -szal központi verziókezelést és NuGet package lockfile-t használni a reprodukálható buildért.

Mit kerülj el: ne írd saját kriptográfiát; ne kapcsold ki a tanúsítvány-ellenőrzést "mert fejlesztésben zavaró" (ez a beállítás előbb-utóbb élesbe kerül); ne bízz semmilyen kliensoldali ellenőrzésben; ne feltételezd, hogy a belső hálózat biztonságos (zero trust: a szolgáltatások közötti hívások is legyenek hitelesítve).

#131 Konkurenciakezelés haladó szinten – adatbázis, throttling, ütközésfeloldás

Verzió: `ExecuteUpdate` : EF Core 7.0; `Parallel.ForEachAsync` : .NET 6; `System.Threading.Lock` : .NET 9 / C# 13.

Ez a fejezet a három meglévő konkurencia-fejezetet köti össze és egészíti ki: "EF Core Concurrency control" (adatbázis, optimista zárolás), "Alkalmazásszintű (in-memory) Concurrency Handling" (memóriabeli megosztott állapot) és "Elosztott lock és háttérfolyamatok több példányban"

(több processz).

A négy szint, ahol a konkurencia jelentkezik

Szint	Probléma	Eszköz
Egy processz, több szál	megosztott memória (Singleton , statikus mező, cache)	lock , Interlocked , SemaphoreSlim , Concurrent* gyűjtemények
Egy adatbázis, több tranzakció	lost update, piszkos olvasás	optimista (RowVersion) vagy pesszimista zárolás, izolációs szint
Több alkalmazáspéldány	egy job kétszer fut	elosztott lock, ütemező klaszter-mód, egyedi index
Több szolgáltatás	elosztott folyamat konzisztenciája	Saga, Outbox/Inbox, idempotencia

Optimista ütközés kezelése – nem elég elkapni, meg is kell oldani

```
public async Task<Result> UpdatePriceAsync(Guid productId, decimal newPrice,
CancellationTokentoken ct)
{
    const int maxAttempts = 3;

    for (var attempt = 1; attempt ≤ maxAttempts; attempt++)
    {
        var product = await _db.Products.FirstAsync(p ⇒ p.Id == productId, ct);
        product.ChangePrice(newPrice);

        try
        {
            await _db.SaveChangesAsync(ct);
            return Result.Success();
        }
        catch (DbUpdateConcurrencyException ex)
        {
            var entry = ex.Entries.Single();
            var databaseValues = await entry.GetDatabaseValuesAsync(ct);

            if (databaseValues is null)
                return Result.Failure("A terméket időközben törölték.");

            // "database wins" újratöltés, majd újra megpróbáljuk az üzleti műveletet
            entry.OriginalValues.SetValues(databaseValues);

            if (attempt == maxAttempts)
                return Result.Failure("A rekordot időközben más módosította, próbálja újra.");
        }
    }

    return Result.Failure("Sikertelen mentés.");
}
```

Három ütközésfeloldási stratégia létezik: **client wins** (a felhasználó értéke felülírja a másikat – csak akkor, ha az adat természete ezt megengedi), **database wins** (a művelet eldobásra kerül, a felhasználó újratölt), és **merge** (mezőnkénti összefésülés, esetleg a felhasználó bevonásával). Egy API-nál a

leggyakoribb helyes válasz a `409 Conflict` az aktuális állapottal együtt, hogy a kliens dönthessen.

Atomi frissítés olvasás nélkül

Ha nincs szükség a domain logikára, a legolcsóbb konkurenciakezelés az, ha az adatbázis végzi a műveletet egyetlen utasításban – így nincs is mit ütköztetni:

```
// EF Core 7+: egyetlen UPDATE utasítás, betöltés és change tracking nélkül
var affected = await _db.Products
    .Where(p => p.Id == productId && p.Stock >= quantity)
    .ExecuteUpdateAsync(s => s.SetProperty(p => p.Stock, p => p.Stock - quantity), ct);

if (affected == 0) return Result.Failure("Nincs elegendő készlet.");
```

Ez a minta ("feltételes UPDATE") készletkezelésnél, egyenleg-levonásnál és számláló-növelésnél a legmegbízhatóbb megoldás. Figyelem: az `ExecuteUpdate` / `ExecuteDelete` **megkerüli a change trackinget**, az interceptorokat és a domain eseményeket – csak ott használd, ahol ez tudatos döntés.

Pesszimista zárolás – amikor tényleg kell

Optimista zárolás akkor jó, ha az ütközés **ritka**. Ha viszont gyakori és az újrapróbálkozás drága (pl. hosszú, több lépéses foglalási folyamat), pesszimista zárolás a helyes:

```
await using var tx = await _db.Database.BeginTransactionAsync(IsolationLevel.ReadCommitted, ct);

// SQL Server: sor szintű írási zár a tranzakció végéig
var seat = await _db.Seats
    .FromSql($"SELECT * FROM Seats WITH (UPDLOCK, ROWLOCK) WHERE Id = {seatId}")
    .SingleAsync(ct);

// PostgreSQL megfelelője: SELECT ... FOR UPDATE
seat.Reserve(customerId);
await _db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

Amire ügyelni kell: **mindig ugyanabban a sorrendben** zárolj az erőforrásokat (különben deadlock); tartsd a tranzakciót a lehető legrövidebb ideig; soha ne hívj külső API-t vagy várj felhasználói bemenetre nyitott tranzakció alatt.

Izolációs szintek – mit engednek meg

Szint	Piszkos olvasás	Nem ismételhető olvasás	Fantom sor
Read Uncommitted	lehet	lehet	lehet
Read Committed (alapértelmezett)	nem	lehet	lehet
Repeatable Read	nem	nem	lehet
Serializable	nem	nem	nem
Snapshot	nem	nem	nem (verziózással, blokkolás nélkül)

A magasabb izoláció több zárat és több deadlockot jelent. SQL Serveren a `READ_COMMITTED_SNAPSHOT` bekapcsolása gyakran a legjobb kompromisszum: az olvasók nem blokkolják az írókat.

Deadlock kezelése

A deadlock nem programozási hiba, hanem üzemi jelenség – az adatbázis az egyik tranzakciót áldozatként megszakítja (SQL Server: 1205-ös hiba). A helyes válasz **újrapróbálkozás**, nem a hiba elnyelése:

```
builder.Services.AddDbContext<AppDbContext>(o => o.UseSqlServer(cs, sql =>
    sql.EnableRetryOnFailure(maxRetryCount: 3, maxRetryDelay: TimeSpan.FromSeconds(5),
        errorNumbersToAdd: [1205])));
```

Fontos: az `EnableRetryOnFailure` (execution strategy) **nem kombinálható automatikusan** kézzel indított tranzakcióval – ilyenkor a teljes tranzakciót az execution strategy-n keresztül kell futtatni:

```
var strategy = _db.Database.CreateExecutionStrategy();
await strategy.ExecuteAsync(async () =>
{
    await using var tx = await _db.Database.BeginTransactionAsync(ct);
    // ... műveletek ...
    await tx.CommitAsync(ct);
});
```

Párhuzamosság korlátozása (throttling)

A korlátlan párhuzamosság önmagában is konkurenciahiba: kimeríti a kapcsolatkészletet vagy megfojtja a külső szolgáltatást.

```
// .NET 6+: beépített párhuzamosság-korlát
await Parallel.ForEachAsync(orderIds,
    new ParallelOptions { MaxDegreeOfParallelism = 8, CancellationToken = ct },
    async (orderId, token) => await ProcessOrderAsync(orderId, token));

// vagy SemaphoreSlim-mel, ha Task.WhenAll kell
using var throttle = new SemaphoreSlim(8);
var tasks = orderIds.Select(async id =>
{
    await throttle.WaitAsync(ct);
    try { return await ProcessOrderAsync(id, ct); }
    finally { throttle.Release(); }
});
var results = await Task.WhenAll(tasks);
```

A `DbContext` nem szálbiztos – soha ne indíts párhuzamos műveleteket ugyanazon a példányon (`Task.WhenAll` két `DbContext` lekérdezéssel `InvalidOperationException`-t dob). Párhuzamos adatbázis-munkához scope-onként külön `DbContext` kell (`IDbContextFactory<T>`).

Egyediség kikényszerítése versenyhelyzetben

A "megnézem, létezik-e, és ha nem, létrehozom" mintát **nem** lehet ellenőrzéssel megoldani – két párhuzamos kérés mindkettő "nem létezik" választ kaphat. Az egyetlen megbízható megoldás az adatbázis egyedi indexe, és a `DbUpdateException` elkapása:

```
try { await _db.SaveChangesAsync(ct); }
catch (DbUpdateException ex) when (ex.InnerException is SqlException { Number: 2601 or 2627 })
{
    return Result.Failure("Ez az e-mail cím már regisztrálva van.");
}
```

Mit kerülj el: ne oldj meg adatbázis-szintű konkurenciát `lock`-kal (több példánynál nem működik); ne emeld az izolációs szintet "biztos, ami biztos" alapon; ne nyelj el `DbUpdateConcurrencyException`-t; ne felejtse el, hogy az újrapróbálkozás csak akkor biztonságos, ha a művelet idempotens.

#132 Async/await haladó szint – ValueTask, thread pool, timeout, fire-and-forget

Verzió: `ValueTask` : .NET Core 2.1 / C# 7; `Task.WaitAsync` : .NET 6;
`CancellationTokenSource.CancelAsync` : .NET 8.

Az "Async/await Best Practices és CancellationToken" és a "Mikor használjunk sync és mikor async API metódusokat" fejezetek az alapokat fedik; itt a napi munkában előkerülő nehezebb esetek következnek.

Mire jó az async valójában – és mire nem

Az `async / await` **nem gyorsítja** az egyes kéréseket, hanem **skálázhatóságot** ad: az I/O várakozás alatt (adatbázis, HTTP, fájl) a szál visszakerül a thread poolba, és más kérést szolgálhat ki. Ebből következik:

- **I/O-kötött** művelet → `async` (adatbázis, hálózat, fájl, üzenetsor).
- **CPU-kötött** művelet → önmagában nem indokolja az aszinkronit; ha egy webkérésben hosszú számítás van, az `Task.Run` **nem** segít (ugyanabból a thread poolból vesz szálakat, csak egy váltással többet) – ilyen háttér folyamatba vagy külön workerbe kell tenni.
- Egy kicsi, gyorsan lefutó szinkron művelet aszinkronná alakítása (`Task.Run` köré csomagolva) rontja a teljesítményt.

Thread pool starvation – a leggyakoribb éles teljesítményprobléma

Ha bárhol a kódban szinkron blokkolás történik egy `async` művelet fölött (`.Result` , `.Wait()` , `.GetAwaiter().GetResult()` , `Task.WhenAll(...).Wait()`), a szál blokkolva várakozik. Terhelés alatt a thread pool kifogy, a válaszidők másodpercekre nőnek, a CPU mégis alacsony marad – ez a klasszikus tünetegyüttes.

```
// TÜNET: alacsony CPU, magas késleltetés, növekvő ThreadPool queue length
// Diagnosztika: dotnet-counters monitor --counters System.Runtime
// → threadpool-queue-length, threadpool-thread-count
```

A javítás mindig ugyanaz: **async all the way** – a végponttól a legelső I/O hívásig végig `await`.

`ValueTask` – mikor van értelme

A `Task` egy heap-en allokkált objektum. Ha egy gyakran hívott metódus **az esetek többségében szinkron módon, azonnal** tud eredményt adni (pl. cache-talalat), a `ValueTask<T>` elkerüli ezt az allokációt:

```
public ValueTask<Product> GetAsync(Guid id, CancellationToken ct)
{
    if (_cache.TryGetValue(id, out var cached))
        return ValueTask.FromResult(cached); // nincs allokáció

    return new ValueTask<Product>(LoadAsync(id, ct)); // ritka ág: valódi async munka
}
```

Szigorú szabályok: egy `ValueTask`-ot **csak egyszer** szabad `await`-elni, nem szabad többször olvasni a `.Result`-jét, és nem szabad párhuzamosan `await`-elni (ha többször kell, `.AsTask()`-kal alakítsd át). Publikus API-n csak akkor válaszd, ha a mérés indokolja – alapértelmezésben `Task` a helyes.

Timeout és lemondás összekapcsolása

```
// .NET 6+: bármely Task időkorlátozása
var result = await GetDataAsync(ct).WaitAsync(TimeSpan.FromSeconds(5), ct);

// Kérés-lemondás és saját timeout együtt
using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(10));
using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(ct, timeoutCts.Token);

try
{
    await _client.GetAsync(url, linkedCts.Token);
}
catch (OperationCanceledException) when (timeoutCts.IsCancellationRequested)
{
    // saját időtúllépés – ez más eset, mint amikor a kliens szakította meg a kérést
    throw new TimeoutException("A külső szolgáltatás nem válaszolt időben.");
}
```

Az `OperationCanceledException` **nem hiba**: ha a kliens lecsatlakozott, ne logold hibaként és ne riasszon rá a monitoring.

Kivételek `Task.WhenAll` esetén

A `Task.WhenAll` `await`-elésekor **csak az első** kivétel dobódik újra, pedig több task is elbukhatott:


```
var tasks = ids.Select(id => ProcessAsync(id, ct)).ToArray();
var all = Task.WhenAll(tasks);

try { await all; }
catch (Exception)
{
    // az ÖSSZES hiba itt érhető el
    foreach (var inner in all.Exception!.InnerExceptions)
        _logger.LogError(inner, "Feldolgozási hiba");
    throw;
}
```

Fire-and-forget – helyesen

A `_ = DoSomethingAsync();` mintában a kivétel elveszik, és az alkalmazás leállása megszakíthatja a félbehagyott munkát. Helyette:

- **Rövid, elveszthető munka** (pl. audit log): `Channel<T>` + `BackgroundService` (lásd "Channels és `IAsyncEnumerable`" fejezet).
- **Nem elveszthető munka:** `Outbox` + üzenetsor (lásd "Outbox Pattern"), vagy ütemező (Quartz.NET).
- Ha mégis elindítasz háttér munkát a kérésből, **soha ne** add tovább a kérés `CancellationToken`-jét (a válasz elküldésekor megszakadna), és mindig csomagold `try/catch`-be.

Aszinkron inicializálás és `IAsyncDisposable`

Konstruktor nem lehet async. Az egyszeri, drága inicializálásra `Lazy<Task<T>>` vagy `AsyncLazy` minta való:

```
private readonly Lazy<Task<Config>> _config;
public ConfigProvider(IConfigLoader loader) =>
    _config = new Lazy<Task<Config>>(() => loader.LoadAsync(),
    LazyThreadSafetyMode.ExecutionAndPublication);

public Task<Config> GetAsync() => _config.Value;    // csak egyszer tölt be, párhuzamos
hívásnál is
```

Aszinkron erőforrás felszabadításához `IAsyncDisposable` + `await using` (a `Dispose()`-ban **soha** ne blokkolj async hívásra).

Szinkron API kényszere

Ha egy interfész (pl. régi könyvtár) szinkron metódust vár, a legkevésbé rossz megoldás nem a `.Result`, hanem az, hogy a hívási lánc egy pontján, kontroll alatt blokkolsz – vagy `AsyncHelper` mintával külön thread poolon futtatod. A helyes irány mindig az interfész async-esítése.

Mit kerülj el: `async void` (kivéve event handler); `Task.Run` webkérésen belüli I/O-hoz; `.Result` / `.Wait()` bárhol az alkalmazáskódban; `CancellationToken` elhagyása I/O metódusokból; `ValueTask` többszöri `await`-elése; háttér munka indítása a kérés tokenjével.

#133 System.Text.Json vs. Newtonsoft.Json – melyiket mikor

Verzió: Newtonsoft.Json (Json.NET): 2008-tól, .NET Framework kora óta; System.Text.Json : .NET Core 3.0-tól, ASP.NET Core alapértelmezett szerializálója .NET Core 3.0 óta.

A .NET-ben ma két elterjedt JSON könyvtár van. A **System.Text.Json (STJ)** a beépített, alapértelmezett megoldás: gyors, kevés allokációval dolgozik, `Span<T>` /UTF-8 alapú, támogatja a forráskód-generátort és a Native AOT-t. A **Newtonsoft.Json (Json.NET)** a régebbi, rendkívül rugalmas, funkciógazdag könyvtár, amely sok olyat tud, amit az STJ szándékosan nem.

Fő különbségek

Szempont	System.Text.Json	Newtonsoft.Json
Teljesítmény / allokáció	lényegesen gyorsabb, kevesebb allokáció	lassabb, több allokáció
Alapértelmezett kis-/nagybetű	érzékeny (opcióval kikapcsolható)	nem érzékeny
Trailing comma, megjegyzés a JSON-ban	alapból hibát ad (opcióval engedhető)	megengedő
<code>DateTime</code> kezelés	szigorú ISO 8601	megengedőbb, sokféle formátum
Számok stringként ("42")	alapból hiba (<code>NumberHandling</code> opcióval)	automatikusan konvertál
Polimorfizmus	<code>[JsonDerivedType]</code> (.NET 7-től)	<code>TypeNameHandling</code> (biztonsági kockázat)
Körkörös referencia	<code>ReferenceHandler.Preserve</code>	<code>PreserveReferencesHandling</code>
Privát mezők / setterek	korlátozott	teljes körű
<code>JsonPath</code> , LINQ to JSON	nincs (csak <code>JsonNode</code> / <code>JsonDocument</code>)	van (<code>JObject</code> , <code>SelectToken</code>)
<code>DataTable</code> , <code>dynamic</code> , <code>ExpandoObject</code>	nem/korlátozott	támogatott
Forráskód-generátor, Native AOT	támogatott	nem
Egyedi szerződés-manipuláció	<code>IJsonTypeInfoResolver</code> (.NET 8)	<code>ContractResolver</code> (érettebb)

Melyiket használd

System.Text.Json – ez az alapértelmezett választás, ha: új projektet írsz; a teljesítmény és a memóriahasználat számít (nagy forgalmú API, üzenetfeldolgozás); Native AOT vagy trimming a cél; a szerződéseket te kontrollárod (saját DTO-k).

Newtonsoft.Json – akkor indokolt, ha: egy meglévő rendszer viselkedését kell pontosan reprodukálni (migrációs kockázat); a bejövő JSON "rendetlen" és megengedő feldolgozás kell; olyan funkció kell, amit az STJ nem tud (`JObject` -alapú dinamikus feldolgozás, `SelectToken` , `DataTable` , összetett `ContractResolver`); JSON Patch (`Microsoft.AspNetCore.JsonPatch` – bár .NET 10-től már van STJ-alapú változat is); harmadik féltől származó könyvtár megköveteli.

Visszakapcsolás ASP.NET Core-ban (ha tényleg kell)

```
dotnet add package Microsoft.AspNetCore.Mvc.NewtonsoftJson
```

```
builder.Services.AddControllers().AddNewtonsoftJson(options =>
{
    options.SerializerSettings.ContractResolver = new CamelCasePropertyNamesContractResolver();
    options.SerializerSettings.NullValueHandling = NullValueHandling.Ignore;
    options.SerializerSettings.ReferenceLoopHandling = ReferenceLoopHandling.Ignore;
});
```

Vegyes használat is lehetséges: az API globálisan STJ-t használ, és csak egyetlen, problémás integrációnál hívsz kézzel `JsonConvert`-et.

Migráció Newtonsoft.Json-ról – a leggyakoribb buktatók

```
var options = new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true, // Newtonsoft
    // alapértelmezett viselkedése
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
    NumberHandling = JsonNumberHandling.AllowReadingFromString,
    ReadCommentHandling = JsonCommentHandling.Skip,
    AllowTrailingCommas = true,
    Converters = { new JsonStringEnumConverter() } // enum név szerint, nem számként
};
```

További eltérések, amik csendben megváltoztatják a viselkedést:

- **Attribútumok nem cserélhetők fel:** `[JsonProperty("name")]` → `[JsonPropertyName("name")]`, `[JsonIgnore]` mindkettőben létezik, de **más névtérben** (`Newtonsoft.Json` vs `System.Text.Json.Serialization`) – ha mindkét csomag jelen van, könnyű rosszat importálni, és a jelölés némán hatástalan lesz.
- Az STJ alaphoz **nem szerializál mezőket**, csak property-eket (`IncludeFields = true` kell hozzá), és a `private set` accessorokat is másképp kezeli.
- Az STJ alapértelmezésben **escape-eli** a nem ASCII karaktereket (az ékezetes betűk `\uXXXX` formában jelennek meg); ha ez zavaró: `Encoder = JavaScriptEncoder.UnsafeRelaxedJsonEscaping` – ez csak akkor biztonságos, ha a kimenet nem kerül közvetlenül HTML-be.
- `TypeNameHandling.All` (`Newtonsoft`) használata deszerializációnál **ismert biztonsági kockázat** (tetszőleges típus példányosítása) – ne vidd át STJ-be sem, helyette explicit `[JsonDerivedType]` diszkriminátor.

Mit kerülj el: ne használd mindkét könyvtárat ugyanazon a szerződésen (eltérő attribútumok, eltérő eredmény); ne migrálj `Newtonsoft`-ról STJ-re szerződéses/jóváhagyó tesztek nélkül (a különbségek csendesek, nem fordítási hibák); ne hozz létre új `JsonSerializerOptions` példányt hívásonként (lásd "System.Text.Json haladó szint" fejezet).

#134 Enterprise szintű globális hibakezelés – teljes felállítás

Verzió: `ExceptionHandler + AddProblemDetails()` : .NET 8-tól; RFC 9457 (a 7807 utódja): 2023;
`IProblemDetailsService` : .NET 7-től.

A "Globális kivételkezelés és `ProblemDetails`" fejezet az alapokat mutatja. Egy éles, nagyvállalati rendszerben a hibakezelés nem egyetlen `try/catch`, hanem **négy összehangolt döntés**: milyen kivétel-hierarchiát használunk, hol keletkezik a hiba, mit lát a kliens, és mit lát az üzemeltetés.

1. Kivétel-hierarchia – a hibatípus, nem a szöveg hordozza a jelentést

```
// Alap: minden szándékos, kezelt alkalmazás-hiba közös őse
public abstract class AppException(string message, Exception? inner = null) :
    Exception(message, inner)
{
    /// Gépileg feldolgozható, stabil hibakód – a kliens ERRE ágazik, nem a szövegre.
    public abstract string ErrorCode { get; }
    public abstract int StatusCode { get; }
    /// Kliensnek megjeleníthető-e a Message, vagy csak generikus szöveg mehet ki
    public virtual bool IsMessageSafeToExpose => true;
    /// Kiegészítő adatok a ProblemDetails "extensions" mezőjébe
    public IDictionary<string, object?> Extensions { get; } = new Dictionary<string, object?>();
}

public sealed class NotFoundException(string resource, object key)
    : AppException($"A(z) {resource} nem található: {key}")
{
    public override string ErrorCode => "resource_not_found";
    public override int StatusCode => StatusCodes.Status404NotFound;
}

public sealed class BusinessRuleViolationException(string code, string message)
    : AppException(message)
{
    public override string ErrorCode => code; // pl. "insufficient_stock"
    public override int StatusCode => StatusCodes.Status422UnprocessableEntity;
}

public sealed class ConcurrencyConflictException(string message, object? currentState = null)
    : AppException(message)
{
    public override string ErrorCode => "concurrency_conflict";
    public override int StatusCode => StatusCodes.Status409Conflict;
    public object? CurrentState { get; } = currentState;
}

public sealed class ExternalServiceException(string service, Exception inner)
    : AppException($"A(z) {service} szolgáltatás hívása megghiúsult.", inner)
{
    public override string ErrorCode => "upstream_unavailable";
    public override int StatusCode => StatusCodes.Status502BadGateway;
    public override bool IsMessageSafeToExpose => false; // ne szivároгjon
    belső topológia
}
}
```

Alapelv: **a domain nem tud a HTTP-ről**. Ha ez zavaró (tisztá Clean Architecture), akkor a domain csak `DomainException`-t dob `ErrorCode`-dal, és a HTTP-státusz leképezése az API rétegben, egy `switch` kifejezésben történik. A Result minta (lásd "Result Pattern / Railway-Oriented Programming" fejezet) az elvárt üzleti hibákra alternatíva; a kivétel a valóban kivételes esetekre marad.

2. Több, láncolt exception handler – felelősségenként egy

A .NET 8 `ExceptionHandler` láncolható: a regisztráció sorrendjében hívódnak, és az első `true`-t visszaadó kezeli le. Így nem egy ezersoros `switch` lesz belőle.

```
builder.Services.AddProblemDetails(options =>
{
    // minden hibaválaszra automatikusan rákerülő közös mezők
    options.CustomizeProblemDetails = ctx =>
    {
        ctx.ProblemDetails.Instance = $"{ctx.HttpContext.Request.Method}
{ctx.HttpContext.Request.Path}";
        ctx.ProblemDetails.Extensions["traceId"] = Activity.Current?.TraceId.ToString()
            ?? ctx.HttpContext.TraceIdentifier;
        ctx.ProblemDetails.Extensions["requestId"] = ctx.HttpContext.TraceIdentifier;
        ctx.ProblemDetails.Extensions["timestamp"] = DateTimeOffset.UtcNow;
    };
});

builder.Services.AddExceptionHandler<ValidationExceptionHandler>(); // 400 + mezőnkénti hibák
builder.Services.AddExceptionHandler<AppExceptionHandler>(); // ismert alkalmazás-hibák
builder.Services.AddExceptionHandler<FallbackExceptionHandler>(); // minden más → 500
```

```

public sealed class AppExceptionHandler(IProblemDetailsService problemDetailsService,
    IHostEnvironment env, ILogger<AppExceptionHandler> logger) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext context, Exception exception,
        CancellationToken ct)
    {
        if (exception is not AppException appException) return false;

        // A kezelt üzleti hiba NEM error szintű log – különben elárasztja a riasztásokat
        logger.LogWarning(exception, "Kezelt alkalmazás-hiba:
{ErrorCode}", appException.ErrorCode);

        context.Response.StatusCode = appException.StatusCode;

        var problem = new ProblemDetails
        {
            Status = appException.StatusCode,
            Title = TitleFor(appException.StatusCode),
            Type = $"https://errors.example.com/{appException.ErrorCode}", // dokumentációs URI
            Detail = appException.IsMessageSafeToExpose
                ? appException.Message
                : "A kérés feldolgozása során hiba történt."
        };
        problem.Extensions["errorCode"] = appException.ErrorCode;
        foreach (var (key, value) in appException.Extensions) problem.Extensions[key] = value;
        if (env.IsDevelopment()) problem.Extensions["exception"] = exception.ToString();

        return await problemDetailsService.TryWriteAsync(new ProblemDetailsContext
        {
            HttpContext = context, ProblemDetails = problem, Exception = exception
        });
    }

    private static string TitleFor(int status) => status switch
    {
        404 => "Nem található", 409 => "Ütközés", 422 => "Feldolgozhatatlan kérés",
        502 => "Külső szolgáltatás hiba", _ => "Hiba"
    };
}

```

A `FallbackExceptionHandler` a valóban váratlan hibákat kezeli: `LogError` **(ez riasztást érdemel)**, a válasz pedig generikus 500-as `ProblemDetails`, kizárólag a `traceId`-vel – hogy a felhasználó ezt bediktálva a support meg tudja találni a pontos kérést a logban.

3. Amit a kliens lát – stabil szerződés

```
{
  "type": "https://errors.example.com/insufficient_stock",
  "title": "Feldolgozhatatlan kérés",
  "status": 422,
  "detail": "A P-123 termékből csak 3 db van készleten.",
  "instance": "POST /api/v1/orders",
  "errorCode": "insufficient_stock",
  "traceId": "0af7651916cd43dd8448eb211c80319c",
  "timestamp": "2026-08-05T10:22:41.881Z",
  "productId": "P-123",
  "available": 3
}
```

Szabályok, amiket érdemes csapatszinten kimondani: a `status` + `errorCode` a **szerződés** (verziózni kell, mint egy API-t); a `detail` **ember számára** szól és lokalizálható; a mezőnkénti validációs hibák a `errors` objektumban mennek (`ValidationProblemDetails`); soha nem kerül ki stack trace, connection string, belső hosztnév, SQL vagy más rendszer nyers hibaüzenete.

4. Amit az üzemeltetés lát

- **Loggolási szint fegyelme:** várt üzleti hiba → `Warning` (vagy `Information`); kezeletlen kivétel → `Error`; a `OperationCanceledException` kliens-lemondásnál → `Debug`, nem hiba.
- **Korreláció:** minden hibalogban benne van a `TraceId` (lásd a következő fejezetet), így a hibaválasz és a log összekapcsolható.
- **Metrika:** `errorCode` dimenzióval (kis kardinalitás!) számláló, hogy riasztható legyen a "hirtelen megnőtt a `payment_declined` aránya".
- **A kivétel maga is telemetria:** `activity?.AddException(ex)` és `RecordException = true` az OTel instrumentációban.

5. A pipeline többi hibaútja

A `UseExceptionHandler()` **nem fog el mindent**. Amire külön gondolni kell:

```
app.UseExceptionHandler();           // kivételek
app.UseStatusCodePages();             // "Üres" 401/403/404 válaszok is kapjanak ProblemDetails-t
```

- A kivételkezelőt a pipeline **legelejére** kell tenni (a middleware sorrendről lásd a "Middleware részletesen" fejezetet), különben az alatta lévő middleware-ek hibái kimaradnak.
- A `Kestrel` szintjén keletkező hibák (túl nagy kérés, hibás protokoll) sosem érnek el a middleware-ig.
- **Háttérfolyamatok és üzenet-fogyasztók** nem részei a HTTP pipeline-nak: ott saját, explicit hibakezelés kell (retry, DLQ – lásd a MassTransit/Kafka fejezeteket), különben egy kezeletlen kivétel megállítja a `BackgroundService`-t (a .NET 6+ alapértelmezés szerint le is állítja a hostot).
- A `startup`-hibák (rossz konfiguráció) `ValidateOnStart()`-tal derüljenek ki, ne futásidőben.

Mit kerülj el: ne kapkodj el kivételt csak azért, hogy `null` -t adj vissza; ne használd a kivételt vezérlési szerkezetként gyakori üzleti útvonalon (drága a stack unwinding – ott a Result minta a helyes); ne fordíts minden hibát 500-ra (a kliens nem tudja, újrapróbálható-e); ne logold ugyanazt a kivételt minden rétegben újra (egyszer, a legfelső, kontextussal rendelkező ponton).

#135 Correlation ID – kérés-követés végponttól végpontig

Verzió: W3C Trace Context (`traceparent`) natív támogatás: .NET 5-től (`Activity` , `DistributedContextPropagator`).

Mi a célja

Egy elosztott rendszerben egyetlen felhasználói művelet több szolgáltatáson, üzenetsoron és háttérfolyamaton megy át. A **correlation ID** az az azonosító, amely ezt az egészet **egy logikai egységként** azonosítja, így:

- Hibakeresés:** egy hibabejelentéshez tartozó összes logsor kikeresése az összes szolgáltatásból, egyetlen szűréssel.
- Támogatás:** a hibaválaszban visszaadott azonosítót a felhasználó bediktálja, és pontosan az ő kérése kerül elő.
- Késleltetés-elemzés:** melyik szolgáltatás melyik lépése volt a lassú (ez már a distributed tracing).
- Auditálás:** ki, mikor, milyen műveletet indított, és annak milyen továbbhatásai lettek (kiküldött üzenetek, másodlagos írások).
- Idempotencia és duplikátum-felismerés** támogatása (bár arra dedikált kulcs való, lásd "Idempotency Pattern API-kban").

Három fogalom, amit nem érdemes összekeverni

Fogalom	Élettartam	Ki adja
TraceId (W3C)	a teljes elosztott művelet	az első szolgáltatás, majd propagálódik
SpanId	egy művelet-lépés (egy szolgáltatás, egy hívás)	minden szolgáltatás maga
Correlation ID (üzleti)	egy üzleti folyamat, akár napokig (pl. rendelés életútja)	az alkalmazás, üzleti azonosítóként

A .NET-ben ma nem kell saját correlation ID mechanizmust írni: a `System.Diagnostics.Activity` a W3C Trace Contextet automatikusan kezeli, a `traceparent` HTTP fejléc a `HttpClient` hívásokon magától utazik, és `Activity.Current.TraceId` mindenhol elérhető. Saját `X-Correlation-ID` fejléc akkor indokolt, ha külső partnerek vagy legacy rendszerek nem támogatják a `traceparent` -et.

Beemelés a logokba (a legfontosabb lépés)

```
// Serilog: minden logsor kapjon TraceId/SpanId-t
builder.Host.UseSerilog((ctx, cfg) => cfg
    .ReadFrom.Configuration(ctx.Configuration)
    .Enrich.FromLogContext()
    .Enrich.WithSpan() // Serilog.Enrichers.Span
    .WriteTo.Console(new CompactJsonFormatter()));
```

```
// Üzleti kontextus hozzáadása a kérés teljes életútjára (scope)
app.Use(async (context, next) =>
{
    var correlationId = context.Request.Headers["X-Correlation-ID"].FirstOrDefault()
        ?? Activity.Current?.TraceId.ToString()
        ?? context.TraceIdentifier;

    // válaszban is adjuk vissza - a kliens/support ezt látja
    context.Response.Headers["X-Correlation-ID"] = correlationId;
    Activity.Current?.SetTag("correlation.id", correlationId);

    using (Serilog.Context.LogContext.PushProperty("CorrelationId", correlationId))
    using (Serilog.Context.LogContext.PushProperty("TenantId",
        context.User.FindFirst("tenant_id")?.Value))
    using (Serilog.Context.LogContext.PushProperty("UserId",
        context.User.FindFirst("sub")?.Value))
    {
        await next();
    }
});
```

Ezt a middleware-t a pipeline **elejére** kell tenni, hogy minden későbbi log és hiba is tartalmazza.

Átvitel üzenetsoron és háttérfolyamaton

A HTTP-vel ellentétben a broker nem propagál automatikusan mindent: a MassTransit és a Wolverine igen (ConversationId / CorrelationId beépített fejlécek), nyers Confluent.Kafka esetén kézzel kell:

```
// Producer: a trace context beírása az üzenet fejlécébe
var propagator = Propagators.DefaultTextMapPropagator;
propagator.Inject(new PropagationContext(Activity.Current!.Context,
    Baggage.Current), message.Headers,
    (headers, key, value) => headers.Add(key, Encoding.UTF8.GetBytes(value)));

// Consumer: kiolvasás és az új Activity szülőjének beállítása
var parentContext = propagator.Extract(default, result.Message.Headers,
    (headers, key) => headers.TryGetLastBytes(key, out var bytes) ?
    [Encoding.UTF8.GetString(bytes)] : []);

using var activity = ActivitySource.StartActivity("process order",
    ActivityKind.Consumer, parentContext.ActivityContext);
```

BackgroundService -ben, ütemezett job-ban nincs bejövő kérés, ezért ott **saját** root Activity -t kell indítani – így a job futása is önálló, kereshető trace lesz.

Kliensoldal és teljes lánc

A böngésző/mobil app is generálhat azonosítót (`X-Request-ID`), amit a backend baggage-be tesz, így a frontend hibajelentés és a backend log összekapcsolható. Az OpenTelemetry `Baggage` API-ja ezt kulcs-érték párként propagálja a teljes láncon – **de** csak nem érzékeny adatot tegyél bele, mert minden downstream szolgáltatás látja.

Mit kerülj el: ne generálj új azonosítót minden szolgáltatásban (akkor nincs mit korrelálni – csak akkor generálj, ha nem jött be); ne tedd személyes adatot vagy tokent a correlation ID-be vagy a baggage-be; ne bízz vakon a kliens által küldött azonosítóban (validáld a formátumot és hosszát – log injection ellen); ne írd kézi propagálást oda, ahol az `Activity` már megteszi.

#136 Object Mother és Test Data Builder – haladó, kettős célú megvalósítás

Verzió: `Guid.CreateVersion7()` (rendezett GUID): .NET 9-től; korábban saját sequential GUID vagy `NewId` (MassTransit) használható.

A "Teszt adat-előállítási patternnek" fejezet az alapokat mutatja. Éles projektben a legnagyobb nyereség egy **kettős célú** builder: ugyanaz a leírás használható tiszta unit tesztben (memóriában, adatbázis nélkül) **és** integrációs/funkcionális tesztben (perzisztálva, valódi adatbázisba).

Azonosító-stratégia: melyik illik enterprise alkalmazáshoz

Részletes összehasonlítás (*ULID, Snowflake, HashIds, ShortGuid, index-hatások*) az "Azonosító-generálási stratégiák .NET-ben" fejezetben.

Megoldás	Előny	Hátrány	Mikor
<code>int / bigint</code> identity	legkisebb, klaszterezett indexre ideális	nem generálható kliensen, kiszivárogtatja a mennyiséget, migráció/merge nehéz	belső, egy adatbázisos rendszer
<code>Guid</code> (v4, random)	kliensen generálható, globálisan egyedi	véletlen sorrend → indexfragmentálódás , lapszétvágás	ha nincs jobb
<code>Guid</code> v7 (időrendezett)	kliensen generálható és rendezett (jó index-lokalitás), beépített	16 bájt, létrehozási időt hordoz	ez az alapértelmezett választás elsődleges kulcsra
<code>NanoID / ULID</code> (string)	rövid, URL-barát, ember által másolható	string kulcs (nagyobb index), külön könyvtár	publikus, URL-ben megjelenő azonosítóra

Enterprise gyakorlat: **belső elsődleges kulcs** = `Guid.CreateVersion7()` (kliensoldali generálás miatt a domain objektum már azonosítóval születik, ami az Outbox és a saga korreláció miatt is fontos), és ha kell **publikus, "beszédes" azonosító, akkor külön mező** (`NanoID` vagy üzleti azonosító, pl. `ORD-2026-000123`). A kettő elválasztása azért jó, mert a publikus azonosító formátuma később változtatható a séma és a külső kulcsok érintése nélkül.

```

public static class Ids
{
    /// Rendezett, kliensen generálható technikai kulcs (index-barát)
    public static Guid NewTechnicalId() ⇒ Guid.CreateVersion7();

    /// Rövid, URL-barát publikus azonosító (NanoID-szerű, ütközésbiztos hosszal)
    private const string Alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"; // Crockford base32:
    nincs I, L, O, U
    public static string NewPublicId(int length = 12)
    {
        var buffer = RandomNumberGenerator.GetBytes(length);
        return string.Create(length, buffer, (span, bytes) ⇒
        {
            for (var i = 0; i < span.Length; i++) span[i] = Alphabet[bytes[i] % Alphabet.Length];
        });
    }
}

```

Object Mother – nevesített, üzleti jelentésű forgatókönyvek

```

/// Object Mother: MIT jelent egy tipikus eset. A "miért ilyen" a névben van.
public static class Orders
{
    public static OrderBuilder ANewDraftOrder() ⇒ new();

    public static OrderBuilder APaidOrderWithTwoItems() ⇒ new OrderBuilder()
        .WithItem(Products.AStandardProduct(), quantity: 2)
        .WithItem(Products.ADiscountedProduct(), quantity: 1)
        .Paid();

    public static OrderBuilder AnOverdueUnpaidOrder() ⇒ new OrderBuilder()
        .WithItem(Products.AStandardProduct(), quantity: 1)
        .PlacedAt(DateTimeOffset.UtcNow.AddDays(-45))
        .AwaitingPayment();

    public static OrderBuilder AnOrderForTenant(Guid tenantId) ⇒ new
    OrderBuilder().ForTenant(tenantId);
}

```

Test Data Builder – kettős célú: memóriában vagy perzisztálva

A kulcsötlet: a `BuildAsync` **opcionális** `DbContext`-et és `CancellationToken`-t fogad. Ha van perzisztencia-kontextus, ment; ha nincs, csak a memóriabeli objektumot adja vissza. Így egy tesztadat-leírás mindhárom teszttrétegben használható.

```

public sealed class OrderBuilder
{
    private Guid _id = Ids.NewTechnicalId();
    private string _publicId = Ids.NewPublicId();
    private Guid _tenantId = TestTenants.Default;
    private Guid _customerId = Ids.NewTechnicalId();
    private DateTimeOffset _placedAt = new(2026, 1, 15, 10, 0, 0, TimeSpan.Zero); //
determinisztikus alap!
    private OrderStatus _status = OrderStatus.Draft;
    private readonly List<(ProductBuilder Product, int Quantity)> _items = [];
    private byte[]? _rowVersion;

    public OrderBuilder WithId(Guid id) { _id = id; return this; }
    public OrderBuilder ForTenant(Guid tenantId) { _tenantId = tenantId; return this; }
    public OrderBuilder ForCustomer(Guid customerId) { _customerId = customerId; return this; }
    public OrderBuilder PlacedAt(DateTimeOffset at) { _placedAt = at; return this; }
    public OrderBuilder WithItem(ProductBuilder product, int quantity) { _items.Add((product,
quantity)); return this; }
    public OrderBuilder Paid() { _status = OrderStatus.Paid; return this; }
    public OrderBuilder AwaitingPayment() { _status = OrderStatus.AwaitingPayment; return this; }
    public OrderBuilder WithConcurrencyToken(byte[] rowVersion) { _rowVersion = rowVersion;
return this; }

    /// Sok elem gyorsan, egyedi adatokkal
    public static IReadOnlyList<OrderBuilder> Many(int count, Action<OrderBuilder, int>?
customize = null) =>
        Enumerable.Range(0, count).Select(i =>
        {
            var builder = new OrderBuilder().PlacedAt(new DateTimeOffset(2026, 1, 1, 0, 0,
0, TimeSpan.Zero).AddDays(i));
            customize?.Invoke(builder, i);
            return builder;
        }).ToList();

    /// UNIT teszt: csak memóriabeli objektum, semmilyen I/O
    public Order Build()
    {
        var order = Order.Rehydrate(_id, _publicId, _tenantId, _customerId, _placedAt, _status);
        foreach (var (product, quantity) in _items)
            order.AddItem(product.Build(), quantity);

        if (_rowVersion is not null) order.SetRowVersionForTest(_rowVersion);
        return order;
    }

    /// INTEGRÁCIÓS / FUNKCIONÁLIS teszt: ugyanaz az objektum, de perzisztálva.
    /// A CancellationToken jelenléte a jelzés, hogy valódi I/O történik.
    public async Task<Order> BuildAsync(AppDbContext db, CancellationToken cancellationToken)
    {
        // a hivatkozott aggregátumok (termék, ügyfél) is jöjjenek létre, ha még nem léteznek
        foreach (var (product, _) in _items)
            await product.EnsurePersistedAsync(db, cancellationToken);

        var order = Build();
        db.Orders.Add(order);
        await db.SaveChangesAsync(cancellationToken);
        db.ChangeTracker.Clear(); // a teszt ne "lássa" a betöltött entitást a tracker
cache-éből
    }
}

```

```

        return order;
    }
}

```

Használat a három tesztrétegben:

```

// 1. UNIT - nincs adatbázis, nincs async
[Fact]
public void Overdue_unpaid_order_can_be_cancelled()
{
    var order = Orders.AnOverdueUnpaidOrder().Build();

    var result = order.CancelDueToNonPayment(asOf: new DateTimeOffset(2026, 3, 1, 0, 0,
0, TimeSpan.Zero));

    result.IsSuccess.Should().BeTrue();
    order.Status.Should().Be(OrderStatus.Cancelled);
}

// 2. INTEGRÁCIÓS - valódi adatbázis, egy szelet/handler tesztelése
[Fact]
public async Task Handler_marks_overdue_orders_as_cancelled()
{
    await using var db = _fixture.CreateDbContext();
    var order = await Orders.AnOverdueUnpaidOrder().BuildAsync(db,
TestContext.Current.CancellationTokens);

    await _handler.HandleAsync(new CancelOverdueOrders(), TestContext.Current.CancellationTokens);

    var reloaded = await db.Orders.FindAsync([order.Id], TestContext.Current.CancellationTokens);
    reloaded!.Status.Should().Be(OrderStatus.Cancelled);
}

// 3. FUNKCIONÁLIS - a teljes API-n keresztül
[Fact]
public async Task Get_order_returns_paid_order_with_items()
{
    await using var db = _fixture.CreateDbContext();
    var order = await Orders.APaidOrderWithTwoItems().BuildAsync(db,
TestContext.Current.CancellationTokens);

    var response = await _client.GetFromJsonAsync<OrderDto>($"api/v1/orders/{order.PublicId}");

    response!.Items.Should().HaveCount(2);
}

```

Gyakorlati szempontok

- **Determinisztikus alapértékek:** fix dátum (`new DateTimeOffset(2026, 1, 15, ...)`), ne `DateTimeOffset.UtcNow` – különben a teszt "néha piros" lesz (pl. hónapfordulón). Ha az alkalmazás időt olvas, injektált `TimeProvider / FakeTimeProvider` (`Microsoft.Extensions.TimeProvider.Testing`) legyen mögötte.
- **Csak a teszt szempontjából lényeges adatot állítsd be** a tesztben; minden más maradjon a builder alapértéke. A teszt így dokumentálja, mi számít.

- **Privát setterek:** a domain invariánsokat ne kerülj meg publikus setterekkel csak a tesztért. Vagy legyen egy `internal static Rehydrate(...)` factory (`InternalsVisibleTo` a tesztprojektnek), vagy a builder is a valódi domain műveleteket hívja (`AddItem` , `Place` , `Pay`) – utóbbi értékesebb, mert magát a domain API-t is használatba veszi.
- **Randomizálás óvatosan:** a Bogus/AutoFixture kényelmes, de véletlen adatokkal a bukó teszt nem reprodukálható – ha használsz, **fix seeddel** (`Randomizer.Seed = new Random(12345)`), és a bukás logolja a seedet.
- **Egyediség:** `Guid` v7 alapértelmezett érték minden builderben – így ugyanaz a mother kétszer meghívva sem üt egyedi indexbe.

Mit kerülsz el: ne készíts egyetlen, mindenre alkalmas `TestData.CreateOrder()` metódust 15 paraméterrel; ne oszd meg ugyanazt a **példányt** tesztek között (a builder legyen új példány minden hívásnál); ne perzisztálsz tesztadatot unit tesztből; ne írsz olyan buildert, ami közben az alkalmazás API-ját is hívja HTTP-n – a tesztadat felépítése ne az éppen tesztelt kódtól függjön.

#137 Testing pyramid felállítása és eldobható adatbázis tesztenként

Verzió: xUnit v3 `TestContext.Current.CancellationToken` : 2024-től; Respawn 6.x: .NET 6+; Testcontainers for .NET: .NET 6+.

A három réteg – mit tesztel, mennyi legyen belőle

Réteg	Mit tesztel	Mit cserél le	Futásidő / teszt	Arány
Unit	domain logika, számítás, szabály, egyetlen osztály viselkedése	mindent (nincs I/O)	< 10 ms	~70%
Integration	egy-egy technikai határ: repository/handler + valódi adatbázis, üzenet-fogyasztó + valódi broker, EF mapping, SQL, migráció	csak a külső, harmadik fél rendszereket	50–500 ms	~20%
Functional (end-to-end az API-n)	egy teljes use case a HTTP végponton át: routing, modellekötés, auth, validáció, tranzakció, válaszformátum	külső fizetés/email/partner API	200 ms – 2 s	~10%

Egy gyakorlati elhatárolás, ami vitákat előz meg: **unit = nincs I/O és nincs `async` I/O; integration = valódi infrastruktúra, de nincs HTTP; functional = a `HttpClient` -en keresztül.**

Projektstruktúra:

```
tests/
  Orders.UnitTests/           # gyors, párhuzamosan fut, minden PR-en
  Orders.IntegrationTests/    # Testcontainers + valódi DB/broker
  Orders.FunctionalTests/     # WebApplicationFactory + valódi DB
  Orders.ArchitectureTests/   # NetArchTest szabályok (lásd a vonatkozó fejezetet)
  Orders.TestKit/            # Object Mother-ök, builderek, fixture-ök (mindhárom
réteg használja)
```

```
# CI: gyors visszajelzés előbb
dotnet test tests/Orders.UnitTests
dotnet test tests/Orders.ArchitectureTests
dotnet test tests/Orders.IntegrationTests
dotnet test tests/Orders.FunctionalTests
```

Adatbázis-izolálás: négy technika, növekvő izolációval és költséggel

Technika	Izoláció	Sebesség	Párhuzamosítható
Tranzakció, amit nem commitolunk	jó	leggyorsabb	igen, de a tesztelt kód nem nyithat saját tranzakciót
Respawn (táblák ürítése teszt után)	jó	gyors	csak ha tesztenként külön adatbázis van
Új adatbázis tesztosztályonként/tesztenként, majd eldobás	teljes	közepes	igen
Új konténer tesztenként	teljes	leglassabb	igen

A harmadik a legjobb kompromisszum enterprise projektben: **egy** adatbázis-szerver konténer indul (Testcontainers), és azon belül **minden tesztosztály (collection) saját, egyedi nevű adatbázist kap**, amit a végén eldob.

Egy konténer, sablon-adatbázis, tesztenként friss adatbázis

A trükk, amivel ez gyors is marad: a migrációkat **egyszer** futtatjuk le egy sablon (template) adatbázison, és utána minden teszt-adatbázis ennek **másolata** – így nem kell újra migrálni (ami a legdrágább lépés).

```

/// Egyszer indul a teljes tesztfutásra (xUnit assembly fixture)
public sealed class DatabaseServerFixture : IAsyncLifetime
{
    private readonly PostgreSQLContainer _container = new PostgreSQLBuilder()
        .WithImage("postgres:16-alpine")
        .WithDatabase("template_db")
        .WithUsername("test").WithPassword("test")
        .WithCleanUp(true)
        .Build();

    public string AdminConnectionString => _container.GetConnectionString();

    public async ValueTask InitializeAsync()
    {
        await _container.StartAsync();

        // a sablon adatbázis egyszeri felépítése a VALÓDI migrációkkal
        var options = new DbContextOptionsBuilder<AppDbContext>
            ().UseNpgsql(AdminConnectionString).Options;
        await using var db = new AppDbContext(options);
        await db.Database.MigrateAsync();

        // opcionális: ritkán változó törzsadat (lookup táblák) is bekerülhet a sablonba
        await SeedReferenceDataAsync(db);
    }

    /// Új, izolált adatbázis a sablon másolásával - migráció nélkül, ezért gyors
    public async Task<string> CreateDatabaseAsync(string name)
    {
        await using var admin = new NpgsqlConnection(AdminConnectionString);
        await admin.OpenAsync();
        await using var cmd = admin.CreateCommand();
        cmd.CommandText = $$$$"CREATE DATABASE \"{name}\" TEMPLATE \"template_db\";$$$";
        await cmd.ExecuteNonQueryAsync();

        var csb = new NpgsqlConnectionStringBuilder(AdminConnectionString) { Database = name };
        return csb.ConnectionString;
    }

    public async Task DropDatabaseAsync(string name)
    {
        await using var admin = new NpgsqlConnection(AdminConnectionString);
        await admin.OpenAsync();
        await using var cmd = admin.CreateCommand();
        // FORCE: a nyitva maradt kapcsolatok lezárása, hogy a DROP ne akadjon el
        cmd.CommandText = $$$$"DROP DATABASE IF EXISTS \"{name}\" WITH (FORCE);$$$";
        await cmd.ExecuteNonQueryAsync();
    }

    public async ValueTask DisposeAsync() => await _container.DisposeAsync();
}

```

SQL Server esetén nincs `TEMPLATE`, de ugyanez elérhető: a sablon adatbázisról készített `BACKUP / RESTORE`, vagy a fájlok leválasztása és `CREATE DATABASE ... FOR ATTACH` másolatból; egyszerűbb (kicsit lassabb) változat: `CREATE DATABASE + Database.MigrateAsync()` tesztosztályonként.

A tesztesztály felállása: friss adatbázis előtte, eldobás után

```
/// Minden tesztesztály (vagy collection) saját adatbázist kap
public class DatabaseFixture(DatabaseServerFixture server) : IAsyncLifetime
{
    private readonly string _databaseName = $"test_{Guid.CreateVersion7():N}";
    public string ConnectionString { get; private set; } = string.Empty;

    public async ValueTask InitializeAsync() =>
        ConnectionString = await server.CreateDatabaseAsync(_databaseName);

    public AppDbContext CreateDbContext() =>
        new(new DbContextOptionsBuilder<AppDbContext>().UseNpgsql(ConnectionString).Options);

    // a teszt után az adatbázis TELJESEN eltűnik - nincs maradék állapot, nincs szivárgás
    // teszt között
    public async ValueTask DisposeAsync() => await server.DropDatabaseAsync(_databaseName);
}
```

```

[Collection(nameof(DatabaseServerCollection))]
public class PlaceOrderTests(DatabaseServerFixture server) : IAsyncLifetime
{
    private DatabaseFixture _db = null!;
    private OrdersApiFactory _factory = null!;
    private HttpClient _client = null!;

    public async ValueTask InitializeAsync()
    {
        _db = new DatabaseFixture(server);
        await _db.InitializeAsync();

        _factory = new OrdersApiFactory(_db.ConnectionString); // lásd
        "WebApplicationFactory" fejezet
        _client = _factory.CreateClient();
    }

    [Fact]
    public async Task Placing_an_order_persists_it_and_publishes_an_event()
    {
        var ct = TestContext.Current.CancellationToken;
        await using var db = _db.CreateDbContext();
        var product = await Products.AStandardProduct().BuildAsync(db, ct);

        var response = await _client.PostAsJsonAsync("/api/v1/orders",
            new { customerId = Guid.CreateVersion7(), lines = new[] { new { productId =
product.Id, quantity = 2 } } }, ct);

        response.StatusCode.Should().Be(HttpStatusCode.Created);

        var harness = _factory.Services.GetTestHarness();
        (await harness.Published.Any<OrderPlaced>()).Should().BeTrue();
    }

    public async ValueTask DisposeAsync()
    {
        _factory.Dispose();
        await _db.DisposeAsync(); // itt törlődik az adatbázis
    }
}

```

Párhuzamosítás és futásidő

- Külön adatbázis tesztosztályonként → a tesztosztályok **párhuzamosan** futhatnak (xUnit ezt alapból megteszi, ha nem ugyanabban a collectionben vannak). Ez a legnagyobb futásidő-nyereség.
- A konténer és a migráció **egyszer** fusson (assembly-szintű fixture), ne tesztenként.
- Adatbázis-tuning tesztkörnyezetre: `fsync=off`, `full_page_writes=off` (PostgreSQL), illetve tmpfs adatkönyvtár – tesztadatnál a tartósság nem szempont, a sebesség igen.
- Ha a teljes készlet így is lassú: külön CI job a lassú rétegeknek, és PR-en csak unit + architektúra + a módosított szelet integrációs tesztjei futnak, a teljes készlet éjszaka/merge után.

Mit kerülj el: ne osszatok meg egyetlen adatbázist a tesztek között "majd figyelünk rá" alapon (ez a flaky tesztek első számú oka); ne hagyj hátra adatbázist a futás után (a `WithCleanup(true)` és a `DROP ... WITH (FORCE)` erről szól); ne indíts konténert tesztmetódusonként; ne tegyél éles connection

stringet tesztkonfigurációba; ne írj funkcionális tesztet olyan esetre, amit egy unit teszt is le tud fedni (a piramis a futásidőről szól).

#138 Middleware részletesen – pipeline, sorrend, saját middleware

Verzió: ASP.NET Core 1.0-tól; `IMiddleware` (factory-alapú): ASP.NET Core 2.0-tól; endpoint filter (Minimal API): .NET 7-től.

Mi a middleware szerepe

A middleware a HTTP kérés-feldolgozás **láncszeme**: minden kérés végigmegy rajta oda- és visszaúton (ezért gyakran "hagyma" vagy "orosz baba" modellként írják le). Egy middleware három dolgot tehet: feldolgozza a kérést és továbbadja (`await next()`), módosítja a választ a visszaúton, vagy **rövidre zárja** a láncot (nem hívja a `next` -et, azonnal válaszol – pl. 401).

Middleware-be az tartozik, ami **minden (vagy sok) kérésre** érvényes, és **nem egy konkrét végpont üzleti dolga** – ezek a keresztirányú aggályok (cross-cutting concerns):

1. Kivételkezelés (`ExceptionHandler`) és status code oldalak.
2. Kérés/válasz naplózás, correlation ID és log-scope beállítása.
3. Autentikáció, autorizáció.
4. HTTPS átirányítás, HSTS, biztonsági fejlécek, CORS.
5. Rate limiting, request size limit, timeouts.
6. Tömörítés (`ResponseCompression`), statikus fájlok kiszolgálása.
7. Routing és endpoint végrehajtás.
8. Multi-tenant kontextus felállítása (lásd "Multi-tenancy" fejezet).
9. Output caching, response caching.
10. Health check és metrika végpontok kiszolgálása.

Amit **nem** ide teszünk: üzleti szabályt, adatbázis-tranzakciót egy konkrét use case-hez, végpont-specifikus validációt (arra endpoint filter / MediatR behavior való), bármit, ami csak egyetlen route-ot érint.

A sorrend a legfontosabb – és nem szabad tippelni

A sorrend **szigorúan** a regisztráció sorrendje. A javasolt, gyakorlatban működő felállítás:

```

app.UseExceptionHandler();           // 1. LEGELSŐ: minden alatta lévő hibáját elfogja
app.UseStatusCodePages();
app.UseHsts();                       // csak produkcióban
app.UseHttpsRedirection();
app.UseResponseCompression();
app.UseStaticFiles();               // routing ELŐTT: statikus fájl ne menjen végig a teljes
pipeline-on
app.UseRequestLocalization();

app.UseRouting();                   // innentől ismert az endpoint metaadata
app.UseCors();                     // routing UTÁN, auth ELŐTT
app.UseRateLimiter();
app.UseAuthentication();           // ki vagy?
app.UseAuthorization();             // szabad-e? (mindig az Authentication UTÁN)
app.UseOutputCache();               // auth után: ne szolgáljunk ki cache-ből védett tartalmat

app.UseMiddleware<CorrelationIdMiddleware>(); // a saját, kontextust felállító middleware-ek
app.UseMiddleware<TenantResolutionMiddleware>();

app.MapControllers();               // endpoint végrehajtás - a lánc vége
app.MapHealthChecks("/health/ready");

```

Tipikus sorrend-hibák és tünetük: a kivételkezelő túl későn → a korábbi middleware hibái nem alakulnak `ProblemDetails`-szé; `UseCors` a `UseRouting` előtt → a preflight kérés nem kapja meg a végpont CORS-policyját; `UseAuthorization` az `UseAuthentication` előtt → mindig 401/403; `UseStaticFiles` a routing után → felesleges munka minden képfájltra; caching az auth előtt → **más felhasználó adata szolgálódik ki.**

Saját middleware három formában

```

// 1. Inline (lambda) - rövid, egyszeri logikára
app.Use(async (context, next) =>
{
    context.Response.Headers["X-API-Version"] = "1.0";
    await next(context);
});

```

```
// 2. Konvenció-alapú osztály: a konstruktorban SINGLETON élettartamú (egyszer jön létre!)
// → Scoped függőséget CSAK az InvokeAsync paraméterként kérj
public class RequestLoggingMiddleware(RequestDelegate next, ILogger<RequestLoggingMiddleware>
logger)
{
    public async Task InvokeAsync(HttpContext context, ITenantContext tenantContext) //
scoped: paraméterként
    {
        var stopwatch = Stopwatch.GetTimestamp();
        try
        {
            await next(context);
        }
        finally
        {
            logger.LogInformation("{Method} {Path} → {StatusCode} ({Elapsed:0.0} ms)",
                context.Request.Method, context.Request.Path, context.Response.StatusCode,
                Stopwatch.GetElapsedTime(stopwatch).TotalMilliseconds);
        }
    }
}
app.UseMiddleware<RequestLoggingMiddleware>();
```

```
// 3. Factory-alapú (IMiddleware): scoped élettartam, DI-ból oldódik fel kérésenként
public class AuditMiddleware(IAuditWriter writer) : IMiddleware // IAuditWriter lehet Scoped
{
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        await next(context);
        await writer.WriteAsync(context.Request.Path, context.Response.StatusCode,
context.RequestAborted);
    }
}
builder.Services.AddScoped<AuditMiddleware>(); // regisztrálni KELL
app.UseMiddleware<AuditMiddleware>();
```

Elágazás és feltételes futtatás

```
app.UseWhen(ctx => ctx.Request.Path.StartsWithSegments("/api"),
    branch => branch.UseMiddleware<ApiKeyMiddleware>()); // csak /api alatt, majd visszatér a
fő láncba

app.Map("/admin", admin => admin.UseMiddleware<AdminOnlyMiddleware>()); // önálló ág, nem
tér vissza

app.MapWhen(ctx => ctx.Request.Headers.ContainsKey("X-Legacy"),
    legacy => legacy.Run(async ctx => await ctx.Response.WriteAsync("Legacy endpoint")));
```

Middleware vs. endpoint filter vs. action filter vs. MediatR behavior

Eszköz	Hatókör	Ismeri az endpointot	Tipikus felhasználás
Middleware	minden kérés (vagy útvonal-ág)	csak <code>UseRouting</code> után	biztonság, log, hibakezelés, tömörítés
Endpoint filter (Minimal API)	egy végpont / route group	igen, a paramétereket is	validáció, végpont-specifikus ellenőrzés
Action filter (MVC)	controller / action	igen	modell-validáció, MVC-specifikus logika
MediatR / Wolverine behavior	egy use case (command/query)	nem (HTTP-független)	tranzakció, validáció, retry, audit

Ökölszabály: ha a logika **HTTP-ről** szól → middleware vagy filter; ha a **use case-ről** → pipeline behavior (így üzenetsorból hívott handlernél is lefut).

Gyakorlati szabályok

- A middleware osztály **egyszer** jön létre (konvenció-alapú esetben): nincs kérésállapot mezőben, minden per-request adat a `HttpContext`-ben vagy scoped szolgáltatásban.
- Ha a válasz törzsét kell olvasni/módosítani (pl. audit), a `Response.Body` streamet ki kell cserélni – ez költséges, csak ott tedd, ahol tényleg kell (és sose logold a teljes választ érzékeny adattal).
- A kérés törzsének többszöri olvasásához `context.Request.EnableBuffering()` szükséges.
- A `next` hívása utáni kódban a válasz fejlécei **már elküldve lehetnek** (`Response.HasStarted`) – fejléc módosítás előtt ezt ellenőrizd, vagy használd az `OnStarting` callbacket.
- Minden middleware adjon tovább `context.RequestAborted` tokenet az általa indított I/O-nak.

Mit kerülj el: ne tegyél üzleti logikát middleware-be; ne blokkolj szinkron módon (`.Result`) middleware-ben (thread pool starvation minden kérésre); ne felejtsd el a `await next()`-et (a kérés csendben "elhal"); ne írd saját autentikációt middleware-ként, ha a beépített séma-alapú megoldás megteszi.

#139 Azonosító-generálási stratégiák .NET-ben – melyiket mikor

Verzió: `Guid.CreateVersion7()` : .NET 9-től; `Guid.NewGuid()` (v4): .NET Framework 1.0-tól; a többi külső NuGet csomag.

Gyakorlati referencia a `Guid`, `NanoID`, `ULID`, `sequential/COMB GUID`, `Snowflake`-jellegű azonosítók és a `sima auto-increment` egész közötti választáshoz.

Összehasonlító táblázat

Típus	Hossz	Időrendben sorolható	Ütközési kockázat	Index-barát	URL/ember-barát	Beépített?	Tipikus csomag
Guid (v4, véletlen)	128 bit / 36 karakter kötőjelekkel	nem	elhanyagolható	gyenge (véletlen beszúrás = lapszétvágás)	nem	igen – System.Guid	–
Guid v7 (időrendezett)	128 bit / 36 karakter	igen (milliszekundum)	elhanyagolható	jó	nem	igen, .NET 9+ – Guid.CreateVersion7()	–
Sequential/COMB GUID (.NET 9 előtt)	128 bit / 36 karakter	igen (közelítőleg)	elhanyagolható	jó	nem	nem	UUIDNext, EF Core NEWSEQUENTIALID(), NHibernate Guid.Comb
ULID	128 bit / 26 karakter (Crockford base32)	igen (milliszekundum)	elhanyagolható	jó	jobb, mint a Guid	nem	Ulid (Csharp), NULid
NanoID	konfigurálható, alpból 21 karakter	nem (csak ha időbélyeg-prefixet tesz el)	nagyon alacsony (ábécével/hosszal állítható)	gyenge, prefix nélkül	igen – rövid, URL-biztos	nem	Nanoid (nanoid-net)
Snowflake-jellegű (64 bites egész)	64 bit / ~19 számjegy	igen	nincs, ha a worker ID-k helyesen vannak beállítva	kiváló (numerikus, monoton)	közepes	nem	IdGen (RobThree), SnowflakeGenerator
Auto-incrementing (adatbázis identity)	32/64 bit	igen	nincs (egy adatbázis esetén)	kiváló	közepes	igen (adatbázis funkció)	–
Hashids / obfuscált azonosító	változó, rövid	nem	nem értelmezhető (kódolás, nem generátor)	nem értelmezhető – csak megjelenítésre	igen	nem	HashidsNet
ShortGuid (base64-kódolt Guid)	22 karakter	a mögöttes Guid-től függ	mint a Guid	mint a Guid	jobb, mint a nyers Guid	nem	saját, vagy CSharpVita mins.ShortGuid

Mi micsoda, egyszerűen

Guid (v4, véletlen) – `Guid.NewGuid()` . Globális egyediség gépek közti koordináció nélkül; a klasszikus .NET/EF Core alapértelmezés. Hátránya: klaszterezett indexbe (SQL Server, MySQL) véletlen értéként beszúrva index-fragmentálódást és lapszétvágást okoz, mert az új értékek a B-fa véletlen pozícióiba kerülnek, nem a végére.

Guid v7 (időrendezett UUID) – .NET 9-től `Guid.CreateVersion7()` . A felső bitekben milliszekundum pontosságú Unix időbélyeget hordoz, így a később generált értékek a korábbiak után rendeződnek. Megszünteti a v4 index-fragmentálódási problémáját, miközben marad sima `Guid / uniqueidentifier` – nincs séma-változás. **Új .NET projektben ez ma a pragmatikus alapértelmezés** GUID-alakú kulcshoz, ha a minimum cél .NET 9+.

Sequential/COMB GUID (.NET 9 előtti megoldás) – a `CreateVersion7` előtt ugyanezt a hasznát adta a `SqlServer.NEWSEQUENTIALID()` , az NHibernate `Guid.Comb` vagy a UUIDNext szekvenciális generátora. Továbbra is érvényes, ha .NET 8-on vagy korábban ragadtál, vagy kifejezetten az SQL Server bájt-sorrendjére van szükség a klaszterezett index miatt.

ULID – 128 bit, mint a `Guid` , de 26 Crockford-base32 karakterként kódolva (kis-nagybetű független, padding nélkül, stringként is lexikografikusan sorolható). Event sourcing / üzenetküldő rendszerekben népszerű, ahol sorolható, kompakt, nem numerikus azonosító kell, ami logban is barátságosabb, mint egy `Guid` . A `Cysharp/Ulid` a gyakori .NET implementáció.

NanoID – nem UUID: rövid, URL-biztos véletlen sztring-generátor (alapból 21 karakter egy 64 szimbólumos ábécéből), tipikusan publikus azonosítókhoz (rövid linkek, slugok, API kulcsok), mert azonos ütközés-ellenállás mellett jóval rövidebb, mint egy `Guid` , és az ábécé/hossz konfigurálható.

Nincs beépített időrendezés – ha sorolhatóság kell, magad teszt el időbélyeget. A standard .NET port a NuGeten `Nanoid` (nanoid-net).

Snowflake-jellegű 64 bites azonosítók – időbélyeg + worker/shard azonosító + milliszekundumon belüli sorszám egy `long` -ba csomagolva. Nagy léptéknél használatos (eredetileg Twitter), ha kompakt, sorolható, közel monoton numerikus azonosító kell több gépen, központi szekvencia/identity oszlop nélkül. A standard .NET könyvtár az `IdGen`; **példányonként egyedi worker/generátor ID kiosztását igényli** – ez egy kis koordinációs költség, ellentétben a `Guid` -dal.

Auto-increment `int / long` (identity/serial oszlop) – a legegyszerűbb megoldás, ha egyetlen írható adatbázis van. Legkisebb kulcs, legjobb index-lokalitás, ember-barát URL-ben és support jegyben; viszont információt szivároztat (sorok száma, növekedés üteme), és nem működik jól offline/kliensoldali generálásnál, illetve multi-master vagy shardolt adatbázisnál.

Hashids – nem generátor, hanem obfuszkátor. Egy meglévő egész/szekvenciális azonosítót kódol visszafejthetően rövid, opak sztringgé megjelenítéshez (hogy a felhasználó ne `/orders/1042` -t lásson). Ne használd tárolt kulcsként – csak a megjelenítési/API határon, ha a szekvenciális azonosítót el akarsz rejteni anélkül, hogy végig `Guid / NanoID`-ra váltanál.

ShortGuid – egy normál Guid 16 bájtyát base64-ként (22 karakter) kódolja a kötőjeles hexa (36 karakter) helyett. Ugyanaz az entrópia és ugyanaz a sorolhatóság, csak rövidebb megjelenítéshez/kopírozáshoz. Akkor hasznos, ha a tárolt típus már Guid , de barátságosabb sztring-forma kell URL-hez/loghoz.

Javaslatok forgatókönyv szerint

Forgatókönyv	Választás
Elsődleges kulcs új EF Core / SQL projektben, .NET 9+	Guid v7 (Guid.CreateVersion7()) – globális, kliensen generálható egyediség + jó klaszterezett-index viselkedés, új függőség és séma-típusváltás nélkül
.NET 8 vagy korábbi, de GUID-alakú, index-barát kulcs kell	Sequential/COMB GUID (UUIDNext szekvenciális mód, NEWSEQUENTIALID() + ValueGeneratedOnAdd() , NHibernate Guid.Comb)
Publikus, rövid azonosító – rövid link, meghívó kód, API kulcs, URL slug	NanoID – rövid, URL-biztos, az ütközési kockázat hosszal/ábécével szabályozható, és nem szivárogtat sorrend-információt, mint az auto-increment
Elosztott / event-sourced rendszer, sorolható, kompakt, logban olvasható azonosítóval	ULID – 26 karakter, sima szövegként rendezhető, nincs kötőjel, nincsenek SQL Server bájt-sorrend furcsaságok
Nagy átbocsátás, sok írócsomópont/shard, kompakt 64 bites numerikus azonosító	Snowflake (IdGen) – numerikus, int -ként indexelhető, közel időrendezett, vízszintesen skálázódik, ha minden csomópont külön worker ID-t kap
Egy adatbázisos CRUD alkalmazás, belső eszköz, admin felület	Auto-increment int / long – legkisebb, leggyorsabban indexelhető, legegyszerűbb; HashIds-szel kombinálva, ha a nyers értéket el akarod rejteni a felhasználó elől
Már mindenhol Guid van, de szebb megjelenítés kell	ShortGuid csak megjelenítésre; a tárolt típus maradjon Guid

Kódpéldák

```
// Véletlen Guid v4 (klasszikus, beépített)
Guid id = Guid.NewGuid();

// Időrendezett Guid v7 (.NET 9+, beépített)
Guid id7 = Guid.CreateVersion7();
// Guid.CreateVersion7(DateTimeOffset.UtcNow) - az időbélyeg explicit megadásához

// NanoId (dotnet add package Nanoid)
using Nanoid;
string shortId = Nanoid.Generate(); // pl. "V1StGXR8_Z5jdHi6B-
myT", 21 karakter
string customId = Nanoid.Generate(alphabet: "0123456789ABCDEF", size: 10);

// ULID (dotnet add package Ulid)
Ulid ulid = Ulid.NewUlid();
string ulidString = ulid.ToString(); //
pl. "01ARZ3NDEKTSV4RRFFQ69G5FAV"

// Snowflake-jellegű (dotnet add package IdGen)
using IdGen;
var generator = new IdGenerator(generatorId: 0); // csomópontonként egyedinek
KELL lennie
long snowflakeId = generator.CreateId();

// Sequential/COMB Guid .NET 9 nélkül (dotnet add package UUIDNext)
using UUIDNext;
Guid sequentialGuid = Uuid.NewSequential();
```

Vegyes stratégia – ami a gyakorlatban a leggyakoribb

Enterprise alkalmazásban jellemzően **nem** egyetlen azonosító-típus van, hanem szerep szerint kettő: a **technikai kulcs** (Guid v7) és a **publikus azonosító** (NanoID, ULID vagy üzleti azonosító, pl. ORD-2026-000123). A kettő szétválasztásának haszna, hogy a publikus azonosító formátuma később megváltoztatható a séma és az idegen kulcsok érintése nélkül – és a belső kulcs sosem kerül a felhasználó szeme elé.

```
public class Order
{
    public Guid Id { get; private set; } = Guid.CreateVersion7(); // technikai kulcs,
index-barát
    public string PublicId { get; private set; } = Nanoid.Generate(size: 12); // URL-ben ez
jelenik meg
}
```

Amire figyelni kell

- **SQL Server klaszterezett index:** a Guid v7 bájt-sorrendje **nem** azonos a NEWSEQUENTIALID() SQL Server-specifikus sorrendjével, ezért a v7 rendezettsége az SQL Server uniqueidentifier összehasonlításában nem érvényesül teljesen. Ha az SQL Server klaszterezett index-lokalitás kritikus, mérd meg, vagy tartsd meg a NEWSEQUENTIALID() -t. PostgreSQL-en (uuid , bájt-sorrend szerinti összehasonlítás) a v7 rendezettsége érvényesül.

- **Az időrendezett azonosító információt szivárogtat:** a `Guid` v7 és a ULID is tartalmazza a létrehozás időpontját, a Snowflake pedig a worker ID-t is. Ha ez üzletileg érzékeny (pl. látszik, mikor jött létre egy fiók), publikus azonosítóként inkább NanoID.
- **Kulcstípus és index-méret:** sztring kulcs (NanoID, ULID sztringként tárolva) nagyobb indexet és lassabb join-t jelent. Ha ULID-ot használsz, `Guid` / `uuid` oszlopban tárold a 128 bitet, és csak megjelenítéskor konvertáld sztringgé.
- **Ütközés-ellenállás NanoID-nál a hossz függvénye:** az alapértelmezett 21 karakter gyakorlatilag ütközésmentes, de ha 8-10 karakterre rövidíted (hogya diktálható legyen), a várható rekordszámhoz kell méretezni, és egyedi indexre + újragenerálási logikára van szükség.
- **Kliensoldali generálás:** `Guid`, ULID, NanoID és Snowflake mind generálható az alkalmazásban (ez fontos az Outbox mintához, a saga korrelációhoz és az idempotencia-kulcsokhoz – lásd a vonatkozó fejezeteket); az auto-increment nem.

Egy mondatban

.NET 9+ esetén az általános célú alapértelmezés a `Guid.CreateVersion7()` (kiváltja azt a legtöbb okot, amiért korábban COMB GUID-hoz vagy `Guid.NewGuid()` -hoz nyúltak); NanoID kifejezetten akkor, ha az azonosító URL-ben jelenik meg vagy ember írja/olvassa; ULID, ha az ökoszisztéma (event sourcing, message broker) már azt preferálja; IdGen/Snowflake, ha sok csomópont generál kompakt numerikus azonosítót; és sima adatbázis auto-increment, ha egyetlen írható adatbázis van, és nincs szükség azon kívüli azonosító-generálásra.